The Shape of Possibility

Memory, Constraint, and Admissible Change

Flyxion
Independent Researcher

Working skeleton, October 2026

Contents

Division of Labour Between the Volumes **xi**

Preface **xiii**

I Spaces of possible action **1**

1 Three Obstructions in a Controlled System **3**
1.1 States, actions, and feasible sets . . . 3
1.2 A common model . . . 4
1.3 The obstructions in the rover . . . 6
1.4 Plan of the volume and conventions . . . 6
1.5 A complete census of the rover . . . 7
1.6 Obligations enlarge the state, and gates read it back . . . 7
1.7 Four sets and how to compare them . . . 8
Exercises . . . 9

2 States, Actions, and Finite Reachability **11**
2.1 Reachability in a finite system . . . 11
2.2 Components are the minimal representation for reachability . . . 12
2.3 Reversibility and the ratchet . . . 12
2.4 Menus and reachable sets . . . 13
2.5 Resources and the expanded graph . . . 13
2.6 The reach profile . . . 14
2.7 Counting trajectories through the condensation . . . 14
2.8 Dead ends and one-way doors . . . 15
Exercises . . . 16

3 Constraints, Convexity, and Viability **17**
3.1 The viability kernel . . . 17
3.2 Viable histories and the kernel . . . 18
3.3 Convexity . . . 19
3.4 Continuous time . . . 19
3.5 Kernels and their algebra . . . 20
3.6 The stopping-distance kernel . . . 20
3.7 Reading the kernel as a design tool . . . 21

Exercises . . . 21

II Stochastic, partially observed, and resource-bounded systems 23

4 Markov Decision Processes and Constrained Variants 25
4.1 Discounted Markov decision processes . . . 25
4.2 Feasibility with certainty . . . 26
4.3 Optimizing under an almost-sure constraint . . . 27
4.4 Constraints on expected cost . . . 27
4.5 The price of certainty in the excursion . . . 28
4.6 Where randomization occurs . . . 29
4.7 Value in the budget as a state variable . . . 29
Exercises . . . 30

5 Partial Observation and Belief-State Sufficiency 31
5.1 Beliefs and their update . . . 31
5.2 The minimal sufficient representation for prediction . . . 32
5.3 Certain safety with partial observation . . . 33
5.4 A machine with a hidden fault . . . 34
5.5 Value of information in a safety game . . . 34
5.6 The value function of the belief MDP . . . 35
Exercises . . . 35

6 Resource-Sensitive Transitions and Petri Nets 37
6.1 Nets and firing . . . 37
6.2 Necessary conditions for reachability . . . 38
6.3 Monotonicity and upward-closed sets . . . 38
6.4 What is known about the complexity . . . 39
6.5 Backward coverability . . . 39
6.6 The coverability tree and unboundedness . . . 40
6.7 Invariants as design constraints . . . 40
Exercises . . . 41

III Histories that change feasibility 43

7 History-Dependent State and Augmentation 45
7.1 Continuation classes and the fibre count . . . 45
7.2 Three augmentations . . . 46
7.3 When no finite augmentation exists . . . 47
7.4 Checking sufficiency of a given description . . . 48
7.5 Rate limits . . . 48
Exercises . . . 49

8 Irreversibility and Path Dependence **51**
8.1 The cost of return . . . 51
8.2 Path dependence . . . 53
8.3 Policies and the components they induce . . . 53
8.4 The reversibility profile . . . 54
8.5 Making a system reversible . . . 54
8.6 Expected cost of return in a stochastic system . . . 55
Exercises . . . 55

9 Endogenous Action Spaces **57**
9.1 Capability systems . . . 57
9.2 The monotone case . . . 58
9.3 Gains and losses . . . 59
9.4 A workshop . . . 60
9.5 Capabilities are complements . . . 61
Exercises . . . 61

10 Establishing a Domain of Choice **63**
10.1 A model of acquisition stages . . . 63
10.2 Existence and computation . . . 64
10.3 What an oracle is worth . . . 65
10.4 Diminishing returns of an established domain . . . 65
10.5 Admissible acquisition under a budget . . . 66
10.6 A three-option instance . . . 67
10.7 The anatomy of the gap to the oracle . . . 67
Exercises . . . 68

IV Measures of possibility **69**

11 Counting Options and Comparing Futures **71**
11.1 Four counts that do not agree . . . 71
11.2 An order that admits incomparability . . . 72
11.3 Simulation and coverage . . . 72
11.4 Growth of feasible histories . . . 73
11.5 Dominance as a comparison of cost vectors . . . 74
11.6 A rate limit as a growth rate . . . 75
Exercises . . . 75

12 Costs of Access and Continuation **77**
12.1 Cost algebras . . . 77
12.2 Several costs at once . . . 78
12.3 Costs that are shared and costs that overlap in time . . . 80
12.4 The duality gap of a single mission . . . 81
12.5 Adding workers can slow a schedule . . . 81
Exercises . . . 82

13 Robustness Under Uncertainty **83**
13.1 Four kinds of guarantee . . . 83
13.2 Robust viability . . . 84
13.3 Chance constraints . . . 85
13.4 Model uncertainty . . . 86
13.5 The price of not knowing the distribution . . . 87
13.6 A horizon allocation in numbers . . . 87
Exercises . . . 88

V Preserving future admissibility **91**

14 Viability Under Evolving Obligations **93**
14.1 The product with an obligation monitor . . . 93
14.2 Slack and the effective constraint set . . . 94
14.3 A corridor . . . 95
14.4 Interaction between obligations . . . 96
14.5 Size of the corridor kernel . . . 97
14.6 When two duties can be held together . . . 97
Exercises . . . 98

15 The Cost of Future Admissibility **101**
15.1 The price of viability . . . 101
15.2 The cost of foresight . . . 102
15.3 Preserving many options . . . 103
15.4 Option capacity of the speed-limited line . . . 104
Exercises . . . 105

16 Corrective Reachability and Fallback Regions **107**
16.1 The recoverable region . . . 107
16.2 Local corrections and trajectory guarantees . . . 108
16.3 Possible and forced recovery . . . 109
16.4 Fallback regions for degradation . . . 109
16.5 A navigation stack as a worked fallback region . . . 111
Exercises . . . 112

VI Adaptive control and institutions **113**

17 Aspect Relegation as Hybrid Switching **115**
17.1 Switched systems with a fault model . . . 115
17.2 When no condition is needed, and when it is not enough . . . 116
17.3 The conditional theorem . . . 116
17.4 The hypotheses are sharp . . . 117
17.5 Monitoring and the cost of detection . . . 118
17.6 The exact region of safe switching parameters . . . 118
Exercises . . . 119

18 Interfaces as Action Abstraction **121**
18.1 Interfaces and admissible reachability . . . 121
18.2 Possible and forced reachability . . . 123
18.3 The smallest interface and the cost in steps . . . 123
Exercises . . . 125

19 Institutional Revision as Augmented Control **127**
19.1 The augmented system . . . 127
19.2 The amendment depth . . . 128
19.3 Entrenchment . . . 129
19.4 Capture of a quorum . . . 130
Exercises . . . 131

20 Proxies and the Deformation of Possibility **133**
20.1 The uniform bound and its sharpness . . . 133
20.2 Transfer is governed by the error on the policies compared . . . 134
20.3 Constrained problems with proxied constraints . . . 135
20.4 Safety need not depend on the proxy . . . 136
20.5 Selection among many options: the winner's overstatement . . . 136
Exercises . . . 137

VII Learning and design **139**

21 Exploration and Unattempted Alternatives **141**
21.1 Three epistemic statuses . . . 141
21.2 When an attempt pays . . . 142
21.3 Safe exploration and the knowledge-indexed kernel . . . 143
21.4 Learning while exploring . . . 145
Exercises . . . 146

22 Maintaining and Acquiring Revision Sufficiency **147**
22.1 Sizes of sufficient representations for families . . . 147
22.2 The cost of a discarded distinction . . . 148
22.3 Many distinctions at once . . . 150
22.4 Counters on a common event . . . 150
22.5 Randomized maintenance . . . 151
Exercises . . . 151

23 Architecture for Revisable Systems **153**
23.1 The design problem . . . 153
23.2 When to commit . . . 154
23.3 The constrained problem . . . 154
23.4 The price of flexibility . . . 155
23.5 A weighted instance . . . 156
Exercises . . . 157

24 Integrated Case Studies and Research Directions **159**
24.1 A resource-limited autonomous system . . . 159
24.2 A governed evidence archive . . . 160
24.3 An adaptive controller . . . 161
24.4 An infrastructure network . . . 162
24.5 Research directions . . . 162
24.6 The vehicle under random costs . . . 163
Exercises . . . 163

A Sufficiency Recap from the First Volume **165**
A.1 Kernels and factorization . . . 165
A.2 Sufficiency . . . 165
A.3 Histories, continuation equivalence, and maintainable representations . . . 166
A.4 Safety and viable histories . . . 166

B Linear Algebra and Convexity **169**
B.1 Spectral radius and growth . . . 169
B.2 Convex sets, support functions, and scalarization . . . 170
B.3 Linear inequalities . . . 170
B.4 Worked examples . . . 170
Exercises . . . 171

C Graph Algorithms **173**
C.1 Components and shortest paths . . . 173
C.2 Computing the viability kernel . . . 173
C.3 Closure and hardness . . . 174
C.4 Worked example of the kernel algorithm . . . 174
Exercises . . . 175

D Dynamical Systems **177**
D.1 Invariance and viability . . . 177
D.2 Lyapunov functions . . . 177
D.3 Contraction . . . 178
D.4 Worked examples . . . 178
Exercises . . . 178

E Optimization **181**
E.1 Linear programming . . . 181
E.2 Dynamic programming . . . 181
E.3 Convex optimization . . . 182
E.4 Submodular maximization . . . 182
E.5 Worked examples . . . 182
Exercises . . . 182

F Stochastic Processes **185**
F.1 Finite Markov chains . . . 185
F.2 Elementary inequalities and laws . . . 185
F.3 Gaussian tails . . . 186
F.4 Worked examples . . . 186
Exercises . . . 186

G Numerical Methods **187**
G.1 Fixed-point iteration for kernels and values . . . 187
G.2 Spectral radius . . . 187
G.3 Monte Carlo certification of chance constraints . . . 188
G.4 Floating-point and thresholds . . . 188
G.5 Worked examples . . . 188
Exercises . . . 189

Division of Labour Between the Volumes

The governing rule is short. *Distinction and Continuation* defines and proves what a representation must preserve. *The Shape of Possibility* studies how actions transform reachable futures and what that transformation costs. When a topic appears in both volumes, the first volume owns its definitions and its sufficiency or impossibility theorems, and the second volume cites those definitions and adds only the dynamics, the optimization, and the computation of regions and costs. Each volume carries a self-contained appendix recapping the statements it needs from the other, so either can be read first.

Topic	**Distinction and Continuation**	**The Shape of Possibility**
Obligations	Semantics, monitors, and temporal specification of commitments that cannot be read from one state (Ch. 14).	Viability under evolving obligations: which regions allow continued compliance (Ch. 14).
Degradation	Characterization of admissible shedding from obligations and dependency structure (Ch. 17).	Computation of recovery and fallback regions and their cost (Ch. 16).
Revision	Revision sufficiency and the revision horizon (Ch. 21).	The cost of maintaining or acquiring revision sufficiency (Ch. 22).
History and state	Sufficient representations, incremental sufficiency, bisimulation (Ch. 5–8).	State augmentation and path-dependent feasibility (Ch. 7).
Interfaces	Delegated authority, boundary contracts, compatibility conditions (Ch. 20).	Action abstraction and simulation: when an interface preserves admissible reachability (Ch. 18).
Identity and provenance	Identity criteria relative to promised operations; lineage (Ch. 11, 15).	Cited only; no independent development.

Every result in either volume carries one of three statuses, displayed in its heading. An *established result* is attributed to its mathematical source. A *new result* is proved in full under assumptions stated beside it. A *proof obligation* is a proposed statement whose proof has not yet survived scrutiny; it is displayed as such and nothing downstream may cite it as a theorem. The companion ledger `proof-spine.tex` lists every load-bearing result with its status, assumptions, and dependencies.

Preface

This volume asks how present actions change the structure of what can subsequently be reached, chosen, or revised. It begins with finite graphs, convex constraints, and viability, adds stochastic, partially observed, and resource-sensitive models, and then develops history-dependent feasibility, the acquisition of new capabilities, and the cost of preserving future alternatives. The final parts give concrete mathematical commitments to adaptive control, interfaces, institutional revision, and proxy optimization, each built on a named formal core with a theorem the chapter is organized around. Results are labelled as established, new, or proof obligations, and no proof obligation is cited as a theorem. Appendix A recaps the sufficiency results needed from the first volume, so this volume can be read first.

Part I

Spaces of possible action

Chapter 1

Three Obstructions in a Controlled System

Question. In how many distinct ways can a proposed continuation become unavailable, and how does a controlled system exhibit each?

Established sources. Factorization of a function through a quotient.

New results. Conservative gating removes admissible futures (Result 1.6); each of the three obstructions occurs in a single running example (Section 1.3).

Proof obligations. None.

Depends on. Nothing.

Most analyses of a controlled system begin with a state space, a menu of actions, and a rule for how actions change the state. The menu is usually written down once. What the system can actually do at a given moment is a smaller set, and what it is entitled to do is smaller again. A fourth set, what a decision procedure can tell from the information it retains, is smaller or larger depending on what was kept. This chapter separates these four and introduces the model used throughout the volume.

1.1 States, actions, and feasible sets

Definition 1.1 (Controlled system)**.** A *controlled system* consists of a set $\mathcal{X}$ of states, a set U of actions, a *feasibility correspondence* $F : \mathcal{X} \to 2^U$ giving the actions that can be executed in each state, and a transition function $\delta : \{(x, u) : u \in F(x)\} \to \mathcal{X}$. The *nominal menu* is U. A *trajectory* from x_0 is a sequence $x_0, u_0, x_1, u_1, \ldots$ with $u_t \in F(x_t)$ and $x_{t+1} = \delta(x_t, u_t)$.

The nominal menu U is what a user interface or a specification lists. The correspondence F is what the physical system allows. The distinction is the first thing to check when someone says that a system "has" an option.

Example 1.2 (A rover with a battery)**.** A rover moves among four positions $0, 1, 2, 3$ on a line, with the dock at position 0. The state is a pair (p, b) of position and battery level $b \in \{0, 1, 2, 3\}$. The nominal menu is $U = \{\text{left}, \text{right}, \text{charge}, \text{sample}, \text{upload}\}$. A move changes p by ± 1 and costs one unit of battery; it is feasible only if $b \geq 1$ and the target position exists. The action charge is feasible only at $p = 0$ and sets $b = 3$. The actions sample and

upload cost no battery and are feasible only at $p = 2$ and $p = 0$ respectively, and neither changes the position or the battery. The nominal menu has five elements at every state, while $F(0, 3) = \{\text{right}, \text{charge}, \text{upload}\}$, $F(1, 0) = \varnothing$, and $F(2, 1) = \{\text{left}, \text{right}, \text{sample}\}$.

Example 1.3 (An unstable scalar system)**.** The state is a real number x and the action is a real number u with $|u| \le 1$, so $F(x) = [-1, 1]$ at every state. The transition is $x_{t+1} = 2x_t + u_t$. The constraint is $|x_t| \le 3$ at every time. The nominal menu is the interval $[-1, 1]$ at every state, and the constraint is satisfied at $x = 2.5$, but from $x = 2.5$ every action leads to $|x_1| \ge 4 > 3$. The state satisfies the constraint and has no feasible action that continues to satisfy it. Chapter 3 treats this system in full.

The second example previews a theme of the volume: that constraints satisfied now do not guarantee that the constraints can be satisfied later. The first example previews another: that the nominal menu and the executable set differ and that the difference depends on retained information.

1.2 A common model

Let E be a set of events and let $\mathcal{H} \subseteq \mathsf{E}^*$ be a prefix-closed set of finite words, the *histories*. Write $h \cdot w$ for the history obtained by appending the word w to h, and $\mathrm{Ext}(h) = \{w : h \cdot w \in \mathcal{H}\}$ for the extensions available after h. Each history determines three things.

- Its *physical state* $x(h) \in \mathcal{X}$, which fixes the set $\mathrm{Reach}(x(h)) \subseteq \mathrm{Ext}(h)$ of continuations that can physically be executed.
- Its *retained representation* $\sigma(h) \in \Sigma$, which is everything a decision procedure is permitted to consult.
- Its *obligations* $\mathrm{Obl}(h)$, a set of commitments that restrict which continuations count as acceptable.

A *proposed continuation* after h is a nonempty set $c \subseteq \mathrm{Ext}(h)$ of words, understood as a specification that any acceptable realization must meet. Its realizations are $\mathrm{Real}(h, c) = c \cap \mathrm{Reach}(x(h))$, and its admissible realizations are those realizations w for which every obligation in $\mathrm{Obl}(h)$ is satisfied along $h \cdot w$. We write $\mathrm{Adm}(h, c)$ for that set.

Definition 1.4 (Three obstructions)**.** A proposed continuation c after h is subject to

(i) the *physical obstruction* $\mathsf{O}_{\mathrm{phys}}(h, c)$ when $\mathrm{Real}(h, c) = \varnothing$;
(ii) the *obligation obstruction* $\mathsf{O}_{\mathrm{obl}}(h, c)$ when $\mathrm{Real}(h, c) \neq \varnothing$ and $\mathrm{Adm}(h, c) = \varnothing$;
(iii) the *representational obstruction* $\mathsf{O}_{\mathrm{rep}}(h, c)$ when there is a history h' with $\sigma(h') = \sigma(h)$ and $\mathbf{1}[\mathrm{Adm}(h', c) \neq \varnothing] \neq \mathbf{1}[\mathrm{Adm}(h, c) \neq \varnothing]$.

The first two are properties of the world and of the commitments in force. The third is a property of the pair $(\sigma, \text{decision})$: it records that the retained representation fails to separate histories that require different answers. The obstructions are analytically distinct but they interact, and Example 1.7 below shows one form of the interaction. They are therefore presented as distinct obstructions and not as mutually exclusive cases.

The representational obstruction

Let $P : \mathcal{H} \to \mathcal{D}$ assign to each history the decision it requires. For admissibility of a fixed c, take $\mathcal{D} = \{0, 1\}$ and $P(h) = \mathbf{1}[\mathrm{Adm}(h, c) \neq \varnothing]$.

Established result 1.5 (Factorization through a representation)**.** *There is a map $\delta : \sigma(\mathcal{H}) \to \mathcal{D}$ with $\delta \circ \sigma = P$ if and only if $\sigma(h_1) = \sigma(h_2)$ implies $P(h_1) = P(h_2)$ for all $h_1, h_2 \in \mathcal{H}$. In particular, if $\sigma(h_1) = \sigma(h_2)$ but $P(h_1) \neq P(h_2)$, then no decision procedure that receives only $\sigma(h)$ returns P correctly on both h_1 and h_2.*

Source: Standard factorization of a function through a quotient; developed in the chapter on equivalence relations and quotient constructions in the first volume.

Proof. If $\delta \circ \sigma = P$ and $\sigma(h_1) = \sigma(h_2)$, then $P(h_1) = \delta(\sigma(h_1)) = \delta(\sigma(h_2)) = P(h_2)$. Conversely, if equal representations force equal decisions, define $\delta(s) = P(h)$ for any h with $\sigma(h) = s$; the hypothesis makes this independent of the choice of h. □

This result establishes the impossibility of *uniformly correct adjudication* from σ. It does not by itself establish that any future becomes unavailable, because a procedure that errs on one of the two histories has not thereby foreclosed anything. Turning the impossibility into a claim about availability needs a further rule that says what the system does when admissibility cannot be established. The following proposition states the simplest such rule and what it yields.

New result 1.6 (Conservative gating removes admissible futures)**.** *Fix a proposed continuation c and call a map $g : \Sigma \to \{\textit{permit}, \textit{refuse}\}$ a* sound gate *if $g(\sigma(h)) = \textit{permit}$ implies $\mathrm{Adm}(h, c) \neq \varnothing$ for every $h \in \mathcal{H}$. If $\sigma(h_1) = \sigma(h_2)$, $\mathrm{Adm}(h_1, c) \neq \varnothing$ and $\mathrm{Adm}(h_2, c) = \varnothing$, then every sound gate refuses c after h_1.*

Assumptions: The gate consults $\sigma(h)$ alone, and soundness is required for all histories in $\mathcal{H}$, not only for those that have occurred.

Proof. Suppose a sound gate permitted c after h_1. Then $g(\sigma(h_1)) = \text{permit}$, so $g(\sigma(h_2)) = \text{permit}$ because $\sigma(h_1) = \sigma(h_2)$, and soundness gives $\mathrm{Adm}(h_2, c) \neq \varnothing$, a contradiction. □

The qualification matters. Proposition 1.6 says that a future which is physically executable and obligation-compliant at h_1 is nevertheless unavailable through any sound gate, and it says this *only* under the policy of refusing what cannot be established. A system that permitted such transitions on a best-effort basis would keep the future available at the price of occasionally acting inadmissibly at h_2. Which trade is acceptable is a design question that the later chapters treat explicitly.

Example 1.7 (Representation concealing an obligation)**.** Let two histories reach the same physical state, so that $\mathrm{Reach}(x(h_1)) = \mathrm{Reach}(x(h_2))$. Suppose h_2 contains an unresolved consent request and h_1 does not, while the retained representation records only the physical state. Then $\sigma(h_1) = \sigma(h_2)$, the continuation c is admissible after h_1 and obstructed by $\mathsf{O}_{\mathrm{obl}}$ after h_2, and Proposition 1.6 shows that a sound gate refuses c after h_1 as well. The obstruction experienced at h_1 is $\mathsf{O}_{\mathrm{rep}}$, even though neither $\mathsf{O}_{\mathrm{phys}}$ nor $\mathsf{O}_{\mathrm{obl}}$ holds there. Here $\mathsf{O}_{\mathrm{rep}}$ is an inherited effect of $\mathsf{O}_{\mathrm{obl}}$ at a different history.

The division of the two volumes

The first volume develops what a representation must preserve for a given family of decisions to be well defined, and proves the corresponding sufficiency and impossibility results. The second volume studies how actions transform the set of reachable futures and what it costs to reach, keep, or restore them. A future can be lost because the physical means to reach it are gone, because obligations prohibit the path, or because the retained representation can no longer distinguish the circumstances in which the path would be admissible. Each of these obstructions recurs throughout both volumes, and the table in the following section records which volume owns which part of the mathematics.

1.3 The obstructions in the rover

Take as events the actions of Example 1.2, so histories are finite sequences of actions and $x(h) = (p, b)$ is the state reached. Then $\mathrm{Reach}(x(h))$ is determined by F.

Physical obstruction. Let $c = \{\text{right right}\}$ be the proposed continuation "move two positions to the right". After the history $h_1 = \text{right right left}$ from the state $(0, 3)$ the rover is at $(1, 0)$, and $\mathrm{Real}(h_1, c) = \varnothing$: neither move can be executed. The obstruction is $\mathsf{O}_{\mathrm{phys}}$.

Obligation obstruction. Suppose a sample taken at position 2 must be uploaded at the dock before another sample is taken. After the history $h_2 = \text{right right sample}$ from $(0, 3)$ the rover is at $(2, 1)$, and the continuation $c = \{\text{sample}\}$ is physically executable there: $\mathrm{Real}(h_2, c) = c$. The obligation incurred by the sample forbids a second one, so $\mathrm{Adm}(h_2, c) = \varnothing$. The obstruction is $\mathsf{O}_{\mathrm{obl}}$.

Representational obstruction. Suppose the rover's controller retains only the pair $\sigma(h) = (p, b)$ and no record of pending uploads. Let $h_3 = \text{right right}$, which reaches $(2, 1)$ without having sampled. Then $\sigma(h_3) = \sigma(h_2)$, the continuation $c = \{\text{sample}\}$ is admissible after h_3 and obstructed after h_2, and Result 1.6 shows that every sound gate based on σ refuses sample after h_3, although neither $\mathsf{O}_{\mathrm{phys}}$ nor $\mathsf{O}_{\mathrm{obl}}$ holds there. The obstruction at h_3 is $\mathsf{O}_{\mathrm{rep}}$, inherited from $\mathsf{O}_{\mathrm{obl}}$ at h_2 as in Example 1.7. Adding one bit, whether an upload is pending, to σ removes the obstruction; Chapter 7 shows that the added bit is exactly what is required.

1.4 Plan of the volume and conventions

Part I treats finite graphs, constraint sets, and viability. Part II adds stochastic transitions, partial observation, and consumable resources. Part III treats feasibility that depends on history, irreversible changes, actions that create other actions, and the establishment of a domain of choice. Part IV introduces measures of possibility and costs of access, and Part V turns to the preservation of future admissibility under obligations. Part VI develops adaptive control, interfaces, institutional revision, and proxies. Part VII treats exploration, the cost of maintaining the distinctions that revision requires, architecture, and four case studies. Appendix A recaps the sufficiency results used from the first volume.

Each chapter begins with a boxed specification listing the question, the established sources, the new results, any proof obligations, and the chapters it depends on. Results are labelled

established, new, or proof obligation, and no proof obligation is cited as a theorem. The recurring exercise sequence asks the reader to construct a model, prove what it preserves, produce a counterexample to a stronger claim, and analyze the effect of revising the constraint set.

1.5 A complete census of the rover

The rover of Example 1.2 is small enough that every set in this chapter can be listed. The nominal menu has 5 actions at each of 16 states, 80 pairs in all. Counting the pairs (x, u) with $u \in F(x)$ gives 30, so 50 of the 80 nominal options are not executable where they are listed. The gap is not uniform. At the dock with a charged battery, $F(0, 3) = \{\text{right}, \text{charge}, \text{upload}\}$ has three of five. At $(1, 0)$ and $(3, 0)$ the feasible set is empty: the rover has a full menu in the specification and no action in the world. At $(2, 0)$ only sample is feasible, and it changes nothing.

The transition graph has 21 edges that change the state and nine self-loops (the actions upload at the dock, charge at $(0, 3)$, and sample at position 2). Its strongly connected components, in the sense of Chapter 2, are 14 in number: one component of three states, $\{(0, 1), (0, 3), (1, 2)\}$, in which the rover can shuttle between the dock and position 1 while recharging, and thirteen singletons. Of the 21 non-loop edges, 18 are irreversible. Starting from $(0, 3)$ the rover can reach six states: $(0, 3), (1, 2), (0, 1), (2, 1), (1, 0), (3, 0)$. Three of them, $(1, 0)$, $(3, 0)$ and the sample site $(2, 1)$ with its single remaining unit of battery, are the places where the first-order description ("it can go anywhere on the line") fails.

Let the constraint be that the rover is never stranded, meaning $b \geq 1$ or $p = 0$. It holds at 13 of the 16 states. Four states leave the viable set: the three stranded states $(1, 0), (2, 0), (3, 0)$ and the state $(3, 1)$, which satisfies the constraint and has a feasible action but whose only action, left, leads to $(2, 0)$. The viability kernel of Chapter 3 has 12 states. The state $(3, 1)$ is the smallest instance of the volume's recurring warning: a constraint can hold at a state and cease to be maintainable from it. (The state $(2, 1)$ remains in the kernel because sample is a self-loop that leaves the rover at $(2, 1)$ with the constraint satisfied; a model in which every action consumes battery would remove it.)

1.6 Obligations enlarge the state, and gates read it back

Add to the rover the obligation of Section 1.3: a sample must be uploaded before another is taken. The obligation is carried by a bit $q \in \{0, 1\}$ that records that a sample is pending, with sample feasible only when $q = 0$ and setting $q = 1$, and upload setting $q = 0$. Starting from $(0, 3, 0)$, the reachable states of the enlarged system number nine:

$$(0, 3, 0),\ (1, 2, 0),\ (0, 1, 0),\ (2, 1, 0),\ (2, 1, 1),\ (1, 0, 0),\ (1, 0, 1),\ (3, 0, 0),\ (3, 0, 1).$$

Their projections to (p, b) are six, and three of the six projections, $(2, 1)$, $(1, 0)$ and $(3, 0)$, are reached with both values of q.

New result 1.8 (A representation without the bit permits no sample)**.** *For the rover started at* $(0, 3)$*, let* $\sigma(h) = (p, b)$ *and let* $c = \{\text{sample}\}$*. Every sound gate for* c *based on* σ *refuses* c *at every history. The refinement* $\sigma'(h) = (p, b, q)$ *permits* c *at exactly the histories whose state is* $(2, 1, 0)$*, for example* right right*, and at no others.*

Assumptions: The rover of Example 1.2 with the pending-upload obligation; histories are the executable action sequences from $(0, 3, 0)$.

Proof. The action sample is physically feasible only at position 2. In the reachable set the only (p, b) with $p = 2$ is $(2, 1)$, which is reached both with $q = 0$ (via right right) and with $q = 1$ (via right right sample). A gate that reads $(p, b) = (2, 1)$ cannot tell the two apart, and by Result 1.6 a sound gate refuses c at the history with $q = 0$ too; at every other reachable (p, b) the action is infeasible, so the gate refuses there as well. With (p, b, q) the representation separates the two histories, and permitting c exactly when $(p, b, q) = (2, 1, 0)$ is sound and permits every history at which c is admissible. □

The example is not contrived to be extreme. It records an ordinary pattern: the only history at which the action is both possible and allowed is indistinguishable, from the retained state alone, from a history at which it is not allowed, and the sound policy is to forbid it. The cost of the missing bit is not a fraction of the options, it is all of them.

New result 1.9 (Cost of removing the representational obstruction)**.** *Let $\sigma : \mathcal{H} \to \Sigma$ be a representation, c a proposed continuation and $P(h) = \mathbf{1}[\mathrm{Adm}(h, c) \neq \varnothing]$. Call a fibre $\sigma^{-1}(s)$* mixed *if P is not constant on it. Then the least number of values of a representation σ' that refines σ and is sufficient for P is $|\sigma(\mathcal{H})| + m$, where m is the number of mixed fibres. The refinement that splits each mixed fibre into its two P-classes attains it.*

Assumptions: $\mathcal{D} = \{0, 1\}$; $\sigma(\mathcal{H})$ finite.

Proof. A refinement σ' of σ has $\ker \sigma' \subseteq \ker \sigma$, so each fibre of σ is a union of fibres of σ'. If σ' is sufficient for P then $\ker \sigma' \subseteq \ker P$, so a mixed fibre contains at least two fibres of σ' (one for each value of P), and every fibre contains at least one. This gives the lower bound $|\sigma(\mathcal{H})| + m$. The representation $\sigma'(h) = (\sigma(h), P(h))$ has exactly one value on each pure fibre and two on each mixed one, is sufficient for P by construction, and refines σ. □

For the rover and the continuation $c = \{\text{sample}\}$ the representation (p, b) takes 6 values on the reachable histories, and only the fibre $(2, 1)$ is mixed: at the other five values either sample is infeasible whatever the pending bit is ($p \neq 2$) or the question never arises. So $m = 1$ and the least sufficient refinement has 7 values, one fewer than the nine reachable states of the enlarged system. The two surplus states are $(1, 0, 1)$ and $(3, 0, 1)$: at the stranded positions no action is feasible, so the pending bit can never matter there, and the enlarged system records a distinction that no continuation can use. The bit at $(2, 1)$ is the whole of what is needed, and nothing more.

1.7 Four sets and how to compare them

For a fixed history h the chapter has distinguished four sets of continuations: the nominal set (everything the specification lists), the realizable set $\mathrm{Real}(h, c)$ (what the world permits), the admissible set $\mathrm{Adm}(h, c)$ (what the commitments permit), and the set that a given gate permits, which depends on $\sigma(h)$ only. The first three are nested, nominal $\supseteq$ realizable $\supseteq$ admissible. The gate's set is nested inside the admissible set only if the gate is sound, and then it can be strictly smaller. A gate that is not required to be sound can permit continuations outside the admissible set; Result 1.6 says that soundness plus a coarse representation forces

strict inclusion on the other side. Almost every question in the two volumes asks which of these inclusions are strict, by how much, and what it would cost to close the gap: Chapters 2 to 3 for the first, the cost chapters of Part IV for the second, and the first volume for the third.

Exercises

Exercise 1.10. In Example 1.2 list $F(p, b)$ for all sixteen states and count the pairs (x, u) with u nominal and $u \notin F(x)$. Describe which states have the largest gap between the nominal menu and the executable set.

Exercise 1.11. Show that for Example 1.3 the set of states at which the constraint is satisfied and some feasible action satisfies it at the next step is $[-2, 2]$. Find the set of states from which the constraint can be maintained for two further steps.

Exercise 1.12. Construct a proposed continuation in the rover example that is subject to $\mathsf{O}_{\mathrm{phys}}$ at one history and to $\mathsf{O}_{\mathrm{obl}}$ at another, and a third history where it is subject to no obstruction.

Exercise 1.13. Show that adding the pending-upload bit to $\sigma(h) = (p, b)$ removes the representational obstruction of this chapter for the continuation $\{\mathrm{sample}\}$, and determine whether it is sufficient for the continuation $\{\mathrm{right}\,\mathrm{sample}\}$.

Exercise 1.14. Show that the representational obstruction is a property of the class of histories sharing a representation: if $\mathsf{O}_{\mathrm{rep}}(h, c)$ holds and $\sigma(h'') = \sigma(h)$ then $\mathsf{O}_{\mathrm{rep}}(h'', c)$ holds.

Exercise 1.15. Verify the census of Section 1.5 by listing $F(p, b)$ for all sixteen states: confirm that there are 30 feasible pairs, nine self-loops, 14 components, and that the viability kernel of the constraint "$b \geq 1$ or $p = 0$" has 12 states.

Exercise 1.16. Modify the rover so that every action, including sample and upload, costs one unit of battery away from the dock. Recompute the kernel of the never-stranded constraint and decide whether $(2, 1)$ remains in it.

Exercise 1.17. Show that Result 1.9 gives $|\sigma(\mathcal{H})| + m$ and not $2|\sigma(\mathcal{H})|$ by exhibiting a representation for which pure fibres are the majority. Why is the bound $2|\sigma(\mathcal{H})|$ attained only if every fibre is mixed?

Exercise 1.18. In the enlarged rover, let the obligation instead be that every sample must be uploaded within three moves. Find a sufficient representation for the admissibility of $\{\mathrm{sample}\}$ and count its values on the reachable histories. Compare with the single bit used above.

Exercise 1.19. Give an example of a gate that is complete (it permits every continuation that is admissible) and not sound, and one that is sound and not complete, for the rover without the pending bit. Which of the two do the stranded states $(1, 0)$ and $(3, 0)$ show to be unavoidable if c is the continuation "return to the dock"?

Chapter 2

States, Actions, and Finite Reachability

Question. Which states can be reached, and which action menus overstate real freedom?

Established sources. Directed graphs, reachability, strongly connected components (Tarjan).

New results. The strongly connected component partition is the minimal sufficient representation for reachability questions (Result 2.3); an action is reversible exactly when its target lies in the component of its source, and component index never decreases along a trajectory (Result 2.5); the size of the menu bounds neither the reachable set nor its growth (Result 2.6).

Proof obligations. None.

Depends on. Chapter 1 and Appendix A.

A system with finitely many states and a feasibility correspondence is a labelled directed graph. Questions about what can be reached are questions about paths in the graph, and the structure of the graph answers them completely. This chapter states that structure, the decomposition into strongly connected components, and uses it to separate three things that are often run together: how many actions are listed, how many states can be reached, and how much of the past can be undone.

2.1 Reachability in a finite system

Definition 2.1 (Transition graph and reachability)**.** Let $(\mathcal{X}, U, F, \delta)$ be a controlled system with $\mathcal{X}$ finite. Its *transition graph* has vertex set $\mathcal{X}$ and an edge $x \xrightarrow{u} y$ whenever $u \in F(x)$ and $y = \delta(x, u)$. A state y is *reachable* from x, written $x \to^* y$, if there is a path (possibly empty) from x to y. The *reachable set* is $R(x) = \{y : x \to^* y\}$.

The relation $\to^*$ is reflexive and transitive. Two states are *mutually reachable* if $x \to^* y$ and $y \to^* x$; this is an equivalence relation whose classes are the *strongly connected components* (SCCs).

Established result 2.2 (Strongly connected components)**.** *The strongly connected components of a finite directed graph with n vertices and m edges can be computed in time $O(n+m)$.*

The condensation *obtained by contracting each component to a vertex is a directed acyclic graph. The relation $x \to^* y$ holds if and only if the component of x precedes or equals the component of y in the condensation.*

Source: Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing* 1 (1972).

Proof. For acyclicity: a directed cycle among components $C_1 \to C_2 \to \cdots \to C_k \to C_1$ would make every state in each C_i mutually reachable with every state in the others, contradicting that they are distinct components. For the reachability statement: a path from x to y projects to a path in the condensation from the component of x to that of y, and conversely a path in the condensation lifts to a path of states by moving within components, which are strongly connected. The running time is that of Tarjan's algorithm, which is cited. □

2.2 Components are the minimal representation for reachability

The question "can the system reach y?" for a fixed y is a function of the current state. A controller that must answer it for every y needs a representation of the current state that supports all of these questions, and the cheapest such representation is the component index.

New result 2.3 (Components are the minimal sufficient representation for reachability)**.** *Let $r : \mathcal{X} \to R$ be a representation of the states of a finite transition graph, and let $\Phi = \{\rho_y : y \in \mathcal{X}\}$ with $\rho_y(x) = \mathbf{1}[x \to^* y]$. Then r is sufficient for every ρ_y if and only if $\ker r \subseteq \ker \kappa$, where $\kappa(x)$ is the component of x. Consequently the component map is a minimal sufficient representation for the family Φ, with exactly as many values as there are components.*

Assumptions: $\mathcal{X}$ finite. Sufficiency in the sense of Appendix A.

Proof. By Result A.3 the least sufficient kernel is $\bigcap_y \ker \rho_y$, so it suffices to show $\bigcap_y \ker \rho_y = \ker \kappa$. If x and x' lie in the same component then $R(x) = R(x')$, so $\rho_y(x) = \rho_y(x')$ for every y. Conversely if $\rho_y(x) = \rho_y(x')$ for all y then $R(x) = R(x')$; since $x \in R(x) = R(x')$ and $x' \in R(x') = R(x)$, the states are mutually reachable. □

The result says that nothing finer than the component structure is needed to decide whether something can be reached, and nothing coarser suffices. A system that tracks position on a map but forgets which one-way doors it has passed has a representation that identifies states in different components, and by the factorization theorem cannot answer reachability questions correctly.

2.3 Reversibility and the ratchet

Definition 2.4 (Reversible action)**.** An edge $x \xrightarrow{u} y$ is *reversible* if $y \to^* x$. Otherwise it is *irreversible.*

New result 2.5 (Irreversible actions are the inter-component edges)**.** *(a) An edge $x \xrightarrow{u} y$ is reversible if and only if x and y lie in the same component.*

(b) Along any trajectory the component never returns to an earlier component of the condensation: if x_s and x_t are states of a trajectory with $s \le t$ and $x_t \to^* x_s$ then they lie in the same component.

(c) If the condensation has c components, any trajectory changes component at most $c-1$ times.

Assumptions: $\mathcal{X}$ finite.

Proof. (a) If $y \to^* x$, then $x \to y \to^* x$ shows mutual reachability. Conversely, same component gives $y \to^* x$. (b) The trajectory gives $x_s \to^* x_t$, and $x_t \to^* x_s$ completes mutual reachability. (c) By Result 2.2 the condensation is acyclic, so component changes follow a path in a DAG, which visits each component at most once and has at most $c-1$ steps. □

Result 2.5 identifies the actions that cannot be undone by any sequence of actions. It does not say how costly the others are to undo; Chapter 8 separates costly reversal from impossible reversal by attaching costs to edges.

2.4 Menus and reachable sets

The nominal menu bounds the immediate branching but not the reach. Two examples make the point.

New result 2.6 (Menu size does not bound reach)**.** *For every $n, m \ge 1$ there is a finite transition graph with states x, y such that x has n feasible actions and $|R(x)| = n+1$, while y has one feasible action and $|R(y)| \ge m$. Moreover for every k there is a graph with a state z having a feasible action at each of its n nominal actions in the menu but whose reachable set after k steps has fewer than n states.*

Assumptions: Finite graphs; the statement is existential.

Proof. Take x with n edges to n distinct sinks (states without outgoing edges). Then $R(x) = \{x\} \cup$ sinks has $n+1$ elements. Take y on a directed cycle of length m. It has one outgoing edge and $R(y)$ contains the whole cycle. For the second statement let z have n edges all leading to the same state z', with z' a sink; the reachable set after any $k \ge 1$ steps is $\{z'\}$. □

A comparison of two states by the sizes of their menus is therefore misleading. The relevant comparison is of reachable sets, and even that needs care: the set $R(x)$ forgets how many steps are needed and at what cost, which Chapter 11 takes up.

2.5 Resources and the expanded graph

A consumable resource enlarges the state. For the rover of Example 1.2 the positional graph on $\{0, 1, 2, 3\}$ ignores battery, while the transition graph of the controlled system has states (p, b).

New result 2.7 (Positional reachability overstates real reachability)**.** *Let $\pi : (p, b) \mapsto p$ be the projection from the expanded graph of Example 1.2 to the positional graph on $\{0, 1, 2, 3\}$ with edges between adjacent positions. For every state (p, b), $\pi(R(p, b)) \subseteq R_{\mathrm{pos}}(p)$, and the inclusion can be strict: $R_{\mathrm{pos}}(2) = \{0, 1, 2, 3\}$, while $R(2, 0) = \{(2, 0)\}$.*

Assumptions: Rover as in Example 1.2: moves cost one unit, charging is possible only at position 0.

Proof. Every path in the expanded graph projects to a path in the positional graph, which gives the inclusion. At $(2, 0)$ no move is feasible since $b = 0$, and the actions sample and upload leave the state unchanged or are infeasible, so nothing but $(2, 0)$ is reachable; the dock, where the battery could be restored, cannot be reached because every move to it needs battery. □

The positional graph is a sufficient representation of the environment's geometry and an insufficient representation of what the rover can do. The difference is the battery, and the rover at $(2, 0)$ is in a component of its own from which nothing else can be reached, although every position looks reachable on the map.

2.6 The reach profile

The reachable set $R(x)$ forgets how long it takes to reach each state. Let $R_k(x)$ be the set of states reachable from x in at most k steps, so $R_0(x) = \{x\}$ and $R(x) = \bigcup_k R_k(x)$. The sequence of sizes $\big(|R_k(x)|\big)_{k\geq 0}$ is the *reach profile* of x.

New result 2.8 (The reach profile stabilizes in at most $n-1$ steps)**.** *In a graph with n states, $R_0(x) \subseteq R_1(x) \subseteq \cdots$ is nondecreasing and $R_k(x) = R(x)$ for every $k \geq n-1$. The bound is attained: on the directed path $v_1 \to v_2 \to \cdots \to v_n$ the profile of v_1 is $1, 2, \ldots, n$. If $R_{k+1}(x) = R_k(x)$ for some k then $R_j(x) = R(x)$ for all $j \geq k$. The* distance $d(x, y) = \min\{k : y \in R_k(x)\}$ *is computed for all y by breadth-first search in time $O(n + m)$.*

Assumptions: Finite graph.

Proof. Monotonicity is the definition. If $y \in R(x)$, a shortest path from x to y visits no state twice (a repeated state could be cut out, giving a shorter path), so it has at most $n-1$ edges and $y \in R_{n-1}(x)$. If $R_{k+1}(x) = R_k(x)$ then $R_k(x)$ is closed under successors, because every successor of a state in $R_k(x)$ lies in $R_{k+1}(x) = R_k(x)$; it contains x, so it contains $R(x)$. For the path graph $R_k(v_1) = \{v_1, \ldots, v_{k+1}\}$. Breadth-first search visits states in order of d and examines each edge once. □

For the rover started at $(0, 3)$ the profile is $1, 2, 4, 6$: after one step the rover can be at $(0, 3)$ or $(1, 2)$, after two steps at four states, and after three steps all six states it will ever reach. Started at $(2, 3)$ it is $1, 3, 5, 8$, and at $(1, 1)$ it is $1, 3, 4, 5, 7, 9$, which reaches its limit only after five steps. The profile distinguishes states whose reachable sets are equal in size: two states can have the same $|R(x)|$ and very different times to reach it. A controller with a horizon of two steps sees 4 states from $(0, 3)$ and 5 from $(2, 3)$, although the long-run counts are 6 and 8.

2.7 Counting trajectories through the condensation

The condensation (Result 2.2) is acyclic, so questions about sequences of component changes are questions about paths in a DAG, and they can be answered by dynamic programming.

New result 2.9 (Reachable components and component paths)**.** *Let the condensation have components $C_1, \ldots, C_c$ numbered in a topological order (every edge goes from a lower to a higher index), and let e_{ij} be the number of edges from C_i to C_j. Then the number N_j of*

distinct paths in the condensation from C_1 to C_j satisfies $N_1 = 1$ and $N_j = \sum_{i<j} e_{ij} N_i$, and the set of components reachable from C_i is computed in $O(c\,(c+m))$ time, or $O(c\,m/w)$ with machine words of w bits.

Assumptions: Finite graph; e_{ij} counts edges between distinct components.

Proof. A path from C_1 to C_j ends with an edge from some C_i with $i < j$, preceded by a path to C_i; these are distinguished by the last edge and the preceding path, giving the recurrence. For reachability, process the components in reverse topological order and set $\mathrm{Reach}(C_i) = \{C_i\} \cup \bigcup_{C_i \to C_j} \mathrm{Reach}(C_j)$, which is a union over at most m edges of sets of at most c elements; with bit sets the unions cost c/w words each. □

Example 2.10 (A funnel)**.** A line has four stages $1 \to 2 \to 3 \to 4$ and each stage i can also be reached from stage 1 by a bypass edge $1 \to i$ for $i = 3, 4$, plus an edge $2 \to 4$. The condensation is the graph itself, and $N_2 = 1$, $N_3 = e_{13}N_1 + e_{23}N_2 = 1 + 1 = 2$ (the bypass and the path through 2), $N_4 = e_{14}N_1 + e_{24}N_2 + e_{34}N_3 = 1 + 1 + 2 = 4$. So there are four distinct component paths to the last stage, and the reachable set of 1 is everything, but a policy that must pass through stage 3 has only two of the four.

2.8 Dead ends and one-way doors

Terminal components deserve their own name because they are where trajectories end. A *dead end* is a terminal component that does not contain the intended goal states.

New result 2.11 (Trajectories settle; every state leads to a terminal component)**.** *In a finite graph every infinite trajectory eventually remains in a single component (not necessarily terminal), and from every state some finite path reaches a terminal component. The set D of states from which a goal set T is unreachable is closed under successors, it is the complement of the backward closure $\mathrm{Pre}^*(T)$, and membership in D is a property of the component: all states of a component are in D or none is.*

Assumptions: Finite graph.

Proof. Along a trajectory the component index never decreases in a topological order (Result 2.5(b)) and takes at most c values, so from some time on it is constant. For the second statement, from any state follow an edge to a different component whenever one exists; the index strictly increases, so this stops after at most $c - 1$ steps at a component with no edge to another, which is terminal. If $x \in D$ and $x \to y$ then $y \in D$, for otherwise $x \to y \to^* T$. The set D is by definition the complement of $\mathrm{Pre}^*(T)$, and mutually reachable states have equal reachable sets, so they agree on whether T is reached. □

In the rover, started at $(0, 3)$ the terminal components reachable are $\{(1, 0)\}$ and $\{(3, 0)\}$, both stranded states, and the intended goal "return to the dock with the sample" is unreachable from them. Take as goal set T the four dock states $(0, b)$. The set D of states from which T cannot be reached has six members: the three stranded states $(1, 0)$, $(2, 0)$, $(3, 0)$, the sample site $(2, 1)$, whose only moves lead to $(1, 0)$ and $(3, 0)$, and the states $(3, 1)$ and $(3, 2)$, from which the rover must pass through $(2, 0)$ or $(2, 1)$ and never gets back. The state $(2, 1)$ shows that a state can lie in the viability kernel, because its self-loop `sample` keeps the constraint, and also in D, because the dock is unreachable from it.

Exercises

Exercise 2.12. Compute the strongly connected components of the graph with edges $a \to b$, $b \to c$, $c \to a$, $c \to d$, $d \to e$, $e \to d$, $e \to f$. Draw the condensation and list the irreversible edges.

Exercise 2.13. Prove that if two states have the same reachable set, they lie in the same component, and that the converse fails for the sets $R_k(x)$ of states reachable in exactly k steps.

Exercise 2.14. In the rover with the charge action available, find all states (p, b) from which a state at position 3 is reachable. Check your answer against the conditions $b \geq 3 - p$ (reach position 3 directly) and $b \geq p$ (return to the dock, charge, and then travel).

Exercise 2.15. A manufacturing line has states “raw”, “cut”, “assembled”, “shipped” with edges raw to cut, cut to assembled, assembled to shipped, and a rework edge from assembled to cut. Compute the components and say which actions are reversible.

Exercise 2.16. Using Result 2.3, describe the minimal representation that a controller needs in order to decide, for each target y, whether the system can still reach y, for the line of the previous exercise. State what the controller cannot decide if it forgets whether the product has shipped.

Exercise 2.17. Compute the reach profile of $(2, 1)$ in the rover and of $(0, 0)$, and explain why $(0, 0)$ has reach size 4 but a profile that stabilizes only at step 3.

Exercise 2.18. Prove that if $R_{k+1}(x) = R_k(x)$ then also $R_{k+1}(y) = R_k(y)$ is *not* implied for $y \in R_k(x)$, by constructing a graph in which the profile of x is flat for one step while that of a successor y is still growing. What is the strongest correct statement about profiles of successors?

Exercise 2.19. Compute N_j for the condensation with components $1, \ldots, 5$ and edges $1 \to 2$ (two parallel edges), $1 \to 3$, $2 \to 4$, $3 \to 4$, $4 \to 5$ (three parallel edges).

Exercise 2.20. Show that the number of strongly connected components of a graph is at least the number of edges of any maximal acyclic subgraph that is a path, and that equality can fail. Give a graph on six vertices with two components and a path of five vertices in it.

Exercise 2.21. Let G be a graph in which every state has out-degree exactly two. Show that G has at least one cycle, and that there are graphs of this kind whose condensation is a single path of components.

Chapter 3

Constraints, Convexity, and Viability

Question. When does continued satisfaction of constraints remain possible?

Established sources. Convex sets, projections, tangent cones (Rockafellar); viability kernels and the Aubin viability theorem; the discrete-time viability kernel as a greatest fixed point.

New results. The viability kernel is the greatest fixed point of the one-step predecessor operator and the set of states with infinite constrained trajectories (Result 3.2); viable histories are the histories that end in the kernel (Result 3.3); for linear systems with convex constraints the kernel is convex (Result 3.4); the unstable scalar system has kernel $[-1, 1]$ inside the constraint set $[-3, 3]$ (Example 3.5).

Proof obligations. None.

Depends on. Chapter 2 and Appendix A.

Chapter 1 observed that a state can satisfy a constraint and have no way to keep satisfying it. This chapter defines the set of states from which continued satisfaction is possible, the viability kernel, shows how to compute it, and treats the case in which constraints and dynamics are convex, where the kernel inherits convexity. The continuous-time statement, the viability theorem of Aubin, is recorded with a worked example.

3.1 The viability kernel

Work with a set-valued dynamics $\Phi : \mathcal{X} \to 2^{\mathcal{X}}$, where $\Phi(x) = \{\delta(x, u) : u \in F(x)\}$ is the set of states reachable in one step, and a *constraint set* $K \subseteq \mathcal{X}$. A *K-trajectory* is an infinite sequence $x_0, x_1, \ldots$ with $x_t \in K$ and $x_{t+1} \in \Phi(x_t)$ for all t. Let $\mathrm{Pre}(W) = \{x : \Phi(x) \cap W \neq \varnothing\}$.

Definition 3.1 (Viability domain and kernel)**.** A set $D \subseteq K$ is a *viability domain* if $D \subseteq \mathrm{Pre}(D)$: from every state of D some one-step successor lies in D. The *viability kernel* is $\mathrm{Viab}(K) = \bigcup\{D \subseteq K : D \text{ a viability domain}\}$.

Established result 3.2 (The viability kernel)**.** *(a)* $\mathrm{Viab}(K)$ *is a viability domain and is the greatest fixed point of the operator* $W \mapsto K \cap \mathrm{Pre}(W)$.

(b) $x \in \mathrm{Viab}(K)$ *if and only if there is a* K*-trajectory starting at* x.

(c) If $\mathcal{X}$ is finite, define $V_0 = K$ and $V_{n+1} = K \cap \mathrm{Pre}(V_n)$. Then $V_0 \supseteq V_1 \supseteq \cdots$ stabilizes after at most $|K|$ steps, at $\mathrm{Viab}(K)$.

Source: Aubin, *Viability Theory* (1991); Saint-Pierre (1994) for the discrete algorithm.

Proof. (a) The union of viability domains is a viability domain: if x lies in a domain D then some successor lies in D and so in the union. Any fixed point $W = K \cap \mathrm{Pre}(W)$ satisfies $W \subseteq K$ and $W \subseteq \mathrm{Pre}(W)$, so is a viability domain contained in $\mathrm{Viab}(K)$; and $\mathrm{Viab}(K) = K \cap \mathrm{Pre}(\mathrm{Viab}(K))$ because $\mathrm{Viab}(K) \subseteq K \cap \mathrm{Pre}(\mathrm{Viab}(K))$ and the right side is a viability domain (call it R; it is contained in K, it contains $\mathrm{Viab}(K)$, and so by monotonicity of Pre we get $R \subseteq \mathrm{Pre}(\mathrm{Viab}(K)) \subseteq \mathrm{Pre}(R)$), hence contained in $\mathrm{Viab}(K)$. So $\mathrm{Viab}(K)$ is the greatest fixed point. (b) If $x \in \mathrm{Viab}(K)$, choose successively $x_{t+1} \in \Phi(x_t) \cap \mathrm{Viab}(K)$, which is possible by (a); this is a K-trajectory. Conversely the set of all states appearing on K-trajectories is a viability domain. (c) By induction $V_{n+1} \subseteq V_n$, using monotonicity of Pre. A decreasing sequence of subsets of a finite set stabilizes after at most $|K|$ strict decreases, at a fixed point V; V is a viability domain, so $V \subseteq \mathrm{Viab}(K)$. Conversely $\mathrm{Viab}(K) \subseteq V_n$ for every n by induction, since $\mathrm{Viab}(K) \subseteq K$ and $\mathrm{Viab}(K) \subseteq \mathrm{Pre}(\mathrm{Viab}(K)) \subseteq \mathrm{Pre}(V_n)$. □

The iteration in (c) is the discrete-time algorithm: remove from K the states with no successor in K, then those with no successor in what remains, and so on. For a finite system with m transitions it runs in time $O(|K|\, m)$ naively and in time $O(m)$ with predecessor counters.

3.2 Viable histories and the kernel

Appendix A defines the viable histories of a safety condition. The following result identifies them in a controlled system.

New result 3.3 (Viable histories are histories that end in the kernel)**.** *Let histories be the finite sequences of actions executable from an initial state x_0, let $\mathcal{S}$ be the set of histories all of whose visited states lie in K, and let $x(h)$ be the state reached by h. Then* $\mathrm{Viable}(\mathcal{S}) = \{h \in \mathcal{S} : x(h) \in \mathrm{Viab}(K)\}$.

Assumptions: Finite action sets (finite branching).

Proof. If $h \in \mathcal{S}$ and $x(h) \in \mathrm{Viab}(K)$ then by Result 3.2(b) there is a K-trajectory from $x(h)$, so h has extensions in $\mathcal{S}$ of every length. Conversely if h is viable then it has extensions in $\mathcal{S}$ of every length, and by finite branching and a compactness argument there is an infinite sequence of actions all of whose finite prefixes extend h within $\mathcal{S}$; the states visited form a K-trajectory from $x(h)$, so $x(h) \in \mathrm{Viab}(K)$ by Result 3.2(b). □

This connects the two languages. In terms of histories, viability is a property of the prefix tree; in terms of states, it is membership in a set computed by a fixed point. The state $x(h)$ is a sufficient representation of h for viability, and the indicator of $\mathrm{Viab}(K)$ composed with x is the viability question. The indicator alone, however, is not maintainable: the next value is not determined by the current indicator and the action, since leaving the kernel depends on where in the kernel the state is. Appendix A gives the representation that is: the continuation quotient of the viability property, whose number of classes is the least size of a maintainable representation sufficient for viability.

3.3 Convexity

Established result 3.4 (Convex linear systems have convex kernels)**.** *Let $\mathcal{X} = \mathbb{R}^d$ and $\Phi(x) = \{Ax + Bu : u \in U\}$ with A, B matrices, $U \subset \mathbb{R}^k$ convex and compact, and $K \subset \mathbb{R}^d$ convex and compact. Then each V_n in the iteration $V_0 = K$, $V_{n+1} = K \cap \mathrm{Pre}(V_n)$ is convex and compact, and $\mathrm{Viab}(K) = \bigcap_n V_n$ is convex and compact.*

Source: Bertsekas and Rhodes (1971); Blanchini, "Set invariance in control" (1999).

Proof. If W is convex and compact then $\mathrm{Pre}(W) = \{x : \exists u \in U,\ Ax + Bu \in W\}$ is the projection onto the x coordinates of $\{(x, u) : u \in U,\ Ax + Bu \in W\}$, which is convex and compact, so $\mathrm{Pre}(W)$ is convex and compact. Intersections of convex compact sets are convex and compact, so every V_n is. The V_n decrease, so $\bigcap_n V_n$ is convex and compact. It contains $\mathrm{Viab}(K)$ by Result 3.2. To show the reverse inclusion, let $x \in \bigcap V_n$. For each n choose $u_n \in U$ with $Ax + Bu_n \in V_n$. By compactness a subsequence of (u_n) converges to some $u \in U$. For each fixed m, $Ax + Bu_n \in V_m$ for all $n \geq m$ and V_m is closed, so $Ax + Bu \in V_m$. Hence $Ax + Bu \in \bigcap V_m$, which shows $\bigcap V_n \subseteq \mathrm{Pre}(\bigcap V_n)$. So $\bigcap V_n$ is a viability domain contained in K and therefore in $\mathrm{Viab}(K)$. □

Example 3.5 (The unstable scalar system)**.** In Example 1.3, $\Phi(x) = [2x - 1, 2x + 1]$ and $K = [-3, 3]$. If $V_n = [-a_n, a_n]$ then $\mathrm{Pre}(V_n) = \{x : [2x - 1, 2x + 1] \cap [-a_n, a_n] \neq \varnothing\} = [-(a_n + 1)/2, (a_n + 1)/2]$, so $a_{n+1} = \min\{3, (a_n + 1)/2\}$ with $a_0 = 3$. This gives $a_1 = 2$, $a_2 = 1.5$, $a_3 = 1.25$, and $a_n \to 1$, the fixed point of $a = (a + 1)/2$. Hence $\mathrm{Viab}(K) = [-1, 1]$. The constraint set $[-3, 3]$ is three times as large as the set of states from which the constraint can be maintained, and every state in $(1, 3]$ satisfies the constraint but eventually leaves K whatever the controller does.

The example illustrates the usual direction of the error: the constraint set is read as the set of safe states, but only the kernel is. A controller designed to respect the constraint at each step, with no look-ahead, will drive the system out of the kernel and then out of K.

3.4 Continuous time

In continuous time the one-step operator is replaced by a tangency condition. Let $\dot{x} \in G(x)$ be a differential inclusion with G set-valued, and let $T_K(x)$ be the tangent cone to a closed set K at x.

Established result 3.6 (Viability theorem)**.** *Let $K \subseteq \mathbb{R}^d$ be closed and let G be upper semi-continuous with nonempty compact convex values and linear growth. Then for every $x \in K$ there is a solution of $\dot{x} \in G(x)$ starting at x and remaining in K for all time if and only if $G(x) \cap T_K(x) \neq \varnothing$ for every $x \in K$.*

Source: Aubin, *Viability Theory* (1991); the condition is due to Nagumo (1942) and Haddad (1981).

Example 3.7 (Continuous-time version of the scalar system)**.** Let $\dot{x} = x + u$ with $|u| \leq 1$, so $G(x) = [x - 1, x + 1]$, and $K = [-3, 3]$. At $x = 3$ the tangent cone is $(-\infty, 0]$ and $G(3) = [2, 4]$, so $G(3) \cap T_K(3) = \varnothing$ and K is not viable. For $K' = [-1, 1]$ the tangent cone at 1 is $(-\infty, 0]$ and $G(1) = [0, 2] \ni 0$, and at -1 the cone is $[0, \infty)$ and $G(-1) = [-2, 0] \ni 0$, while interior points have the full line as cone; so K' satisfies the condition and is viable. The viability kernel of K is $[-1, 1]$, as in discrete time.

3.5 Kernels and their algebra

The kernel is a function of the constraint set and of the dynamics, and the way it depends on each is regular enough to state.

New result 3.8 (Monotonicity and the intersection inequality)**.** *(a) If $K \subseteq K'$ then* $\mathrm{Viab}(K) \subseteq \mathrm{Viab}(K')$*. (b) If $\Phi(x) \subseteq \Phi'(x)$ for all x then* $\mathrm{Viab}_{\Phi}(K) \subseteq \mathrm{Viab}_{\Phi'}(K)$*: more freedom enlarges the kernel. (c)* $\mathrm{Viab}(K_1 \cap K_2) \subseteq \mathrm{Viab}(K_1) \cap \mathrm{Viab}(K_2)$*, and the inclusion can be strict. (d)* $\mathrm{Viab}(K_1) \cup \mathrm{Viab}(K_2) \subseteq \mathrm{Viab}(K_1 \cup K_2)$*, and this can be strict as well.*

Assumptions: Set-valued dynamics Φ on an arbitrary state set.

Proof. (a) A viability domain inside K is a viability domain inside K'. (b) A domain for Φ is a domain for Φ' since its successors persist. (c) By (a) applied to $K_1 \cap K_2 \subseteq K_i$. For strictness take $X = \{x, y, z\}$ with $\Phi(x) = \{y, z\}$, $\Phi(y) = \{y\}$, $\Phi(z) = \{z\}$, and $K_1 = \{x, y\}$, $K_2 = \{x, z\}$. Then $\mathrm{Viab}(K_1) = K_1$ (the trajectory $x, y, y, \dots$), $\mathrm{Viab}(K_2) = K_2$ (the trajectory $x, z, z, \dots$), so x lies in both kernels; but $K_1 \cap K_2 = \{x\}$ and x has no successor in it, so $\mathrm{Viab}(K_1 \cap K_2) = \varnothing$. (d) By (a) applied to $K_i \subseteq K_1 \cup K_2$. Strictness: a state alternating between two constraint sets, $\Phi(a) = \{b\}$, $\Phi(b) = \{a\}$, $K_1 = \{a\}$, $K_2 = \{b\}$: each kernel is empty, the kernel of the union is $\{a, b\}$. □

Part (c) is the formal reason that satisfying two requirements at once is harder than satisfying each separately, and part (d) is the reason that alternating among constraint sets, as a schedule does, can be viable when no single constraint is.

3.6 The stopping-distance kernel

The unstable scalar system has a kernel that is a rescaling of the constraint. A two-dimensional example shows a kernel whose boundary is curved, and it is the standard picture of why constraints on position imply constraints on speed.

Let the state be (x, v) with position $x \in \{0, \dots, N\}$ and velocity $v \in \{-V, \dots, V\}$, the dynamics $x_{t+1} = x_t + v_t$, $v_{t+1} = v_t + u_t$ with $u_t \in \{-1, 0, 1\}$ subject to $|v_{t+1}| \le V$, and the constraint $0 \le x_t \le N$ at every time.

New result 3.9 (Viability kernel of the speed-limited line)**.** *The viability kernel is*

$$\mathrm{Viab}(K) = \Big\{(x,v):\ v \ge 0,\ x + \tfrac{v(v+1)}{2} \le N\Big\} \ \cup\ \Big\{(x,v):\ v \le 0,\ x - \tfrac{|v|(|v|+1)}{2} \ge 0\Big\},$$

intersected with $0 \le x \le N$. In words, the state is viable iff the vehicle can brake to rest within the interval, the least stopping displacement from speed $|v|$ being $|v|(|v|+1)/2$.

Assumptions: Integer positions and velocities; the velocity bound V does not restrict braking.

Proof. Sufficiency. Let $v \ge 0$ and $x + v(v+1)/2 \le N$. Brake with $u = -1$ until $v = 0$: the positions visited are $x,\ x+v,\ x+v+(v-1), \dots,\ x+v(v+1)/2$, each at most N and at least $x \ge 0$. At rest, take $u = 0$ forever. The case $v \le 0$ is symmetric. *Necessity.* Suppose $v \ge 1$ and the trajectory stays in $[0, N]$ forever. A trajectory with unbounded rightward drift leaves, so the velocity must become ≤ 0 at some first time j. For $0 \le k < j$ we have $v_k \ge v - k$ since the velocity falls by at most one per step, and $v_k > 0$ for $k < j$, so $j \ge v$ and the position at

time v is $x_v = x + \sum_{k<v} v_k \geq x + \sum_{k<v}(v-k) = x + v(v+1)/2$. This must be at most N. The case $v \leq -1$ is symmetric and $v = 0$ is covered by both. □

A script that runs the iteration of Result 3.2(c) on $N = 10$, $V = 6$ confirms this: the sizes of $V_0, V_1, \ldots$ are $143, 101, 71, 61, 59, 59$, so the kernel has 59 of the 143 states and the iteration stabilizes after four steps. The states removed first are those one step from leaving, and the removal propagates backward along the braking curve. The boundary $x = N - v(v+1)/2$ is the discrete form of the continuous-time stopping curve $x = N - v^2/2$; the difference is the linear term, which accounts for the velocity applied before the first braking step.

3.7 Reading the kernel as a design tool

A kernel is useful to a designer in three ways, each used again in later chapters. First, it is a *certificate*: membership in a viability domain is checked by a one-step condition, and a domain inside K proves viability without iteration. Second, it is a *filter* on actions: at $x \in \mathrm{Viab}(K)$ an action is *safe* if it has a successor in $\mathrm{Viab}(K)$, and the set of safe actions is nonempty; a controller that chooses safe actions never leaves K (this is Result 3.2(b) read as a recipe). Third, it is a *measure of the slack* in the specification: the ratio of the size of the kernel to that of the constraint set (here $59/143 \approx 0.41$ for the speed-limited line, $1/3$ for the unstable scalar system) says how much of what the specification calls acceptable is acceptable in fact. The constructions of Chapters 14 to 16 are refinements of these three uses.

Exercises

Exercise 3.10. Compute $\mathrm{Viab}(K)$ for the rover of Example 1.2 with $K = \{(p, b) : b \geq 1\}$, with and without the charge action.

Exercise 3.11. For $\Phi(x) = [2x - 1, 2x + 1]$ show that $[-1, 1]$ is a viability domain and that $(1, 3]$ contains no viability domain contained in $K = [-3, 3]$.

Exercise 3.12. Give an example with K convex and Φ with nonconvex values (for instance a control set of two points) for which $\mathrm{Viab}(K)$ is not convex.

Exercise 3.13. Prove that for a finite system the predecessor-counter algorithm computes $\mathrm{Viab}(K)$ in time linear in the number of transitions: maintain for each state in K the number of successors still in the candidate set, and remove states whose count reaches zero.

Exercise 3.14. For the discrete system $x_{t+1} = x_t + u_t$ with $u_t \in \{-1, 0, 1\}$ and $K = \{0, 1, 2\}$, compute the iteration V_n and $\mathrm{Viab}(K)$. Then change the dynamics to $x_{t+1} = 2x_t + u_t$ on the integers and recompute.

Exercise 3.15. In the example for Result 3.8(c), change $\Phi(x)$ to $\{y\}$ and recompute the three kernels. Which part of the inequality becomes an equality, and what does this say about the role of the choice at x?

Exercise 3.16. Compute the kernel of the speed-limited line for $N = 4$, $V = 3$ directly from the iteration and check Result 3.9. How many states does the kernel have?

Exercise 3.17. Replace the control set $\{-1, 0, 1\}$ by $\{-2, -1, 0, 1, 2\}$ in the speed-limited line. Find the new stopping displacement from speed v and state the kernel.

Exercise 3.18. Let the line be circular (positions modulo $N + 1$). Without a constraint on position what is the kernel of the constraint $|v| \le V$, and what is the kernel if a forbidden zone $\{x = 3\}$ is added?

Exercise 3.19. Show that a state $x \in K \setminus \mathrm{Viab}(K)$ is removed at stage n of the iteration iff the shortest forced exit has length n, and relate n to the number of braking steps in Result 3.9.

Part II

Stochastic, partially observed, and resource-bounded systems

Chapter 4

Markov Decision Processes and Constrained Variants

Question. How do feasibility and optimality interact under stochastic transitions?

Established sources. Discounted MDPs (Puterman); constrained MDPs (Altman).

New results. Almost-sure safety is decided by the greatest fixed point of a support-based predecessor operator (Result 4.3); optimizing under an almost-sure constraint is the same as optimizing the restricted process on that kernel (Result 4.5); expected-cost constraints can require randomization (Example 4.7) and make the optimal action depend on remaining budget.

Proof obligations. None.

Depends on. Chapter 3 and Appendix A.

When transitions are stochastic, two questions that were one in the deterministic setting separate. One is whether a constraint can be kept, with what assurance. The other is how well the system performs while keeping it. This chapter treats the interaction. It begins with the standard theory of discounted Markov decision processes, introduces the stochastic analogue of the viability kernel for constraints that must hold with certainty, proves that optimizing under such a constraint reduces to optimizing a restricted process, and then treats constraints on expected cost, where the reduction fails and the state must be enlarged.

4.1 Discounted Markov decision processes

Definition 4.1 (MDP)**.** A finite *Markov decision process* is a tuple (S, A, P, R, γ) with finite state and action sets S, A, transition probabilities $P(s' \mid s, a)$, reward $R : S \times A \to \mathbb{R}$, and discount $\gamma \in [0, 1)$. A *policy* π chooses, for each finite history of states and actions, a distribution over actions. The value of π from s is $V^\pi(s) = \mathbb{E}_{\pi,s}\big[\sum_{t\geq 0} \gamma^t R(s_t, a_t)\big]$ and the optimal value is $V^*(s) = \sup_\pi V^\pi(s)$.

Established result 4.2 (Bellman optimality)**.** *The operator* $(TV)(s) = \max_a \big[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a)V(s')\big]$ *is a* γ*-contraction in the supremum norm, so it has a unique fixed point* V^**, and value iteration* $V_{n+1} = TV_n$ *converges to* V^* *geometrically. A stationary deterministic policy that chooses in each state an action attaining the maximum in* $(TV^*)(s)$ *is optimal.*

Source: Puterman, *Markov Decision Processes* (1994), Chapter 6.

Proof of the contraction property. For V, W and each s,

$$|(TV)(s) - (TW)(s)| \le \max_a \gamma \sum_{s'} P(s' \mid s,a)\,|V(s') - W(s')| \le \gamma \|V - W\|_\infty,$$

using that the maximum of functions differs by at most the maximum difference. Banach's fixed point theorem gives the rest of the first claim. That the fixed point is the optimal value and that a greedy policy attains it is the standard verification argument cited above. □

In an MDP the current state is a sufficient representation of the history in the sense of Appendix A: the distribution of the future given the history depends on the history only through the state, and the optimal action depends on the state alone. This is what is lost when the state is only partly observed (Chapter 5) or when the objective is constrained in expectation (the last section of this chapter).

4.2 Feasibility with certainty

Let $K \subseteq S$ be a constraint set. Say that a policy is *almost surely safe* from s if $\Pr_{\pi,s}[s_t \in K \text{ for all } t] = 1$, and write $\Pi_K(s)$ for the set of such policies. The condition is the stochastic counterpart of the viability condition of Chapter 3 with certainty replaced by probability one. Its solution involves the *support* of the transition law, the set of successors with positive probability. Let $\operatorname{supp} P(s,a) = \{s' : P(s' \mid s,a) > 0\}$ and

$$\operatorname{Pre}_{\forall\exists}(W) = \{s \in S : \exists a \in A,\ \operatorname{supp} P(s,a) \subseteq W\}.$$

New result 4.3 (Almost-sure safety is a support-based viability kernel)**.** *Let W^* be the greatest set with $W^* \subseteq K \cap \operatorname{Pre}_{\forall\exists}(W^*)$, obtained as the limit of $W_0 = K$, $W_{n+1} = K \cap \operatorname{Pre}_{\forall\exists}(W_n)$. Then $\Pi_K(s) \neq \varnothing$ if and only if $s \in W^*$. For $s \in W^*$, any policy that selects at each state $s' \in W^*$ an action a with $\operatorname{supp} P(s',a) \subseteq W^*$ is almost surely safe, in fact safe on every sample path.*

Assumptions: Finite S and A; policies may depend on the full history.

Proof. The sets W_n decrease and stabilize on a finite set, at the greatest fixed point W^* (the argument is that of Result 3.2(c) for the operator $K \cap \operatorname{Pre}_{\forall\exists}$). ($\Leftarrow$) If $s \in W^*$, the stationary policy that picks at each $s' \in W^*$ an action with support inside W^* moves only within $W^* \subseteq K$, so every sample path stays in K. ($\Rightarrow$) Let $\pi \in \Pi_K(s)$ and let W be the set of states visited with positive probability at some time under π from s. Then $s \in W$ and $W \subseteq K$, since a state visited with positive probability that lay outside K would contradict almost sure safety. Take $s' \in W$ and a history of positive probability ending in s' at time t. The policy puts positive mass on some action a at that history. Every $s'' \in \operatorname{supp} P(s',a)$ is visited at time $t+1$ with positive probability, so $s'' \in W$. Thus $\operatorname{supp} P(s',a) \subseteq W$, which shows $W \subseteq \operatorname{Pre}_{\forall\exists}(W)$, and $W \subseteq K$ gives $W \subseteq W^*$. Hence $s \in W^*$. □

The theorem says that almost-sure safety is blind to the magnitudes of the probabilities: only which transitions can occur matters. Any action that has even a small chance of leaving W^* is excluded at that state, because over an unbounded horizon the small chance is eventually

realized. This is the stochastic counterpart of the observation in Chapter 3 that the constraint set is not the set of safe states.

Example 4.4 (A field excursion)**.** States D (dock), F (field), X (stranded). At D the actions are wait (stay at D with probability 1, reward 0) and go (move to F, reward 0). At F the actions are return (move to D, reward 0) and sample (reward 1; stay at F with probability 0.9, move to X with probability 0.1). The state X is absorbing with reward 0. Let $K = \{D, F\}$. Then $W_1 = \{D, F\}$ since wait and return have supports inside K, and $W^* = \{D, F\}$. At F the action sample is not in the restricted set, since its support contains X. The unconstrained optimum samples at F repeatedly, with positive value; the almost-sure constraint forbids it, and the optimal almost-surely safe value is 0.

4.3 Optimizing under an almost-sure constraint

New result 4.5 (Reduction to the restricted process)**.** *Let W^* be as in Result 4.3 and define the restricted MDP M' with state set W^*, action sets $A'(s) = \{a : \operatorname{supp} P(s,a) \subseteq W^*\}$ (nonempty for $s \in W^*$), and the same P, R, γ restricted. Then for every $s \in W^*$,*

$$\sup_{\pi \in \Pi_K(s)} V^\pi(s) \;=\; V^*_{M'}(s).$$

Consequently the optimal almost-surely safe value is attained by a stationary deterministic policy on W^*, obtained by value iteration on M'.

Assumptions: As in Result 4.3.

Proof. Every policy of M' (extended arbitrarily outside W^*) stays in $W^* \subseteq K$ on every sample path, so it belongs to $\Pi_K(s)$ and has the same value. Conversely let $\pi \in \Pi_K(s)$. By the proof of Result 4.3 the set W of states visited with positive probability satisfies $W \subseteq W^*$, and at every history of positive probability ending in s' every action in the support of π has $\operatorname{supp} P(s', a) \subseteq W \subseteq W^*$, so lies in $A'(s')$. Changing π on histories of probability zero does not change $V^\pi(s)$, so π has the same value as a policy of M'. Hence the suprema agree, and the last claim is Result 4.2 for M'. □

The result separates the two questions cleanly. Feasibility is settled first, by the kernel, and performance is optimized afterwards over what feasibility leaves. A policy that optimizes performance first and is then corrected for safety is in general not optimal under the constraint; the correct order is the one in the theorem.

4.4 Constraints on expected cost

A different kind of constraint bounds an expected accumulated cost. Let $C : S \times A \to \mathbb{R}_{\geq 0}$ be a cost function and $d \geq 0$ a budget. The *constrained problem* is to maximize $V^\pi(s_0)$ subject to $\mathbb{E}_{\pi, s_0}\left[\sum_t \gamma^t C(s_t, a_t)\right] \leq d$.

Established result 4.6 (Randomization and occupation measures)**.** *The constrained problem is equivalent to a linear program over the discounted occupation measures $\rho(s,a) = \sum_t \gamma^t \Pr_{\pi, s_0}[s_t = s, a_t = a]$, and its optimal policy may have to be randomized.*

Source: Altman, *Constrained Markov Decision Processes* (1999).

Example 4.7 (A constraint that forces randomization)**.** One state s with two actions: a_1 with reward 1 and cost 1, a_2 with reward 0 and cost 0, and s is left unchanged by both. Let the budget be $d = 1/(2(1-\gamma))$. A stationary policy that chooses a_1 with probability q has reward and cost both equal to $q/(1-\gamma)$, so it is feasible if and only if $q \le 1/2$, and the best feasible value is attained at $q = 1/2$ and equals $1/(2(1-\gamma))$. The stationary deterministic policies are $q = 0$, with value 0, and $q = 1$, which is infeasible. No stationary deterministic policy attains the optimum.

There is a second effect. In the unconstrained problem the optimal action depends on the state alone. In the constrained problem it depends on the budget already spent: at the same state, a system that has used most of its budget must act differently from one that has used none. The state is therefore not a sufficient representation of the history for the constrained optimum. The remedy is to enlarge it, adding the remaining budget, and this is the first instance of the construction studied in Chapter 7: the augmented state is sufficient for the constrained problem exactly when the added component is sufficient for the feasibility question.

4.5 The price of certainty in the excursion

Take Example 4.4 with discount $\gamma = 0.9$. The unconstrained value at the field satisfies $V(F) = 1 + \gamma \cdot 0.9\, V(F)$, because sampling pays 1 and the rover survives with probability 0.9 to sample again, so $V(F) = 1/(1-0.81) \approx 5.263$ and, going out from the dock, $V(D) = \gamma V(F) \approx 4.737$. The alternative at F, to return, is worth $\gamma V(D) \approx 4.263$, so sampling is optimal. Under the almost-sure constraint the optimal value is 0. The gap of 4.737 is the price of certainty, and the interesting design question is what lies between the extremes.

Consider the policy that leaves the dock, samples a fixed number n of times, returns if it has survived, and then waits at the dock forever. Its value is $\sum_{k=0}^{n-1} \gamma^{1+k} 0.9^k$ and its probability of ending stranded is $1 - 0.9^n$. For $n = 1, \dots, 8$ the values are 0.900, 1.629, 2.220, 2.698, 3.085, 3.399, 3.653, 3.859 and the ruin probabilities are 0.100, 0.190, 0.271, 0.344, 0.410, 0.469, 0.522, 0.570. Every increment in value is paid for by an increment in the probability of ruin. The nth sample adds $0.9 \cdot 0.81^{n-1}$ to the value and $0.1 \cdot 0.9^{n-1}$ to the probability of ruin, so the value gained per unit of added risk is 9, 8.1, 7.29, ..., falling by the factor 0.9 at each further sample: later samples are worse bargains, but each still buys something, and no sample is free of risk. Among these policies, a chance constraint requiring ruin probability at most 0.2 selects $n = 2$, with value 1.629 against 4.737 for the unconstrained optimum; the almost-sure formulation selects $n = 0$. (A randomized choice between $n = 2$ and $n = 3$ that meets the constraint with equality would do slightly better, which is the phenomenon of Result 4.9 below.)

New result 4.8 (Repeated exposure drives ruin to one)**.** *Let π be a policy and $t_0 \ge 0$ such that, at every history of positive probability of length $t \ge t_0$ that ends in K, the action chosen has probability at least $p > 0$ of leading outside K. Then $\Pr_\pi[s_t \in K \text{ for } t = 0, \dots, t_0 + T] \le (1-p)^T$ for all $T \ge 0$, so $\Pr[s_t \in K \text{ for all } t] = 0$.*

Assumptions: As stated; K is the constraint set.

Proof. Let E_T be the event that the first t_0+T steps remain in K. Conditional on any history in E_T the next step leaves K with probability at least p, so $\Pr[E_{T+1} \mid \text{history}] \le 1-p$, and by induction $\Pr[E_T] \le (1-p)^T$. The event "in K forever" is contained in every E_T. □

The proposition is the quantitative form of the remark after Result 4.3: a small, constant risk per period becomes certain ruin over a long horizon, and a risk that is tolerable for one excursion is not tolerable for an unbounded program of excursions. With $p = 0.1$ the probability of surviving 50 exposures is $0.9^{50} \approx 0.005$.

4.6 Where randomization occurs

Example 4.7 shows that the constrained optimum can need randomization. How much randomization it needs is limited by the number of constraints.

New result 4.9 (One constraint, one randomized state)**.** *For the constrained problem with a single expected-cost constraint there is an optimal stationary policy that randomizes in at most one state; with k constraints, in at most k states.*

Assumptions: Finite MDP; the problem is feasible and has an optimal solution.

Proof. By the occupation-measure formulation (Altman) the problem is the linear program $\max \sum R\rho$ subject to $\sum_a \rho(s',a) - \gamma \sum_{s,a} P(s' \mid s,a)\rho(s,a) = \mu_0(s')$ for each s', $\sum C\rho + z = d$ with a slack $z \ge 0$, and $\rho, z \ge 0$. An optimal solution exists at a vertex, which has at most $|S|+1$ positive coordinates because the system has $|S|+1$ equality constraints. Let S' be the set of states with positive total occupation $\sum_a \rho(s,a)$. Every pair (s,a) with $\rho(s,a) > 0$ has $s \in S'$ and, since inflow to a state forces positive occupation there, all its successors with positive probability lie in S'; so the columns of the positive coordinates are supported on the $|S'|$ flow rows of S' and the budget row, and a vertex has at most $|S'|+1$ of them. Each state of S' has at least one positive coordinate, so the number of states of S' with two or more is at most $(|S'|+1) - |S'| = 1$. The stationary policy $\pi(a \mid s) = \rho(s,a)/\sum_{a'} \rho(s,a')$ on S' reproduces ρ and randomizes in at most one state. With k constraints the same count gives at most k. □

The result explains the shape of Example 4.7, where the single state randomizes, and it gives a design rule: a system with a single budget needs a single coin. The policy is deterministic everywhere except at one state, and its placement is part of the solution of the linear program.

4.7 Value in the budget as a state variable

The remedy named at the end of the chapter, adding the remaining budget to the state, can be made explicit in the finite-horizon deterministic-cost case, where it is exact. Let costs be nonnegative integers and the budget a nonnegative integer d with a hard (path-wise) constraint $\sum_t C(s_t,a_t) \le d$. The augmented state (s,β), with β the remaining budget, evolves by $\beta' = \beta - C(s,a)$ and an action is allowed only if $C(s,a) \le \beta$. The value function satisfies

$$V_n(s,\beta) = \max_{a:\, C(s,a)\le\beta} \Big[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V_{n-1}\big(s', \beta - C(s,a)\big) \Big],$$

which is the Bellman recursion on the product, an MDP with $|S|(d+1)$ states. The optimal action at the same s depends on β through the allowed set and the continuation value. For expected-cost constraints there is no exact finite augmentation (Result 7.5 of Chapter 7), and the budget becomes a real-valued state whose value function is concave in β (Result E.2).

Exercises

Exercise 4.10. Compute W^* for the excursion of Example 4.4 when $K = \{D, F, X\}$ and when $K = \{D\}$.

Exercise 4.11. Modify Example 4.4 so that sample moves to X with probability 0.1 and returns to D with probability 0.9. Recompute W^*, the restricted action sets, and the optimal almost-surely safe value for $\gamma = 1/2$ and reward 1 for sample.

Exercise 4.12. Show that if the support of a transition law is replaced by the whole successor set whenever the probability is positive but arbitrarily small, the sets W_n do not change, and conclude that the kernel W^* depends on P only through its supports.

Exercise 4.13. Give a two-state MDP in which the optimal unconstrained policy leaves W^* with positive probability, and compare its value with the optimal value of Result 4.5.

Exercise 4.14. For Example 4.7, construct a deterministic nonstationary policy whose cost does not exceed d and whose value is within γ^N of the randomized optimum for some N, for $\gamma = 1/2$. Explain why non-stationarity of this kind amounts to adding a clock to the state.

Exercise 4.15. Verify the table of values and ruin probabilities in Section 4.5 for $n = 3$, and find the largest n with ruin probability at most 0.3 and the corresponding value.

Exercise 4.16. In the excursion with $\gamma = 0.9$, add an action recharge at F that costs nothing and returns to F with the reward 0.5 and no risk. Compute the unconstrained optimum, the almost-surely safe optimum, and the policy that mixes the two to achieve ruin probability at most 0.2 over a single excursion.

Exercise 4.17. Prove Result 4.8 for a policy that exposes itself at independent times with different risks $p_1, p_2, \ldots$ and show that $\Pr[\text{stay}] \le \prod(1 - p_i)$, which is 0 iff $\sum p_i = \infty$. State the connection to the second Borel–Cantelli lemma in Appendix F.

Exercise 4.18. For a two-state MDP with a single cost constraint, write the occupation-measure linear program explicitly, find a basic optimal solution, and identify the state in which the policy randomizes.

Exercise 4.19. Find the augmented-state recursion of Section 4.7 for a two-state, two-action MDP with integer costs 0 and 1 and budget $d = 2$, horizon 3, and compute V_3 at every augmented state.

Chapter 5

Partial Observation and Belief-State Sufficiency

Question. When does a belief state serve as a sufficient representation for feasibility and decision?

Established sources. POMDPs and belief-state sufficiency (Åström; Smallwood and Sondik; Kaelbling, Littman, and Cassandra); predictive state representations (Littman, Sutton, and Singh).

New results. The belief is a maintainable representation; the coarsest representation sufficient for prediction is the quotient by the span of the prediction vectors, so the belief is minimal exactly under a rank condition (Result 5.3); almost-sure safety is decided on belief supports (Result 5.5).

Proof obligations. Conditions under which the belief is a minimal sufficient representation for a stated policy family beyond prediction.

Depends on. Chapter 4 and Appendix A.

When the controller cannot see the state, it acts on what it has seen. The history of its actions and observations is the raw record, and a belief, the posterior distribution over the hidden state, is the standard summary. This chapter asks in what sense the belief is a sufficient representation, using the vocabulary of the first volume, and what is the least representation that does the same work. The answer is a rank condition, and it also delivers the form in which feasibility with certainty can be decided without tracking probabilities at all.

5.1 Beliefs and their update

Definition 5.1 (POMDP and belief)**.** A finite *POMDP* consists of finite sets S, A, O of hidden states, actions, and observations, transition probabilities $P(s' \mid s, a)$, observation probabilities $Z(o \mid s')$ (the observation depends on the state reached), an initial distribution b_0 on S, and rewards as in Chapter 4. A history is $h = (a_1, o_1, \ldots, a_t, o_t)$. The *belief* $b(h) \in \Delta(S)$ is the posterior of the hidden state s_t given h, under the prior b_0.

For an action a and observation o let $T_{a,o}$ be the matrix with entries $T_{a,o}(s, s') = P(s' \mid s, a)Z(o \mid s')$. For a row vector b, the probability of o after action a is $\Pr(o \mid b, a) = bT_{a,o}\mathbf{1}$.

Established result 5.2 (Belief update and sufficiency)**.** *If* $\Pr(o \mid b(h), a) > 0$ *then* $b(h\,a\,o) = \tau(b(h), a, o) := \dfrac{b(h)T_{a,o}}{b(h)T_{a,o}\mathbf{1}}$*. The map* $h \mapsto b(h)$ *is therefore a maintainable representation of the positive-probability histories in the sense of Appendix A. The conditional law of all future observations given* h *and any future action sequence is a function of* $b(h)$*.*

Source: Åström (1965); Smallwood and Sondik (1973).

Proof. By Bayes' rule, $\Pr(s_{t+1} = s', o \mid h, a) = \sum_s b(h)(s)P(s' \mid s, a)Z(o \mid s') = (b(h)T_{a,o})(s')$, and dividing by the total over s' gives the update. Since the update depends only on $b(h), a, o$, equal beliefs give equal beliefs after the same event, so $\ker b$ is a right congruence. For future observations, the joint probability of $(o_1, \ldots, o_k)$ after actions $(a_1, \ldots, a_k)$ is $b(h)T_{a_1,o_1} \cdots T_{a_k,o_k}\mathbf{1}$, a function of $b(h)$. □

5.2 The minimal sufficient representation for prediction

For a word $w = (a_1, o_1, \ldots, a_k, o_k)$ let $q_w = T_{a_1,o_1} \cdots T_{a_k,o_k}\mathbf{1} \in \mathbb{R}^S$, so that the probability of observing $o_1, \ldots, o_k$ after taking $a_1, \ldots, a_k$ from belief b is $b \cdot q_w$. Let $Q = \operatorname{span}\{q_w\}$ over all words, including the empty word with $q_\varepsilon = \mathbf{1}$.

Call two positive-probability histories *predictively equivalent*, $h \approx h'$, if $b(h) \cdot q_w = b(h') \cdot q_w$ for all words w. This is the continuation equivalence of Appendix A for the property that records the distribution of future observations.

New result 5.3 (Predictive equivalence and the rank condition)**.** *(a)* $h \approx h'$ *if and only if* $b(h) - b(h') \perp Q$*.*

(b) $\approx$ *is a right congruence on positive-probability histories, and it is the coarsest maintainable representation of the history that is sufficient for predicting future observations. The belief map is a finer maintainable representation, and its kernel is contained in* $\approx$*.*

(c) The minimal number of real parameters of such a representation is $\dim Q$*, attained by the projection of the belief onto* Q*. In particular the belief is a minimal sufficient representation, for every set* B *of reachable beliefs, if and only if* $(B - B) \cap Q^\perp = \{0\}$*, and for all beliefs if* $Q = \mathbb{R}^S$*.*

Assumptions: Finite POMDP; histories of positive probability.

Proof. (a) is the definition, since $b \cdot q_w = b' \cdot q_w$ for all w means $(b - b') \perp \operatorname{span}\{q_w\}$. (b) Suppose $h \approx h'$ and $\Pr(o \mid b(h), a) > 0$. Taking the word (a, o) gives $b(h) \cdot q_{(a,o)} = b(h') \cdot q_{(a,o)}$, so $\Pr(o \mid b(h'), a) = \Pr(o \mid b(h), a) > 0$. For any word w, $\tau(b, a, o) \cdot q_w = (b \cdot q_{(a,o)w})/\Pr(o \mid b, a)$ where $(a, o)w$ is the concatenated word, and the numerators and denominators agree for $b(h), b(h')$. Hence $h\,a\,o \approx h'\,a\,o$. The relation is by definition the continuation equivalence of the prediction property, so by Result A.5 it is the coarsest right congruence inside the kernel of prediction, which is the coarsest maintainable sufficient representation. Equal beliefs give equal predictions, so $\ker b \subseteq \approx$. (c) The map $b \mapsto (b \cdot q_w)_w$ factors through the orthogonal projection onto Q and is injective on it, so predictive classes correspond to points of the projection, and the minimal dimension is $\dim Q$. The belief is injective modulo $\approx$ on B exactly when two beliefs in B with the same predictions are equal, that is, when $(B - B) \cap Q^\perp = \{0\}$. □

The rank condition is the familiar observability condition of linear systems theory, for a system whose output is a probability. When it fails, the belief carries more than the future depends on, and the excess is the component of the belief in $Q^{\perp}$.

Example 5.4 (Observationally identical hidden states)**.** Let $S = \{1, 2, 3\}$ with a single action, $Z(o \mid s') = 1$ for the unique observation o in every state (a blind system) and transition probabilities arbitrary. Then $q_w = \mathbf{1}$ for every word w, $Q = \operatorname{span}\{\mathbf{1}\}$ has dimension 1, and all beliefs are predictively equivalent. The belief carries two parameters of information that no future observation can reveal, and the minimal representation for prediction has one value. If, however, the reward depends on the hidden state, the question that matters is no longer prediction of observations, and the minimal representation is that for the prediction of rewards; Exercise 5.11 asks for this extension.

5.3 Certain safety with partial observation

Predictive structure matters for what the system can predict. For feasibility with certainty only which states are possible matters, not how probable they are.

For a set $\Sigma \subseteq S$ of states, an action a, and an observation o let $\mathrm{Post}(\Sigma, a, o) = \{s' : \exists s \in \Sigma,\ P(s' \mid s, a) > 0,\ Z(o \mid s') > 0\}$. The *support* of the belief after a history is $\Sigma(h) = \operatorname{supp} b(h)$, and $\Sigma(h\,a\,o) = \mathrm{Post}(\Sigma(h), a, o)$ whenever the latter is nonempty. A policy is *observation-based* if it chooses actions as a function of the history of actions and observations. Let $\mathrm{Obs}(\Sigma, a) = \{o : \mathrm{Post}(\Sigma, a, o) \neq \varnothing\}$.

New result 5.5 (Almost-sure safety is decided on supports)**.** *Let $K \subseteq S$ and let $\Sigma_0 = \operatorname{supp} b_0$. There is an observation-based policy under which $\Pr[s_t \in K \text{ for all } t] = 1$ if and only if Σ_0 belongs to the greatest family $\mathcal{W}$ of subsets of K satisfying: for each $\Sigma \in \mathcal{W}$ there is an action a such that $\mathrm{Post}(\Sigma, a, o) \in \mathcal{W}$ for every $o \in \mathrm{Obs}(\Sigma, a)$. The supports form a maintainable representation that is sufficient for this question.*

Assumptions: Finite POMDP.

Proof. A state $s \in \Sigma(h)$ has positive posterior probability at the end of h, so it is visited with positive probability. Hence an observation-based policy is almost surely safe if and only if every positive-probability history has $\Sigma(h) \subseteq K$. The support process is a deterministic function of the history, evolving by $\Sigma \mapsto \mathrm{Post}(\Sigma, a, o)$, so the question is a safety condition in the deterministic system whose states are supports and whose transitions are chosen by the controller (the action) and by the environment (the observation). The greatest family $\mathcal{W}$ is the winning region of this safety game by the argument of Result 3.2: if $\Sigma_0 \in \mathcal{W}$ the controller plays the stated actions and the support stays in $\mathcal{W} \subseteq 2^K$; conversely the set of supports visited with positive probability under a safe policy is such a family. Supports update by a function of the previous support, the action, and the observation, so the representation is maintainable; it is sufficient because the winning region is a property of the support. □

The support construction has up to $2^{|S|}$ states. Lower bounds for safety with imperfect information show that this exponential cannot in general be avoided. The belief itself, by contrast, is a point in a continuum and gives no finite description of the same question.

5.4 A machine with a hidden fault

A machine is either good (G) or faulty (B). The action run produces output and moves G to B with probability 0.1, while a faulty machine stays faulty; the action inspect leaves the state unchanged and returns the observation alarm with probability 0.6 if the machine is faulty and 0.1 if it is good. The observation after run is uninformative. From the prior belief $b_0(G) = 1$, one run gives $b(G) = 0.9$ and a second 0.81. An inspection with the outcome alarm updates by Bayes' rule: with $b(G) = 0.81$, the unnormalized masses are $0.81 \cdot 0.1 = 0.081$ for G and $0.19 \cdot 0.6 = 0.114$ for B, so $b(G) = 0.081/0.195 \approx 0.415$. With outcome ok the masses are $0.81 \cdot 0.9 = 0.729$ and $0.19 \cdot 0.4 = 0.076$, so $b(G) \approx 0.906$. The belief is a single number, and in this example it is minimal: the observation vectors $q_{\text{inspect,alarm}} = (0.1, 0.6)$ and $q_\varepsilon = (1, 1)$ are linearly independent, so $\dim Q = 2$ and, with $S = \{G, B\}$, the rank condition of Result 5.3 holds.

When the rank condition fails, the belief is redundant. Let $S = \{1, 2, 3\}$ with one action that leaves the state unchanged, and observations o_1 in state 1 and o_2 in states 2 and 3, each with probability one. Then $T_{o_1} = \operatorname{diag}(1, 0, 0)$ and $T_{o_2} = \operatorname{diag}(0, 1, 1)$, so $q_{o_1} = (1, 0, 0)$, $q_{o_2} = (0, 1, 1)$, every longer word with both observations gives 0, and $Q = \operatorname{span}\{(1, 0, 0), (0, 1, 1)\}$ has dimension 2. The beliefs $(0, \frac{1}{2}, \frac{1}{2})$ and $(0, 1, 0)$ are predictively equivalent, since $b \cdot q_w$ agrees for all w, although they are different beliefs: the representation $(b_1,\ b_2 + b_3)$ is minimal, and since the components of a belief sum to one it amounts to the single free number b_1, in agreement with $\dim Q = 2$ for the unnormalized vectors. If the reward distinguishes states 2 and 3, then the equivalence is finer, since a reward that depends on the state adds the reward vector to the span of the predictive vectors, and the vector $(0, 1, -1)$-direction then separates the two beliefs.

5.5 Value of information in a safety game

Result 5.5 says that almost-sure safety is a game on supports. The game structure makes visible something that the state-based theory of Chapter 4 cannot: an action whose only purpose is to observe can make a hopeless support winning.

Example 5.6 (Peeking)**.** Let $S = \{a, b, c\}$, $K = \{a, b\}$, and let the actions be L, R and peek. Action L keeps a at a and sends b to c; action R keeps b at b and sends a to c; peek keeps both states and returns the observation o_a in a and o_b in b. The observation after L and R is the same in a and b. The prior support is $\Sigma_0 = \{a, b\}$. No single action is safe on Σ_0: L and R each have a successor $c \notin K$ in one of the states, and peek keeps both states. But peek is safe, and $\operatorname{Post}(\Sigma_0, \text{peek}, o_a) = \{a\}$, $\operatorname{Post}(\Sigma_0, \text{peek}, o_b) = \{b\}$, each of which is winning by repeating L or R respectively. So $\Sigma_0 \in \mathcal{W}$, although $\mathcal{W}$ computed without the action peek would contain only $\{a\}$ and $\{b\}$. In the game the safety is obtained by acquiring information, then committing.

New result 5.7 (The winning supports form a down-closed family)**.** *If $\Sigma \in \mathcal{W}$ and $\Sigma' \subseteq \Sigma$ with $\Sigma' \neq \varnothing$ then $\Sigma' \in \mathcal{W}$. The family $\mathcal{W}$ need not be closed under union: in Example 5.6 without the action* peek*, $\{a\}$ and $\{b\}$ are winning and $\{a, b\}$ is not.*

Assumptions: Finite POMDP; $\mathcal{W}$ as in Result 5.5.

Proof. Let $\mathcal{W}' = \mathcal{W} \cup \{\Sigma' \neq \varnothing : \Sigma' \subseteq \Sigma \text{ for some } \Sigma \in \mathcal{W}\}$. For $\Sigma' \subseteq \Sigma \in \mathcal{W}$ use the action a that is good for Σ: for each observation o possible from Σ', $\operatorname{Post}(\Sigma', a, o) \subseteq \operatorname{Post}(\Sigma, a, o) \in \mathcal{W}$,

so it belongs to $\mathcal{W}'$. Thus $\mathcal{W}'$ satisfies the defining condition and is contained in the greatest such family, which is $\mathcal{W}$. For the failure of union closure, see the example. □

The result shows that the winning region is described by its maximal elements, and the value of the observing action is the difference between the maximal elements with and without it. Deciding whether $\Sigma_0 \in \mathcal{W}$ is EXPTIME-complete in general (Chatterjee, Doyen, Henzinger and Raskin, on games with imperfect information); the exponential is in the number of hidden states because supports are subsets.

5.6 The value function of the belief MDP

Beyond feasibility, the belief supports a value function with a useful shape.

Established result 5.8 (Piecewise-linear convexity)**.** *For a finite-horizon POMDP with rewards $R(s,a)$, the optimal n-step value as a function of the belief is $V_n(b) = \max_{\alpha \in \Gamma_n} b \cdot \alpha$ for a finite set $\Gamma_n \subset \mathbb{R}^S$ of vectors; hence it is convex and piecewise linear.*

Source: Smallwood and Sondik (1973).

Proof. For $n = 0$ take $\Gamma_0 = \{0\}$. Suppose the claim holds for $n-1$. For an action a, $r_a(s) = R(s,a)$, and with $V_{n-1}(b') = \max_{\alpha \in \Gamma_{n-1}} b' \cdot \alpha$,

$$V_n(b) = \max_a \Big[b \cdot r_a + \gamma \sum_o \Pr(o \mid b,a) V_{n-1}(\tau(b,a,o)) \Big] = \max_a \Big[b \cdot r_a + \gamma \sum_o \max_{\alpha \in \Gamma_{n-1}} b\, T_{a,o} \alpha \Big],$$

because $\Pr(o \mid b,a)\,\tau(b,a,o) = b T_{a,o}$ is linear in b and V_{n-1} is positively homogeneous in the unnormalized belief. Each inner maximum is a maximum of linear functions of b, and a sum of maxima of linear functions is the maximum of the sums over choices of α for each o, still a maximum of finitely many linear functions. Adding $b \cdot r_a$ and maximizing over a gives the form with Γ_n of size at most $|A| |\Gamma_{n-1}|^{|O|}$. □

The vectors in Γ_n grow doubly exponentially in the horizon in the worst case, which is why exact solution of POMDPs is limited to small problems; the representation of the value function is as large as the sufficient statistic is small.

Exercises

Exercise 5.9. For a two-state sensor with $S = \{L, H\}$, one action, $Z(o_H \mid H) = 0.8$, $Z(o_H \mid L) = 0.3$, and transitions that leave the state unchanged, compute Q and show that the belief is minimal. Compute the belief after the observations o_H, o_H, o_L from the uniform prior.

Exercise 5.10. Show that if two hidden states have identical transition rows and identical observation probabilities, then $e_1 - e_2 \in Q^{\perp}$, and conclude that the belief is not minimal on any set of beliefs containing two that differ only by moving mass between them.

Exercise 5.11. Extend q_w to include expected rewards by adding vectors r_w whose entry at s is the expected total reward over the word times the probability of its observations, both from s, and state and prove the analogue of Result 5.3 for the joint prediction of observations and rewards.

Exercise 5.12. Construct a POMDP with $|S| = 3$ and a constraint set K for which the family $\mathcal{W}$ of Result 5.5 contains a support Σ although no single state of Σ would be safe under a policy that does not use observations.

Exercise 5.13. Prove that the support is a coarsening of the belief, and describe the kernel of the support map in terms of which states carry positive probability. Show that the support is not a sufficient representation for expected reward.

Exercise 5.14. For the machine of Section 5.4 compute the belief after run, inspect with outcome alarm, run, and verify that the update is independent of the order of normalization.

Exercise 5.15. In Example 5.6 add a fourth action that is a noisy peek whose observation is correct with probability 0.9. Is $\{a, b\}$ still in $\mathcal{W}$? What does this say about safety with certainty and imperfect sensors?

Exercise 5.16. Compute Γ_1 and Γ_2 for the machine, with reward 1 for run in G, 0 for run in B, and 0 for inspect, and plot V_2 as a function of $b(G)$ (a piecewise-linear convex function).

Exercise 5.17. Prove that in the three-state example of Section 5.4 with no action-dependence the representation $(b_1, b_2 + b_3)$ is maintainable, by computing its update under the observations o_1, o_2.

Exercise 5.18. Show that Result 5.7 fails if $\mathcal{W}$ is replaced by the family of supports from which a policy achieves safety with probability at least $1/2$ (not almost surely), by giving an example of a union that is winning although a part is not.

Chapter 6

Resource-Sensitive Transitions and Petri Nets

Question. How do consumable resources change which transitions are executable?

Established sources. Petri nets (Murata); decidability of reachability (Mayr, Kosaraju); EXPSPACE-hardness of coverability (Lipton).

New results. The state equation is necessary but not sufficient for reachability (Result 6.2); place invariants bound the reachable markings (Result 6.3); the markings from which a net can run forever form an upward-closed set with a finite basis (Result 6.6).

Proof obligations. None.

Depends on. Chapter 2.

A resource is consumed when it is used. A transition that requires a token cannot fire when the token is absent, and firing may remove the token for later transitions. Petri nets model exactly this, and they do so with a finite description of what may be an infinite state space. This chapter states the model, proves the elementary invariants that constrain reachability, shows that the markings from which the system can keep running form an upward-closed set, and records what is known about the cost of deciding reachability.

6.1 Nets and firing

Definition 6.1 (Petri net)**.** A *Petri net* consists of finite sets P of places and T of transitions and weight functions $\mathrm{Pre}, \mathrm{Post} : T \to \mathbb{N}^P$. A *marking* is an element $m \in \mathbb{N}^P$. A transition t is *enabled* at m if $m \geq \mathrm{Pre}(t)$ componentwise, and firing it yields $m' = m - \mathrm{Pre}(t) + \mathrm{Post}(t)$, written $m \xrightarrow{t} m'$. The *incidence matrix* is the $P \times T$ matrix C with columns $\mathrm{Post}(t) - \mathrm{Pre}(t)$. For a firing sequence σ the *Parikh vector* $\#\sigma \in \mathbb{N}^T$ counts occurrences.

The nominal menu is T; the executable set at m is $\{t : m \geq \mathrm{Pre}(t)\}$. The rover of Example 1.2 is a net with places $p_0, \ldots, p_3$ (one token marks the position), b and d (battery charge and deficit, with $b + d = 3$), transitions m_{ij} for adjacent positions consuming p_i and b and producing p_j and d, and a charging transition consuming p_0 and d and producing p_0 and b (one unit at a time).

6.2 Necessary conditions for reachability

New result 6.2 (The state equation)**.** *If $m \xrightarrow{\sigma} m'$ then $m' = m + C\,\#\sigma$. Hence m' is reachable from m only if the equation $m' = m + Cx$ has a solution $x \in \mathbb{N}^T$. The converse fails: in the net with places p, q, s, transitions $t_1 : p \to q + s$ and $t_2 : q \to p$, the equation from $m = (0, 0, 0)$ to $m' = (0, 0, 1)$ is solved by $x = (1, 1)$, but no transition is enabled at m, so m' is not reachable.*

Assumptions: Weights are nonnegative integers.

Proof. By induction on $|\sigma|$: each firing adds the column $\mathrm{Post}(t) - \mathrm{Pre}(t)$. For the counterexample $C\,(1, 1)^{\mathsf{T}}$ is the sum of $(-1, 1, 1)$ and $(1, -1, 0)$, which is $(0, 0, 1)$. At $(0, 0, 0)$ neither t_1 (needing p) nor t_2 (needing q) is enabled. □

The failing instance has a cycle that would be fine if it could start; the solution of the equation does not respect the order in which tokens become available. This is the sense in which resources impose an ordering that linear algebra alone cannot see.

New result 6.3 (Place invariants)**.** *If $y \in \mathbb{Z}^P$ satisfies $y^{\mathsf{T}}C = 0$ then $y \cdot m$ is constant on every set of markings reachable from one another. If $y \geq 0$ and $y \cdot m_0 = k$ then every reachable marking has support in $\{p : y_p > 0\}$ with $y \cdot m = k$, so every place with $y_p > 0$ is bounded by $\lfloor k/y_p \rfloor$.*

Assumptions: Weights are nonnegative integers.

Proof. By Result 6.2, $y \cdot m' = y \cdot m + y^{\mathsf{T}}C\,\#\sigma = y \cdot m$. If $y \geq 0$ then $y_p m'(p) \leq y \cdot m' = k$ for each p. □

For the rover net, $y = (1, 1, 1, 1, 0, 0)$ and $y' = (0, 0, 0, 0, 1, 1)$ are invariants: every move transition removes a position token and adds one, and every charge unit moves a token between d and b. With the initial marking of one position token and $b + d = 3$, every reachable marking has exactly one position token and $b + d = 3$, so at most $4 \cdot 4 = 16$ markings are reachable, the state space of Example 1.2. The net is bounded and Chapter 2 applies to its marking graph.

6.3 Monotonicity and upward-closed sets

New result 6.4 (Monotonicity)**.** *If $m \xrightarrow{\sigma} m'$ and $x \in \mathbb{N}^P$ then $m + x \xrightarrow{\sigma} m' + x$. Consequently, if a net can fire an infinite sequence from m it can from every $m + x$, and if a marking $\geq g$ can be reached from m then one can be reached from $m + x$.*

Proof. If $m \geq \mathrm{Pre}(t)$ then $m + x \geq \mathrm{Pre}(t)$, and the result of firing is $m + x - \mathrm{Pre}(t) + \mathrm{Post}(t)$. Induct on $|\sigma|$. □

Established result 6.5 (Dickson's lemma)**.** *Every upward-closed subset of $\mathbb{N}^P$ has finitely many minimal elements, and is the set of markings above one of them.*

Source: Dickson (1913).

New result 6.6 (The viability set of a net is upward closed)**.** *The set $M_\infty = \{m \in \mathbb{N}^P :$ an infinite firing sequence is enabled at $m\}$ is upward closed and so has a finite basis of minimal markings. Likewise for every target $g \in \mathbb{N}^P$ the set of markings from which a marking $\geq g$ can be reached is upward closed with a finite basis.*

Assumptions: The set of markings is $\mathbb{N}^P$ with the componentwise order.

Proof. Upward closure is Result 6.4; the finite basis is Result 6.5. □

The set M_∞ is the viability kernel of the constraint set $\mathbb{N}^P$ in the sense of Chapter 3: the markings from which operation can continue forever. The theorem says that it is characterized by finitely many minimal resource endowments, a finite list of resource vectors each of which suffices and below none of which operation eventually stalls. The same holds for the markings from which a goal can be covered, which is the set of initial endowments that make a task feasible.

6.4 What is known about the complexity

Established result 6.7 (Decidability and complexity)**.** *Reachability of a marking in a Petri net is decidable. Coverability, deciding whether a marking above a given one is reachable, is EXPSPACE-complete. Boundedness and the existence of an infinite firing sequence are decidable.*

Source: Mayr (1981) and Kosaraju (1982) for decidability of reachability; Lipton (1976) for EXPSPACE-hardness and Rackoff (1978) for the matching upper bound for coverability; Karp and Miller (1969) for boundedness. Recent work shows reachability is Ackermann-complete (Czerwiński and Orlikowski; Leroux, 2021).

The practical consequence is that exact reachability questions for resource-sensitive systems are expensive in general, while the structural statements of this chapter, invariants and upward closure, come cheaply and often settle the question that matters.

6.5 Backward coverability

Result 6.6 says that the set of markings from which a goal can be covered has a finite basis, and it can be computed. For an upward-closed set U with basis B and a transition t, the markings from which one firing of t lands in U form an upward-closed set whose basis is obtained by a one-line formula.

Established result 6.8 (Backward coverability)**.** *For $u \in \mathbb{N}^P$ and a transition t, the least marking m with $m \xrightarrow{t} m'$ and $m' \geq u$ is $\mathrm{pb}(t,u) = \mathrm{Pre}(t) + \max\big(0,\ u - \mathrm{Post}(t)\big)$, with maximum taken componentwise. Define $B_0 = \{g\}$ and $B_{k+1} = \min\big(B_k \cup \{\mathrm{pb}(t,u) : u \in B_k,\ t \in T\}\big)$. Then B_k is the basis of the set of markings from which g can be covered by at most k firings, the sequence stabilizes after finitely many steps, and the limit is the basis of the set of markings from which a marking $\geq g$ is reachable.*

Source: Abdulla, Čerāns, Jonsson and Tsay (1996); Esparza, Finkel and Mayr (also for the general framework of well-structured systems).

Proof. The two conditions $m \geq \mathrm{Pre}(t)$ and $m - \mathrm{Pre}(t) + \mathrm{Post}(t) \geq u$ together say $m \geq \mathrm{Pre}(t)$ and $m \geq \mathrm{Pre}(t) + u - \mathrm{Post}(t)$, which is $m \geq \mathrm{pb}(t,u)$. Let U_k be the set of markings above some element of B_k. Then $U_0 \subseteq U_1 \subseteq \cdots$ is an increasing chain of upward-closed sets, and U_k is exactly the set of markings from which g is covered in at most k firings. If the chain were

strictly increasing forever, each step would add a new minimal element not above any earlier one, giving an infinite sequence in $\mathbb{N}^P$ with no element above an earlier one, which Dickson's lemma (Result 6.5) forbids. So $U_k = U_{k+1}$ for some k, and then U_k is closed under predecessors and equals the limit. □

Example 6.9 (Two coins make a coin)**.** A net with places A, B, C and two transitions: t_1 consumes two tokens from A and produces one in B; t_2 consumes one token from B and produces one in each of A and C. The goal is to cover $C \geq 1$. Starting from $g = (0,0,1)$ (in coordinates (A,B,C)): $\mathrm{pb}(t_1, g) = (2,0,0) + \max(0, (0,0,1) - (0,1,0)) = (2,0,1)$, which is above g and is discarded; $\mathrm{pb}(t_2, g) = (0,1,0) + \max(0, (0,0,1) - (1,0,1)) = (0,1,0)$. So $B_1 = \{(0,0,1), (0,1,0)\}$. In the next round, $\mathrm{pb}(t_1, (0,1,0)) = (2,0,0) + \max(0, (0,1,0) - (0,1,0)) = (2,0,0)$, and $B_2 = \{(0,0,1), (0,1,0), (2,0,0)\}$. Further rounds add nothing, so the basis is $\{C,\ B,\ 2A\}$: C can be covered from a marking iff it already holds a C-token, a B-token (fire t_2), or two A-tokens (fire t_1 then t_2). A brute-force search over all markings with at most five tokens per place agrees with this basis.

6.6 The coverability tree and unboundedness

When the net can be unbounded, the marking graph is infinite and exploration does not terminate. The Karp–Miller construction repairs this by acceleration.

Established result 6.10 (Karp–Miller coverability tree)**.** *Extend $\mathbb{N}$ to $\mathbb{N} \cup \{\omega\}$ with $\omega + k = \omega$ and $\omega \geq n$. Build a tree from the root m_0: for a node labelled m and each enabled transition t, the child is labelled $m' = m - \mathrm{Pre}(t) + \mathrm{Post}(t)$, except that if an ancestor m'' satisfies $m'' \leq m'$ and $m'' \neq m'$, every place p with $m''(p) < m'(p)$ is set to ω in the child, and branches stop when a node repeats an ancestor's label. The tree is finite. A place is unbounded iff some node has ω in that place, and a marking $\geq g$ is coverable iff some node label is $\geq g$.*

Source: Karp and Miller (1969).

Example 6.11 (A source)**.** One place A initially marked and one place B, and a transition t consuming nothing from B and one token from A while producing a token in A and a token in B. The root is $(1,0)$; its child is $(1,1)$, which is above the ancestor $(1,0)$ in place B, so it becomes $(1,\omega)$, whose child is again $(1,\omega)$ and stops. The tree has three nodes. The place B is unbounded, A is bounded by 1, and every marking $(1,k)$ is coverable.

In the worst case the tree is very large: its size is not bounded by any primitive recursive function of the size of the net (see Figueira, Figueira, Schmitz and Schnoebelen 2011 for the Ackermannian bounds), although coverability itself is decidable in exponential space (Result 6.7). The backward algorithm of the previous section often does much better in practice because it works with minimal elements only.

6.7 Invariants as design constraints

Place invariants (Result 6.3) are sometimes read as properties of a given net and sometimes as specifications to be enforced when a net is designed. The second reading gives a design rule. If a conservation law $y \cdot m = k$ is required (for example, the total number of tokens representing

a licence pool is fixed), then every transition must satisfy $y \cdot (\mathrm{Post}(t) - \mathrm{Pre}(t)) = 0$, a linear condition on the transition weights that can be checked locally, transition by transition. A violation by any one transition breaks the conservation for the whole system, and a violating transition can be found in time linear in the size of the net. The dual use is diagnostic: the failure of an expected invariant at a measured marking, $y \cdot m \neq k$, where $k = y \cdot m_0$ is computed from the modelled initial marking, certifies that either a transition not in the model has occurred or the measurement or the initial marking is wrong, since no firing sequence of the modelled net from m_0 can produce the marking.

Exercises

Exercise 6.12. Verify that $y = (1, 1, 1, 1, 0, 0)$ and $y' = (0, 0, 0, 0, 1, 1)$ are invariants of the rover net. Use them to list the reachable markings and compare with the sixteen states of Example 1.2.

Exercise 6.13. Add a transition that creates a spare battery token with no input, and show that the net becomes unbounded. Which invariant fails?

Exercise 6.14. Prove that if a net is bounded then the set of reachable markings from any m is finite, and conclude that reachability is decidable by exploration of the marking graph of Chapter 2.

Exercise 6.15. Find the minimal markings of M_∞ for the net with one place p and one transition consuming one token from p and producing one token in p, and for the net with a transition consuming two tokens and producing one.

Exercise 6.16. Show that the set of markings from which a transition t can ever fire is upward closed, and compute its basis for the net of Result 6.2 with $t = t_2$.

Exercise 6.17. Run the backward algorithm by hand for the net of Example 6.9 with the goal $(0, 0, 2)$ and find the basis.

Exercise 6.18. Build the Karp–Miller tree for the net with places A, B, initial marking $(1, 0)$, and two transitions $t_1 : A \mapsto A + B$ and $t_2 : B \mapsto A$. Identify the unbounded places.

Exercise 6.19. Prove that the formula for $\mathrm{pb}(t, u)$ in Result 6.8 gives the *least* predecessor and not merely a predecessor, and give a net in which a predecessor not above $\mathrm{pb}(t, u)$ would be enabled but would not reach u.

Exercise 6.20. For the rover net, add a transition that leaks a battery token with no input (the battery self-discharges). Show that the set of markings from which the rover can run forever (without consuming the last token) is smaller than before and compute its basis.

Exercise 6.21. Show that the set of markings from which a given marking g is exactly reachable is not in general upward closed, by an example in a one-place net.

Part III

Histories that change feasibility

Chapter 7

History-Dependent State and Augmentation

Question. When does a purported state description omit information needed to determine feasible continuation?

Established sources. State augmentation; Myhill–Nerode.

New results. A state description is sufficient for feasible continuation exactly when its kernel lies inside continuation equivalence (Result 7.1); the least augmentation has as many values as the largest number of continuation classes in a single state fibre; hard budgets, counters of bounded use, and pending obligations are augmentations of this kind, and unbounded counters admit none (Result 7.5).

Proof obligations. None.

Depends on. Chapter 1 and Appendix A.

A state description is a promise. It says that two histories ending in the same state have the same future, so that nothing about how the state was reached need be remembered. The promise is often kept in the model that a designer first writes down and broken in the system that is later built: a charger that may be used a fixed number of times, a permit that was granted once, an upload that is still owed. In each case the nominal state is the same after two histories and the feasible continuations are not. This chapter states precisely what has been omitted, what it costs to put it back, and when it cannot be put back at finite cost.

7.1 Continuation classes and the fibre count

Fix an alphabet E of events and a prefix-closed set $\mathcal{H} \subseteq \mathsf{E}^*$ of histories, as in Appendix A. For $h \in \mathcal{H}$ write $\mathrm{Fut}(h) = \{w : hw \in \mathcal{H}\}$ for the set of feasible continuations. Two histories are *continuation equivalent*, $h \approx h'$, if $\mathrm{Fut}(h) = \mathrm{Fut}(h')$. This is the continuation equivalence of Appendix A for the trivial property, and by Result A.5 it is the coarsest right congruence on $\mathcal{H}$ that respects feasibility.

A *state description* is a function $x : \mathcal{H} \to X$ that is maintainable: $x(h) = x(h')$ implies $x(he) = x(h'e)$ whenever $he, h'e \in \mathcal{H}$. It is *sufficient for feasible continuation* if $\ker x \subseteq \approx$, that is, if $x(h) = x(h')$ implies $\mathrm{Fut}(h) = \mathrm{Fut}(h')$. For a value $\xi \in x(\mathcal{H})$ the *fibre* is $x^{-1}(\xi)$, and $\nu(\xi)$ denotes the number of classes of $\approx$ that meet the fibre. The *fibre count* is $N(x) = \sup_\xi \nu(\xi)$.

New result 7.1 (Augmentation)**.** *Let x be a state description, $g : \mathcal{H} \to G$ a function, and write $(x, g)(h) = (x(h), g(h))$.*

(a) x is sufficient for feasible continuation if and only if $N(x) = 1$.

(b) If (x, g) is maintainable, it is sufficient if and only if g takes different values on histories of the same fibre that lie in different classes of $\approx$.

(c) If $N(x) < \infty$ there is a g with $|g(\mathcal{H})| = N(x)$ such that (x, g) is maintainable and sufficient, and every g with (x, g) maintainable and sufficient has $|g(\mathcal{H})| \geq N(x)$. If $N(x) = \infty$ no finite g exists.

Assumptions: The state description x is maintainable.

Proof. (a) $N(x) = 1$ says that every fibre lies inside one class of $\approx$, which is $\ker x \subseteq \approx$. (b) is the definition of sufficiency read fibre by fibre: $(x, g)(h) = (x, g)(h')$ means $x(h) = x(h')$ and $g(h) = g(h')$, and then $h \approx h'$ is required.

(c) For the lower bound let ξ be a fibre value with $\nu(\xi) = N$ classes, and let $h_1, \ldots, h_N$ be histories of the fibre in distinct classes. By (b) the values $g(h_1), \ldots, g(h_N)$ are pairwise distinct. For the upper bound enumerate, for each value ξ, the classes of $\approx$ that meet the fibre $x^{-1}(\xi)$ by indices in $\{1, \ldots, \nu(\xi)\}$ and let $g(h)$ be the index of the class of h among those of the fibre of $x(h)$. Then g takes at most $N(x)$ values, and by construction two histories with the same (x, g) lie in the same class, so (x, g) is sufficient. It is maintainable: if $(x, g)(h) = (x, g)(h')$ then $h \approx h'$, so $he \approx h'e$ because $\approx$ is a right congruence, and $x(he) = x(h'e)$ because x is maintainable; the class of he and the value $x(he)$ together fix $g(he)$, and these coincide for $h'e$. If $N(x) = \infty$ the lower-bound argument produces arbitrarily many distinct values of g. □

The fibre count measures the information that the state description omits, and the theorem says that exactly this much, no more and no less, must be added. The augmentation of Appendix A in terms of the whole equivalence uses $|\mathcal{H}/\!\approx|$ values in total; the fibre count is smaller because the state description x is kept and already carries part of the distinction. For example if x already separates all classes then $N(x) = 1$ and nothing is added.

7.2 Three augmentations

Example 7.2 (A charger of bounded use)**.** Consider a variant of the rover of Example 1.2 in which one event `charge`, available only at the dock, restores the battery to 3, and in which the charger may be used at most m times in the whole life of the rover. The state description $x = (p, b)$ of position and charge is maintainable. For $0 \leq j \leq m$ let $h_j = (\text{right left charge})^j$. Each h_j ends in the state $(0, 3)$, and $\mathrm{Fut}(h_j)$ contains $(\text{right left charge})^k$ exactly when $k \leq m - j$. So the $m + 1$ histories lie in one fibre and in distinct classes, and $N(x) \geq m + 1$. A counter of charges used, capped at m, makes the description sufficient, and Result 7.1 shows that no augmentation with fewer than $m + 1$ values does.

Example 7.3 (A pending obligation)**.** In the rover of Example 1.2, after `sample` an upload is owed. The histories `sample` and `sample upload` end in the same state, and Fut differs because only the first permits and requires an upload. The fibre count is 2, and one bit, whether an upload is pending, is the least augmentation. This is the bit that Chapter 1 added to remove the representational obstruction.

New result 7.4 (Hard budgets)**.** *Let (S, A, P) be a finite process with integer costs $C(s, a) \in \{0, 1, 2, \dots\}$ and a budget $d \in \mathbb{N}$, and call a history* within budget *if its accumulated cost does not exceed d. Let $\mathcal{H}$ consist of the within-budget histories of (S, A, P) and let $x(h) = s$ be the final state. Then:*

(a) The remaining budget $r(h) = d - \mathit{cost}(h)$ makes (x, r) maintainable and sufficient for feasible continuation, so the product process on $S \times \{0, \dots, d\}$ is an exact model of the budgeted process.

(b) The set W^* of Result 4.3 computed in the product process is upward closed in the remaining budget: if $(s, r) \in W^*$ and $r' \geq r$ then $(s, r') \in W^*$.

(c) The remaining budget cannot be dropped: for every d there is a process with a single state in which $N(x) = d + 1$.

Assumptions: Finite state and action sets; integer costs; a history is within budget if every prefix is.

Proof. (a) The remaining budget after he is $r(h) - C$, which depends only on $r(h)$ and the event e taken, and the event is feasible exactly when $r(h) \geq C$. The final state updates as in P. So (x, r) is maintainable, and the feasible continuations of h are determined by $(x(h), r(h))$.

(b) Let $W_0 = \{(s, r) : s \in K, r \geq 0\}$ and $W_{n+1} = W_0 \cap \mathrm{Pre}_{\forall\exists}(W_n)$. W_0 is upward closed in r. Suppose W_n is, and let $(s, r) \in W_{n+1}$ with action a such that every $(s', r - C(s, a))$ with $s' \in \operatorname{supp} P(s, a)$ lies in W_n. For $r' \geq r$ the same action leads to $(s', r' - C(s, a))$, which has remaining budget at least as large, so lies in W_n by hypothesis, and $r' \geq 0$. So W_{n+1} is upward closed, and so is the intersection W^* of the decreasing sequence.

(c) Take one state, one action of cost 1 and the budget d. Then $\mathrm{Fut}(h)$ is the set of words of length at most $d - |h|$, and the $d+1$ histories of length $0, \dots, d$ lie in one fibre and in distinct classes. □

Part (b) is why a hard budget is tractable: for each state there is a threshold remaining budget above which feasibility holds, so the kernel is described by at most one number per state, though the number of the augmented states is larger. The constraint on expected cost of Chapter 4 is the continuous analogue: the remaining budget there is a real number, and the augmented state has a continuum fibre.

7.3 When no finite augmentation exists

New result 7.5 (Unbounded counters admit no finite augmentation)**.** *Let $\mathsf{E} = \{a, b\}$ and $\mathcal{H} = \{h :$ every prefix of h has at least as many a's as b's$\}$. For the constant state description x the fibre count is infinite, and no finite state description (maintainable function with finitely many values) is sufficient for feasible continuation.*

Assumptions: None.

Proof. Let $h_n = a^n$. Then $\mathrm{Fut}(h_n)$ contains b^n but not b^{n+1}, so the histories h_n are pairwise in distinct classes of $\approx$ and all lie in the one fibre of the constant description. If x' is a maintainable function with finitely many values that is sufficient, by Result A.5 it has at least as many values as $\approx$ has classes, and the number of classes is infinite. □

The system is a counter: each a adds a unit of permission and each b spends one. The result says that a system whose resources are unbounded cannot be described by a finite

state, however large, and that the set of histories it allows is not a regular language. In the vocabulary of Chapter 6, the augmenting quantity is the marking of an unbounded place. Finite representations then require either a bound, imposed by a cap on the resource that makes some histories infeasible, or an infinite state space with a finite description, such as that of a Petri net.

7.4 Checking sufficiency of a given description

A designer who proposes a state description needs a way to test it. For finite histories the test is algorithmic.

New result 7.6 (Deciding sufficiency)**.** *Let $\mathcal{H}$ be given by a deterministic automaton with n states (all states live) over a finite alphabet of size σ, and let x be a maintainable description given by a map from the automaton states to X that is compatible with the transitions. Then $N(x) = 1$, equivalently sufficiency for feasible continuation, is decided in time $O(n\sigma \log n)$ by computing the minimal automaton, and when $N(x) > 1$ the algorithm returns two histories in the same fibre and a shortest continuation that separates them.*

Assumptions: Finite automaton with every state reachable and live; x is a function of the automaton state, compatible with the transitions.

Proof. Minimize the automaton by Hopcroft's algorithm in time $O(n\sigma \log n)$; the classes of states of the minimal automaton are the classes of $\approx$ (Result A.5). The description x is sufficient iff each fibre meets exactly one class, which is checked by one pass labelling each state with its class and its x-value. If a fibre meets two classes, take two states from it in different classes; breadth-first search in the product of the minimal automaton with itself finds a shortest word on which they disagree on feasibility. □

The test is cheap compared with the cost of the failure it prevents. A system whose designers could not state the state description precisely enough to run the test has not, in the sense of this chapter, promised anything.

7.5 Rate limits

A rate limit says that at most k uses are allowed in any window of w consecutive steps. It is the augmentation problem for a rule that mixes a budget with a clock. At each step the event is `use` or `idle`. The raw state is the set $S \subseteq \{1, \dots, w-1\}$ of ages of the uses still inside the window, with $|S| \le k$; a use is allowed iff $|S| < k$.

New result 7.7 (Irrelevant ages)**.** *Write the elements of S as $a_1 < \cdots < a_j$. The element a_i affects feasible continuation only if $a_i \le w - 1 - (k - i)$. Removing all elements that violate this inequality (they form the oldest elements of S) gives a state with the same set of feasible continuations. The normalized states are the sets with $j \le k$ elements satisfying $a_i \le w - 1 - k + i$, and there are*

$$\sum_{j=0}^{k} \binom{w-1-k+j}{j} = \binom{w}{k}$$

of them for $1 \le k < w$. Hence the least maintainable sufficient description has at most $\binom{w}{k}$ values.

Assumptions: $1 \le k < w$.

Proof. Consider a_i and a future step at which it is still in the window, that is, after t steps with $a_i + t \le w - 1$, so within the next $w - 1 - a_i$ steps; it is the i-th youngest of the old uses and the $i - 1$ younger ones outlast it. A use is blocked by a_i at some step only if, at that step, removing a_i would allow it, i.e. the number of alive old uses together with the new uses made since is exactly k with a_i and $k - 1$ without. At that step i old uses (at least) are alive, so the number of new uses made earlier is $k - i$, and these need $k - i$ separate steps before the blocked attempt, which must occur while a_i is alive: $k - i + 1 \le w - a_i$, which is $a_i \le w - 1 - (k - i)$. If the inequality fails, a_i never blocks a use that would otherwise be allowed, and the same holds for every older element. Dropping them changes no feasibility, and the count is the number of sequences $1 \le a_1 < \cdots < a_j$ with $a_i \le m + i$ for $m = w - 1 - k$. Setting $d_i = a_i - i$ gives nondecreasing sequences $0 \le d_1 \le \cdots \le d_j \le m$, of which there are $\binom{m+j}{j}$, and $\sum_{j=0}^{k} \binom{m+j}{j} = \binom{m+k+1}{k} = \binom{w}{k}$ by the hockey-stick identity. □

An exhaustive computation of the fibre count by partition refinement for $w \le 6$ and every k with $1 \le k \le 4$ gives $\binom{w}{k}$ exactly (for $k \ge w$ the rule never binds and the count is 1): for $w = 4$ the counts are $4, 6, 4$ for $k = 1, 2, 3$; for $w = 5$ they are $5, 10, 10, 5$; for $w = 6$ they are $6, 15, 20, 15$ for $k = 1, \ldots, 4$. The normal form is therefore not only sufficient but, in the cases checked, minimal. The shape of the answer is worth recording: the memory needed to enforce a rate limit of k per w steps is $\log_2 \binom{w}{k} \approx k \log_2(w/k) + k \log_2 e$ bits when $k \ll w$. Recording the times of the last k uses as an ordered list would cost about $k \log_2 w$ bits; the difference, about $\log_2 k! \approx k \log_2(k/e)$ bits, is the cost of remembering which use was which, and the normal form does not pay it because only the set of ages matters.

Exercises

Exercise 7.8. In Example 7.2 give the classes of $\approx$ explicitly for $m = 2$, and exhibit a maintainable description of exactly the right size.

Exercise 7.9. Show that two successive augmentations (x, g_1) and $((x, g_1), g_2)$ can be replaced by a single augmentation of no greater size than the one given by Result 7.1.

Exercise 7.10. In the hard-budget setting suppose all costs are multiples of $q > 1$ and d is not. Show that the fibre count is smaller than $d + 1$ and compute it.

Exercise 7.11. Describe the sufficient augmentation for a rule that a permit must be renewed within k steps of each use, and compute its size as a function of k.

Exercise 7.12. Prove that if x is a state description and $x' = f \circ x$ is a maintainable coarsening of it, then $N(x') \ge N(x)$, and give an example where the inequality is strict.

Exercise 7.13. Run the normalization of Result 7.7 by hand for $w = 5$, $k = 2$ and list the 10 normalized states. Verify that the raw states $\varnothing$ and $\{4\}$ have the same feasible continuations.

Exercise 7.14. For $k = 1$ prove directly that the least sufficient description for the rate limit "at most one use in any w consecutive steps" has exactly w values (the number of steps since the last use, capped at w).

Exercise 7.15. Prove the lower bound of Result 7.7 for $k = w-1$: show that the w normalized states are pairwise separated by continuations.

Exercise 7.16. Describe the sufficient description for a rule requiring a minimum gap of g steps between any two uses and a lifetime limit of m uses, and count its values. Compare with the product of the two separate descriptions.

Exercise 7.17. Apply Result 7.6 to the description $x(h) =$ "number of uses in the last w steps" for the rate limit, and give a pair of histories in the same fibre that it fails to separate, with a separating continuation, for $w = 4$, $k = 2$.

Chapter 8

Irreversibility and Path Dependence

Question. How is costly reversal distinguished from impossible reversal?

Established sources. Absorbing sets; hysteresis; weighted shortest paths (Dijkstra); finite Markov chains (Norris).

New results. Return cost is a quasi-metric whose finite part is the reversibility relation of Chapter 2 (Result 8.1); no budget above $|X|-1$ unit steps buys reversibility that a smaller budget does not (Result 8.2); path dependence is the failure of the state to be sufficient (Result 8.5); a policy can make a reversible move irreversible in effect and never the reverse (Result 8.7).

Proof obligations. None.

Depends on. Chapter 2 and Chapter 7.

Chapter 2 separated edges that can be undone by some sequence of actions from edges that cannot, and gave the first the name reversible without saying what undoing costs. A move that can be undone at a price that no one will pay differs little in practice from one that cannot, and a move that is undone at a negligible price differs little from one that never happened. This chapter attaches costs, shows that reversal cost has the structure of an asymmetric distance, and relates it to path dependence, the property that two histories reaching the same place leave different futures behind.

8.1 The cost of return

Let G be a finite transition graph on X whose edges carry costs $c(x \xrightarrow{u} y) \geq 0$. The cost of a path is the sum of its edge costs. Define the *return cost*

$$\rho(x, y) = \min\{\mathrm{cost}(\pi) : \pi \text{ a path from } x \text{ to } y\},$$

with $\rho(x, y) = \infty$ if there is no such path and $\rho(x, x) = 0$. The cost of reversing an edge $x \to y$ is $\rho(y, x)$. The shortest paths, and hence ρ, are computed for all pairs in polynomial time by Dijkstra's algorithm run from each source.

New result 8.1 (Return cost is a quasi-metric)**.** *(a) $\rho(x, x) = 0$ and $\rho(x, z) \leq \rho(x, y) + \rho(y, z)$ for all x, y, z, with the convention $\infty + t = \infty$.*

(b) $\rho(x, y) < \infty$ if and only if $y \in R(x)$, the set reachable from x. An edge $x \to y$ is reversible in the sense of Chapter 2 if and only if $\rho(y, x) < \infty$.

(c) *The symmetrization* $\sigma(x, y) = \rho(x, y) + \rho(y, x)$ *is symmetric, satisfies the triangle inequality, and is finite exactly when* x *and* y *lie in the same strongly connected component. On each component it is a pseudometric.*

Assumptions: Nonnegative edge costs; X finite.

Proof. (a) Concatenating a cheapest path from x to y with one from y to z gives a path from x to z of cost $\rho(x, y) + \rho(y, z)$, so the minimum is at most this; if either term is infinite the inequality is trivial. (b) is the definition of reachability together with Result 2.5. (c) Symmetry is evident. For the triangle inequality add the two instances $\rho(x, z) \le \rho(x, y) + \rho(y, z)$ and $\rho(z, x) \le \rho(z, y) + \rho(y, x)$ to get $\sigma(x, z) \le \sigma(x, y) + \sigma(y, z)$. Finiteness of σ means that each of x, y is reachable from the other. □

The return cost is not symmetric. A well in which descent is cheap and climbing is dear gives $\rho(\text{top}, \text{bottom}) \ll \rho(\text{bottom}, \text{top})$, and the structure is that of a quasi-metric rather than of a metric: the length of a round trip is symmetric, but the price of each leg is not. In a graph of unit costs ρ is the directed graph distance.

New result 8.2 (A budget above the number of states buys no further reversibility)**.** *For a budget* β *say that an edge* $x \to y$ *is* β-reversible *if* $\rho(y, x) \le \beta$*. The* β*-reversible edges increase with* β *and for unit costs the set of* β*-reversible edges for* $\beta = |X| - 1$ *is the set of all reversible edges. For general nonnegative costs the same holds with* $\beta = (|X| - 1)\, c_{\max}$*, where* $c_{\max}$ *is the largest edge cost.*

Assumptions: Finite X.

Proof. Monotonicity in β is clear. If $y \to^* x$ then there is a path from y to x visiting no state twice, since a repeated state can be cut out without raising cost, and such a path has at most $|X| - 1$ edges, so cost at most $(|X| - 1)c_{\max}$. Hence $\rho(y, x) \le (|X| - 1)c_{\max}$ for every reversible edge. □

The result separates costly reversal from impossible reversal on a finite system by a threshold: an edge is impossible to reverse if and only if no finite budget reverses it, and in a system with n states and unit costs it is enough to test the budget $n - 1$. The distinction is therefore never a matter of degree in a fixed finite system, though it is a matter of degree across a family of systems whose size grows, which is where the difference between costly and impossible matters in practice.

Example 8.3 (A well)**.** States $0, 1, \ldots, n$, with edges $i \to i-1$ of cost 1 for $1 \le i \le n$ (descent) and $i \to i + 1$ of cost k for $0 \le i < n$ (ascent). State 0 also has a waiting edge to itself of cost 0. The graph is strongly connected, so every edge is reversible. Descending from n to 0 costs n and returning costs nk. For $k = 1$ the system is a symmetric chain; as $k \to \infty$ the ascent becomes unaffordable at any fixed budget, and at $k = \infty$ (edges removed) the system is a chain of $n + 1$ singleton components and every descent is irreversible. The family interpolates between reversible and irreversible without a discontinuity in ρ until the edge is deleted.

8.2 Path dependence

Definition 8.4 (Path dependence)**.** A system with histories $\mathcal{H}$ and a state description x is *path dependent* if there are histories h, h' with $x(h) = x(h')$ and $\mathrm{Fut}(h) \neq \mathrm{Fut}(h')$.

New result 8.5 (Path dependence is insufficiency of the state)**.** *A system is path dependent with respect to a maintainable state description x if and only if x is not sufficient for feasible continuation, that is, if and only if the fibre count $N(x) > 1$ in the sense of Chapter 7. In a path-dependent system the quantity that distinguishes h and h' is a function of the histories that is not a function of x, and its least encoding has $N(x)$ values.*

Assumptions: As in Result 7.1.

Proof. This is Result 7.1(a), (c) with Definition 8.4. □

Example 8.6 (A relay with hysteresis)**.** An input level $v \in \{0, 1, 2\}$ is applied at each step. A hidden output $y \in \{\mathrm{lo}, \mathrm{hi}\}$ is set to hi when $v = 2$, to lo when $v = 0$, and left unchanged when $v = 1$. The action `run` is feasible exactly when $y = \mathrm{hi}$. Take the state x to be the last input level. The histories 2 1 and 0 1 both have $x = 1$, but `run` is feasible after the first and not after the second. So the system is path dependent with fibre count 2 at $x = 1$, and the hidden bit y is the minimal augmentation. Changing y from hi to lo requires a step at level 0 and from lo to hi a step at level 2, so the feasibility of `run` depends on the last extreme input and not on the present one. This is the memory of a hysteresis loop in the vocabulary of control: the present input does not determine the output.

Irreversibility and path dependence are related but distinct. Irreversibility is a property of the transition graph, of the existence of paths. Path dependence is a property of the relation between a state description and the histories, of whether the description has dropped something that matters. A system with an irreversible edge can be path-independent in the state description that already records the component, and a system with no irreversible edge, the relay above, can be path dependent.

8.3 Policies and the components they induce

A stationary policy selects one feasible action in each state; it induces a subgraph with the same states and, at each state, the chosen edges (one per state in the deterministic case).

New result 8.7 (Policies refine components)**.** *Let G_π be the subgraph induced by a stationary policy π of a deterministic system, or by the actions in the support of a stochastic policy. Every strongly connected component of G_π is contained in a component of G. Hence an edge irreversible in G is irreversible in G_π, while an edge reversible in G can be irreversible in G_π.*

Assumptions: X finite.

Proof. A path of G_π is a path of G, so if x, y are mutually reachable in G_π they are in G. For the last claim take the well of Example 8.3 with finite k and the policy that always descends: its subgraph has no edge ascending, and every descent is irreversible in G_π though reversible in G. □

Established result 8.8 (Absorption in a finite Markov chain)**.** *Consider a finite Markov chain. With probability one the chain eventually enters a bottom strongly connected component (a component with no edge leaving it) and stays there forever; the expected time to do so is finite.*

Source: Norris, *Markov Chains* (1997), Chapter 1.

For a system under a stationary policy this says that, whatever the reversible structure of G, long-run behaviour is concentrated on the bottom components of G_π, and the reversible region of G outside them is traversed only finitely often. A policy, by selecting among feasible actions, selects which part of the possible reversibility is exercised, and by Result 8.7 it can only reduce it. The reversibility that a system has is thus a property of the feasibility structure, an upper bound on what any policy can retain, and the reversibility that a system retains is a property of the policy it follows.

8.4 The reversibility profile

Result 8.2 says that a finite system has a threshold beyond which no reversal becomes affordable. Below the threshold the picture is graded, and a convenient summary is the fraction of edges that can be reversed within a budget.

Definition 8.9 (Reversibility profile)**.** For a graph with edge set E (self-loops excluded) and return cost ρ the *reversibility profile* is $\mathrm{Rv}(\beta) = |\{(x \to y) \in E : \rho(y, x) \le \beta\}|/|E|$ for $\beta \ge 0$. It is a nondecreasing step function, with values changing only at the finitely many distinct values of ρ, and its limit is the fraction of reversible edges.

For the well of Example 8.3 (ignoring the waiting edge) the descents $i \to i-1$ have reversal cost $\rho(i-1, i) = k$ and the ascents have reversal cost 1. So $\mathrm{Rv}(\beta) = 0$ for $\beta < 1$, $\frac{1}{2}$ for $1 \le \beta < k$, and 1 for $\beta \ge k$. For the rover, with each move or charge costing one unit, the three edges of the component $\{(0,1),(0,3),(1,2)\}$ of Section 1.5 each have reversal cost 2: the edge $(0,3) \to (1,2)$ is undone by `left` and then `charge`, the edge $(1,2) \to (0,1)$ by `charge` and then `right`, and the edge $(0,1) \to (0,3)$ by `right` and then `left`. The other 18 non-loop edges, such as $(1,2) \to (2,1)$, have infinite reversal cost. The profile of the rover graph is therefore 0 for $\beta < 2$ and 3/21 for $\beta \ge 2$, and stays there: a design in which most of the edges leave the component is a design in which most of the moves are final.

8.5 Making a system reversible

If irreversible edges are costly, a designer may add edges, that is, new capabilities that undo. The minimum number of edges needed has a clean answer.

Established result 8.10 (Strong connectivity augmentation)**.** *Let G be a directed graph whose condensation has s source components (no incoming edge from another component) and t sink components (no outgoing edge), not counting the case where G is already strongly connected. The least number of edges that must be added to make G strongly connected is* $\max(s, t)$*, except that components that are both source and sink (isolated components) count in both s and t.*

Source: Eswaran and Tarjan, "Augmentation problems," *SIAM Journal on Computing* 5 (1976).

For the chain $1 \to 2 \to 3 \to 4$ there is one source (the state 1) and one sink (the state 4), so one edge suffices, from 4 to 1. For three disjoint chains the count is three. The theorem says that the cost of making a system fully reversible is determined by the endpoints of its irreversible structure and not by its internal length: undoing a one-way process of any length takes a single return path, provided that path can be traversed. The practical obstacle is not the number of edges but their feasibility, which is the question of Chapter 9.

8.6 Expected cost of return in a stochastic system

When transitions are random, the cost of return is a random variable, and its expectation solves a linear system.

New result 8.11 (Expected return cost in a Markov chain)**.** *Let a finite Markov chain have transition matrix P and a nonnegative cost $c(s)$ per step spent in state s, and let B be a target set reached with probability one from every state outside it. The vector $h(s)$ of expected total cost until first entering B satisfies $h = 0$ on B and $h(s) = c(s) + \sum_{s'} P(s, s')h(s')$ otherwise, and is the unique solution $h = (I - Q)^{-1}c$, where Q is P restricted to B^c. If the target is reached with probability less than one from some state, $h(s) = \infty$ there.*

Assumptions: Finite chain.

Proof. Condition on the first step. When B is reached with probability one from every state outside it, $Q^n \to 0$, so $I - Q$ is invertible with inverse $\sum_n Q^n$, which gives uniqueness and the formula; $h = \sum_n Q^n c$ is the expected total cost. The remaining case lies outside the statement but completes it: if $c > 0$ outside B and from some state the target is missed with positive probability then the chain is trapped forever outside B with positive probability and, since the cost is positive in every state of the trap, the expected cost is infinite (with costs allowed to vanish in the trap the expectation could be finite). □

Example 8.12 (Returning with an unreliable ascent)**.** In the well with $n = 2$ suppose the ascent $i \to i+1$ succeeds with probability q and otherwise leaves the system at i, each attempt costing 1. The expected cost of ascending one level is $1/q$ by the geometric law, so from level 0 to level 2 it is $2/q$. For $q = 1/2$ the expected cost of returning is 4, against 2 if the ascent were reliable; with $q = 0.1$ it is 20. The expected cost diverges as $q \to 0$, and at $q = 0$ the descent is irreversible, which is the sense in which reversibility can be seen as a limit of expected costs.

Exercises

Exercise 8.13. Compute ρ for all pairs of the well of Example 8.3 with $n = 3$ and $k = 2$, and verify the triangle inequality on a triple of your choice.

Exercise 8.14. Give a graph in which ρ is not a metric on any component because $\rho(x, y) \neq \rho(y, x)$ for all $x \neq y$ in a component of size at least three, and compute σ.

Exercise 8.15. Show that the bound of Result 8.2 is attained: give a strongly connected graph on n states with unit costs in which some edge has $\rho(y, x) = n - 1$.

Exercise 8.16. For the relay of Example 8.6, compute the continuation classes and verify that the bit y together with the last input gives a sufficient description. How many values are required if the input range is $\{0, \dots, m\}$ with thresholds at 0 and m?

Exercise 8.17. Prove that if a stationary policy induces a graph G_π whose bottom components are singletons, then under that policy every state is left forever with probability one unless it is absorbing, and relate this to the ratchet of Result 2.5.

Exercise 8.18. Compute the reversibility profile of the well with $n = 3$, $k = 3$ including the waiting edge at 0, and sketch it.

Exercise 8.19. Apply Result 8.10 to the graph with edges $a \to b$, $a \to c$, $b \to d$, $c \to d$, $e \to f$. Count sources and sinks and give a set of added edges of minimum size that makes the graph strongly connected.

Exercise 8.20. Show that adding $\max(s, t)$ edges cannot in general be done by adding edges within the sink components, and that the added edges must go from sink components to source components.

Exercise 8.21. In Example 8.12 compute the probability that the return completes within m attempts for $q = 1/2$, and the expected cost of a round trip (descent cost 1 per level, ascent as above) from level 2.

Exercise 8.22. Prove the first half of Result 8.8 for a chain with a single bottom component: the chain enters it with probability one, using the fact that from every state there is a path to the component and the finite state space.

Chapter 9

Endogenous Action Spaces

Question. How are systems modelled in which actions create, destroy, or transform the available actions?

Established sources. Control systems with state-dependent action sets; delete-free planning (Bylander); Horn closure.

New results. When actions only add capabilities the reachable capability sets have a unique maximum, the closure, computed in time linear in the size of the rules (Result 9.2); shortest acquisition plans have length at most the number of capabilities (Result 9.3); nominal, eventual, and current menus are nested and the gaps between them are measurable (Result 9.4); when actions also remove capabilities, reachability is PSPACE-complete.

Proof obligations. Reachability for endogenous action spaces with both gains and losses under restricted structure.

Depends on. Chapters 2 and 7.

In the systems of the previous chapters the menu of actions was part of the specification: at each state the controller chooses among actions that are there. A great deal of what matters in practice does not have this form. A tool must be acquired before it can be used, a skill trained before it can be exercised, a permission granted before a request can be filed, and each of these acquisitions is itself the result of acting. The action space is then a state variable. This chapter gives a model for it, proves what can be said when acquisitions only accumulate, and separates what is nominally listed, what can eventually be done, and what can be done now.

9.1 Capability systems

Definition 9.1 (Capability system)**.** A *capability system* consists of a finite set Cap of capabilities and a finite set Act of actions. An action a has a precondition $\mathrm{req}(a) \subseteq \mathrm{Cap}$, a set $\mathrm{gain}(a) \subseteq \mathrm{Cap}$ of capabilities it creates, and a set $\mathrm{lose}(a) \subseteq \mathrm{Cap}$ of capabilities it destroys. A *configuration* is a set $C \subseteq \mathrm{Cap}$ of capabilities held. The action a is *enabled* at C if $\mathrm{req}(a) \subseteq C$, and then it leads to $C' = (C \setminus \mathrm{lose}(a)) \cup \mathrm{gain}(a)$. The system is *monotone* if $\mathrm{lose}(a) = \varnothing$ for every a.

Examples are of this form. A hammer is a capability required by the action *drive nail*; the action *buy hammer*, with the precondition that money is held, gains it. A licence is a capability

required by *file request*, gained by *pass test*, which requires the capability *identity document*, gained by *request document*, which requires *birth certificate*. A consumable tool is a capability lost by the action that uses it.

The enabled actions at a configuration C are the *current menu* $\mathrm{Cur}(C) = \{a : \mathrm{req}(a) \subseteq C\}$. The *nominal menu* is Act. The *eventual menu* $\mathrm{Ev}(C)$ is the set of actions enabled at some configuration reachable from C.

9.2 The monotone case

For a configuration C define the closure $\mathrm{Cl}(C)$ as the least set containing C and closed under the rule: if $\mathrm{req}(a) \subseteq D$ then $\mathrm{gain}(a) \subseteq D$. Equivalently $\mathrm{Cl}(C)$ is the limit of $C_0 = C$, $C_{k+1} = C_k \cup \bigcup\{\mathrm{gain}(a) : \mathrm{req}(a) \subseteq C_k\}$.

New result 9.2 (Closure in monotone systems)**.** *In a monotone capability system:*

(a) Every configuration reachable from C is contained in $\mathrm{Cl}(C)$, and $\mathrm{Cl}(C)$ is itself reachable from C. Hence the family of reachable configurations has a unique maximum.

(b) Cl is extensive ($C \subseteq \mathrm{Cl}(C)$), monotone ($C \subseteq D$ implies $\mathrm{Cl}(C) \subseteq \mathrm{Cl}(D)$) and idempotent.

(c) $\mathrm{Cl}(C)$ is computed in time $O\big(\sum_a(|\mathrm{req}(a)| + |\mathrm{gain}(a)|)\big)$.

(d) A capability c can be acquired from C if and only if $c \in \mathrm{Cl}(C)$.

Assumptions: $\mathrm{lose}(a) = \varnothing$ for all a.

Proof. (a) By induction on the length of a path, each reachable configuration lies in every set closed under the rule that contains C, in particular in $\mathrm{Cl}(C)$. For the converse, the sequence C_k above increases and stabilizes; each step from C_k to C_{k+1} is realized by firing the enabled actions one after another, and since enabledness is preserved as the configuration grows (monotonicity of $\mathrm{req}(a) \subseteq D$ in D), the actions can be fired in turn. (b) Extensiveness and idempotence are immediate from the definition. For monotonicity, $\mathrm{Cl}(D)$ contains C and is closed under the rule, so contains $\mathrm{Cl}(C)$. (c) Keep for each action a counter of unmet preconditions, and a queue of newly acquired capabilities; processing a capability decrements the counters of the actions that require it and fires an action when its counter reaches zero. Each requirement and each gain is processed once. (d) follows from (a). □

New result 9.3 (Short acquisition plans)**.** *In a monotone system, if a capability c can be acquired from C then it can be acquired by a sequence of at most $|\mathrm{Cap} \setminus C|$ actions, each of which gains a capability not previously held. This bound is attained.*

Assumptions: Monotone.

Proof. Take a shortest sequence of actions leading from C to a configuration containing c. If some action gained nothing new, deleting it leaves the configurations after it unchanged, so the later actions are still enabled and the sequence is shorter, contradicting minimality. So each action gains a new capability, and there are at most $|\mathrm{Cap} \setminus C|$ new capabilities. For attainment take capabilities $c_1, \ldots, c_n$, with an action a_i requiring c_{i-1} and gaining c_i ($c_0 \in C$); the only plan to obtain c_n is $a_1, \ldots, a_n$. □

New result 9.4 (Nominal, eventual, and current menus)**.** *For every configuration C of a monotone system,* $\mathrm{Cur}(C) \subseteq \mathrm{Ev}(C) \subseteq \mathrm{Act}$ *and* $\mathrm{Ev}(C) = \mathrm{Cur}(\mathrm{Cl}(C))$. *Each inclusion can be*

strict, and the length of the shortest plan enabling an action $a \in \mathrm{Ev}(C)$ is the distance *of a from C; it is at most $|\mathrm{Cap} \setminus C|$.*

Assumptions: Monotone.

Proof. The first inclusion holds because C is reachable from itself; the second is trivial. Since the reachable configurations are contained in $\mathrm{Cl}(C)$ and $\mathrm{Cl}(C)$ is reachable, an action is enabled somewhere reachable iff it is enabled at $\mathrm{Cl}(C)$, since enabledness is monotone in the configuration. Strictness: with $\mathrm{Cap} = \{c_1, c_2, c_3\}$, $C = \varnothing$, an action a_1 with no requirement gaining c_1, an action a_2 requiring c_1 and gaining c_2, and an action a_3 requiring the capability c_3, which no action gains, one finds $\mathrm{Cur} = \{a_1\}$, $\mathrm{Ev} = \{a_1, a_2\}$, $\mathrm{Act} = \{a_1, a_2, a_3\}$. The distance bound is Result 9.3. □

The result separates three notions that the list of actions blurs. The nominal menu says what the system has been designed to do. The eventual menu says what it can come to do. The current menu says what it can do now. A system can have a large nominal menu and a small eventual one, as when an action depends on a capability that no action creates, and the distance quantifies how far the eventual menu is from the current one.

Example 9.5 (Permission chains)**.** A licence application requires an identity document, which requires a birth certificate, which requires a form request. If each requirement is a capability gained by exactly one action, the plan to file the application has four steps, and the distance of *file application* from the empty configuration is four. A policy that adds one further prerequisite to the chain lengthens the shortest plan by one without changing the nominal menu at all, which is how administrative burden acts on freedom: through distance, not through the list.

9.3 Gains and losses

Established result 9.6 (Planning with deletions)**.** *For capability systems in which actions may lose capabilities, deciding whether a given capability can be obtained from a given configuration is PSPACE-complete. Without deletions, deciding whether a shortest plan of at most a given length exists is nonetheless NP-complete, even though deciding reachability is polynomial by Result 9.2.*

Source: Bylander (1994), *The computational complexity of propositional STRIPS planning.*

The established result shows that the monotone case is exceptional in two ways: closure removes the order of actions from the question of reachability, and with deletions the order matters and the problem becomes a search in a space of configurations of size $2^{|\mathrm{Cap}|}$. A consumable capability (a tool lost by use, a permission that expires) is a loss. If the capability may be held in multiple units, the system is a Petri net as in Chapter 6, and the markings are the multiplicities.

Example 9.7 (Training)**.** A skill that is gained after k practice sessions is not a capability in the sense of Definition 9.1, because the gain depends on a count. It is modelled by capabilities $t_1, \ldots, t_k$ with a practice action gaining t_i from t_{i-1}, and the skill is t_k. The monotone result applies and gives distance k. A skill that decays, so that t_i is lost when unused, is a loss and takes the system outside the monotone case.

9.4 A workshop

A small capability system shows the quantities of this chapter together. The capabilities are *money, tools, wood, id, permit, saw, shed, stock, fuel, kiln*, and the eight actions, written as req $\Rightarrow$ gain, are:

buy tools	money $\Rightarrow$ tools	build shed	tools, wood, permit $\Rightarrow$ shed
buy wood	money $\Rightarrow$ wood	make saw	tools, wood $\Rightarrow$ saw
get id	money $\Rightarrow$ id	make stock	saw, wood $\Rightarrow$ stock
get permit	id, money $\Rightarrow$ permit	fire kiln	shed, stock, fuel $\Rightarrow$ kiln

Starting from the configuration $C = \{\text{money}\}$, the closure iteration defined above adds in successive rounds

$$\{\text{wood, id, tools}\}, \quad \{\text{saw, permit}\}, \quad \{\text{shed, stock}\},$$

and stops, so $\mathrm{Cl}(C)$ has eight of the ten capabilities. The action *fire kiln* is never enabled: it requires *fuel*, which no action creates. The three menus are therefore the current menu {buy tools, buy wood, get id} of size three, the eventual menu of size seven, and the nominal menu of size eight. The gap between the nominal and eventual menus is a single action, and it is caused by a single missing capability; adding *fuel* to the starting configuration enlarges the closure by two capabilities (*fuel* itself and *kiln*) and the eventual menu to eight.

The *depth* of a capability is the round in which it first appears: one for *wood, id, tools*; two for *saw, permit*; three for *shed, stock*. If actions take one unit of time and independent actions can run in parallel, the earliest time to the shed is its depth, three. A single worker, performing one action at a time, needs the five actions *buy tools, buy wood, get id, get permit, build shed*, so the shortest sequential plan to the shed has length five. The two numbers answer different questions, and the difference is the available parallelism.

New result 9.8 (Depth and plan length)**.** *In a monotone capability system let $d(c)$ be the round of the closure iteration at which $c \in \mathrm{Cl}(C)$ first appears ($d = 0$ on C). Then $d(c)$ is the length of the shortest parallel plan for c (the minimum number of rounds, each round firing any set of enabled actions), and $d(c) \le \ell(c) \le |\mathrm{Cap} \setminus C|$, where $\ell(c)$ is the length of a shortest sequential plan. Both bounds are attained.*

Assumptions: Monotone system.

Proof. A parallel plan with r rounds acquires everything in C_r of the closure iteration by induction on r: round j can fire only actions whose requirements lie in the capabilities acquired by round $j-1$, which are contained in C_{j-1}. Conversely the iteration is itself a parallel plan. So the shortest parallel plan for c has $d(c)$ rounds. A sequential plan of ℓ actions is a parallel plan of ℓ rounds with one action per round, so $\ell \ge d(c)$ by the first part. The upper bound is Result 9.3. For attainment of the first bound take a chain, where parallelism does not help and $d = \ell$; for the second take n capabilities gained independently from the starting configuration and one further capability requiring all of them, where $d = 2$ for the last capability and the shortest sequential plan has $n + 1 = |\mathrm{Cap} \setminus C|$ actions. □

9.5 Capabilities are complements

How much is a capability worth? A natural measure is the increase in the size of the closure. In the workshop, starting from the empty configuration, *tools* alone gives a closure of size one, *wood* alone gives a closure of size one, and the pair gives size four (tools, wood, saw, stock, since the saw needs both and the stock needs the saw and the wood). So the marginal value of *wood* given *tools* is three, against one given nothing. The closure size is not submodular: capabilities that are useful only in combination are complements, and the value of the second exceeds the value of the first.

New result 9.9 (The closure size is neither submodular nor supermodular)**.** *The set function $f(D) = |\mathrm{Cl}(D)|$ is monotone, and in general it is neither submodular nor supermodular. In the workshop $f(\{\text{tools}, \text{wood}\}) - f(\{\text{tools}\}) = 3 > 1 = f(\{\text{wood}\}) - f(\varnothing)$, so the marginal value of a capability can rise as others are added, and f is not submodular. With capabilities c_1, c_2, g and actions $a_1 : c_1 \Rightarrow g$, $a_2 : c_2 \Rightarrow g$ the marginal value can fall: $f(\{c_2\}) - f(\varnothing) = 2 > 1 = f(\{c_1, c_2\}) - f(\{c_1\})$, so f is not supermodular.*

Assumptions: Monotone systems.

Proof. Monotonicity of f is Result 9.2(b). In the workshop, $f(\varnothing) = 0$, $f(\{\text{tools}\}) = 1$, $f(\{\text{wood}\}) = 1$ (no action fires from either alone) and $f(\{\text{tools}, \text{wood}\}) = 4$ (adding the saw, then the stock). For the second system, $f(\varnothing) = 0$, $f(\{c_1\}) = f(\{c_2\}) = 2$ (the capability and g), $f(\{c_1, c_2\}) = 3$. □

The consequence for planning is that greedy acquisition by marginal closure gain is not safe. Complementary capabilities must be acquired in groups, the substitutes in the other direction should be acquired only once, and a good plan reasons about the hypergraph of requirements, not about capabilities one at a time.

Exercises

Exercise 9.10. Compute $\mathrm{Cl}(\varnothing)$ and the distances of all actions in a system with three capabilities and four actions of your choice, in which one action is never enabled.

Exercise 9.11. Show that adding a loss to a single action of a monotone system can make the family of reachable configurations have two incomparable maximal elements.

Exercise 9.12. Show that in a monotone system there is a shortest plan to any acquirable capability in which no action is repeated, and explain why repetition is useless in the monotone case but not in Example 9.7 if the skill is modelled by a count instead of by capabilities $t_1, \ldots, t_k$.

Exercise 9.13. In Example 9.5, suppose each step has a waiting time and the steps can be started in parallel when their requirements are met. Compute the time to file as a longest path in a directed acyclic graph.

Exercise 9.14. State and prove the analogue of Result 9.2(d) when each action also has a cost and one wishes to know the least total cost at which a capability is acquirable, for the case in which the requirement hypergraph is a tree. Explain where the proof breaks if two actions share a prerequisite.

Exercise 9.15. Verify the closure rounds, the three menus and the depths of the workshop of Section 9.4. Then add an action *buy fuel* with requirement money and gain fuel, and recompute the eventual menu and the depth of kiln.

Exercise 9.16. Prove that in a monotone system the depth function d satisfies $d(c) = 1 + \min_{a:\, c \in \mathrm{gain}(a)} \max_{c' \in \mathrm{req}(a)} d(c')$ for $c \notin C$, with $d = 0$ on C and the minimum over the empty set equal to ∞.

Exercise 9.17. Give a capability system in which the shortest sequential plan to a capability is n but the shortest parallel plan is 2, and one in which the two coincide for every capability.

Exercise 9.18. Show that every capability system, monotone or not, has a configuration graph on $2^{|\mathrm{Cap}|}$ vertices, and explain why for monotone systems the strongly connected components of this graph are singletons.

Exercise 9.19. In the workshop, suppose *build shed* also loses *wood*. Describe the reachable configurations from $\{\text{money}\}$. Is *stock* still acquirable, and does the order of actions now matter?

Chapter 10

Establishing a Domain of Choice

Question. How are choices evaluated when the domain from which one chooses must first be established?

Established sources. Sequential search with costly inspection (Weitzman's Pandora's box); optimal stopping.

New results. A model of acquisition stages with a non-anticipation condition (Definition 10.1); existence and computation of optimal non-anticipating and budget-admissible acquisition policies by backward induction on a finite sufficient state (Results 10.2 and 10.7); the value of an oracle that need not search bounds every policy from above and is strictly above the optimum in the stated example (Result 10.5); the value of an established domain is submodular in the domain (Result 10.6).

Proof obligations. None.

Depends on. Chapter 9 and Chapter 7.

Textbook decision theory evaluates a choice from a menu that is given. In most of the systems of interest the menu is not given but made: options are found, inspected, qualified, or negotiated before they can be chosen, and each of these steps has a cost and an outcome that was not known before it was taken. Establishing the domain is then part of the decision, and a rule that evaluates options as though the whole domain were already established evaluates a different problem. This chapter gives a formal model of acquisition stages in which a choice is made from the options established so far, states the condition under which a policy does not use information it has not yet paid for, and proves what can be said about the existence, computation, and value of such policies.

10.1 A model of acquisition stages

There is a finite set $I = \{1, \dots, n\}$ of candidate options. Option i has a value $X_i \geq 0$ that is unknown until option i is *inspected*, and an inspection cost $c_i > 0$. The values $X_1, \dots, X_n$ are random variables with finite support, and in the first results independent. Inspecting an option is the acquisition of the capability to choose it, in the sense of Chapter 9, with the difference that what is acquired has a value that is revealed only on acquisition. The *established domain* after some inspections is the set $S \subseteq I$ of options inspected so far.

Definition 10.1 (Acquisition policy)**.** An *acquisition policy* π specifies, for each finite sequence of inspected options together with their revealed values (an *observed history*), either to stop,

or to inspect an option not yet inspected. When it stops it selects an option of the established domain with the greatest observed value, or selects nothing if the domain is empty. The payoff is $\max_{i\in S} X_i - \sum_{i\in S} c_i$, where S is the final domain and the maximum over the empty set is 0. The policy is *non-anticipating* if its decision at each stage depends only on the observed history, so that it is a function on the finite set of observed histories and uses no value that has not been revealed.

Definition 10.1 is a restriction on policies, not on the world: a policy that inspects option i only when X_i is large uses X_i before paying for it. Such a policy is feasible as a mathematical function on the underlying probability space and is not available to any agent. Some of the apparent power of decision rules in the literature comes from implicitly using anticipating policies, by evaluating options at their eventual values and charging only for those chosen.

10.2 Existence and computation

For a set $U \subseteq I$ of uninspected options and a current best value $m \geq 0$ (equal to 0 if no option has been inspected) define

$$V(U,m) = \max\Big\{m,\ \max_{i\in U}\big[-c_i + \mathbb{E}\,V\big(U\setminus\{i\}, \max(m, X_i)\big)\big]\Big\}, \qquad V(\varnothing, m) = m,$$

where the cost already paid is accounted for outside V.

New result 10.2 (Backward induction for acquisition)**.** *If $X_1, \ldots, X_n$ are independent with finite supports then the pair (U, m) is a sufficient state for non-anticipating policies: the optimal continuation from any observed history depends on the history only through the set U of uninspected options and the current best value m. The function V is the optimal value of continuation, and the optimal value of the problem is $V^* = V(I, 0)$. An optimal non-anticipating policy exists and is the policy that stops at (U, m) when the first term attains the maximum and otherwise inspects an option attaining the second.*

Assumptions: Independent X_i with finite supports; $c_i > 0$.

Proof. Induct on $|U|$. For $U = \varnothing$ the only decision is to stop and the value is m. For $|U| = k+1$, an observed history ending in state (U, m) determines the conditional law of the uninspected values, which by independence is their unconditional law, whatever the history. The policy must stop, with value m, or inspect some $i \in U$, paying c_i and observing X_i, after which the state is $(U \setminus \{i\}, \max(m, X_i))$ and by the induction hypothesis the best continuation has value V at that state. The best value is the larger of these alternatives, which is the recursion. Finiteness of the supports makes the expectations finite sums and the set of non-anticipating policies finite, so a maximizer exists, and the policy described attains V at every state by construction. □

Without independence the posterior law of the uninspected values depends on the inspected values, and the sufficient state is the posterior itself, a point in a continuum, as in Chapter 5.

Established result 10.3 (Weitzman's index policy)**.** *Under the assumptions of Result 10.2, let z_i be the* reservation value *of option i, the solution of $\mathbb{E}\,(X_i - z_i)^+ = c_i$. An optimal policy inspects the uninspected options in decreasing order of z_i and stops as soon as the current best value m is at least the largest reservation value among the options not yet inspected.*

Source: Weitzman (1979), *Optimal search for the best alternative.*

The index policy shows that the dynamic program has a simple solution: each option can be assessed separately by its reservation value, without regard to the others, and the complexity of the problem is the complexity of sorting.

Example 10.4 (Two options)**.** Let $n = 2$, each X_i equal to 0 or 10 with probability $\frac{1}{2}$, and $c_1 = c_2 = 1$. The reservation value solves $\frac{1}{2}(10 - z) = 1$, so $z = 8$ for both. By Result 10.3 the optimal policy inspects option 1, stops if $X_1 = 10$, and otherwise inspects option 2. The recursion gives $V(\{2\}, 10) = \max\{10, -1 + 10\} = 10$ and $V(\{2\}, 0) = \max\{0, -1 + 5\} = 4$, so $V^* = -1 + \frac{1}{2} \cdot 10 + \frac{1}{2} \cdot 4 = 6$.

10.3 What an oracle is worth

An oracle that knows all values in advance can choose the option i maximizing $X_i - c_i$ and pay only c_i, or abstain.

New result 10.5 (Oracle bound)**.** *For every acquisition policy, anticipating or not,*

$$\mathbb{E}[\textit{payoff}] \;\le\; V^{\mathrm{orc}} := \mathbb{E}\Big[\max\big(0, \max_i (X_i - c_i)\big)\Big].$$

In Example 10.4 $V^{\mathrm{orc}} = 6.75$ *and* $V^* = 6$*, so the optimal non-anticipating policy falls short of the oracle by* 0.75*, a loss attributable to inspection costs that the oracle avoids and that are decomposed in the proof.*

Assumptions: None for the bound; Example 10.4 for the numbers.

Proof. The payoff of a policy ending with domain S and selected option $j \in S$ is $X_j - \sum_{i \in S} c_i \le X_j - c_j \le \max_i (X_i - c_i)$ because all costs are positive, and if $S = \varnothing$ the payoff is 0. In both cases the payoff is at most $\max(0, \max_i (X_i - c_i))$ pointwise, and taking expectations gives the bound. In the example the oracle obtains 9 whenever at least one value equals 10, which has probability $\frac{3}{4}$, and 0 otherwise, so $V^{\mathrm{orc}} = 6.75$. The optimal policy of Example 10.4 attains 9 with probability $\frac{1}{2}$ (when $X_1 = 10$), pays both costs in the remaining half, obtaining 8 if $X_2 = 10$ and -2 otherwise, so its value is $\frac{1}{2} \cdot 9 + \frac{1}{4} \cdot 8 + \frac{1}{4} \cdot (-2) = 6$. The difference 0.75 is the sum of two losses: in the event $X_1 = 0, X_2 = 10$ (probability $\frac{1}{4}$) the policy pays an extra 1 relative to the oracle, and in the event $X_1 = X_2 = 0$ (probability $\frac{1}{4}$) it loses 2 where the oracle abstains and pays nothing, and $\frac{1}{4} \cdot 1 + \frac{1}{4} \cdot 2 = 0.75$. □

The gap is the price of having to establish the domain by paying for it in ignorance of what it contains. It is a property of the information structure and not of the agent: the bound holds for every policy, and the gap measures how much of the bound a policy is able to reach without using unrevealed values.

10.4 Diminishing returns of an established domain

Let $v(S) = \mathbb{E}\big[\max_{i \in S} X_i\big]$ for $S \subseteq I$, with $v(\varnothing) = 0$, be the value of choosing from the established domain S when the choice is made at the end and costs are ignored.

New result 10.6 (Submodularity of domain value)**.** *For nonnegative random variables X_i, not necessarily independent, v is monotone and submodular: for $S \subseteq T \subseteq I$ and $i \notin T$,*

$$0 \le v(T \cup \{i\}) - v(T) \le v(S \cup \{i\}) - v(S) \le v(\{i\}) = \mathbb{E}X_i.$$

Assumptions: $X_i \ge 0$ with finite expectation.

Proof. Write $M_S = \max_{j \in S} X_j$, with $M_\varnothing = 0$. Then $\max(M_S, X_i) - M_S = (X_i - M_S)^+$, so $v(S \cup \{i\}) - v(S) = \mathbb{E}(X_i - M_S)^+$. Since $S \subseteq T$ gives $M_S \le M_T$, we have $(X_i - M_T)^+ \le (X_i - M_S)^+ \le X_i$ pointwise, which gives the three inequalities after taking expectations, and the first is nonnegativity of $(\cdot)^+$. □

The consequence is that the marginal value of establishing a further option is at most its stand-alone value and shrinks as the domain grows. An option that would have been the best of a small domain may add almost nothing to a large one. Exercise 10.10 asks for the corresponding statement for the net value with costs.

10.5 Admissible acquisition under a budget

Suppose total inspection cost may not exceed a budget B on any path. The policy is then *admissible* if every observed history has total cost at most B. The empty policy, which inspects nothing, is admissible.

New result 10.7 (Budget-admissible acquisition)**.** *Under the assumptions of Result 10.2 and a budget B, the triple (U, m, r), where r is the remaining budget, is a sufficient state, and the value*

$$V_B(U, m, r) = \max\Big\{m,\ \max_{i \in U:\, c_i \le r} \big[-c_i + \mathbb{E}\, V_B\big(U \setminus \{i\}, \max(m, X_i), r - c_i\big)\big]\Big\}$$

is the optimal value of an admissible non-anticipating policy. An optimal admissible policy exists, and V_B is nondecreasing in r with $V_B(I, 0, B) \le V^*$.

Assumptions: Independent X_i with finite supports; $c_i > 0$; $B \ge 0$.

Proof. The proof of Result 10.2 applies with the action set at (U, m, r) restricted to $\{i \in U : c_i \le r\}$ and the remaining budget updated by $r \mapsto r - c_i$; this is the product construction of Result 7.4. Monotonicity in r: the action set at (U, m, r') contains that at (U, m, r) for $r' \ge r$, and the successor states have budgets that are larger, so by induction on $|U|$ the right-hand side does not decrease. The unconstrained value is the value with $r \ge \sum_i c_i$, which is at least $V_B(I, 0, B)$ by the same monotonicity. □

The budget enters as a state variable and so is an augmentation in the sense of Chapter 7. Without it the state (U, m) would be insufficient: two histories with the same inspected set and the same best value but different amounts spent would have different feasible continuations.

10.6 A three-option instance

Three options differ in kind. Option A is a sure thing: $X_A = 6$ at inspection cost $c_A = 0.5$. Option B is a long shot: $X_B = 20$ with probability 0.2 and 0 otherwise, at cost $c_B = 1$. Option C is as in Example 10.4: $X_C \in \{0, 10\}$ with probability one half each, cost $c_C = 1$. The reservation values are found by solving $\mathbb{E}(X - z)^+ = c$: for A, $6 - z = 0.5$ gives $z_A = 5.5$; for B, $0.2(20 - z) = 1$ gives $z_B = 15$; for C, $\frac{1}{2}(10 - z) = 1$ gives $z_C = 8$. The order of the means is A, C, B $(6, 5, 4)$, while the order of the indices is B, C, A. Weitzman's rule therefore inspects the long shot first, although it has the smallest mean, and stops as soon as the best value found is at least the next index. Direct dynamic programming over the states (U, m) gives the optimal value $V^* = 8.4$, and the value of the index policy, computed by following its order and stopping rule, is also 8.4. The oracle bound is $V^{\mathrm{orc}} = 9.6$, so the cost of establishing the domain in ignorance is 1.2. For three identical options of the type of C the values are $V^* = 7.0$ and $V^{\mathrm{orc}} = 7.875$.

The numbers carry the point of the chapter. A rule that ranks options by their mean values, or by their values net of inspection cost, would inspect A first and, stopping at the first value of at least 6, would end with expected payoff $6 - 0.5 = 5.5$ against 8.4 for the index policy, because it undervalues the information that a high-variance option can supply.

New result 10.8 (Spread raises the reservation value)**.** *Let Y be a mean-preserving spread of X, in the sense that $\mathbb{E}\,\phi(X) \le \mathbb{E}\,\phi(Y)$ for every convex ϕ (equivalently, $X \le_{\mathrm{cx}} Y$). Then for every inspection cost $c > 0$ the reservation values satisfy $z_X \le z_Y$.*

Assumptions: Integrable random variables with $\mathbb{E}(X - z)^+ \to 0$ as $z \to \infty$ and continuous in z.

Proof. The function $\phi_z(x) = (x - z)^+$ is convex, so $g_X(z) := \mathbb{E}(X - z)^+ \le \mathbb{E}(Y - z)^+ = g_Y(z)$ for every z. Both g_X and g_Y are continuous, nonincreasing, and tend to 0 at infinity, and $g(z) = c$ has a unique solution wherever g is strictly decreasing at the level c; otherwise z_X and z_Y denote the largest solutions. At $z = z_X$ we have $g_Y(z_X) \ge g_X(z_X) = c$. If $z_Y < z_X$, then $g_Y(z_X) < c$ because z_Y is the largest solution and g_Y is nonincreasing, a contradiction. So $z_Y \ge z_X$. □

The result says that, in a search with recall, risk is attractive: of two options with the same mean and cost, the riskier is inspected first, because the downside is truncated at the value of the best option already in hand while the upside is not. It also explains why institutions that search for candidates, ideas, or sites value diversity of the pool: a more dispersed option has a higher index.

10.7 The anatomy of the gap to the oracle

The gap $V^{\mathrm{orc}} - V^*$ has two sources, which the proof of the oracle bound separates. The first is the cost of inspections that turn out not to matter: the oracle pays only for the option it selects, whereas any non-anticipating policy pays for each option it inspects before knowing whether it is the best. The second is the cost of stopping or continuing without knowing the unseen values. For n identical options of type C the oracle value is $9\big(1 - 2^{-n}\big)$, since it collects $10 - 1 = 9$ unless every value is zero, while the optimal policy inspects sequentially and stops at the first 10, which gives $V_k = 4 + \frac{1}{2}V_{k-1}$ with $V_0 = 0$ and hence $V_n = 8\big(1 - 2^{-n}\big)$. The gap

is therefore $1-2^{-n}$: it equals 0.75 for $n=2$ and 0.875 for $n=3$, and tends to 1. The number has a direct reading. The optimal policy makes $2-2^{1-n}$ inspections in expectation, of which exactly one finds the 10 with probability $1-2^{-n}$, so the expected number of inspections that did not matter is $1-2^{-n}$ and each costs 1. No policy can close the gap, because no policy can know in advance which inspection will find the 10.

Exercises

Exercise 10.9. Compute V^* and V^{orc} for four options with the same distribution as in Example 10.4 and costs $c_i = 1$, and confirm that the gap is $1-2^{-4}$.

Exercise 10.10. Show that $w(S) = v(S) - \sum_{i \in S} c_i$ is submodular and not necessarily monotone, and give a distribution for which the maximizer of w over subsets of I is a proper nonempty subset.

Exercise 10.11. Give an example in which the optimal policy of Result 10.2 inspects options in an order that is not decreasing in $\mathbb{E}X_i$, and confirm that Weitzman's reservation values explain the order.

Exercise 10.12. Show that if the values are perfectly correlated, $X_i = X$ for all i, then inspecting more than one option is never optimal, and compute the sufficient state.

Exercise 10.13. Construct an anticipating policy that attains V^{orc} in Example 10.4 and verify that it is not a function of the observed history.

Exercise 10.14. Verify by hand the reservation values $z_A = 5.5$, $z_B = 15$, $z_C = 8$ in Section 10.6, and compute the expected payoff of the index policy that inspects in the order B, C, A and stops as soon as the best value found is at least the next index. Show that it equals 8.4, and that the order A, C, B with the same stopping rule gives 5.5.

Exercise 10.15. Prove that if X is deterministic with value v and cost $c < v$ then $z = v - c$ and the option is inspected first iff z exceeds the indices of all others. Explain why a deterministic option need never be inspected if its value is known.

Exercise 10.16. Using Result 10.8, order the options $X_1 \in \{4, 6\}$, $X_2 \in \{0, 10\}$, $X_3 = 5$ (each equiprobable, cost 0.5) by reservation value, and compute the three values.

Exercise 10.17. Derive the gap $V^{\mathrm{orc}} - V^*$ for n identical options of type C in closed form by solving the recursion $V_k = -1 + \frac{1}{2} \cdot 10 + \frac{1}{2} V_{k-1}$ with $V_0 = 0$ (the value with k options left and a best value of 0), and check $n = 2, 3$.

Part IV

Measures of possibility

Chapter 11

Counting Options, Reachability, and Comparing Futures

Question. Why does a larger nominal option set not imply greater practical freedom, and how are futures compared without forcing a scalar ranking?

Established sources. Simulation preorders (Milner); Perron–Frobenius theory; set inclusion and dominance.

New results. Cardinality, volume, and graph distance of reachable sets each misrank resource-bounded freedom (Result 11.1); the budgeted dominance order is a preorder with incomparable pairs, and weighted scalar scores can reverse any incomparable pair (Result 11.2); simulation preserves coverage of outcomes (Result 11.4); the growth rate of feasible histories is the largest spectral radius among reachable components (Result 11.6).

Proof obligations. None.

Depends on. Chapters 2 and 6.

A designer asked whether one situation leaves more freedom than another is tempted to count: the actions on the menu, the states that can be reached, the volume of the reachable region, the number of steps to a goal. Each count answers a real question and none answers the question asked. This chapter separates the counts by explicit constructions, replaces the single number by an order that admits incomparable situations, relates the order to simulation, and isolates the one count, the growth rate of feasible histories, that has a clean structural characterization.

11.1 Four counts that do not agree

Let the transition graph of Chapter 2 carry nonnegative edge costs and let ρ be the return cost of Chapter 8. For a budget $\beta \geq 0$ the *budgeted reach* of x is $R_\beta(x) = \{y : \rho(x, y) \leq \beta\}$, and $R(x) = \bigcup_\beta R_\beta(x)$ is the reach of Chapter 2. For continuous systems $R^{(k)}(x)$ denotes the set reachable in exactly k steps.

New result 11.1 (The counts disagree with budgeted freedom)**.** *(a)* Cardinality. *For every n there are states x, y with $|R(x)| > n\,|R(y)|$ and a budget β with $|R_\beta(x)| < |R_\beta(y)|$.*

(b) Volume. *There are systems A, B on $\mathbb{R}^2$ with the same initial state such that the reachable set of A in k steps has positive area, that of B has area zero, and B reaches points at a distance from the origin that A cannot reach in k steps.*

(c) Graph distance. *There are states x, y and a target g with the same graph distance from both and a budget β such that $g \in R_\beta(y)$ and $g \notin R_\beta(x)$.*

Assumptions: Constructions as stated.

Proof. (a) Let x have edges of cost 10 to $3n$ distinct new states, and y have one edge of cost 1 to a state y_1 which has one edge of cost 1 to y_2. Then $|R(x)| = 3n + 1 > 3n = n|R(y)|$ since $|R(y)| = 3$, while $|R_5(x)| = 1 < 3 = |R_5(y)|$.

(b) Let A be $z^+ = z + u$ with $u \in [-1, 1]^2$ and let B be $z^+ = z + u\, e_1$ with $u \in [-M, M]$ and $e_1 = (1, 0)$, both from $z_0 = 0$. After k steps the reachable set of A is $[-k, k]^2$ of area $4k^2$, and that of B is the segment $[-Mk, Mk] \times \{0\}$, of area zero. For $M > 2$ the point $(Mk, 0)$ lies outside $[-k, k]^2$ and B reaches it, while A reaches (k, k) and B does not.

(c) Let $y \to m \to g$ with costs $1, 1$ and $x \to m' \to g$ with costs $1, 100$. Both have graph distance 2 to g, and for $\beta = 10$ we have $g \in R_{10}(y)$ and $g \notin R_{10}(x)$. □

The point of the three parts is that a count that is fixed by the structure of the graph or the region ignores the resources needed to exercise the options. Which count is appropriate depends on what resource is scarce, and the budget β is the means of stating it.

11.2 An order that admits incomparability

To compare states by what they can bring about, fix a set $O \subseteq X$ of outcomes of interest. For states x, y write $x \preceq y$ if $R_\beta(x) \cap O \subseteq R_\beta(y) \cap O$ for every budget β (the *dominance* order).

New result 11.2 (Dominance is a preorder with incomparable pairs)**.** *The relation $\preceq$ is reflexive and transitive. It need not be total: there are states x, y with neither $x \preceq y$ nor $y \preceq x$. For such a pair, and any nonnegative weights $w(\beta)$ on a finite set of budgets, the score $s_w(x) = \sum_\beta w(\beta)\, |R_\beta(x) \cap O|$ can rank x above y for some weights and y above x for others.*

Assumptions: None beyond the definitions.

Proof. Reflexivity and transitivity are those of inclusion, budget by budget. For incomparability let $O = \{a, b_1, \dots, b_{10}\}$, let x have one edge of cost 1 to a and let y have edges of cost 5 to each b_j. Then $R_1(x) \cap O = \{a\} \not\subseteq R_1(y) \cap O = \varnothing$ and $R_5(y) \cap O = \{b_1, \dots, b_{10}\} \not\subseteq R_5(x) \cap O = \{a\}$. With budgets $\{1, 5\}$ the scores are $s_w(x) = w(1) + w(5)$ and $s_w(y) = 10\, w(5)$, so $s_w(x) > s_w(y)$ for $w = (1, 0)$ and $s_w(x) < s_w(y)$ for $w = (0, 1)$. □

Every finite preorder can be extended to a total preorder, so a scalar ranking consistent with dominance exists. The result says that the extension is not determined by the system. It depends on a weighting of budgets, that is, on which resource levels the user cares about, and two analysts who use different weights can disagree on incomparable pairs without either making a mistake.

11.3 Simulation and coverage

Let the transition graph have labelled edges $x \xrightarrow{u} x'$. A relation $\mathcal{S} \subseteq X \times X$ is a *simulation* if whenever $x\, \mathcal{S}\, y$ and $x \xrightarrow{u} x'$ there is y' with $y \xrightarrow{u} y'$ and $x'\, \mathcal{S}\, y'$. Write $x \sqsubseteq y$ if some simulation relates them.

Established result 11.3 (Simulation preorder)**.** *The relation $\sqsubseteq$ is the largest simulation, is a preorder, and implies inclusion of the sets of label sequences feasible from the two states.*

Source: Milner, *Communication and Concurrency* (1989).

New result 11.4 (Simulation preserves coverage)**.** *If $x \sqsubseteq y$ then for every $x' \in R(x)$ there is $y' \in R(y)$ with $x' \sqsubseteq y'$. If simulation is moreover required to respect a labelling $\lambda : X \to \Lambda$ of outcomes ($x \mathcal{S} y \Rightarrow \lambda(x) = \lambda(y)$), then every outcome label reachable from x is reachable from y. Inclusion $R(x) \subseteq R(y)$ is neither implied by nor implies $x \sqsubseteq y$.*

Assumptions: Labelled graph.

Proof. Induct on the length of a path from x to x': the step from x is matched by a step from y, the successors stay related, and the induction hypothesis applies. The labelled version follows because related states have equal labels. For the last claim, in a graph with two disjoint components $x \xrightarrow{a} x_1$ and $y \xrightarrow{a} y_1$ with x_1, y_1 having no moves, $x \sqsubseteq y$ but $R(x) \not\subseteq R(y)$; and in a graph with $x \xrightarrow{a} x$ and $y \xrightarrow{b} x$ we have $R(x) = \{x\} \subseteq R(y) = \{y, x\}$ while $x \not\sqsubseteq y$ because y has no a-move. □

Simulation is the right comparison when futures are to be compared as processes: it says that whatever x can do, y can do while remaining ahead in the same sense. Inclusion of reachable sets compares only which states appear, not which sequences of actions get there.

11.4 Growth of feasible histories

Let A be the matrix with A_{xy} the number of actions leading from x to y. The number of feasible action sequences of length k from x is $N_k(x) = e_x^{\mathsf{T}} A^k \mathbf{1}$. Define the *growth rate* $g(x) = \limsup_k N_k(x)^{1/k}$.

Established result 11.5 (Perron–Frobenius)**.** *For an irreducible nonnegative matrix B the spectral radius $\lambda(B)$ is an eigenvalue with a strictly positive eigenvector, and $\lambda(B) \geq 1$ when B is a nonzero integer matrix.*

Source: Horn and Johnson, *Matrix Analysis*, Chapter 8.

New result 11.6 (Growth rate and components)**.** *Let $\mathcal{C}(x)$ be the set of strongly connected components reachable from x, and for a component C let A_C be the restriction of A to C. Then*

$$g(x) = \max_{C \in \mathcal{C}(x)} \lambda(A_C),$$

with the convention that the maximum is 0 when every reachable component is a single state without a self-loop. Consequently $g(x) \leq d_{\max}$, the maximum number of feasible actions at a state, $g(x) \geq 1$ if and only if a cycle is reachable from x, and a state has finitely many feasible histories if and only if $g(x) = 0$.

Assumptions: Finite state and action sets.

Proof. Restrict A to $R(x)$ and order the components topologically; the restriction is block upper triangular with diagonal blocks A_C, so its eigenvalues are those of the blocks, and its spectral radius λ is $\max_C \lambda(A_C)$. By the Jordan form there is a polynomial p with $\|A^k\| \le p(k)\lambda^k$ on $R(x)$, hence $g(x) \le \lambda$. For the reverse inequality let C attain the maximum and let $v > 0$ be a Perron vector of A_C (Result 11.5, when $|C| > 1$ or $A_C \neq 0$). From a state $y \in C$ reachable from x in m steps, walks of length k staying in C number at least $e_y^{\mathsf{T}} A_C^k \mathbf{1} \ge e_y^{\mathsf{T}} A_C^k v / \max v = \lambda^k v_y / \max v$, and $N_{m+k}(x) \ge$ this, so $g(x) \ge \lambda$. Row sums of A are at most $d_{\max}$, so $\lambda \le \|A\|_\infty \le d_{\max}$. A cycle makes some A_C nonzero integer-valued, so $\lambda(A_C) \ge 1$ by Result 11.5; if no cycle is reachable then every reachable A_C is nilpotent, $g(x) = 0$, and the number of feasible histories is finite because $A^k = 0$ on $R(x)$ for $k \ge |R(x)|$. □

The growth rate is a structural count, and it is the one that respects the decomposition of Chapter 2: it is determined by the strongly connected components that can be reached, not by the menu at x. The menu size bounds it from above and is attained only when every reachable state has the full menu and the transitions are as mixing as the menu allows. Exercise 11.12 gives two systems with the same menu size and different growth rates.

11.5 Dominance as a comparison of cost vectors

The dominance order has a concrete form. For an outcome $o \in O$ write $\rho_x(o) = \rho(x, o) \in [0, \infty]$ for the least budget at which o is reachable from x, and collect these into the cost vector $\rho_x = (\rho_x(o))_{o \in O}$.

New result 11.7 (Dominance is the componentwise order on cost vectors)**.** *For states x, y we have $x \preceq y$ if and only if $\rho_y(o) \le \rho_x(o)$ for every $o \in O$. Consequently the maximal elements of $\preceq$ among a finite set of states are the Pareto-minimal cost vectors, and two states are incomparable exactly when each is strictly cheaper than the other for some outcome.*

Assumptions: Finite graph with nonnegative costs.

Proof. Suppose $x \preceq y$ and $o \in O$. If $\rho_x(o) = \infty$ there is nothing to prove. Otherwise $o \in R_\beta(x)$ for $\beta = \rho_x(o)$, hence $o \in R_\beta(y)$, which says $\rho_y(o) \le \rho_x(o)$. Conversely, if the cost inequalities hold and $o \in R_\beta(x) \cap O$, then $\rho_y(o) \le \rho_x(o) \le \beta$, so $o \in R_\beta(y)$. □

A three-state illustration fixes ideas. Let A, B, C be states and a, b be two outcomes, with cost vectors $\rho_A = (1, 9)$, $\rho_B = (9, 1)$ and $\rho_C = (4, 4)$. No state dominates another: A is cheapest for a, B for b, and C is intermediate in both, so all three are Pareto-minimal. A scalar score $w_a \rho(\cdot, a) + w_b \rho(\cdot, b)$, with lower better, orders the states as follows, writing $t = w_b / w_a$. State C is best exactly when $3/5 < t < 5/3$, because C beats A iff $4 + 4t < 1 + 9t$ iff $t > 3/5$ and beats B iff $4 + 4t < 9 + t$ iff $t < 5/3$; for smaller t state A is best and for larger t state B is best. A direct evaluation confirms it: at $t = 1$ the scores are $10, 10, 8$, at $t = 0.5$ they are $5.5, 9.5, 6$, and at $t = 3$ they are $28, 12, 16$. The middle state, which would be chosen by almost no one who cares about one outcome only, is the unique optimum for every analyst whose weights are within a factor of $5/3$ of equal. This is the usual pattern of a Pareto frontier: the structure supplies the frontier, and the weights select a point on it.

New result 11.8 (Simulation is monotone for label growth)**.** *Let $L_k(x)$ be the number of distinct label sequences of length k feasible from x and $g_L(x) = \limsup_k L_k(x)^{1/k}$. If $x \sqsubseteq y$*

then $L_k(x) \le L_k(y)$ for all k, hence $g_L(x) \le g_L(y)$. For deterministic graphs $L_k = N_k$ and $g_L = g$.

Assumptions: Finite labelled graph.

Proof. By Result 11.3 every label sequence feasible from x is feasible from y, so the set counted by $L_k(x)$ is contained in that counted by $L_k(y)$. Taking kth roots and the limit superior preserves the inequality. In a deterministic graph a label sequence determines the path, so $L_k = N_k$. □

The result has a design reading. If a redesigned system simulates the original (every action sequence the old system allowed remains available), the growth of its futures cannot fall, and a lower growth rate certifies that simulation fails. It does not give a converse: two systems can have the same growth rate without either simulating the other.

11.6 A rate limit as a growth rate

The growth rate quantifies what the rate limits of Chapter 7 cost in options per step. Take two actions, a (always available) and b (available only when not used at the previous step). The state is whether b was just used: from *free*, a leads to *free* and b to *cool*; from *cool*, a leads to *free*. The matrix is $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$, and direct multiplication gives $N_k(\text{free}) = 2, 3, 5, 8, 13, 21, 34, 55, 89, 144$ for $k = 1, \dots, 10$ (the Fibonacci numbers) and $N_k(\text{cool}) = 1, 2, 3, 5, 8, 13, 21, 34, 55, 89$. The spectral radius is $\varphi = (1+\sqrt{5})/2 \approx 1.618$, against $d_{\max} = 2$ for an unrestricted pair of actions, so the restriction costs $\log_2 2 - \log_2 \varphi \approx 0.306$ bits of freedom per step. If a cooling period of two steps is required after each use of b, the characteristic equation becomes $\lambda^3 = \lambda^2 + 1$ and the growth rate falls to 1.4656. The numbers are small but the structural point is general: a local restriction on the menu, repeated at every step, compounds into a multiplicative reduction of the number of futures, and the reduction is read from the spectrum of the restricted transition matrix.

Exercises

Exercise 11.9. Construct, for $n = 2$, the graphs of Result 11.1(a) with explicit numbers, and check each inequality.

Exercise 11.10. Give two distinct states that dominate each other, $x \preceq y$ and $y \preceq x$, and conclude that dominance is a preorder and not a partial order on states.

Exercise 11.11. Prove that $\sqsubseteq$ is a preorder by showing that the identity relation is a simulation and that the composition of two simulations is a simulation.

Exercise 11.12. Compute the growth rate of the graph on two states with two parallel actions a, b from each state to itself and compare it with the graph on two states where each action goes to the other state and the other state has a single self-loop. Both have maximum menu 2.

Exercise 11.13. Show that for a deterministic labelled graph the number of distinct reachable states within k steps is at most $N_k(x)$ summed over lengths, and give a graph in which the ratio of the two grows without bound.

Exercise 11.14. Verify the cost-vector comparison of Section 11.5: compute the scores of A, B, C at $t = 0.6$ and $t = 5/3$, and show that at the two endpoints C ties with A and with B respectively.

Exercise 11.15. Prove that if ρ_x and ρ_y are cost vectors and $x \preceq y$, then $\max_o \rho_y(o) \leq \max_o \rho_x(o)$ and $\sum_o \rho_y(o) \leq \sum_o \rho_x(o)$, so that any monotone aggregation of costs respects dominance. Explain why this does not determine a total order.

Exercise 11.16. Derive the Fibonacci recurrence $N_{k+2}(\text{free}) = N_{k+1}(\text{free}) + N_k(\text{free})$ from the transition graph of Section 11.6, and show that $N_k(\text{free}) = F_{k+2}$.

Exercise 11.17. Show that the three-state cooling construction in which b requires two idle steps has growth rate the real root of $\lambda^3 = \lambda^2 + 1$, and bound it between 1.46 and 1.47 by evaluating the polynomial.

Exercise 11.18. Give two deterministic systems with equal growth rates such that neither simulates the other, so that equality of g does not imply mutual simulation.

Chapter 12

Costs of Access and Continuation

Question. How do energy, time, coordination, information, and permission enter as distinct costs?

Established sources. Algebraic path problems (Lehmann); multi-objective shortest paths; weight-constrained paths (Garey and Johnson); list scheduling (Graham).

New results. Different costs combine along a path by different operations, and the same graph yields different optimal paths under additive and bottleneck costs (Result 12.2); componentwise optima are not jointly achievable (Result 12.3); scalar weights decide feasibility only for mixtures of paths (Result 12.4); the Pareto front can be exponentially large (Result 12.5); integer vector budgets are decided exactly by a product graph (Result 12.6); costs of shared acquisitions are subadditive (Result 12.7) and time and energy of a task graph obey different bounds (Result 12.8).

Proof obligations. None.

Depends on. Chapters 6 and 11.

Chapter 11 introduced a budget β without saying what it is a budget of. In practice a transition costs several things at once: energy that is spent and may or may not be replenished, time that passes, effort of coordination with others who must also act, information that must be acquired before the transition is possible, and permission that must be obtained. These are not one currency under different names. They combine along a path in different ways, they are consumed from different stocks, and there is no exchange rate fixed by the system that would let one be traded for another. This chapter makes the distinctions precise and records what survives when several costs are carried at once.

12.1 Cost algebras

A *cost algebra* is a semiring $(K, \oplus, \otimes, \mathbf{0}, \mathbf{1})$ in which the cost of a path is the product $\otimes$ of its edge costs and the cost of reaching y from x is the sum $\oplus$ over paths. Three examples cover the main cases.

- *Additive*: $(\mathbb{R}_{\geq 0} \cup \{\infty\}, \min, +, \infty, 0)$. Energy spent and time elapsed along a single sequence of actions add.
- *Bottleneck*: $(\mathbb{R}_{\geq 0} \cup \{\infty\}, \min, \max, \infty, 0)$. The requirement that is the most demanding along a path governs: the peak power drawn, the highest clearance level required, the slowest of several parties that must agree.

- *Boolean*: $(\{0,1\}, \vee, \wedge, 0, 1)$. Permission is available or it is not.

Established result 12.1 (Algebraic path problems)**.** *If the semiring is closed (infinite sums of the relevant kind exist and distribute), the path costs between all pairs of an n-vertex graph are computed in $O(n^3)$ semiring operations by the Floyd–Warshall–Kleene recursion. Dijkstra's algorithm computes single-source costs when the semiring is totally ordered and $\otimes$ is monotone.*

Source: Lehmann (1977); Tarjan (1981).

New result 12.2 (The algebra changes the optimal path)**.** *There is a graph with two paths from s to t for which the path optimal in the additive algebra is not optimal in the bottleneck algebra and conversely. Under the Boolean algebra the two paths are equally good.*

Assumptions: Edge costs as stated.

Proof. Path P_1 is a single edge of cost 5 and path P_2 consists of three edges of cost 2. Under $(\min, +)$ the costs are 5 and 6, so P_1 is optimal. Under $(\min, \max)$ they are 5 and 2, so P_2 is optimal. Under the Boolean algebra every edge is permitted and both paths have value 1. □

Every graph question about cost must therefore state, along with the numbers, how the numbers combine. A requirement stated as a peak, such as a maximum current, is not satisfied by minimizing total energy, and a requirement stated as a total is not satisfied by minimizing a peak.

12.2 Several costs at once

Let each edge carry a cost vector $c(e) \in \mathbb{R}^m_{\geq 0}$, with every component additive. A path has cost $c(\pi) = \sum_{e \in \pi} c(e)$, and a vector budget $\beta \in \mathbb{R}^m_{\geq 0}$ is met by a path if $c(\pi) \leq \beta$ componentwise. The set of vector-budgeted reachable states is $R_\beta(x) = \{y : \exists \pi : x \to y,\ c(\pi) \leq \beta\}$. For states x, y let $P(x, y) \subset \mathbb{R}^m_{\geq 0}$ be the set of path costs.

New result 12.3 (Componentwise optima are not jointly achievable)**.** *Let $\underline{c}_j(x, y) = \min_{p \in P(x,y)} p_j$ and $\underline{c} = (\underline{c}_1, \ldots, \underline{c}_m)$. Then $\underline{c} \leq p$ for every $p \in P(x, y)$, so $\underline{c} \leq \beta$ is necessary for $y \in R_\beta(x)$. It is not sufficient: there is a graph with $P(x, y) = \{(1, 5), (5, 1)\}$, for which $\underline{c} = (1, 1) \leq (2, 2) = \beta$ but $y \notin R_\beta(x)$.*

Assumptions: None.

Proof. The inequality is the definition of the componentwise minimum. In the example two parallel edges from x to y have costs $(1, 5)$ and $(5, 1)$, and neither is below $(2, 2)$. □

A tempting remedy is to choose weights $w \geq 0$ and charge $w \cdot c(e)$ to a single scalar budget. The next result says exactly what that remedy decides.

New result 12.4 (Scalar weights decide the feasibility of mixtures)**.** *Let $P \subset \mathbb{R}^m_{\geq 0}$ be a finite set of path costs and $\beta \in \mathbb{R}^m_{\geq 0}$. Then*

$$\min_{p \in P} w \cdot p \leq w \cdot \beta \quad \text{for all } w \geq 0 \iff \beta \in \operatorname{conv}(P) + \mathbb{R}^m_{\geq 0}.$$

That is, scalar budgets with all weights decide whether some convex combination of path costs meets the vector budget, not whether a single path does. For $P = \{(1, 5), (5, 1)\}$ the budget

$(3,3)$ *is the midpoint of the two costs, so every weight vector accepts it, although neither path meets it, whereas the budget* $(2,2)$ *is rejected by the weight vector* $w=(1,1)$.

Assumptions: Finite P.

Proof. ($\Leftarrow$) If $\beta=\sum_i \lambda_i p_i + z$ with λ a probability vector and $z \geq 0$ then $w\cdot\beta \geq \sum_i \lambda_i w\cdot p_i \geq \min_p w\cdot p$. ($\Rightarrow$) Suppose $\beta \notin K := \operatorname{conv}(P)+\mathbb{R}^m_{\geq 0}$. The set K is convex and closed, being the sum of a compact set and a closed cone, so there is w with $w\cdot\beta < \inf_{q\in K} w\cdot q$. If some $w_j<0$ the infimum is $-\infty$ along $q+te_j$, which is impossible, so $w\geq 0$. For $w\geq 0$ the infimum over K is attained on P, so $\min_p w\cdot p > w\cdot\beta$, contradicting the hypothesis. □

Scalarization is thus exactly right when the mission is repeated and costs average, as in the constrained Markov decision processes of Chapter 4, where randomization over policies realizes the convex combination. It overstates feasibility when the mission is performed once.

New result 12.5 (The Pareto front can be exponentially large)**.** *For every* n *there is a graph with* $n+1$ *vertices and* $2n$ *edges and cost vectors in* $\mathbb{N}^2$ *for which* $P(s,t)$ *has* 2^n *pairwise incomparable elements.*

Assumptions: Two cost components.

Proof. Take vertices $v_0,\dots,v_n$ with two parallel edges from v_i to v_{i+1} of costs $(2^i,0)$ and $(0,2^i)$, and $s=v_0$, $t=v_n$. A path chooses for each i one of the two edges; if S is the set of indices where the first edge is chosen, the path cost is $(a, 2^n-1-a)$ with $a=\sum_{i\in S}2^i$, and a ranges over $\{0,\dots,2^n-1\}$ with each value realized exactly once. All 2^n vectors have the same component sum, so if $p\leq q$ componentwise then $p=q$, and the vectors are pairwise incomparable. □

The front cannot in general be carried as a state, so a controller that needs to know what it can still afford must carry the remaining budget, as in Chapter 7.

New result 12.6 (Exact decision for integer vector budgets)**.** *Let costs and the budget be integer vectors. Define the product graph on* $X\times\prod_{j=1}^m\{0,\dots,\beta_j\}$ *with an edge* $(x,r)\to(y,r-c(e))$ *for each edge* $e: x\to y$ *with* $c(e)\leq r$. *Then* $y\in R_\beta(x)$ *if and only if some state* (y,r) *is reachable from* (x,β) *in the product, and the product has at most* $|X|\prod_j(\beta_j+1)$ *vertices.*

Assumptions: Integer costs and budgets.

Proof. A path in the product from (x,β) projects to a path in X whose cost is $\beta-r\leq\beta$, where $r\geq 0$ is the final remaining budget. Conversely a path in X of cost $p\leq\beta$ lifts, step by step, since the remaining budget after each prefix is β minus the prefix cost, which is nonnegative. □

The size of the product is polynomial in the numerical values of the budgets and exponential in their length when they are written in binary. This is not an artifact of the construction: deciding whether a path meets two budgets simultaneously is NP-complete, the weight-constrained shortest path problem (Garey and Johnson, ND30), so a vector budget is a source of intractability that a single additive budget is not.

12.3 Costs that are shared and costs that overlap in time

Information and permission are acquired, and once acquired they serve every later use. A cost that is paid for the first use and not again is not additive over a collection of tasks.

New result 12.7 (Joint acquisition is subadditive)**.** *In a monotone capability system (Chapter 9) with a cost $\kappa(a) \geq 0$ for each action, each action being paid for once however often it is used, let $J(T)$ denote the least total cost of a set of actions whose closure from C_0 contains the set T of capabilities. Then J is monotone, $J(S \cup T) \leq J(S) + J(T)$, and $J(T) \leq \sum_{c \in T} J(\{c\})$ with strict inequality possible.*

Assumptions: Monotone capability system with action costs.

Proof. If $S \subseteq T$ every set of actions acquiring T acquires S, so $J(S) \leq J(T)$. The union of an action set acquiring S and one acquiring T acquires both, at cost at most the sum. For strictness take capabilities c_1, c_2 each gained by a distinct action of cost 1 that requires the same capability c_0 gained by an action of cost 10 from $C_0 = \varnothing$. Then $J(\{c_1\}) = J(\{c_2\}) = 11$ and $J(\{c_1, c_2\}) = 12$. □

Computing J is the optimal delete-free planning problem, NP-hard by the result cited in Chapter 9. The consequence for accounting is that the cost of a collection of tasks cannot be obtained by summing task by task when tasks share prerequisites, and that the sum overcharges exactly the shared part.

Time differs from energy in how concurrent work is counted. Let a task graph be a directed acyclic graph with durations $d_v > 0$ on vertices, let L be the length of a longest path (the critical path), and let $E = \sum_v d_v$ be the total work, which is the energy if each unit of busy time costs one unit of energy.

New result 12.8 (Time and energy of a task graph)**.** *The time to complete the tasks with unlimited parallelism is exactly L, while the energy is E however many workers are used. With p identical workers the completion time T_p of any schedule obeys $T_p \geq \max(L, E/p)$, and a greedy list schedule achieves $T_p \leq E/p + (1 - 1/p)L$.*

Assumptions: Tasks are non-preemptive and a task starts only after its predecessors finish.

Proof. Each path must be executed in sequence, so no schedule finishes before L, and the schedule that starts every task as soon as its predecessors finish takes exactly L. p workers perform at most pT_p units of work in time T_p, so $T_p \geq E/p$. The upper bound is Graham's analysis of list scheduling (1966). □

The two costs are measured on one graph and optimized by different quantities: time by the longest path, energy by the sum. Adding workers lowers time toward $\max(L, E/p)$ and does not lower energy at all, so a design question of the form "how much faster for how much more" cannot be answered by a single exchange rate.

12.4 The duality gap of a single mission

Result 12.4 says what scalar weights decide. The same fact can be stated as an optimization bound with a numerical gap. Let a mission have three available paths with energy and time costs $(0, 10)$, $(10, 0)$ and $(6, 6)$, and let the time be limited to $B = 6$. The least energy among paths that meet the limit is 6, attained by the third path. The Lagrangian relaxation prices time at $\lambda \geq 0$ and charges $c_1 + \lambda(c_2 - B)$, giving the lower bound $\max_{\lambda \geq 0} \min_p \{p_1 + \lambda(p_2 - 6)\}$. The three terms are 4λ, $10 - 6\lambda$ and 6, the inner minimum is the smaller of the first two and 6, the first two cross at $\lambda = 1$ with value 4, and so the dual value is $4 < 6$. The gap of 2 is the price of insisting on a single path: the mixture that takes the second path with probability 0.4 and the first with probability 0.6 has expected cost $(4, 6)$, meets the limit on average, and attains the dual value 4.

New result 12.9 (Lagrangian bound equals the mixture optimum)**.** *For a finite set $P \subset \mathbb{R}^2_{\geq 0}$ of path costs and a limit B on the second component, the Lagrangian dual value equals the least expected first component over probability mixtures of paths with expected second component at most B, whenever some path has second component at most B. The least first component over single paths is at least this value and can exceed it.*

Assumptions: Finite path set, feasibility of at least one single path.

Proof. The mixture problem is the linear program $\min\{\sum_i \lambda_i p_{i1} : \sum_i \lambda_i = 1, \lambda \geq 0, \sum_i \lambda_i p_{i2} \leq B\}$. It is feasible by assumption, and a linear program with a feasible bounded problem has zero duality gap; its dual with multiplier $\lambda \geq 0$ on the time constraint is exactly $\max_\lambda \min_i \{p_{i1} + \lambda(p_{i2} - B)\}$. Single paths are mixtures with a point mass, so the single-path optimum is at least the mixture optimum. The example above gives $6 > 4$. □

12.5 Adding workers can slow a schedule

Result 12.8 bounds the completion time of a greedy schedule but does not claim that it falls as workers are added. For list scheduling it can rise. Take nine tasks of durations $3, 2, 2, 2, 4, 4, 4, 4, 9$, with T_1 preceding T_9 and T_4 preceding each of T_5, T_6, T_7, T_8, and let the list order be $T_1, \ldots, T_9$. A simulation of the list rule gives completion time 12 with three workers and 15 with four. The critical path is T_4, T_5 of length 6 against T_1, T_9 of length 12, so $L = 12$ and the three-worker schedule is optimal, while the four-worker schedule is 25% longer. With three workers, T_1, T_2, T_3 start at time 0 and the worker freed by T_1 at time 3 takes T_9 at once, since $T_4, T_5, \ldots$ are served earlier but are not yet all ready; T_9 then finishes at 12. With four workers, $T_1, \ldots, T_4$ all start at time 0, and when T_2, T_3, T_4 finish at time 2 the three free workers take T_5, T_6, T_7, which precede T_9 in the list. The worker freed by T_1 at time 3 takes T_8, so T_9 cannot start until time 6 and finishes at 15. The total work is 34 either way. The anomaly is that the priority list, which was acceptable for three workers, uses the extra capacity on short tasks and delays the critical task. Extra capacity helps only if the priority rule is critical-path aware, and the example is a concrete reason that the bound $T_p \leq E/p + (1 - 1/p)L$ is an upper bound on the damage and not a description of it.

Exercises

Exercise 12.10. Give a graph on which three different paths are optimal in the additive, bottleneck, and fewest-edges senses respectively.

Exercise 12.11. For $P = \{(1,5),(5,1),(4,4)\}$ compute the set $\mathrm{conv}(P) + \mathbb{R}^2_{\geq 0}$ and list the budgets β accepted by all scalar weights but met by no single path.

Exercise 12.12. In the graph of Result 12.5 with $n = 3$ list all eight cost vectors, and give a vector budget accepted by exactly three of them.

Exercise 12.13. Show that the product of Result 12.6 can be pruned to the states (y, r) that are not dominated by another reachable (y, r') with $r' \geq r$, and explain why the pruned set is the Pareto front of Result 12.5 in disguise.

Exercise 12.14. For the task graph with tasks a, b, c, d of durations $2, 3, 1, 4$ where d depends on a and b, and c is independent, compute L, E, and the bounds of Result 12.8 for $p = 2$.

Exercise 12.15. For the three paths of Section 12.4 compute the dual value for the limit $B = 5$ and for $B = 7$, and compare it with the best single-path value.

Exercise 12.16. Prove that the Lagrangian dual value in Result 12.9 is nonincreasing in B and equals the minimum first component over all paths once B is at least the largest second component.

Exercise 12.17. Verify by drawing the schedules that the nine-task example of Section 12.5 completes at time 12 with three workers and 15 with four under the list order $T_1, \ldots, T_9$, and find a priority order for which four workers finish at time 12, and show that no schedule can finish before 12.

Exercise 12.18. Show that a cost algebra in which $\otimes$ is max and $\oplus$ is min makes the Pareto front of Result 12.5 a single point when only the bottleneck on one component is tracked, and explain what this says about which costs create combinatorial difficulty.

Chapter 13

Robustness Under Uncertainty

Question. Which guarantees survive disturbances and model uncertainty, and under which assumptions?

Established sources. Robust viability and invariance; chance constraints (Prékopa); Gaussian deterministic equivalents (Charnes and Cooper).

New results. The robust kernel is the greatest fixed point of the robust predecessor operator for arbitrary state spaces (Result 13.1); it shrinks as the disturbance set grows and can vanish abruptly (Results 13.2 and 13.3); chance constraints have deterministic equivalents for Gaussian margins and compose by Boole's inequality (Results 13.4 and 13.5); with unbounded full-support noise no policy can remain in a bounded region forever with positive probability (Result 13.6); an unknown constant parameter can be strictly easier than a time-varying one (Result 13.7).

Proof obligations. None.

Depends on. Chapters 3 and 4.

A guarantee is a statement about all the circumstances in some class, and the class is part of the guarantee. A system that keeps its constraint against every disturbance of size at most one has said nothing about a disturbance of size two; a system that keeps it with probability 0.99 at each step has said nothing about the probability of keeping it for a year. This chapter lists the main classes, proves for each what can be asserted, and shows where each assertion stops.

13.1 Four kinds of guarantee

The guarantees that appear in practice differ in what they assume about the disturbance and in what they promise.

(i) *Robust* (certain) guarantee: the constraint holds for every disturbance sequence in a stated set. This is viability against an adversary.

(ii) *Almost-sure* guarantee: the constraint holds with probability one. For finite supports this is a robust guarantee against the support (Chapter 4).

(iii) *Chance* guarantee: the constraint holds with at least a stated probability, over a stated horizon.

(iv) *Expected* guarantee: a cost or violation measure has bounded expectation (Chapter 4).

Each is weaker than the one before in the sense that a policy that meets (i) over a set containing the support meets (ii), (iii), and (iv) in the corresponding forms, and none is strictly more realistic than another: the choice is a statement about what is known of the disturbance.

13.2 Robust viability

Consider a system $x_{t+1} = f(x_t, u_t, w_t)$ with $u_t \in F(x_t)$ chosen by a controller and $w_t \in \mathcal{W}(x_t, u_t)$ chosen by an adversary, where $\mathcal{W}$ assigns a nonempty set to each state and action. Both choose after seeing the history: u_t may depend on $x_0, \dots, x_t$, and w_t may depend on everything. Given a constraint set K define the *robust predecessor*

$$\mathrm{Pre}_{\mathrm{rob}}(Z) = \{x : \exists u \in F(x)\ \forall w \in \mathcal{W}(x,u) :\ f(x,u,w) \in Z\}.$$

A state is *robustly viable* if some policy keeps $x_t \in K$ for all t against every adversary.

New result 13.1 (The robust kernel)**.** *Let $\mathcal{V}$ be the greatest set with $\mathcal{V} \subseteq K \cap \mathrm{Pre}_{\mathrm{rob}}(\mathcal{V})$ (it exists by monotonicity of the operator, as the union of all such sets). Then a state is robustly viable if and only if it lies in $\mathcal{V}$. This holds for arbitrary state spaces and disturbance sets.*

Assumptions: Policies may depend on the history; the adversary may choose any $w_t \in \mathcal{W}(x_t, u_t)$.

Proof. ($\Leftarrow$) From $x_0 \in \mathcal{V}$ the controller chooses at each state of $\mathcal{V}$ an action u with $f(x,u,w) \in \mathcal{V}$ for all $w \in \mathcal{W}(x,u)$; the state then stays in $\mathcal{V} \subseteq K$ forever, whatever the adversary does. ($\Rightarrow$) Let a policy keep the state in K against every adversary, and let S be the set of states x_t that occur at some time t for some adversary. Then $x_0 \in S \subseteq K$. For $s \in S$ fix a history at which s occurs, and let u be the action the policy takes there. Every $w \in \mathcal{W}(s,u)$ is an admissible next disturbance, so $f(s,u,w)$ occurs one step later for some adversary and lies in S. Hence $s \in \mathrm{Pre}_{\mathrm{rob}}(S)$, so $S \subseteq K \cap \mathrm{Pre}_{\mathrm{rob}}(S)$ and $S \subseteq \mathcal{V}$. □

For a finite system with disturbance set the support of the transition law, this is the almost-sure kernel of Result 4.3; with a singleton disturbance set it is the viability kernel of Chapter 3.

New result 13.2 (Larger disturbance sets shrink the kernel)**.** *If $\mathcal{W}(x,u) \subseteq \mathcal{W}'(x,u)$ for all x, u then the robust kernel for $\mathcal{W}'$ is contained in the robust kernel for $\mathcal{W}$.*

Assumptions: Same f, F, K.

Proof. $\mathrm{Pre}'_{\mathrm{rob}}(Z) \subseteq \mathrm{Pre}_{\mathrm{rob}}(Z)$ for every Z, since a condition required of all w in a larger set is required of all w in a smaller one. The operator $Z \mapsto K \cap \mathrm{Pre}_{\mathrm{rob}}(Z)$ is monotone, so the greatest fixed point of the smaller operator contains that of the larger. □

The cost of robustness is the difference of kernels, and it need not be gradual.

New result 13.3 (A threshold for tolerable disturbance)**.** *Consider the system of Example 1.3, $x_{t+1} = 2x_t + u_t + w_t$ with $|u_t| \le 1$, $|w_t| \le \delta$, and $K = [-3, 3]$. For $0 \le \delta \le \frac{1}{2}$ the robust kernel is $[-(1-\delta), 1-\delta]$. For $\delta > \frac{1}{2}$ it is empty. The kernel therefore shrinks continuously to the interval $[-\frac{1}{2}, \frac{1}{2}]$ as $\delta \uparrow \frac{1}{2}$ and is empty immediately beyond it.*

Assumptions: As stated.

Proof. For a given x the best action minimizes $\max_{|w|\le\delta}|2x+u+w| = \min_{|u|\le 1}|2x+u|+\delta = \max(2|x|-1,0)+\delta$. Let $\mathcal{W}_0$ be any set with $\mathcal{W}_0 \subseteq K \cap \mathrm{Pre}_{\mathrm{rob}}(\mathcal{W}_0)$, assumed nonempty, and let $a = \sup_{x\in\mathcal{W}_0}|x| \le 3$. Every $x \in \mathcal{W}_0$ must have $\max(2|x|-1,0)+\delta \le a$, which gives $\delta \le a$ and $2|x|-1+\delta \le a$. Taking the supremum over x in the second inequality gives $2a-1+\delta \le a$, so $a \le 1-\delta$, and then $\delta \le a \le 1-\delta$ forces $\delta \le \frac{1}{2}$. So for $\delta > \frac{1}{2}$ no nonempty such set exists, and for $\delta \le \frac{1}{2}$ every such set lies in $[-(1-\delta), 1-\delta]$. Conversely this interval is a fixed point: for $|x| \le 1-\delta$ we have $\max(2|x|-1,0)+\delta \le \max(1-2\delta,0)+\delta = 1-\delta$ when $\delta \le \frac{1}{2}$. By Result 13.1 it is the kernel. □

At $\delta = \frac{1}{2}$ the system can still keep $[-\frac{1}{2}, \frac{1}{2}]$ invariant; for any larger disturbance the controller cannot cancel the growth of the state at the equilibrium, and the whole kernel disappears at once. This is the usual shape of a stability threshold, and it means that a safety margin cannot be estimated by watching the kernel shrink: the kernel can look comfortable until a parameter crosses a boundary.

13.3 Chance constraints

A chance constraint requires $\Pr[g(x) \le 0] \ge 1-\varepsilon$ for a risk level $\varepsilon \in (0,1)$. For a single linear constraint on a Gaussian quantity the requirement has a closed form.

Established result 13.4 (Gaussian deterministic equivalent)**.** *If $x \sim \mathcal{N}(\mu, \Sigma)$, $a \neq 0$ and $\varepsilon \in (0,1)$, then*

$$\Pr[a^{\mathsf{T}}x \le b] \ge 1-\varepsilon \iff a^{\mathsf{T}}\mu + \Phi^{-1}(1-\varepsilon)\sqrt{a^{\mathsf{T}}\Sigma a} \le b,$$

where Φ is the standard normal distribution function. For $\varepsilon < \frac{1}{2}$ the right-hand side is a convex (second-order cone) constraint in $(\mu, \Sigma^{1/2})$.

Source: Charnes and Cooper (1959); Prékopa, *Stochastic Programming* (1995).

Proof. $a^{\mathsf{T}}x \sim \mathcal{N}(a^{\mathsf{T}}\mu, a^{\mathsf{T}}\Sigma a)$, so $\Pr[a^{\mathsf{T}}x \le b] = \Phi\big((b-a^{\mathsf{T}}\mu)/\sigma\big)$ with $\sigma^2 = a^{\mathsf{T}}\Sigma a$ (positive since Σ is positive definite), and Φ is increasing. □

The closed form is the reason Gaussian margins are common in practice. The margin $\Phi^{-1}(1-\varepsilon)\sigma$ grows with the standard deviation, not with the variance, and with the risk level logarithmically.

Constraints over several steps are combined by Boole's inequality.

New result 13.5 (Allocating risk over a horizon)**.** *If $\Pr[A_t^c] \le \varepsilon_t$ for $t = 1, \dots, T$, where A_t is the event that the constraint holds at time t, then $\Pr[\bigcap_{t\le T} A_t] \ge 1 - \sum_t \varepsilon_t$. If the failures are independent events of probability exactly p then $\Pr[\bigcap_{t\le T} A_t] = (1-p)^T$, so keeping it at least $1-\varepsilon$ requires $p \le 1-(1-\varepsilon)^{1/T} \le \varepsilon/T$, and the constraint on the per-step risk gets tighter in proportion to the horizon.*

Assumptions: As stated.

Proof. Boole's inequality gives $\Pr[\bigcup_t A_t^c] \le \sum_t \Pr[A_t^c]$. For independent failures $\Pr[\bigcap A_t] = \prod(1-p) = (1-p)^T$, and $(1-p)^T \ge 1-\varepsilon$ is equivalent to $p \le 1-(1-\varepsilon)^{1/T}$. By Bernoulli's inequality $(1-\varepsilon/T)^T \ge 1-\varepsilon$, so $p = \varepsilon/T$ suffices, and $1-(1-\varepsilon)^{1/T} \le \varepsilon/T$ because $(1-\varepsilon)^{1/T} \ge 1-\varepsilon/T$ by concavity of $t \mapsto t^{1/T}$. □

Boole's inequality is conservative when failures are positively correlated and tight when they are disjoint. The per-step allocation ε_t may be unequal: a schedule with $\sum \varepsilon_t < \infty$, such as $\varepsilon_t = \varepsilon\, 2^{-t}$, controls the probability of ever failing over an infinite horizon, at the price of margins that grow without bound as $t \to \infty$.

New result 13.6 (No forever-safety under unbounded noise)**.** *Let $x_{t+1} = f(x_t, u_t) + w_t$ where u_t takes values in a bounded set, f is bounded on $K \times U$ with K bounded, and the w_t are independent of the past and identically distributed with $\Pr[|w_t| > R] > 0$ for every R. Then there is $q > 0$ such that for every policy and every initial state, $\Pr[x_1, \ldots, x_T \in K] \le (1-q)^T$. In particular $\Pr[x_t \in K \text{ for all } t] = 0$.*

Assumptions: As stated; Gaussian noise satisfies the tail assumption.

Proof. Let $M = \sup_{K\times U} |f|$, $D = \sup_{x\in K} |x|$, $R = M + D$ and $q = \Pr[|w_t| > R] > 0$. On the event $\{x_t \in K\}$, if $|w_t| > R$ then $|x_{t+1}| \ge |w_t| - |f(x_t, u_t)| > R - M = D$, so $x_{t+1} \notin K$. The action u_t depends only on $x_0, \ldots, x_t$, hence on $w_0, \ldots, w_{t-1}$, and is independent of w_t, so $\Pr[x_{t+1} \notin K \mid x_0, \ldots, x_t] \ge q$ on $\{x_t \in K\}$. Hence $\Pr[x_1, \ldots, x_T \in K] \le (1 - q) \Pr[x_1, \ldots, x_{T-1} \in K]$ and by induction $\le (1-q)^T$. □

The consequence is a division of labour among the guarantees. Certain guarantees over infinite horizons are available only for disturbances with bounded support, as in Result 13.3. With unbounded noise the achievable statements are finite-horizon chance constraints, bounds on the expected time to violation, or constraints that are enforced by a mechanism that can reset the system (Chapter 7); a claim of lasting safety under Gaussian noise with a bounded constraint set is a claim about a finite horizon in disguise.

13.4 Model uncertainty

Disturbances vary from step to step. Parametric uncertainty is different: a parameter $\theta \in \Theta$ is unknown but fixed, and the controller can learn from the states it observes. A guarantee against a time-varying parameter is a guarantee against a fixed one, but not conversely.

New result 13.7 (Constant parameters are easier than varying ones)**.** *Let $x_{t+1} = f(x_t, u_t, \theta_t)$ with $\theta_t \in \Theta$. Let $\mathcal{V}_{\text{var}}$ be the robust kernel when the adversary chooses θ_t freely at each step and $\mathcal{V}_{\text{const}}$ the set of states from which a history-dependent policy keeps the state in K for every constant sequence $\theta_t \equiv \theta$. Then $\mathcal{V}_{\text{var}} \subseteq \mathcal{V}_{\text{const}}$, and the inclusion can be strict.*

Assumptions: History-dependent policies.

Proof. A policy that keeps the state in K against all sequences keeps it against constant sequences. For strictness take finite states $s_0, s_1, s_2, \text{bad}$, with K everything except bad and $\Theta = \{1, 2\}$, and actions $\{1, 2\}$. From s_0 every action leads to s_θ. From s_j ($j = 1, 2$) action k leads to s_k if $\theta = k$ and to bad otherwise. Under a constant parameter θ the controller, on observing s_θ, plays θ forever, and the state stays in $\{s_0, s_1, s_2\}$ forever. Against a time-varying adversary, whatever action k is played at s_j the adversary chooses $\theta \neq k$ and reaches bad. So $\mathcal{V}_{\text{var}} = \varnothing$ and $\mathcal{V}_{\text{const}} = \{s_0, s_1, s_2\}$. □

The controller in the example learns the parameter from the state it reaches, so its information is a belief, in the sense of Chapter 5, over Θ: initially the support Θ, afterwards a singleton. The sufficient representation for feasibility is therefore the pair of the current state and the set of parameters consistent with the history.

13.5 The price of not knowing the distribution

The Gaussian margin of Result 13.4 depends on the distribution being Gaussian. When only the mean and variance of the constrained quantity are known, the guarantee has to hold for every distribution with those two moments, and the margin is larger.

Established result 13.8 (Distribution-free chance constraint)**.** *Let Y have mean μ and variance σ^2, and let $\varepsilon \in (0,1)$. Then $\Pr[Y \le b] \ge 1-\varepsilon$ for every distribution with these moments if and only if $b \ge \mu + \sigma\sqrt{(1-\varepsilon)/\varepsilon}$.*

Source: Cantelli (1928); El Ghaoui, Oks and Oustry (2003); Calafiore and El Ghaoui (2006).

Proof. For $b > \mu$ Cantelli's inequality gives $\Pr[Y > b] \le \sigma^2/(\sigma^2 + (b-\mu)^2)$, which is at most ε exactly when $(b-\mu)^2 \ge \sigma^2(1-\varepsilon)/\varepsilon$. The bound cannot be improved. The two-point distribution that places mass ε at $\mu + \sigma\sqrt{(1-\varepsilon)/\varepsilon}$ and mass $1-\varepsilon$ at $\mu - \sigma\sqrt{\varepsilon/(1-\varepsilon)}$ has mean μ, variance σ^2 and $\Pr[Y \ge b] = \varepsilon$ when b is the upper atom. If $(b-\mu)^2 < \sigma^2(1-\varepsilon)/\varepsilon$, moving the upper atom to a point slightly above b and adjusting the masses to keep the mean and variance gives a distribution with $\Pr[Y > b] > \varepsilon$, so no smaller margin works for all distributions with this mean and variance. □

The margins in units of σ for several risk levels are as follows (values computed and rounded).

ε	Gaussian $\Phi^{-1}(1-\varepsilon)$	Distribution-free $\sqrt{(1-\varepsilon)/\varepsilon}$
0.1	1.282	3.000
0.05	1.645	4.359
0.01	2.326	9.950
0.001	3.090	31.607

At $\varepsilon = 0.05$ the distribution-free margin is 2.65 times the Gaussian one, and at $\varepsilon = 0.001$ it is about ten times as large. The difference is the price of ignorance of the shape of the disturbance, and the table shows that the price rises steeply with the demanded confidence: a designer who asserts a rare-event guarantee on the strength of two moments is asserting a very conservative one, and a designer who adopts the Gaussian margin has made a distributional assumption that is the entire content of the claim.

13.6 A horizon allocation in numbers

Result 13.5 says how the per-step risk must be reduced. For a total risk of $\varepsilon = 0.05$ over $T = 10$ independent steps, the exact per-step bound is $p = 1 - 0.95^{1/10} = 0.00512$, slightly looser than the Boole allocation $\varepsilon/T = 0.005$, and the corresponding Gaussian margins are $\Phi^{-1}(1-p) = 2.568\sigma$ and $\Phi^{-1}(1-0.005) = 2.576\sigma$, against 1.645σ for a single step. Ten steps

cost about one additional standard deviation of margin, and the increase is logarithmic in the horizon, which is why Gaussian chance constraints over moderate horizons are affordable and why the Boole allocation is a reasonable approximation when failures are close to independent.

New result 13.9 (Exit time under bounded Gaussian noise)**.** *Let $x_{t+1} = ax_t + w_t$ with $|a| < 1$, $w_t \sim \mathcal{N}(0,1)$ independent, and $K = [-L, L]$. Let P be the sub-Markov operator on K with kernel $p(x, y) = \phi(y - ax)$ for $y \in K$. Then P has a simple eigenvalue $\lambda_* \in (0, 1)$ equal to its spectral radius, with a positive eigenfunction, and for every initial state $x_0 \in K$ the survival probability satisfies $\Pr[x_1, \dots, x_T \in K] = c(x_0)\lambda_*^T(1 + o(1))$ with $c(x_0) > 0$. The exit time τ is asymptotically geometric: $\Pr[\tau > T + 1 \mid \tau > T] \to \lambda_*$.*

Assumptions: The kernel is strictly positive and continuous on $K \times K$.

Proof. The kernel $p(x, y)$ is continuous and strictly positive on the compact square $K \times K$, so P is a compact, strictly positive operator on $C(K)$. The Krein–Rutman theorem (the infinite-dimensional Perron–Frobenius theorem) gives a simple positive eigenvalue λ_* equal to the spectral radius, with a strictly positive eigenfunction e and a positive left eigenmeasure ν, and all other eigenvalues have strictly smaller modulus. Hence $P^T\mathbf{1} \sim \lambda_*^T\, e\, \nu(\mathbf{1})/\nu(e)$, which is the asserted form with $c = e(x_0)\nu(\mathbf{1})/\nu(e)$. Since P is dominated by a Markov kernel with strict loss, $\lambda_* < 1$. □

For $a = \frac{1}{2}$, $L = 3$ and $x_0 = 0$, a quadrature of the operator on a grid of 1201 points gives the survival probabilities 0.9973, 0.9659, 0.9261, 0.8513, 0.6612, 0.4339, 0.1869 at $T = 1, 5, 10, 20, 50, 100, 200$, and the dominant eigenvalue $\lambda_* \approx 0.9916$, so the expected exit time is about $1/(1 - \lambda_*) \approx 119$ steps. A system that violates its constraint with probability 0.27% at the first step is therefore expected to violate it within about a hundred steps, and the survival probability after two hundred is under one in five. The numbers show the content of Result 13.6: the per-step risk is small, the horizon is long, and the product is not.

Exercises

Exercise 13.10. Compute the robust kernel for the system of Result 13.3 with $K = [-1/2, 1/2]$ and $\delta = 1/4$.

Exercise 13.11. For the Gaussian constraint $\Pr[x \le 5] \ge 0.95$ with $x \sim \mathcal{N}(\mu, 4)$, find the largest μ, using $\Phi^{-1}(0.95) \approx 1.645$.

Exercise 13.12. Show that for the time-varying disturbance $|w_t| \le \delta$ in Result 13.3 and the constant unknown $w_t \equiv \theta$ with $|\theta| \le \delta$, the two kernels coincide, and explain why learning θ after the first step does not enlarge the kernel.

Exercise 13.13. In Result 13.6 replace the Gaussian noise by noise uniform on $[-1, 1]$ and show by example that a policy may remain in a bounded set forever with probability one, which of the assumptions has failed.

Exercise 13.14. Construct a risk schedule ε_t with $\sum_t \varepsilon_t = \varepsilon$ and compute the margin $\Phi^{-1}(1 - \varepsilon_t)\sigma$ for $\varepsilon_t = \varepsilon 2^{-t}$, showing it grows without bound.

Exercise 13.15. Verify the entries of the margin table for $\varepsilon = 0.1$ and $\varepsilon = 0.01$, and construct the two-point distribution of Result 13.8 explicitly for $\mu = 0$, $\sigma = 1$, $\varepsilon = 0.1$.

Exercise 13.16. For $\varepsilon = 0.05$ and $T = 20$ compute the exact per-step risk $1 - (1 - \varepsilon)^{1/T}$ and the Boole allocation ε/T, and the corresponding Gaussian margins, using $\Phi^{-1}(0.9975) \approx 2.807$.

Exercise 13.17. In Result 13.9 show that if $a = 0$ the survival probability is exactly q^T with $q = \Pr[|w| \le L]$, and that $\lambda_* = q$ in that case. Compute q for $L = 3$.

Exercise 13.18. Show that the eigenvalue λ_* of Result 13.9 increases with L and tends to 1 as $L \to \infty$, and explain what this says about the relation between margin and expected time to violation.

Exercise 13.19. Explain why a controller that has learned a constant disturbance parameter after one step (Result 13.7) improves the guarantee against that parameter but not against the unbounded noise of Result 13.6.

Part V

Preserving future admissibility

Chapter 14

Viability Under Evolving Obligations

Question. How do present actions generate duties that restrict later transitions, and which regions allow continued compliance?

Established sources. Viability kernels (Chapter 3); bounded-response obligations and their monitors (Chapter 14 of the first volume).

New results. Compliant viability is the viability kernel of the product of the physical system with the obligation monitor (Result 14.2); the kernel is monotone in the slack of pending obligations (Result 14.3); a trigger action is admissible exactly when its successor lies in the kernel (Result 14.4), which in a corridor example gives an explicit limit on where an obligation may be incurred (Result 14.6); admissibility of a trigger depends on obligations already pending (Result 14.7); no gate that consults the monitor alone is both sound and complete when the limit depends on position (Result 14.8).

Proof obligations. None.

Depends on. Chapters 3 and 7.

An action that is physically available and violates no constraint now may still be one that the system should not take, because it creates a duty that the system will be unable to discharge. A rover that samples far from its dock is bound to return within a deadline; a service that accepts an order is bound to ship; a clinician who begins a procedure is bound to complete it. The constraint set that governs the next action is therefore not fixed in advance but is updated by what has been done, and the regions from which compliance can be continued are the viability kernel of a system whose state includes the record of what is owed. This chapter builds that system for the bounded-response obligations of the first volume and proves what its kernel looks like.

14.1 The product with an obligation monitor

Definition 14.1 (Controlled system with bounded-response obligations)**.** Let a finite controlled system consist of a finite state set X, a finite action set U, a feasibility correspondence $F : X \to 2^U$, a deterministic transition $\delta : X \times U \to X$ defined for $u \in F(x)$, and a constraint set $K \subseteq X$. There are r obligations indexed by $i = 1, \dots, r$, each with a set $A_i \subseteq U$ of *trigger*

actions, a set $B_i \subseteq U$ of *response* actions with $A_i \cap B_i = \varnothing$, and a deadline $k_i \geq 1$: after a trigger the system owes a response within the next k_i steps.

A *monitor state* is a vector $c = (c_1, \ldots, c_r)$ with $c_i \in \{0, 1, \ldots, k_i\}$, where $c_i = 0$ means that obligation i is idle and $c_i > 0$ means that a response is owed and c_i steps remain, counting the next one. Taking action u at monitor state c gives the new coordinate c_i' as follows.

- If $u \in B_i$ then $c_i' = 0$.
- If $u \notin B_i$ and $c_i \geq 2$ then $c_i' = c_i - 1$.
- If $u \notin B_i$ and $c_i = 1$ then the action is *non-compliant*: the deadline has passed without a response.
- If $c_i = 0$ then $c_i' = k_i$ if $u \in A_i$ and $c_i' = 0$ otherwise.

A trigger issued while the obligation is pending is absorbed. The *compliant product* has state set $X \times C$ with $C = \prod_i \{0, \ldots, k_i\}$, and a transition $(x, c) \xrightarrow{u} (\delta(x, u), c')$ for each $u \in F(x)$ that is compliant at every coordinate.

The monitor is that of the first volume for the disjoint fragment: its size is $\prod_i (k_i + 1)$ together with a sink for violation, and the sink is represented here by the exclusion of the non-compliant action. A state (x, c) is *compliantly viable* if there is an infinite run of the compliant product from (x, c) that stays in $K \times C$.

New result 14.2 (The compliant kernel)**.** *The set $\mathcal{V}$ of compliantly viable states is the greatest set $Z \subseteq K \times C$ such that every $(x, c) \in Z$ has a compliant action $u \in F(x)$ with $(\delta(x, u), c') \in Z$. It is computed by the decreasing iteration of Result 3.2 on the compliant product, which has $|X| \prod_i (k_i + 1)$ states and stabilizes after at most that many rounds.*

Assumptions: Finite X and U; deterministic transitions.

Proof. The compliant product is a finite controlled system with the constraint set $K \times C$, and compliant viability is viability in it. The statement is Result 3.2. □

A state x is thus acceptable to occupy when it has no debt if $(x, \mathbf{0}) \in \mathcal{V}$, and when its debts are those of c if $(x, c) \in \mathcal{V}$. The section $\mathcal{V}_c = \{x : (x, c) \in \mathcal{V}\}$ is the effective constraint set in force when the monitor reads c.

14.2 Slack and the effective constraint set

Order the coordinates of the monitor by *slack*: write $s_i(c) = c_i$ if $c_i > 0$ and $s_i(c) = \infty$ if $c_i = 0$. More slack is better: an idle obligation is the least demanding, and a pending one is more demanding the smaller its counter.

New result 14.3 (Viability is monotone in slack)**.** *If $(x, c) \in \mathcal{V}$ and $s_i(c') \geq s_i(c)$ for all i, then $(x, c') \in \mathcal{V}$. Consequently each section $\mathcal{V}_c$ is monotone: $\mathcal{V}_c \subseteq \mathcal{V}_{c'}$ when c' has at least as much slack as c.*

Assumptions: As in Definition 14.1.

Proof. Let $Z_0 = K \times C$ and $Z_{n+1} = Z_0 \cap \mathrm{Pre}(Z_n)$, where Pre is the predecessor operator of the compliant product, so that $\mathcal{V} = \bigcap_n Z_n$. We show by induction that every Z_n is upward closed in slack. Z_0 is. Suppose Z_n is, and let $(x, c) \in Z_{n+1}$ with a compliant action u leading into Z_n, and let c' have at least the slack of c. We compare the effect of u on the two monitor states

coordinate by coordinate. If $u \in B_i$ both coordinates become 0. Otherwise, if $c_i = 0$ then $c'_i = 0$ too and both coordinates evolve identically, since the update depends only on whether the coordinate is idle and whether $u \in A_i$. If $c_i > 0$ and $c'_i = 0$ (so $s_i(c') = \infty$), after u the new coordinate of c is $c_i - 1$ (compliantly, so $c_i \geq 2$), and that of c' is k_i or 0, both with slack at least $c_i - 1$ because $c_i \leq k_i$. If both are pending with $c'_i \geq c_i$, then $c_i \geq 2$ gives $c'_i \geq 2$ and the new coordinates satisfy $c'_i - 1 \geq c_i - 1$. In every case u is compliant at c' and leads to a state with at least the slack of the successor of (x, c), hence into Z_n. So $(x, c') \in Z_{n+1}$. □

The kernel contains the idle sections in the largest form, and as obligations become urgent the effective constraint set shrinks. The first volume described this as a constraint that changes with the history; the result says it changes monotonically with the urgency of what is owed.

New result 14.4 (A trigger is admissible when its successor is viable)**.** *Let u be a trigger of obligation i, with $u \in F(x)$, taken at a compliant monitor state c with $c_i = 0$. The action is part of some compliant infinite run from (x, c) if and only if its successor $(\delta(x, u), c') \in \mathcal{V}$ (and $\delta(x, u) \in K$). In particular the set of states at which the system can* incur *obligation i at idle is $\{x : (\delta(x, u), c') \in \mathcal{V}\}$, and it is in general a proper subset of $\mathcal{V}_c$.*

Assumptions: As in Definition 14.1.

Proof. A compliant infinite run that begins with u is a compliant infinite run from the successor, which exists exactly when the successor is in $\mathcal{V}$. The strictness is shown in Result 14.6. □

14.3 A corridor

Example 14.5 (Where an obligation may be incurred)**.** A rover moves along positions $X = \{0, 1, \ldots, L\}$ with actions `left`, `right`, `stay`, `sample`, and `upload`. Moves change the position by one (not past the ends), `stay` and `sample` leave the position unchanged, and `upload` is feasible only at position 0 and leaves it unchanged. There is one obligation with trigger $A = \{\texttt{sample}\}$, response $B = \{\texttt{upload}\}$, and deadline k. The constraint set is $K = X$.

New result 14.6 (The corridor kernel)**.** *In Example 14.5 the kernel is $\mathcal{V} = \{(p, 0)\} \cup \{(p, c) : 1 \leq c \leq k,\ c \geq p + 1\}$. Consequently the action `sample` is admissible at position p with the obligation idle if and only if $p \leq k - 1$. For $k \geq L + 1$ it is admissible everywhere, and for $k \leq L$ it is refused at the positions $k, \ldots, L$ although those positions are viable when idle.*

Assumptions: Example 14.5.

Proof. Let S denote the set on the right. Idle states are in S because `stay` keeps the system idle. For $(p, c) \in S$ pending: if $p = 0$, `upload` leads to $(0, 0) \in S$; if $p \geq 1$, `left` leads to $(p-1, c-1)$ with $c-1 \geq p \geq (p-1)+1$ and $c-1 \geq 1$ because $c \geq p+1 \geq 2$, so the successor is in S. Thus $S \subseteq \mathrm{Pre}(S)$ and $S \subseteq \mathcal{V}$. Conversely let $1 \leq c \leq p$. Any compliant run must contain an `upload` at one of the next c steps, say at step $j \leq c$, which requires the position to be 0 before that step. Reaching 0 from p takes at least p moves, all of which precede the upload, so $j \geq p + 1 > c$, a contradiction. So such states are not viable. By Result 14.4, `sample` at $(p, 0)$ leads to (p, k), which lies in $\mathcal{V}$ if and only if $k \geq p + 1$. □

The result is concrete evidence for the claim that the constraint set is updated by the monitor. From the idle state, every position is viable: the rover can always decline to sample. Yet the one action that incurs the debt is permitted only within $k-1$ steps of the dock, and a rover at a distant position that samples has committed itself to a violation.

14.4 Interaction between obligations

New result 14.7 (Admissibility of a trigger depends on what is pending)**.** *Extend Example 14.5 by a second obligation with trigger* `alarm`*, which leaves the position unchanged, response* `inspect`*, feasible only at position L, and deadline k_2. Take $L = 3$, $k_1 = k_2 = 3$. Then:*

(a) At position 1 with both obligations idle, each of `sample` *and* `alarm` *is admissible: its successor is in the kernel of the system with the other obligation removed.*

(b) After `sample` *at position 1 the state $(1; 3, 0)$ is in $\mathcal{V}$, but* `alarm` *is then not admissible: its successor $(1; 2, 3) \notin \mathcal{V}$.*

(c) Hence the kernel of the product is strictly smaller than the intersection of the kernels of the two single-obligation systems.

Assumptions: As stated.

Proof. (a) `sample` at 1 gives $(1, 3)$ with $3 \geq p + 1 = 2$, viable by Result 14.6. For `alarm` alone the rover must reach $L = 3$ and inspect within 3 steps: from position 1 it takes two moves and the `inspect`, three steps, which is allowed with deadline 3. (b) From $(1; 3, 0)$ the sequence `left`, `upload` discharges the first obligation, so the state is viable. `alarm` at that state yields $(1; 2, 3)$ (the first counter decreases to 2 and the second is set to 3). In this state the first obligation can only be discharged by `left` followed by `upload` at step 2, since an upload needs position 0. After these two steps the position is 0 and the second counter has fallen to 1, so the `inspect`, which needs position 3, cannot be performed on the next step and the second deadline is missed. Any other plan misses the first deadline, so the state is not viable. (c) The state $(1; 2, 3)$ is viable in each single-obligation system (by the arguments for (a)) and not in the product. □

This is the joint obstruction of the first volume in dynamic form: each duty is dischargeable alone, and the two are not dischargeable together because they pull the system to opposite ends of the corridor. Incurring both is a decision about the product, and the monitor that tracks each duty separately does not know it.

New result 14.8 (No monitor-only gate)**.** *In Example 14.5 with $1 \leq k \leq L$, no gate that decides whether to permit* `sample` *as a function of the monitor state alone is both sound (it never permits a trigger whose successor is not in $\mathcal{V}$) and complete (it permits every trigger whose successor is in $\mathcal{V}$), when positions are not observable to the gate.*

Assumptions: The gate sees only the monitor state, so it acts identically at all positions.

Proof. At monitor state **0** the gate permits or refuses. If it permits, it permits at position $L \geq k$, which is not admissible by Result 14.6, so it is unsound. If it refuses, it refuses at position 0, which is admissible since $0 \leq k - 1$, so it is incomplete. □

The result is the representational obstruction of Chapter 1 for this system: the monitor state fails to separate histories whose continuations differ in admissibility, because the continuation depends on the physical state as well. Adding the position to the monitor, that is, working in the product, removes the obstruction at the cost of $|X|$ times as many states.

14.5 The size of the kernel and the share of positions that can incur

Result 14.6 determines the kernel in closed form, so its size and the share of idle positions from which the obligation may be incurred can be written down.

New result 14.9 (Counting the corridor kernel)**.** *In Example 14.5 the kernel has*

$$|\mathcal{V}| = (L+1) + \sum_{c=1}^{k} \min(c,\, L+1)$$

states, of which $L+1$ are idle, and the share of idle positions from which `sample` *is admissible is* $\min(k, L+1)/(L+1)$.

Assumptions: Example 14.5.

Proof. By Result 14.6 the pending states in $\mathcal{V}$ with counter c are those with $p \le c-1$ and $p \le L$, which number $\min(c, L+1)$; summing over $c = 1, \dots, k$ and adding the $L+1$ idle states gives the count. Sample is admissible at $p \le k-1$ and $p \le L$, which is $\min(k, L+1)$ positions. □

A direct computation of the decreasing iteration on the compliant product confirms the formula: for $(L, k) = (3, 3)$ the kernel has 10 states and `sample` is admissible at positions $0, 1, 2$; for $(4, 3)$ it has 11 states with admissible positions $0, 1, 2$; for $(5, 2)$ it has 9 states with admissible positions $0, 1$. The formulas give $4 + (1 + 2 + 3) = 10$, $5 + (1 + 2 + 3) = 11$ and $6 + (1 + 2) = 9$. In the third case a rover in a corridor of six positions may incur the duty from only one third of them, and the fraction falls in proportion to the deadline.

14.6 When two duties can be held together

Result 14.7 exhibits one pair of obligations whose joint incurrence fails. The condition can be stated in general for the corridor with responses at the two ends.

New result 14.10 (Admissibility of the second trigger)**.** *In the corridor with positions $0, \dots, L$, an obligation with response* `upload` *at position 0, and an obligation with trigger* `alarm` *and response* `inspect` *at position L with deadline k_2, suppose the first obligation is pending with counter $c_1 \ge 2$ at position p and the second is idle. The action* `alarm` *has its successor in the kernel if and only if one of the following holds:*

*(a) (*discharge the first duty, then the second*)* $c_1 - 1 \ge p + 1$ *and* $k_2 \ge p + L + 2$*;*

*(b) (*discharge the second duty, then the first*)* $k_2 \ge L - p + 1$ *and* $c_1 - 1 \ge 2L - p + 2$.

Assumptions: Moves cost one step each, responses cost one step, and later triggers may be refused.

Proof. After `alarm` the counters are $c_1 - 1$ and k_2 at the same position p. The successor is in the kernel exactly when both duties can be discharged within their remaining times, since after both are discharged the rover can stay idle forever. The only decisions are the order of the two responses. In the order (a), the upload is performed at step $p+1$ (after p moves), which must not exceed $c_1 - 1$; the inspection then needs L moves and one more step, so it is performed at step $p + 1 + L + 1$, which must not exceed k_2. In the order (b) the inspection is performed at step $L-p+1$, which must not exceed k_2, and the upload at step $(L-p+1)+L+1 = 2L-p+2$, which must not exceed $c_1 - 1$. Waiting never helps, and interleaving the two journeys reaches no response earlier than going straight to the first target. □

The criterion was checked against the decreasing iteration for every combination of $L \in \{1, \dots, 4\}$, $k_1 \in \{2, \dots, 7\}$, $k_2 \in \{1, \dots, 9\}$, every position and every counter $c_1 \geq 2$ (2646 cases), with no disagreement. For the case of Result 14.7, with $L = 3$, $p = 1$ and $c_1 = 3$, condition (a) reads $2 \geq 2$ and $k_2 \geq 6$, and condition (b) reads $k_2 \geq 3$ and $2 \geq 7$; so `alarm` is admissible exactly when $k_2 \geq 6$, which the iteration confirms (the successor is outside the kernel for $k_2 = 1, \dots, 5$ and inside it for $k_2 = 6, \dots, 9$). The general pattern is that a second duty is compatible with a pending first duty when its deadline covers the whole detour: the journey to the first response, the journey from there to the second, and the two responses themselves. A deadline shorter than that detour cannot be repaired by any policy, because the geometry of the corridor fixes the time the detour takes.

Exercises

Exercise 14.11. For Example 14.5 with $L = 4$ and $k = 3$, list the positions at which `sample` is admissible and the pending states (p, c) in $\mathcal{V}$.

Exercise 14.12. Modify the corridor so that `upload` takes effect only if the battery is not empty, with battery decreasing by one per move, and compute the kernel for battery capacity b and deadline k as a function of position and battery.

Exercise 14.13. Prove that in Definition 14.1 absorbing a trigger while pending can be replaced by restarting the counter at k_i without changing which states are in the kernel, or give a system in which the choice matters.

Exercise 14.14. In Result 14.7, find the least deadline k_2 for which `alarm` remains admissible after `sample` at position 1, and describe the general pattern.

Exercise 14.15. Show that the number of monitor states in the compliant product is $\prod_i (k_i + 1)$, compute it for r obligations of deadline 2, and explain why the exponential dependence on r cannot be avoided by a smaller monitor in the disjoint fragment, by the exact count of Chapter 14 of the first volume.

Exercise 14.16. Verify the count $|\mathcal{V}|$ of Result 14.9 for $(L, k) = (2, 5)$ by listing the pending states, and show that for $k \geq L+1$ the formula gives $(L+1)(L+2)/2+(L+1)+(k-L-1)(L+1)$.

Exercise 14.17. Using Result 14.10, find for $L = 4$, $p = 2$, $c_1 = 5$ the least deadline k_2 for which `alarm` is admissible, and say which of the two orders is used.

Exercise 14.18. Show that order (b) of Result 14.10 is feasible only when the first counter is at least $2L - p + 3$ before the alarm, and deduce that when $c_1 \le k_1 \le L + 1$ only the order (a) is ever used.

Exercise 14.19. Generalize Result 14.10 to three obligations with responses at positions 0, $L/2$ and L in a corridor with L even, and describe the set of orderings that must be tested.

Exercise 14.20. Explain why the product-graph computation of Result 14.2 would find the admissibility rule of Result 14.10 without being told it, and what the closed form adds that the computation does not.

Chapter 15

The Cost of Future Admissibility

Question. What does it cost to preserve at least one acceptable path, and what does it cost to preserve a broad range of alternatives?

Established sources. Constrained optimal control; discounted dynamic programming; viability kernels (Chapter 3).

New results. Optimizing subject to staying in the constraint set is optimizing on the viability kernel with the actions that stay in it (Result 15.1); the price of viability is zero exactly when an unconstrained optimum already stays in the kernel and can be arbitrarily large otherwise (Result 15.2); lookahead of $|X|-1$ steps is necessary and sufficient to detect non-viability (Result 15.3); preserving m options at every time is viability in a nested family of kernels whose prices are nondecreasing in m (Results 15.4 and 15.5).

Proof obligations. None.

Depends on. Chapter 14.

Chapter 3 established when a path that stays acceptable forever exists. Whether the system takes such a path is a separate matter, because the cheapest, fastest, or most profitable behaviour often leaves the acceptable region, and because keeping the possibility of staying acceptable is not the same thing as keeping many possibilities. This chapter prices the two. The first price is the cost of keeping at least one acceptable continuation; the second is the cost of keeping several.

15.1 The price of viability

Let X be finite, with feasibility F, deterministic transition δ, and constraint set K, as in Chapter 14. Let $\ell : X \times U \to \mathbb{R}_{\geq 0}$ be a stage cost and $\gamma \in [0, 1)$ a discount factor. A feasible action sequence $u_0, u_1, \ldots$ from x_0 generates a trajectory and a discounted cost $\sum_t \gamma^t \ell(x_t, u_t)$. Define the unconstrained optimum

$$J(x) = \min\Big\{ \textstyle\sum_t \gamma^t \ell(x_t, u_t) \Big\}$$

over all feasible sequences from x (the minimum exists since the system is finite), and the viable optimum $J_K(x)$ as the minimum over feasible sequences whose trajectory stays in K at all times, with $J_K(x) = \infty$ if there is none. The *price of viability* at x is $\pi(x) = J_K(x) - J(x) \in [0, \infty]$.

New result 15.1 (Viable optimization is optimization on the kernel)**.** *Let $\mathcal{K} = \mathrm{Viab}(K)$ and let $F_{\mathcal{K}}(x) = \{u \in F(x) : \delta(x,u) \in \mathcal{K}\}$ for $x \in \mathcal{K}$. Then $F_{\mathcal{K}}(x) \neq \varnothing$ for $x \in \mathcal{K}$, and J_K is, on $\mathcal{K}$, the optimal value of the finite discounted problem with states $\mathcal{K}$, actions $F_{\mathcal{K}}$ and cost ℓ; elsewhere $J_K = \infty$. It is the unique fixed point of $(TV)(x) = \min_{u \in F_{\mathcal{K}}(x)}[\ell(x,u) + \gamma V(\delta(x,u))]$ on $\mathcal{K}$, computed by value iteration.*

Assumptions: Finite X; $0 \leq \gamma < 1$.

Proof. If $x \in \mathcal{K}$ then by Result 3.2 some action leads into $\mathcal{K}$, so $F_{\mathcal{K}}(x) \neq \varnothing$. A trajectory from x that stays in K forever has every one of its states in $\mathcal{K}$, since the tail starting at each state is a K-trajectory (Result 3.2(b)); so each of its actions belongs to $F_{\mathcal{K}}$ at the state where it is taken. Conversely every action sequence that uses only actions of $F_{\mathcal{K}}$ from $x \in \mathcal{K}$ stays in $\mathcal{K} \subseteq K$. Hence the sequences counted by J_K are exactly those of the restricted problem, and for $x \notin \mathcal{K}$ there are none. The restricted problem is a finite discounted deterministic problem, so Result 4.2 applies. □

This is the deterministic form of Result 4.5. Feasibility first, then performance on what feasibility leaves.

New result 15.2 (The price of viability)**.** *(a) $\pi(x) = 0$ if and only if some unconstrained optimal trajectory from x stays in K.*
(b) $\pi(x) = \infty$ if and only if $x \notin \mathrm{Viab}(K)$.
(c) For every $B > 0$ there is a system and a state $x \in \mathrm{Viab}(K)$ with $\pi(x) > B$.

Assumptions: As in Result 15.1.

Proof. (a) $J_K(x) = J(x)$ if and only if a K-trajectory attains the unconstrained minimum. (b) is Result 15.1, since J is finite. (c) Take states s, d, g with $K = \{s, g\}$. At s the action `fast` leads to d at cost 0, the action `slow` leads to g at cost c, and the action `wait` returns to s at cost c. At d the only action is a self-loop of cost 0, and at g the only action is a self-loop of cost c. Then $J(s) = 0$ by going to d and staying. The state s is in the kernel by waiting forever. A K-trajectory from s pays c at every step, since each of `wait`, `slow`, and the loop at g costs c, so $J_K(s) = c/(1-\gamma)$ and $\pi(s) = c/(1-\gamma)$, which exceeds B for large c. □

The first claim is the pleasant case: the optimum of the problem already respects the constraint, and viability is free. The third is the general condition, since nothing in the definition of the kernel relates to the stage cost. A system that is never asked to price its constraints can look as though it satisfies them for free.

15.2 The cost of foresight

A controller that checks only a bounded horizon cannot distinguish a state that has a long-lived future from one whose future ends. Say that x is *N-step viable* if some feasible sequence of N actions from x has a trajectory that stays in K at every one of its $N+1$ states. By the iteration of Result 3.2(c), the N-step viable states are exactly the set V_N of that iteration.

New result 15.3 (Lookahead needed to detect non-viability)**.** *If $|X| = n$, then $V_n = \mathrm{Viab}(K)$: a state is viable if and only if it is n-step viable. For every $n \geq 1$ there is a system with n states and a state that is $(n-1)$-step viable and not viable.*

Assumptions: Finite X.

Proof. The first claim is Result 3.2(c). For the second take states $x_0, \dots, x_{n-1}$ with a single action from x_i to x_{i+1} for $i < n-1$, no action at x_{n-1}, and $K = X$. The trajectory $x_0, \dots, x_{n-1}$ has $n-1$ actions and stays in K, so x_0 is $(n-1)$-step viable, while no trajectory has n actions, so $x_0 \notin V_n = \mathrm{Viab}(K) = \varnothing$. □

A planner that looks fewer than $n-1$ steps ahead in a system of n states may therefore accept a state with no future. The bound is tight, and it grows with the number of states and not with the connectivity of the graph.

15.3 Preserving many options

Preserving viability preserves one acceptable continuation. A system that must keep several, so as to be able to respond to the failure of any one, has to do something stronger. For $m \geq 1$ let

$$\mathrm{Pre}_m(Z) = \big\{x : \ \big|\{u \in F(x) : \delta(x,u) \in Z\}\big| \geq m\big\},$$

the set of states with at least m distinct feasible actions leading into Z. The *m-option kernel* W_m is the greatest set $Z \subseteq K$ with $Z \subseteq \mathrm{Pre}_m(Z)$.

New result 15.4 (The m-option kernels)**.** *(a) $W_1 = \mathrm{Viab}(K)$ and $W_{m+1} \subseteq W_m$. The sets W_m are computed by the decreasing iteration $Z_0 = K$, $Z_{j+1} = K \cap \mathrm{Pre}_m(Z_j)$, which stabilizes after at most $|K|$ steps.*

(b) Every state of W_m has at least m^k feasible action sequences of length k whose trajectories stay in W_m, for every k. Hence the growth rate of feasible histories (Chapter 11) from a state of W_m is at least m.

(c) Define $m^*(x) = \max\{m : x \in W_m\}$ (the option capacity*, 0 if $x \notin W_1$). A state from which m options can be preserved forever has $m^*(x) \geq m$.*

Assumptions: Finite X and U.

Proof. (a) $\mathrm{Pre}_1 = \mathrm{Pre}$, so W_1 is the greatest fixed point of Chapter 3. Since $\mathrm{Pre}_{m+1}(Z) \subseteq \mathrm{Pre}_m(Z)$, a set $Z \subseteq K \cap \mathrm{Pre}_{m+1}(Z)$ also satisfies $Z \subseteq K \cap \mathrm{Pre}_m(Z)$, hence lies in W_m as the union of all such sets; so $W_{m+1} \subseteq W_m$. The operator is monotone, so the iteration is decreasing and stabilizes at the greatest fixed point on a finite set, after at most $|K|$ strict decreases. (b) Induct on k: each $x \in W_m$ has m distinct actions leading into W_m, and from each successor there are at least m^{k-1} sequences; sequences with different first actions are different, giving $m \cdot m^{k-1}$. The growth rate is at least $\lim m^{k/k} = m$. (c) is the definition. □

New result 15.5 (The price of options is nondecreasing)**.** *Let $J_m(x)$ be the minimum discounted cost over feasible sequences from x whose trajectory stays in W_m at every time, with $J_m = \infty$ if none exists. Then $J(x) \leq J_1(x) \leq J_2(x) \leq \cdots$ for every x, and $J_m(x) = \infty$ for $m > m^*(x)$. The price $\pi_m = J_m - J$ is therefore nondecreasing in m. Moreover each π_m can be computed as in Result 15.1, with W_m in place of* $\mathrm{Viab}(K)$*, and the strict inequalities can occur.*

Assumptions: As above.

Proof. $W_{m+1} \subseteq W_m \subseteq K$, so every feasible sequence whose trajectory stays in W_{m+1} stays in W_m, and the minimum over a smaller set is no smaller. For $x \notin W_m$ there is no trajectory in W_m from x, so $J_m(x) = \infty$. The computation is that of Result 15.1, applied to the system with constraint set W_m, whose kernel is W_m itself because $W_m \subseteq \mathrm{Pre}_m(W_m) \subseteq \mathrm{Pre}(W_m)$. For strictness see the example below. □

Example 15.6 (One narrow route and two wide ones)**.** States s, n, w with $K = \{s, n, w\}$. At s the actions are `narrow`, leading to n at cost 0, and $\texttt{wide}_1$, $\texttt{wide}_2$, both leading to w at cost 2. At n the only action is a self-loop of cost 0. At w there are two self-loop actions of cost 0. Then $W_1 = \{s, n, w\}$. In W_2 the state n is absent, since it has one action; w has two self-loop actions into w, so stays; s has the two wide actions into w, so stays: $W_2 = \{s, w\}$. $W_3 = \varnothing$, since w has only two actions. Hence $J(s) = J_1(s) = 0$, $J_2(s) = 2$, and $J_3(s) = \infty$: preserving one acceptable path is free, preserving two costs 2, and preserving three is impossible, and $m^*(s) = 2$.

The example shows what the narrow route is: it is viable, indeed cheapest, and it commits the system to a single continuation. The price of keeping a second continuation open is the cost of the wide route. The option capacity m^* is a structural number, computed by iteration, that says how much redundancy the system can ever have from a state.

15.4 Option capacity of the speed-limited line

The speed-limited line of Chapter 3 has states (x, v) with $0 \le x \le N$ and $|v| \le V$, controls $u \in \{-1, 0, 1\}$, the update $x^+ = x + v$, $v^+ = v + u$, and actions with $|v + u| > V$ infeasible. For $N = 10$ and $V = 6$ the viability kernel W_1 has 59 of the 143 states. The next question is how many of those states can keep two options open forever.

New result 15.7 (No second option on the unit-control line)**.** *For the speed-limited line with controls $\{-1, 0, 1\}$ the two-option kernel is empty, $W_2 = \varnothing$, for every N and V. Hence $m^*(x) = 1$ at every viable state: the system can preserve one acceptable continuation but never two.*

Assumptions: The dynamics above; $V \ge 1$.

Proof. Suppose $Z = W_2 \neq \varnothing$ and let $M = \max\{v : (x, v) \in Z\}$. If $M \ge 1$, take $(x, M) \in Z$. Its feasible successors have velocities $M - 1, M, M + 1$, and those in Z have velocity at most M, so the two successors required by $Z \subseteq \mathrm{Pre}_2(Z)$ are exactly those with velocities $M - 1$ and M. In particular $(x + M, M) \in Z$, and repeating the argument gives $(x + jM, M) \in Z$ for every j, which leaves $[0, N]$ for $j > N/M$, a contradiction. Hence $M \le 0$, and by symmetry $\min\{v\} \ge 0$, so every state of Z has $v = 0$. But a state $(x, 0)$ has only one successor with velocity 0, so it does not have two successors in Z, a contradiction. □

The cause is inertia combined with unit authority: any state with nonzero speed is committed to drifting in one direction, and the only states with no drift offer a single way of staying put. More authority changes the answer. With controls $\{-2, \ldots, 2\}$ (same N, V, with the same feasibility rule) the iteration gives kernels of sizes 77, 59 and 0 for $m = 1, 2, 3$, so that $m^* \le 2$ and exactly the 59 states of W_2 can preserve two options. The numbers say that doubling the control authority both enlarges the viability kernel (from 59 to 77 states) and

creates a redundancy that did not exist: a controller that is required to keep a fallback open, in the sense of Result 15.4, needs more actuation than one that is required only to remain viable. Redundancy of options is a design requirement that has to be paid for in actuation, and the price π_2 of Result 15.5 is infinite on the unit-control line and finite on the double-control line.

Exercises

Exercise 15.8. Compute W_1, W_2, W_3 for the graph on $\{a, b, c\}$ in which every state has actions to the other two states and a self-loop, and compute m^*.

Exercise 15.9. In Example 15.6 let the two wide actions have cost $c > 0$ in place of 2. Show that W_1, W_2 and $m^*(s)$ do not depend on c while $J_2(s) = c$, so that the price of a second option is unbounded although the structure is unchanged. Then let the wide actions cost $c_1 \neq c_2$ and decide whether $J_2(s)$ is min, max or the sum of the two costs, justifying the answer from the definition of J_2.

Exercise 15.10. Show that $m^*(x)$ can be strictly smaller than the maximum number of feasible actions at x, and give a system in which $m^*(x) = 1$ although every state has at least two feasible actions.

Exercise 15.11. Show that adding to F an action that leaves K and cannot return leaves the kernel and J_K unchanged, and can increase the price of viability, and explain why a larger menu can make a constraint more expensive.

Exercise 15.12. Suppose actions leading into $\mathrm{Viab}(K)$ have a one-time cost q per newly incurred obligation (Chapter 14). Write the restricted Bellman operator on the compliant product and say how the price of viability depends on the deadline.

Exercise 15.13. Compute W_1 and W_2 by hand for the unit-control line with $N = 2$ and $V = 1$, and check Result 15.7.

Exercise 15.14. Modify Result 15.7 so that the controller also has a brake `hold` that sets $v^+ = 0$ from any state. Show that W_2 is then nonempty, and find the states of W_2 when $N = 3$, $V = 1$.

Exercise 15.15. Prove that $W_m(\Sigma') \supseteq W_m(\Sigma)$ when the system Σ' has every feasible action of Σ together with additional ones, and explain why this contrasts with Exercise 15.11 on a larger menu making a constraint costly.

Exercise 15.16. Give a system with $m^*(x) = 2$ in which every viable trajectory from x eventually enters a state of $m^* = 1$, so that the capacity of x is not preserved by any policy.

Chapter 16

Corrective Reachability, Degradation, and Fallback Regions

Question. Does a path exist from an undesirable state to an acceptable region, and at what cost?

Established sources. Backward reachability and attractors; potential functions; graceful degradation (Chapter 17 of the first volume).

New results. The recoverable region is the least fixed point of the backward step and is closed under predecessors (Result 16.1); a local progress function guarantees recovery, the step distance is one, and greedy descent of other potentials, even with bounded lookahead, can leave the recoverable region irreversibly (Results 16.2 and 16.3); forced recovery against an adversary is a smaller region than possible recovery (Result 16.4); fallback regions for the shedding sets are down-closed families that are polynomial to decide under a unique support and NP-complete in general (Result 16.5).

Proof obligations. None.

Depends on. Chapters 2, 3, and 14.

Preserving viability is a statement about the future of a state that has not yet gone wrong. Real systems go wrong: a constraint is violated, a component is lost, a deadline is missed. The question then is whether the system can be brought back to an acceptable region, how long it takes, and what the system must not do while trying. This chapter treats the question for a finite system, compares recovery by local corrections with recovery by trajectory-level guarantees, and computes the regions of fault patterns from which the degradation of Chapter 17 of the first volume can still deliver the required service.

16.1 The recoverable region

Let the transition graph be as in Chapter 2 and let $T \subseteq X$ be a target set of acceptable states. For $Z \subseteq X$ put $\mathrm{Pre}(Z) = \{x : \exists u \in F(x),\ \delta(x,u) \in Z\}$ as in Chapter 3.

New result 16.1 (The recoverable region)**.** *Let* $\mathrm{Rec}_0 = T$ *and* $\mathrm{Rec}_{k+1} = \mathrm{Rec}_k \cup \mathrm{Pre}(\mathrm{Rec}_k)$*, and let* $\mathrm{Rec}(T) = \bigcup_k \mathrm{Rec}_k$*.*

(a) $\mathrm{Rec}(T)$ *is the least set containing* T *and closed under* Pre*, and equals* $\{x : R(x) \cap T \neq \varnothing\}$*.*

(b) $x \in \mathrm{Rec}_k$ *if and only if* T *is reachable from* x *in at most* k *steps, so* $d_T(x) = \min\{k : x \in \mathrm{Rec}_k\}$ *is the step distance to* T*. The iteration stabilizes after at most* $|X| - 1$ *steps.*

(c) $\mathrm{Rec}(T)$ *is closed under predecessors: if* $x \to y$ *and* $y \in \mathrm{Rec}(T)$ *then* $x \in \mathrm{Rec}(T)$*. Its complement is closed under successors.*

(d) With nonnegative edge costs, the cheapest cost of reaching T *from every state is computed by Dijkstra's algorithm on the reversed graph from a source joined to all of* T*, in time* $O(m + n \log n)$*.*

Assumptions: Finite X.

Proof. (b) By induction on k: $x \in \mathrm{Rec}_{k+1}$ if and only if $x \in T$ or some successor lies in Rec_k, that is, if and only if T is reachable in at most $k+1$ steps. A shortest path to T visits no state twice, hence has at most $|X| - 1$ edges, so the iteration is stationary after $|X| - 1$ steps. (a) follows from (b): $\mathrm{Rec}(T)$ contains T, is closed under Pre, and every closed set containing T contains each Rec_k by induction. (c) If $x \to y$ then anything reachable from y is reachable from x. (d) is Result 12.1 (or the standard algorithm) applied to the reversed graph. □

The complement of $\mathrm{Rec}(T)$ is the set of states from which the acceptable region is out of reach: a trap in the language of Chapter 8. It is a union of components that can only reach each other, and the recoverable region is the union of the components that can reach T. Every action leaving the recoverable region for its complement is an irreversible edge of the ratchet.

16.2 Local corrections and trajectory guarantees

A common control practice is to reduce an error measure: choose, at each step, an action that decreases some function φ measuring distance from acceptability. The guarantee that such a practice provides is as follows.

New result 16.2 (Progress functions)**.** *Let* $\varphi : X \to \mathbb{N}$ *have* $\varphi(x) = 0$ *exactly on* T*, and suppose that every* $x \notin T$ *has a feasible action* u *with* $\varphi(\delta(x, u)) < \varphi(x)$ *(local progress). Then* $X \subseteq \mathrm{Rec}(T)$ *and* $d_T(x) \le \varphi(x)$ *for all* x*, and any policy that chooses a progress action at every state reaches* T *within* $\varphi(x)$ *steps. Conversely on* $\mathrm{Rec}(T)$ *the step distance* d_T *itself is a progress function, with progress by exactly one step.*

Assumptions: Finite X.

Proof. A progress action sequence strictly decreases φ in $\mathbb{N}$ until $\varphi = 0$, that is, until it reaches T, in at most $\varphi(x)$ steps. For $x \in \mathrm{Rec}(T) \setminus T$ a shortest path to T starts with an action to a state of distance $d_T(x) - 1$. □

The theorem gives a sufficient condition for local rules to work: the potential must have the progress property at every state. The next result shows that without it local rules can fail, and that more lookahead does not remove the possibility.

New result 16.3 (Greedy descent can leave the recoverable region)**.** *For every* $L \ge 1$ *there is a finite system, a target* $T = \{t\}$*, a function* φ *with* $\varphi^{-1}(0) = T$*, and a state* $a \in \mathrm{Rec}(T)$ *such that the* L*-step greedy rule, which chooses the action from which some continuation of* L *steps reaches the smallest value of* φ*, moves to a state outside* $\mathrm{Rec}(T)$*, from which* T *can never be reached.*

Assumptions: Constructed.

Proof. Take states $a, b_1, \dots, b_{L+1}, c_1, \dots, c_{L+1}, t$. Edges: $a \to b_1$, $b_j \to b_{j+1}$ for $j \le L$, $b_{L+1} \to b_{L+1}$ (a self-loop); $a \to c_1$, $c_j \to c_{j+1}$ for $j \le L$, $c_{L+1} \to t$, $t \to t$. Let $\varphi(t) = 0$, $\varphi(a) = L + 2$, $\varphi(b_j) = L + 2 - j$, and $\varphi(c_j) = L + 2 + j$. Then $a \in \mathrm{Rec}(T)$ via the c-chain, in $L + 2$ steps. The L-step lookahead from a through b_1 reaches b_L with $\varphi = 2$, while through c_1 it reaches c_L with $\varphi = L+2+L > 2$; since t is $L+2$ steps away along the c-chain it is not seen. The greedy rule therefore chooses b_1. The states b_j have only the b-chain and its self-loop as successors, so none of them can reach t: the edge $a \to b_1$ is an irreversible edge out of $\mathrm{Rec}(T)$. □

The construction is the failure that the ratchet predicts: a locally improving move can be an irreversible one, and no fixed horizon of local information is enough to detect it. The remedy is not a longer horizon but the structure of Chapter 2: knowing which edges cross out of $\mathrm{Rec}(T)$.

16.3 Possible and forced recovery

When the transition is chosen partly by an adversary or by the environment, reachability splits. Let the transition be a set-valued $\delta(x, u) \subseteq X$ with the adversary choosing the successor, and define

$$\mathrm{Pre}_{\exists\forall}(Z) = \{x : \exists u \in F(x),\ \delta(x, u) \subseteq Z\}.$$

The *attractor* of T is $\mathrm{Attr}(T) = \bigcup_k \mathrm{Attr}_k$ with $\mathrm{Attr}_0 = T$ and $\mathrm{Attr}_{k+1} = \mathrm{Attr}_k \cup \mathrm{Pre}_{\exists\forall}(\mathrm{Attr}_k)$. The possibly recoverable region $\mathrm{Rec}(T)$ is defined as before with $\mathrm{Pre}(Z) = \{x : \exists u,\ \delta(x, u) \cap Z \neq \varnothing\}$.

New result 16.4 (Forced recovery is smaller than possible recovery)**.** $\mathrm{Attr}(T) \subseteq \mathrm{Rec}(T)$*. The controller has a strategy that reaches T against every adversary within k steps if and only if $x \in \mathrm{Attr}_k$, and then a stationary strategy suffices. The inclusion can be strict.*

Assumptions: Finite X and U; the adversary chooses a successor after seeing the controller's action.

Proof. Since $\delta(x, u) \subseteq Z$ with $\delta(x, u)$ nonempty implies $\delta(x, u) \cap Z \neq \varnothing$, each $\mathrm{Attr}_k \subseteq \mathrm{Rec}_k$. For the characterization, induct on k. $\mathrm{Attr}_0 = T$. If $x \in \mathrm{Attr}_{k+1} \setminus T$, the action u with $\delta(x, u) \subseteq \mathrm{Attr}_k$ forces the next state into Attr_k, from which a strategy reaches T in k more steps. Conversely a strategy that forces T within $k + 1$ steps from $x \notin T$ has a first action u all of whose successors are forced within k steps, hence lie in Attr_k. The choice of u depends on the state and the rank k only, so it is stationary on ranks. For strictness take x with one action leading to $\{t, z\}$ and z with a self-loop only: $x \in \mathrm{Rec}(\{t\})$ but $x \notin \mathrm{Attr}(\{t\})$. □

The difference between the two regions is the difference between a statement that recovery is possible and one that it is guaranteed. The recoverable region is the right object for a system whose environment is neutral, and the attractor for one whose environment is adversarial or unmodelled.

16.4 Fallback regions for degradation

Return to the dependency systems of Chapter 17 of the first volume. Capabilities C with rules $\mathcal{R}$ and a required set Req are given. Each capability c carries a load $\ell(c) \ge 0$ (the resource it uses when running), and the system has capacity Γ. After a set $F \subseteq C$ of capabilities is lost

to faults, the system may shed further capabilities so that what it still provides is $\mathrm{Prov}(D)$ for a shed set $D \supseteq F$. A configuration is a *fallback* for F if D is admissible ($\mathrm{Req} \subseteq \mathrm{Prov}(D)$) and the load $\ell(\mathrm{Prov}(D)) = \sum_{c \in \mathrm{Prov}(D)} \ell(c) \le \Gamma$. The *fallback region* is $\mathrm{FB}(\Gamma) = \{F \subseteq C :$ a fallback exists for $F\}$.

New result 16.5 (Fallback regions)**.** *(a)* $F \in \mathrm{FB}(\Gamma)$ *if and only if there is a support* S *with* $S \cap F = \varnothing$ *and* $\ell(S) \le \Gamma$. *Hence* $\mathrm{FB}(\Gamma) = \bigcup_{S:\ \ell(S) \le \Gamma} 2^{C \setminus S}$ *over supports* S*, and* $\mathrm{FB}(\Gamma)$ *is closed under taking subsets and increasing in* Γ.

(b) If every capability heads exactly one rule and $\mathrm{Req} \subseteq \mathrm{Prov}(\varnothing)$*, with unique minimal support* S^**, then* $\mathrm{FB}(\Gamma) = 2^{C \setminus S^*}$ *if* $\ell(S^*) \le \Gamma$ *and* $\mathrm{FB}(\Gamma) = \varnothing$ *otherwise; membership is decided in linear time.*

(c) Deciding whether $F \in \mathrm{FB}(\Gamma)$ *is NP-complete, even with* $F = \varnothing$.

(d) The number of maximal elements of $\mathrm{FB}(\Gamma)$ *can be exponential in* $|C|$.

Assumptions: Finite C and $\mathcal{R}$; loads nonnegative.

Proof. (a) Let $D \supseteq F$ be admissible with $\ell(\mathrm{Prov}(D)) \le \Gamma$. By the characterization of admissible shedding in Chapter 17 of the first volume (part (b)), $S = \mathrm{Prov}(D)$ is a support with $S \cap D = \varnothing$, so $S \cap F = \varnothing$. Conversely let S be a support with $S \cap F = \varnothing$ and $\ell(S) \le \Gamma$, and put $D = C \setminus S \supseteq F$. By the same characterization $D \cap S = \varnothing$ gives $S \subseteq \mathrm{Prov}(D)$, and the defining operator never adds a member of D, so $\mathrm{Prov}(D) \subseteq C \setminus D = S$. Thus $\mathrm{Prov}(D) = S$, D is admissible, and the load is $\ell(S) \le \Gamma$. The union form and the closure properties follow.

(b) By part (c) of that characterization every support contains S^*, so the least load of a support is $\ell(S^*)$, and a support disjoint from F exists if and only if $S^* \cap F = \varnothing$. Membership is the test $S^* \cap F = \varnothing$ and $\ell(S^*) \le \Gamma$, and S^* is computed by one pass over the derivation of Req.

(c) Membership is in NP: a support S is a certificate, and the conditions (self-supporting, contains Req, disjoint from F, load at most Γ) are checked in polynomial time. For hardness reduce from SET COVER: given a universe $\{1, \dots, q\}$, sets $s_1, \dots, s_p \subseteq \{1, \dots, q\}$ and k, let $C = \{r\} \cup \{e_j\} \cup \{s_i\}$, with the single rule $(r, \{e_1, \dots, e_q\})$, rules $(e_j, \{s_i\})$ for each i with $j \in s_i$, and $(s_i, \varnothing)$; let $\mathrm{Req} = \{r\}$, $\ell(s_i) = 1$ and $\ell = 0$ elsewhere, $F = \varnothing$, $\Gamma = k$. A support contains r, all e_j, and a set of the s_i that covers every j; conversely any cover together with r and the e_j is a support. The least load of a support is the least size of a cover, so $\varnothing \in \mathrm{FB}(k)$ if and only if a cover of size at most k exists.

(d) Take $C = \{r\} \cup \{e_i, p_i, q_i : 1 \le i \le n\}$, the rule $(r, \{e_1, \dots, e_n\})$, rules $(e_i, \{p_i\})$ and $(e_i, \{q_i\})$, primitives p_i, q_i with load 1, and load 0 elsewhere, $\mathrm{Req} = \{r\}$, $\Gamma = n$. The minimal supports choose p_i or q_i for each i, 2^n in all, each of load n, and by (a) their complements are 2^n distinct maximal elements of $\mathrm{FB}(n)$ (no complement contains another, since the supports are incomparable). □

The fallback region says which fault patterns the system can survive by degrading; it is the object that a scalar capacity number tried and failed to capture in Chapter 17 of the first volume. Part (b) shows when the number suffices, part (d) why the region cannot be tabulated, and part (c) why even its emptiness is a hard question.

16.5 A navigation stack as a worked fallback region

Take seven capabilities: `nav` (required), `pos` and `map`, and four primitives `gps`, `imu`, `odo`, `lidar`. The rules are `nav` ← `pos`, `nav` ← `map`, `pos` ← `gps`, `pos` ← {`imu`, `odo`}, `map` ← `lidar`, and the primitives have empty bodies. The loads are 3 for `gps`, 2 for `imu`, 1 for `odo`, 4 for `lidar` and 0 for the rest. The minimal supports of Req = {`nav`} are

$$S_1 = \{\texttt{nav}, \texttt{pos}, \texttt{gps}\} \qquad \text{(load 3)},$$
$$S_2 = \{\texttt{nav}, \texttt{pos}, \texttt{imu}, \texttt{odo}\} \qquad \text{(load 3)},$$
$$S_3 = \{\texttt{nav}, \texttt{map}, \texttt{lidar}\} \qquad \text{(load 4)}.$$

By Result 16.5(a), FB(Γ) is empty for $\Gamma < 3$, equals $2^{C\setminus S_1} \cup 2^{C\setminus S_2}$ for $3 \le \Gamma < 4$, and adds $2^{C\setminus S_3}$ from $\Gamma = 4$ on. Counting by inclusion and exclusion, $|2^{C\setminus S_1}| = 16$, $|2^{C\setminus S_2}| = 8$ and their intersection has 4 members, so $|\mathrm{FB}(3)| = 20$; adding S_3 gives $|\mathrm{FB}(4)| = 31$ of the 128 fault patterns, and the region does not grow for any larger Γ because no further minimal support exists. An exhaustive computation over all fault sets and all shed sets confirms these counts, and gives their distribution by size: for $\Gamma = 3$ the numbers of tolerated fault sets of sizes $0, 1, 2, 3, 4$ are $1, 5, 8, 5, 1$, and for $\Gamma \ge 4$ they are $1, 6, 13, 9, 2$. The least load needed after losing `gps` is 3 (use `imu` and `odo`), after losing `gps` and `imu` it is 4 (fall back to `lidar`), and after losing `gps` and `lidar` it is 3.

Two features deserve attention. First, the region is not a function of the number of faults: at $\Gamma = 3$ the pair {`gps`,`lidar`} is tolerated and the pair {`gps`,`imu`} is not. Second, raising the capacity from 3 to 4 enlarges the region by eleven fault patterns, among them one of size one: the loss of `pos`, which no support of load 3 survives, is tolerated at $\Gamma = 4$ through the `lidar` route. Nothing is gained beyond 4.

New result 16.6 (Fault tolerance is a transversal number)**.** *Let $P \subseteq C$ be the set of capabilities that can fail, and let $\mathcal{S}_\Gamma$ be the family of supports of load at most Γ. Every fault set $F \subseteq P$ with $|F| \le t$ lies in* FB(Γ) *if and only if no set of at most t elements of P meets every member of $\mathcal{S}_\Gamma$. Hence the largest such t is $\tau_P(\Gamma) - 1$, where $\tau_P(\Gamma)$ is the least size of a subset of P meeting all supports in $\mathcal{S}_\Gamma$ (taken to be ∞ if some support avoids P).*

Assumptions: Finite system; faults are subsets of P.

Proof. By Result 16.5(a), $F \in \mathrm{FB}(\Gamma)$ if and only if some support of load at most Γ is disjoint from F, that is, if and only if F does not meet every member of $\mathcal{S}_\Gamma$. Therefore a fault set of size at most t outside FB(Γ) exists exactly when a set of at most t elements of P meets all supports. The largest t with no such set is $\tau_P(\Gamma) - 1$. □

In the navigation stack with $P = \{\texttt{gps}, \texttt{imu}, \texttt{odo}, \texttt{lidar}\}$ the supports of load at most 3 meet P in {`gps`} and {`imu`, `odo`}, which needs two elements to meet both, so $t = 1$; at $\Gamma = 4$ the third member {`lidar`} raises the transversal number to three and $t = 2$. If the faults may also strike `nav` (which belongs to every support) the transversal number is one and no fault is tolerated, which is why the fault classes that a design claims to survive must be stated. The result converts a question about the region into a hitting-set problem, and by Result 16.5(c) the hitting-set problem is hard in general, so the number t cannot be expected to be cheap to compute for large systems; for the small, hand-built dependency systems in which a designer reasons about redundancy it is a short enumeration.

Exercises

Exercise 16.7. Compute Rec_k and d_T on the graph with edges $1 \to 2 \to 3 \to 4$, $3 \to 1$, $5 \to 4$, $6 \to 6$ and $T = \{4\}$, and identify the trap.

Exercise 16.8. Show that every progress function φ satisfies $d_T(x) \le \varphi(x)$ on X, so that d_T is the pointwise smallest progress function, and give a system in which a progress function exceeds d_T at a state.

Exercise 16.9. In Result 16.3 modify the construction so that the escape route c is cheaper in cost than the dead-end route, and explain why a cost-aware greedy rule still fails.

Exercise 16.10. Give a system in which $\mathrm{Attr}(T) = \varnothing$ although $\mathrm{Rec}(T) = X$, and compute the attractor when the adversary may choose only among successors that are not in T.

Exercise 16.11. For the dependency system of Result 16.5(d) with $n = 2$, list the four maximal elements of $\mathrm{FB}(2)$, and compute the largest t such that every fault set of size at most t lies in $\mathrm{FB}(2)$.

Exercise 16.12. For the navigation stack, compute the least load of a fallback for $F = \{\texttt{imu}\}$, $F = \{\texttt{odo}, \texttt{lidar}\}$ and $F = \{\texttt{gps}, \texttt{odo}, \texttt{lidar}\}$, and state which lie in $\mathrm{FB}(3)$.

Exercise 16.13. Add a rule $\texttt{pos} \leftarrow \{\texttt{wheel}\}$ with a new primitive `wheel` of load 2. Recompute the minimal supports, the break points of $\mathrm{FB}(\Gamma)$ in Γ, and the fault tolerance t at each.

Exercise 16.14. Prove that $\Gamma \mapsto \tau_P(\Gamma)$ is nondecreasing, takes finitely many values, and changes only at loads of minimal supports.

Exercise 16.15. Give a dependency system in which $\tau_P(\Gamma)$ increases by two at a single breakpoint, and explain what is added by the new minimal support.

Part VI

Adaptive control and institutions

Chapter 17

Aspect Relegation as Hybrid Switching

Question. When does handing a task to an automatic controller preserve the safety constraint, and when does it conceal errors?

Established sources. Switched systems and dwell-time analysis (Liberzon; Hespanha and Morse); hybrid automata; the robust kernel of Chapter 13.

New results. When both controllers keep one common set invariant, no condition on switching is needed (Result 17.2); when they keep different sets invariant, switching alone can leave the safe set (Result 17.3); under a common core, a handover property, a latency bound, and a dwell time, the safe set is preserved against faults of the automatic controller (Result 17.4); the hypotheses are sharp in a worked family (Result 17.5); the tolerable latency is a graph distance and fixes the least monitoring rate (Result 17.6).

Proof obligations. None. The conditional statement proposed in the outline is proved here in a corrected form.

Depends on. Chapters 3, 13, and 7.

To relegate an aspect of a task is to stop attending to it and let a procedure run: an autopilot holds altitude, a clerk's template files the form, a script rotates the keys. Relegation is a benefit when the procedure keeps the constraint without supervision and a hazard when it fails without notice. The question of the outline is when the system as a whole keeps its safe set under switching between supervised and relegated operation. The answer has three parts, which this chapter proves in turn: a case in which no condition on the switching is needed, a case in which switching alone is enough to break safety, and the conditions, on latency and dwell time, that restore it when the automatic controller can fail.

17.1 Switched systems with a fault model

Fix a finite controlled system (X, U, F, δ) and a safe set $K \subseteq X$ as in Chapter 3. An *explicit* controller κ_E and an *automatic* controller κ_A are state feedbacks, $\kappa(x) \in F(x)$. For a set S write S *invariant under* κ if $\delta(x, \kappa(x)) \in S$ for every $x \in S$. For $Z \subseteq X$ let $R_j(Z)$ be the set of states reachable from Z in exactly j steps by arbitrary feasible actions, with $R_0(Z) = Z$.

Definition 17.1 (Hybrid execution)**.** A configuration is a state x together with a mode: E (explicit controller active, with a counter $m \geq 0$ of consecutive steps spent in E), A_{ok}

(automatic controller active and nonfaulty), or A_{fault} (automatic controller active and faulty, with a counter $j \geq 1$ of faulty steps taken). In E the action is $\kappa_E(x)$; in A_{ok} it is $\kappa_A(x)$; in A_{fault} it is an arbitrary $u \in F(x)$. The mode changes as follows.

- *Fault*: from A_{ok} the fault may begin at any time.
- *Detection with latency* $d \geq 1$: from A_{fault} the system switches to E at some time, and at the latest after d faulty steps. From A_{ok} it may also switch to E at any time (a false alarm).
- *Handover with dwell time* $\tau \geq 0$: from E the system may switch to A_{ok} only after $m \geq \tau$.

An execution is any sequence consistent with these rules. The *latency* and the *dwell time* are thus constraints on the switching signal, and the fault is chosen by an adversary.

The adversary of the definition chooses when the fault begins, what the faulty controller does, and when a permitted switch happens; the guarantees below hold for every such choice.

17.2 When no condition is needed, and when it is not enough

New result 17.2 (A common invariant set survives any switching)**.** *If $S \subseteq K$ is invariant under both κ_E and κ_A, then every execution from S in which the automatic controller never faults stays in S, for every switching signal, with no condition on dwell time or latency.*

Assumptions: No faults.

Proof. Induct on the length: if $x_t \in S$ then $x_{t+1} = \delta(x_t, \kappa(x_t)) \in S$ for whichever $\kappa \in \{\kappa_E, \kappa_A\}$ is active. □

The statement of the outline is thus true and vacuous in its strong form: if both controllers keep the safe set invariant, the switching signal need not be restricted. Dwell time and latency enter only when the hypothesis is weakened to what is true in practice.

New result 17.3 (Distinct invariant sets do not compose)**.** *There is a system and controllers κ_1, κ_2 with sets $S_1, S_2 \subseteq K$ such that S_i is invariant under κ_i, but execution that switches from κ_1 to κ_2 at a state of S_1 leaves K at the first step after the switch, whatever the dwell time.*

Assumptions: Constructed.

Proof. Take $X = \{a, b, c\}$, $K = \{a, b\}$, $U = \{\text{stay}, \text{go}\}$ with $\delta(\cdot, \text{stay})$ the identity and $\delta(a, \text{go}) = \delta(b, \text{go}) = c$. Let $\kappa_1(a) = \text{stay}$, $\kappa_1(b) = \text{go}$, $\kappa_2(a) = \text{go}$, $\kappa_2(b) = \text{stay}$. Then $S_1 = \{a\}$ is invariant under κ_1 and $S_2 = \{b\}$ under κ_2. From a under κ_1, a switch to κ_2 yields the action go and the state $c \notin K$. A dwell time restricts how soon one may switch away from a controller, not what the controller does after the switch. □

17.3 The conditional theorem

The following hypotheses describe a relegated system whose controllers keep different sets invariant, but share a core to which the explicit controller can return the system, and whose automatic controller can be allowed to fail for up to d steps without leaving the region the explicit controller can recover.

New result 17.4 (Invariant preservation under relegation)**.** *Let $S_c \subseteq S_E \subseteq K$ satisfy:*

(H1) S_E *is invariant under* κ_E*, and* S_c *is invariant under both* κ_E *and* κ_A*.*
*(H2) (*Handover*) For every* $x \in S_E$ *the trajectory of* κ_E *from* x *is in* S_c *at time* τ*.*
*(H3) (*Latency*)* $R_j(S_c) \subseteq S_E$ *for every* $0 \le j \le d$*.*
Then every execution of Definition 17.1 with latency d *and dwell time* τ *that starts in mode* A_{ok} *at a state of* S_c*, or in mode* E *with* $m = 0$ *at a state of* S_E*, stays in* K *forever, whatever the fault does.*

Assumptions: Finite system; κ_A is arbitrary outside S_c; the fault is arbitrary within the feasible actions.

Proof. We show that every reachable configuration is of one of three kinds: (a) mode A_{ok} with $x \in S_c$; (b) mode A_{fault} with counter $j \le d$ and $x \in R_j(S_c)$; (c) mode E with counter m, $x \in S_E$, and either $m \ge \tau$ and $x \in S_c$, or $m < \tau$ and the κ_E-trajectory from x is in S_c after $\tau - m$ further steps. All three imply $x \in K$. In kind (c) with $m = 0$ the second alternative is exactly (H2) applied to $x \in S_E$ (for $\tau = 0$ it reads $x \in S_c$). The initial configurations are of these kinds.

(a) $\to$ (a) by κ_A, since S_c is κ_A-invariant. (a) $\to$ (b) with $j = 1$: the state is in $R_1(S_c)$. (a) $\to$ (c) by a false alarm: $x \in S_c \subseteq S_E$, and (H2) gives the alternative required at $m = 0$. (b), $j \to j + 1 \le d$: the state is in $R_{j+1}(S_c)$. (b) $\to$ (c): by (H3) $x \in R_j(S_c) \subseteq S_E$, and (H2) gives the alternative at $m = 0$; detection occurs by $j = d$, so no faulty step number exceeds d. (c) $\to$ (c): the state stays in S_E by (H1), and the alternative is propagated: a trajectory in S_c after $\tau - m$ steps is in S_c after $\tau - m - 1$ steps from its successor, and once in S_c the state stays there because S_c is κ_E-invariant. (c) $\to$ (a) is permitted only when $m \ge \tau$, so $x \in S_c$. □

The theorem shows what the dwell time is for: it is the time that the explicit controller needs to bring the state from the larger set it can recover (H1) to the core where the automatic controller is safe (H2). The latency is the time the automatic controller can spend misbehaving before the explicit controller takes over, and (H3) says that the state, after that much arbitrary behaviour, is still in the set from which the explicit controller can act. The case $S_E = S_c = S$, $d = \tau = 0$ is Result 17.2.

17.4 The hypotheses are sharp

New result 17.5 (Sharpness of dwell time and latency)**.** *Let* $X = \{-N-d-1, \dots, N+d+1\}$ *with* $N \ge 2d \ge 2$*,* $U = \{-1, 0, 1\}$*,* $\delta(x, u) = x + u$ *(truncated at the ends),* $K = [-N, N]$*. Let* $\kappa_E(x) = -\operatorname{sign}(x)$*,* $\kappa_A(x) = +1$ *if* $x < N - d$ *and* 0 *otherwise,* $S_E = K$*, and* $S_c = [-(N-d), N-d]$*.*

(a) (H1)–(H3) hold with latency d *and* $\tau = d$*, so the theorem guarantees safety.*
(b) With dwell time $d - 1$ *there is an execution that leaves* K*.*
(c) With latency $d + 1$ *there is an execution that leaves* K*.*

Assumptions: As stated.

Proof. (a) κ_E does not increase $|x|$, so $S_E = K$ is invariant and so is S_c. κ_A keeps S_c: for $x < N - d$ the next state is at most $N - d$, and at $x = N - d$ it stays. From $x \in K$ the controller κ_E reaches $|x| \le N - d$ within d steps (H2 with $\tau = d$). Arbitrary actions move the state by at most one per step, so $R_j(S_c) \subseteq [-(N-d) - j, N - d + j] \subseteq K$ for $j \le d$ (H3).

(b) Start in mode E at $x = N$. After $d - 1$ steps $x = N - d + 1 \notin S_c$. A switch to A_{ok} is permitted. Then κ_A gives 0 and the state stays; the fault begins and takes d steps of $+1$ (within latency d), reaching $N - d + 1 + d = N + 1 \notin K$.

(c) Start in A_{ok} at $x = N - d \in S_c$ and let the fault take $d + 1$ steps of $+1$, reaching $N + 1 \notin K$. □

17.5 Monitoring and the cost of detection

The latency is a property of the monitor: a monitor that samples the performance signal every s steps leaves up to s faulty steps undetected. Let the *exit distance* be $\delta^* = \min\{j : R_j(S_c) \not\subseteq S_E\} \in \mathbb{N} \cup \{\infty\}$, the least number of arbitrary steps that can carry the state from the core out of the recoverable set.

New result 17.6 (Tolerable latency is a graph distance)**.** *(H3) holds with latency d if and only if $d \le \delta^* - 1$. A monitor with sampling period s and cost c_{m} per sample therefore preserves safety if and only if $s \le \delta^* - 1$, at cost rate $c_{\mathrm{m}}/s \ge c_{\mathrm{m}}/(\delta^* - 1)$ per step. The exit distance is computed by a breadth-first search from S_c in time linear in the size of the transition graph. If $\delta^* \le 1$ no monitor can satisfy (H3), and if $\delta^* = \infty$ no monitoring is needed.*

Assumptions: Sampling detects a fault at the next sampling instant.

Proof. (H3) says that no $R_j(S_c)$ with $j \le d$ leaves S_E, that is, $d < \delta^*$. A fault that begins just after a sample is detected at the next one, so up to s faulty steps occur, and $d = s$. The rest is the definition of δ^* as a shortest-path distance to the complement of S_E. □

The requirement is that the monitor look at the system often enough that a fault cannot take it out of the recoverable set between looks. Where the performance signal does not reveal the fault at all (a fault that keeps every sampled performance value above its threshold), no finite latency exists, and the error is concealed: Exercise 17.9 asks for the construction.

17.6 The exact region of safe switching parameters

Result 17.5 shows that the choices $\tau = d$ and latency d are safe and that one step less of dwell, or one step more of latency, is not. The two parameters can be varied independently, and the set of pairs for which the system is safe has a simple description.

New result 17.7 (The safe region in the latency–dwell plane)**.** *In the family of Result 17.5 with $N \ge 2d \ge 2$, let the hybrid execution have latency $\ell \ge 0$ and dwell time $\tau \ge 0$, and start either in mode A_{ok} at a state of S_c or in mode E with $m = 0$ at a state of K. Every execution stays in K if and only if $\ell \le d$ and $\tau \ge \ell$.*

Assumptions: As in Result 17.5; the adversary controls the fault, its duration up to ℓ steps, and the switches the rules permit.

Proof. Sufficiency. Let $\ell \le d$ and $\tau \ge \ell$. Put $S_c' = [-(N - \ell), N - \ell]$, which contains S_c, and $S_E = K$. The controller κ_E leaves K and S_c' invariant, and κ_A leaves S_c' invariant: for $x < N - d$ it moves to at most $N - d \le N - \ell$, and for $x \ge N - d$ it holds the state. From $x \in K$ the controller κ_E reaches $|x| \le N - \ell$ within $\ell \le \tau$ steps, which is (H2) with S_c', and

$R_j(S'_c) \subseteq K$ for $j \le \ell$, which is (H3). Result 17.4 applies, with the starting states of the statement lying in $S_c \subseteq S'_c$ or in K.

Necessity. If $\ell \ge d+1$, start in A_{ok} at $x = N - d \in S_c$ and let the fault take $d+1$ steps of $+1$, reaching $N+1$. If $1 \le \ell \le d$ and $\tau \le \ell - 1$, start in E at $x = N$, run κ_E for τ steps to $x = N-\tau \ge N-\ell+1 \ge N-d+1$, switch to A_{ok} (permitted since $m = \tau$), where κ_A holds the state because $x \ge N-d$, begin the fault, and take ℓ steps of $+1$, reaching $N-\tau+\ell \ge N+1$. □

The characterization was checked by solving the safety game over the configuration graph (state, mode, counter) exhaustively, for $(N, d) = (10, 3)$, $(8, 2)$, $(12, 4)$ and $(6, 3)$ and for all ℓ, τ up to $d+2$, and the safe pairs agreed with the statement in every case. For $(N, d) = (10, 3)$ the safe pairs, with latency down the side and dwell time across, are as follows (S marks safe, a dot unsafe).

$\ell \backslash \tau$	0	1	2	3	4	5	6
0	S	S	S	S	S	S	S
1	·	S	S	S	S	S	S
2	·	·	S	S	S	S	S
3	·	·	·	S	S	S	S
4–6	·	·	·	·	·	·	·

The region is a triangle truncated at $\ell = d$. Its design reading is a pair of trade-offs. The monitor can be slower only up to the exit distance, no matter how long the dwell, because a latency beyond d lets the fault carry the state out before the explicit controller acts. Within that bound a quicker monitor lowers the dwell time that is needed, in proportion: one step of latency costs one step of dwell, and each step of dwell is a step of supervised operation that the relegation was meant to avoid. The cheapest safe configuration in terms of supervised time is therefore the fastest monitor that can be afforded, and the cost of attention shows up as the sampling rate of the monitor and not as the dwell time.

Exercises

Exercise 17.8. In the system of Result 17.5 compute δ^* and the least cost rate of a monitor with sampling cost c_{m}.

Exercise 17.9. Let the monitor flag a fault only when a performance function $p : X \to \mathbb{R}$ falls below a threshold θ. Give a system and a fault whose trajectory leaves S_E while $p \ge \theta$ at every state until the exit, and show that no latency bound holds.

Exercise 17.10. Show that Result 17.4 remains true if the automatic controller is allowed to be nondeterministic within a set of feasible actions that keep S_c invariant, and fails if the set may include an action leaving S_c while nonfaulty.

Exercise 17.11. Replace the false alarm rule by the requirement that a switch from A_{ok} to E occurs only after a fault; state which of the hypotheses can be dropped.

Exercise 17.12. For two relegated aspects sharing one explicit controller, with cores S_c^1, S_c^2, formulate the joint latency hypothesis and show that the tolerable latency of each is reduced by the other's faults in general.

Exercise 17.13. Using Result 17.7, compute for $(N, d) = (10, 3)$ the least total of latency plus dwell for which the system is safe and the starting latency is at least 1, and the number of safe pairs (ℓ, τ) with $\ell, \tau \leq 6$.

Exercise 17.14. Suppose that monitoring at latency ℓ costs c_{m}/ℓ per step and that each step of latency costs ah per step in expected loss, where h is the fault rate and a the loss per step of undetected fault. Show that the cost rate $c_{\mathrm{m}}/\ell + ah\ell$ is minimized over real $\ell > 0$ at $\ell^{*} = \sqrt{c_{\mathrm{m}}/(ah)}$, and that over the integers $\ell \leq \ell_{\max}$ allowed by Result 17.7 the minimizer is the better of $\lfloor \ell^{*} \rfloor$ and $\lceil \ell^{*} \rceil$ if $\ell^{*} \leq \ell_{\max}$ and is $\ell_{\max}$ otherwise.

Exercise 17.15. Prove the claim in the necessity argument that κ_A holds the state at $x \geq N - d$, and show by an example that for x between $N - d$ and N the controller κ_A is not itself safe against a fault.

Exercise 17.16. Replace κ_A by $\kappa'_A(x) = +1$ for $x < N - d'$ with $d' \neq d$, and find the safe region.

Chapter 18

Interfaces as Action Abstraction

Question. When does an interface preserve admissible reachability, as opposed to merely displaying a pre-existing action space?

Established sources. Simulation and bisimulation (Milner); weak simulation; interface automata (de Alfaro and Henzinger).

New results. An interface preserves admissible reachability from every admissible state if and only if the identity relation is a weak simulation in both directions on the admissible region (Result 18.2); neither direction holds in general, and strong one-step simulation is sufficient but not necessary; an interface can preserve possible reachability and destroy forced reachability (Result 18.6).

Proof obligations. None. The characterization proposed in the outline is proved here, with both directions.

Depends on. Chapters 2 and 11.

An interface presents some actions and hides others. A button groups several steps into one; a menu omits a rarely used command; a form walks the user through a sequence and forbids the rest. The designer asks whether the interface lets the user do what the system can do, and nothing more, within the region of states that the user is permitted to occupy. The question is not about what the interface looks like but about the reachability relation it induces, and the vocabulary of simulation applies. This chapter makes the question exact and answers it with a characterization, and then shows what the characterization does not capture: the user's ability to force an outcome rather than merely to make it possible.

18.1 Interfaces and admissible reachability

Definition 18.1 (Concrete system, admissible region, interface)**.** A *concrete system* is a finite set X of states, a finite set C of concrete actions, and a relation $x \xrightarrow{c} y$. An *admissible region* is a set $\mathrm{Ad} \subseteq X$. An *interface* presents a finite set A of actions, each with a *denotation* $[\![a]\!] \subseteq C^*$, a nonempty set of concrete action words. A single concrete action is a word of length one, and a macro is a longer word. The presented step is $x \xrightarrow{a}_{\mathrm{p}} y$ if for some $w \in [\![a]\!]$ there is a concrete path from x to y labelled w.

The presented step is existential: if the interface is used, some denoted behaviour is run, and the user is able to reach y by a when some behaviour does reach it. The macro can pass through states in between, and these need not be admissible.

Define on Ad the concrete admissible step relation $S_{\mathrm{c}} = \{(x, y) \in \mathrm{Ad}^2 : \exists c,\ x \xrightarrow{c} y\}$ and the presented admissible step relation $S_{\mathrm{p}} = \{(x, y) \in \mathrm{Ad}^2 : \exists a,\ x \xrightarrow{a}_{\mathrm{p}} y\}$ (only the endpoints are required to be admissible). The *admissible reach* of $x \in \mathrm{Ad}$ is $R_{\mathrm{c}}^{\mathrm{Ad}}(x) = \{y : (x, y) \in S_{\mathrm{c}}^*\}$ in the concrete system, and $R_{\mathrm{p}}^{\mathrm{Ad}}(x) = \{y : (x, y) \in S_{\mathrm{p}}^*\}$ under the interface, where * is the reflexive and transitive closure. The interface *preserves admissible reachability* if $R_{\mathrm{p}}^{\mathrm{Ad}}(x) = R_{\mathrm{c}}^{\mathrm{Ad}}(x)$ for every $x \in \mathrm{Ad}$.

New result 18.2 (Characterization of preservation)**.** *The interface preserves admissible reachability if and only if both of the following hold.*

*(a) (*Completeness*; the interface hides nothing admissible.) Every admissible concrete step is matched by a finite sequence of admissible presented steps:* $S_{\mathrm{c}} \subseteq S_{\mathrm{p}}^*$.

*(b) (*Soundness*; the interface shows nothing inadmissible.) Every admissible presented step is matched by a finite sequence of admissible concrete steps:* $S_{\mathrm{p}} \subseteq S_{\mathrm{c}}^*$.

That is, the identity relation on Ad *is a weak simulation of the concrete system by the interface, and of the interface by the concrete system. One-step matching,* $S_{\mathrm{c}} \subseteq S_{\mathrm{p}}$ *and* $S_{\mathrm{p}} \subseteq S_{\mathrm{c}}$*, is sufficient and is not necessary.*

Assumptions: Finite X and C.

Proof. Reflexive transitive closure is monotone and idempotent. ($\Leftarrow$) From (a), $S_{\mathrm{c}}^* \subseteq (S_{\mathrm{p}}^*)^* = S_{\mathrm{p}}^*$; from (b), $S_{\mathrm{p}}^* \subseteq S_{\mathrm{c}}^*$. So the closures coincide and so do the reaches. ($\Rightarrow$) If the reaches coincide for every $x \in \mathrm{Ad}$ then $S_{\mathrm{p}}^* = S_{\mathrm{c}}^*$, and $S_{\mathrm{c}} \subseteq S_{\mathrm{c}}^* = S_{\mathrm{p}}^*$ and $S_{\mathrm{p}} \subseteq S_{\mathrm{p}}^* = S_{\mathrm{c}}^*$. One-step matching implies (a) and (b) since $S \subseteq S^*$. It is not necessary: with $z \in \mathrm{Ad}$, $S_{\mathrm{c}} = \{(x, y), (x, z), (z, y)\}$ and $S_{\mathrm{p}} = \{(x, z), (z, y)\}$, the closures coincide though $(x, y) \notin S_{\mathrm{p}}$. □

The characterization has the shape that the outline anticipated: a simulation on the admissible region. It shows that both directions are needed and that they are independent of each other. They are also interpretable. Completeness is the property that a user of the interface is not locked out of an admissible state that a user of the concrete system could reach, and failures of completeness are the hidden options of Chapter 11. Soundness is the property that the interface does not display, as available, a state that cannot be reached admissibly.

Example 18.3 (A macro through an inadmissible state)**.** States d, v, s, p, u with concrete actions `validate`: $d \to v$, `skip`: $d \to u$, `save-unchecked`: $u \to s$, and `publish`: $s \to p$. The admissible region is $\mathrm{Ad} = \{d, v, s, p\}$, excluding u (a draft saved without validation). The concrete admissible reach of d is $\{d, v\}$, since s can only be reached through u. An interface offers the presented action *quick save* with denotation {`skip save-unchecked`} and *publish* as a single action. Then $(d, s) \in S_{\mathrm{p}}$, and the interface's admissible reach of d is $\{d, v, s, p\}$, which contains states not admissibly reachable: soundness fails, although every presented endpoint is in Ad. The failure is in the middle of the macro.

Example 18.4 (A missing button)**.** In the same concrete system without `skip` and `save-unchecked`, add `save`: $v \to s$. An interface that presents `validate` and `publish` but not `save` has $S_{\mathrm{p}} = \{(d, v), (s, p)\}$, so the admissible reach of d is $\{d, v\}$ and that of d in the concrete system is $\{d, v, s, p\}$. Completeness fails: the pair $(v, s) \in S_{\mathrm{c}}$ is not in S_{p}^*. The interface displays a subset of a pre-existing action space and loses the states beyond v.

Example 18.5 (Admissible region matters)**.** Let a concrete step $x \xrightarrow{c} y$ with $y \notin \mathrm{Ad}$ be hidden by the interface. The pair $(x, y) \notin S_{\mathrm{c}}$ because $y \notin \mathrm{Ad}$, so completeness imposes no

requirement on it. The interface may hide any step that leaves the admissible region without losing reachability within it, and may need to hide it for soundness.

18.2 Possible and forced reachability

The characterization concerns what the user can reach if the system cooperates. When the interface merges actions with different outcomes, the user can make an outcome possible but not certain. Let the concrete system be deterministic, $x \xrightarrow{c} f(x,c)$, and consider the *demonic* reading of an interface whose denotations are single letters: the presented action a at x is executed by the system by choosing some $c \in [\![a]\!]$ adversarially, with outcome set $\mathrm{Out}(x,a) = \{f(x,c) : c \in [\![a]\!]\}$. For a target $T \subseteq \mathrm{Ad}$ let $\mathrm{Rec_c}(T)$ be the recoverable region of Chapter 16 in the concrete system and $\mathrm{Attr_p}(T)$ the attractor of T in the interface, where the interface action leads to $\mathrm{Out}(x,a)$ and the adversary picks the successor.

New result 18.6 (An interface can preserve possible reachability and destroy forced reachability)**.** *For every deterministic concrete system and interface with single-letter denotations,* $\mathrm{Attr_p}(T) \subseteq \mathrm{Rec_c}(T)$. *The inclusion can be strict even when the interface is complete and sound in the sense of Result 18.2 with respect to possible reachability.*

Assumptions: Deterministic concrete system; single-letter denotations.

Proof. Induct on the rank: if $\mathrm{Out}(x,a) \subseteq \mathrm{Attr_p^k}$ then every $c \in [\![a]\!]$ leads to a state in $\mathrm{Attr_p^k} \subseteq \mathrm{Rec_c}(T)$ by the induction hypothesis, and $[\![a]\!] \neq \varnothing$, so $x \in \mathrm{Rec_c}(T)$. For strictness take states x, t, z and two concrete actions $c_1 : x \to t$ and $c_2 : x \to z$, with z a trap, and an interface with the single presented action a denoting $\{c_1, c_2\}$ at x. The interface's possible reach from x contains t and z, as does the concrete reach, so possible reachability is preserved; but $\mathrm{Out}(x,a) = \{t,z\}$, so $x \notin \mathrm{Attr_p}(\{t\})$ while $x \in \mathrm{Rec_c}(\{t\})$. □

The second example shows the cost of grouping: the user of the interface cannot choose between the outcomes that the grouped actions produce. This is the point at which an interface is more than a display of a pre-existing menu. An interface that merges two concrete actions has changed the system that the user controls, even if it is complete and sound for possible reachability, and the right notion of preservation for control is a bisimulation-like condition on outcomes rather than a simulation.

18.3 The smallest interface and the cost in steps

Preservation of admissible reachability fixes the closure S_p^* and leaves the interface free to choose which steps to present. Since the soundness half of Result 18.2 only requires $S_\mathrm{p} \subseteq S_\mathrm{c}^*$, a macro may present any pair of the concrete closure, whether or not a single concrete step joins it. The smallest interface that preserves admissible reachability is therefore the smallest graph with the same transitive closure as the admissible concrete graph.

Established result 18.7 (Minimum equivalent graph)**.** *Let G be a finite directed graph, let G^c be its condensation (a DAG whose nodes are the strongly connected components), and let*

$\mathrm{tr}(G^{\mathrm{c}})$ *be the transitive reduction of the condensation. The least number of edges of a graph on the same vertex set with the same transitive closure as* G *is*

$$\sum_{C:\ |C|\geq 2} |C| \ + \ |E(\mathrm{tr}(G^{\mathrm{c}}))|,$$

attained by a directed cycle through each nontrivial component together with one edge for each edge of the reduction of the condensation.

Source: Aho, Garey and Ullman, *The transitive reduction of a directed graph*, SIAM J. Comput. (1972).

In the vocabulary of the chapter the number is the minimum number of presented actions of single-step type that preserve admissible reachability; macros may stand for longer denotations without changing it. An example fixes the numbers. Consider a document workflow with states draft (d), review (r), rejected (x), approved (a), published (p) and archived (z), all admissible, and concrete steps $d \to r$, $r \to a$, $r \to x$, $x \to d$, $a \to p$, $p \to z$, and three shortcuts $d \to z$, $a \to z$, $r \to p$. The strongly connected components are $\{d, r, x\}$ and the singletons a, p, z, and the reduction of the condensation has the three edges $\{d, r, x\} \to a \to p \to z$. The minimum number of presented steps is therefore $3 + 3 = 6$, against 9 concrete steps and 18 ordered pairs of distinct states in the transitive closure. The interfaces of sizes 6, 9 and 18 all preserve admissible reachability. They differ in the cost to the user, measured in presented steps. A computation of shortest paths gives, for the minimal interface of six steps, a total of 40 presented steps summed over all reachable ordered pairs and a longest required path of 5 (from x through d, r, a, p to z); for the nine concrete steps the total is 29 and the longest path 3; and for the closure every pair is one step apart, total 18 for the off-diagonal pairs and longest path 1.

New result 18.8 (Size and effort trade against each other)**.** *Among interfaces that preserve admissible reachability, let* m *be the number of presented steps and let* W *be the sum over reachable ordered pairs of the least number of presented steps connecting them. Then* $m \geq m_{\min}$ *with* $m_{\min}$ *as in Result 18.7, and* $W \geq W_{\min}$*, the number of reachable ordered pairs of distinct states. Both bounds are attained, but not by the same interface in general; for the workflow above* $(m, W) = (6, 40)$ *for the minimal interface and* $(m, W) = (18, 18)$ *for the closure, and adding shortcuts to the minimal interface lowers* W *monotonically as* m *grows.*

Assumptions: As in Result 18.2.

Proof. The bound on m is Result 18.7. Each reachable ordered pair of distinct states needs at least one presented step, so $W \geq W_{\min}$, with equality exactly when every such pair is directly presented, which is the closure. A presented step is added by adding a pair of the closure, which cannot lengthen any shortest path, so W is nonincreasing as steps are added. □

The design reading is that preservation of reachability is a weak requirement: it asks that nothing be hidden or invented, and it does not ask how many steps the user needs. A minimal interface that preserves reachability can force a path of five presented steps where a single shortcut would do, and the cost to the user is invisible to the characterization. The interface designer is therefore choosing a point on a curve between the size of the menu and the length of the journey, and the characterization of the chapter marks the set of admissible points without selecting among them.

Exercises

Exercise 18.9. In Example 18.3 give a second interface, with the same endpoints, that is sound, by routing *quick save* through `validate` and `save`, and add the concrete action `save` to the system.

Exercise 18.10. Show that the composition of two interfaces that each preserve admissible reachability, with denotations composed by substitution, preserves it, by showing that equality of closures is preserved under composition.

Exercise 18.11. Find an interface with $S_{\mathrm{c}} \subseteq S_{\mathrm{p}}^{*}$ and $S_{\mathrm{p}} \not\subseteq S_{\mathrm{c}}^{*}$ in which every presented denotation has length one, and say what it displays.

Exercise 18.12. State the bisimulation-like condition under which the attractor of every target is preserved by an interface with single-letter denotations, and prove that it implies $\mathrm{Attr}_{\mathrm{p}}(T) = \mathrm{Rec}_{\mathrm{c}}(T)$ for deterministic concrete systems.

Exercise 18.13. Relate the characterization to the strongly connected component partition of Chapter 2: show that the interface preserves admissible reachability if and only if it preserves the condensation of the admissible subgraph.

Exercise 18.14. For the workflow of Section 18.3 give an interface with seven presented steps that preserves admissible reachability and has a smaller total W than the minimal interface, and compute W and the longest path.

Exercise 18.15. Show that for a strongly connected admissible graph on n states the minimal interface has n presented steps, and that its longest path is $n - 1$ presented steps. Give the interface with $2n - 2$ steps (a star through a hub) and compute its longest path.

Exercise 18.16. Explain why Result 18.7 is no longer polynomial to apply if the interface may present only concrete steps and not macros (the smallest subgraph with the same closure).

Exercise 18.17. Show that adding a presented step that is in S_{c}^{*} but not in S_{c} never destroys soundness, and give an example in which it destroys the property that the presented step is a single concrete action, and say what the user loses.

Chapter 19

Institutional Revision as Augmented Control

Question. How is permissible revision of rules distinguished from revision that is technically available to a powerful actor?

Established sources. Control systems with parameters; robust viability (Chapter 13); return cost (Chapter 8).

New results. An augmented system whose state includes the governing rule; its kernel is the greatest family of per-rule sections each of which is a viability domain relative to the intersection of the sections of the rule's amendments (Result 19.2); a constraint is preserved against every amender with fewer amendments than the amendment depth, and the depth is a shortest path in the revision graph (Result 19.3); with unrestricted revision the depth is one (Result 19.4); amending the amendment procedure is a path in the same graph (Example 19.5).

Proof obligations. None.

Depends on. Chapters 3, 13, and 9.

Rules govern what actors may do, and rules can be changed. Whether a constraint that the rules protect will survive depends on who can change the rules, and how. A powerful actor that can rewrite any rule at any time makes every protection provisional; a procedure that permits only certain amendments, each costing something, makes some protections durable. The difference between revision that is permitted and revision that is merely possible is the difference between two revision relations on the set of rules, and the consequences of the difference can be stated and computed once the rule is made part of the state.

19.1 The augmented system

Definition 19.1 (Rule system)**.** A *rule system* consists of a finite state set X, a finite action set U, a transition $\delta : X \times U \to X$, a finite set $\mathcal{R}$ of rules, and for each rule r a feasibility set $F_r(x) \subseteq U$ for every state x and a set $\mathrm{Rev}(r) \subseteq \mathcal{R}$ of rules to which r may be amended in one step, with $r \in \mathrm{Rev}(r)$. A *constitutional constraint* is a set $C \subseteq X$. The *augmented system* has states (x, r). In each step the controller chooses $u \in F_r(x)$, the *amender* chooses $r' \in \mathrm{Rev}(r)$, and the next state is $(\delta(x, u), r')$.

The rule r governs which actions are feasible; it also carries its own revision set. Hence the procedure of revision is part of the rule, and amending the procedure is a step in the revision graph, a case treated below.

A set $S \subseteq X$ is *admitted* by a rule r if every $x \in S$ has a feasible action $u \in F_r(x)$ with $\delta(x,u) \in S$ (that is, S is a viability domain for the system governed by r alone). The *revision graph* has vertex set $\mathcal{R}$ and an edge $r \to r'$ for $r' \in \mathrm{Rev}(r)$, $r' \neq r$. Let $\mathrm{Rev}^{\leq k}(r)$ be the rules reachable from r by at most k amendments and $\mathrm{Rev}^{*}(r)$ those reachable by any number.

New result 19.2 (The kernel of the augmented system)**.** *A state (x,r) is* robustly viable *if the controller has a strategy that keeps $x_t \in C$ for all t against every amender. The set of robustly viable states is $\mathcal{V} = \{(x,r) : x \in Z_r\}$ where $(Z_r)_{r\in\mathcal{R}}$ is the greatest family of subsets of C such that*

$$\forall r\ \forall x \in Z_r\ \exists u \in F_r(x) :\ \delta(x,u) \in \bigcap_{r' \in \mathrm{Rev}(r)} Z_{r'}.$$

If $\mathrm{Rev} \subseteq \mathrm{Rev}'$ *(pointwise) then every section for* Rev' *is contained in the corresponding section for* Rev*: permitting more revisions can only shrink what is protected.*

Assumptions: Finite X and $\mathcal{R}$; the amender chooses after seeing the controller's action.

Proof. The augmented system is a system of the form of Chapter 13 with state (x,r), control $u \in F_r(x)$, disturbance $r' \in \mathrm{Rev}(r)$, and constraint $C \times \mathcal{R}$. By Result 13.1 the robustly viable set is the greatest set $\mathcal{Z} \subseteq C \times \mathcal{R}$ such that every $(x,r) \in \mathcal{Z}$ has $u \in F_r(x)$ with $(\delta(x,u), r') \in \mathcal{Z}$ for all $r' \in \mathrm{Rev}(r)$. Writing $\mathcal{Z}$ by its sections Z_r gives the displayed condition. Monotonicity is Result 13.2. □

The displayed condition is the content of the theorem. The section for a rule r is not a viability domain for r alone but for r relative to the intersection of the sections of its amendments: a state is safe under r only if the controller can move to a state that is safe under every rule to which r can be amended. Protection flows backward along the revision graph.

19.2 The amendment depth

New result 19.3 (Protection against bounded amendment)**.** *Let $S \subseteq C$ and $r_0 \in \mathcal{R}$. Define the* amendment depth

$$\mathrm{dep}_S(r_0) = \min\{k : \mathrm{Rev}^{\leq k}(r_0) \text{ contains a rule that does not admit } S\} \in \mathbb{N} \cup \{\infty\},$$

which is the length of a shortest path in the revision graph from r_0 to the set of rules not admitting S, and is computed by breadth-first search in time linear in the size of the graph after checking the admission of S for each rule. Then for every $x_0 \in S$ there is a controller strategy that keeps $x_t \in S$ for all t against every amender who makes fewer than $\mathrm{dep}_S(r_0)$ amendments, where an amendment is a step with $r_{t+1} \neq r_t$. If amendments carry costs $w(r,r') \geq 0$, the same holds against every amender whose total cost is less than the weighted distance from r_0 to the rules not admitting S.

Assumptions: Finite system; the amender may amend only along the revision graph.

Proof. Write a_t for the number of amendments made before time t. We claim $x_t \in S$ and $r_t \in \mathrm{Rev}^{\le a_t}(r_0)$ for all t, and that r_t admits S. Initially $a_0 = 0$ and $r_0 \in \mathrm{Rev}^{\le 0}(r_0)$ admits S since $\mathrm{dep}_S(r_0) \ge 1$. Inductively, as $a_t < \mathrm{dep}_S(r_0)$ the rule $r_t \in \mathrm{Rev}^{\le a_t}(r_0)$ admits S, so the controller chooses $u_t \in F_{r_t}(x_t)$ with $\delta(x_t, u_t) \in S$. The next rule r_{t+1} equals r_t or is an amendment of r_t, lying in $\mathrm{Rev}^{\le a_t+1}(r_0)$ when an amendment is made; in both cases $a_{t+1} \le \mathrm{dep}_S(r_0) - 1$ keeps r_{t+1} inside the set of rules that admit S, by the definition of depth. For weighted costs replace the number of amendments by the accumulated cost and $\mathrm{Rev}^{\le k}$ by the set of rules within weighted distance less than the budget; the same induction applies, with the weighted distance computed by Dijkstra's algorithm as in Chapter 8. □

The depth is the number of amendments that an actor must be able to make before the protection of S may fail. It is a property of the rule system and not of the actor, and it turns the contrast of the outline into a number: a revision relation is more entrenching than another, with respect to a constraint, if the depth of the former is larger.

New result 19.4 (Unrestricted revision gives depth one)**.** *If* $\mathrm{Rev}(r) = \mathcal{R}$ *for every* r *(every rule can be amended into every rule at once), then* $\mathrm{dep}_S(r_0) = 1$ *unless every rule admits* S*, in which case* $\mathrm{dep}_S(r_0) = \infty$*. Moreover, if some rule* $r_\perp$ *is such that* $F_{r_\perp}(x) \cap \{u : \delta(x,u) \in S\} = \varnothing$ *for some* $x \in S$*, a single amendment to* $r_\perp$ *at a time when the state is* x *forces the state out of* S*, so that* S *is not preserved against an amender with the power of one amendment who can choose the time.*

Assumptions: As stated.

Proof. With total revision every rule is within one amendment of r_0, so the first claim is the definition. For the second, if the state is $x \in S$ and the amender changes the rule to $r_\perp$ at time t, then at time $t+1$ every feasible action leaves S. □

The corollary is the sense in which revision that is merely technically available makes every protection provisional: when anything may become anything, only a constraint that every conceivable rule respects is safe, which is no protection at all against rules designed to violate it.

19.3 Entrenchment

Example 19.5 (Layers of entrenchment)**.** Let $X = \{s_1, s_2, b\}$, $C = \{s_1, s_2\}$, $U = \{\texttt{stay}, \texttt{swap}, \texttt{force}\}$ with δ: `stay` fixes every state, `swap` exchanges s_1, s_2 and fixes b, and `force` sends s_1, s_2 to b. Let $\mathcal{R} = \{r_0, r_1, r_2\}$ with $F_{r_0} = F_{r_1} = \{\texttt{stay}, \texttt{swap}\}$ and $F_{r_2} = \{\texttt{force}\}$. The rules r_0, r_1 admit C, and r_2 does not.

(a) Revision graph $r_0 \to r_1 \to r_2$. The depth is 2. Against one amendment C is preserved. Against two, the amender changes r_0 to r_1 and then to r_2; from s_1 the controller can only `stay` or `swap` while the rules are r_0, r_1, and under r_2 the action `force` sends the state to $b \notin C$.

(b) Entrenching r_1: let $\mathrm{Rev}(r_1) = \{r_1\}$. Now $\mathrm{Rev}^*(r_0) = \{r_0, r_1\}$, every rule in it admits C, and C is preserved against any number of amendments (Result 19.3, depth ∞).

(c) Entrenching the amendment procedure: introduce a rule r_1' with the same feasibility as r_1 and $\mathrm{Rev}(r_1') = \{r_1', r_2\}$, and let $\mathrm{Rev}(r_1) = \{r_1, r_1'\}$. The depth of C at r_0 is 3: the

amender must first amend the amendment procedure of r_1 (to r_1') before the amendment to r_2 becomes possible.

Part (c) is the point of the clause of the outline concerning constraints on revising the revision constraints: no new machinery is needed. A rule carries its revision set, and amending the revision set is an edge of the revision graph, so every layer of entrenchment appears as an extra edge on the shortest path to the unsafe rules. The depth thus counts the layers.

New result 19.6 (Depth can be made arbitrary)**.** *For every $k \geq 1$ there is a rule system with $\mathrm{dep}_C(r_0) = k$. Conversely the depth is at most the number of rules.*

Assumptions: Constructed.

Proof. Take rules $r_0, \ldots, r_k$ with $\mathrm{Rev}(r_i) = \{r_i, r_{i+1}\}$ for $i < k$, $\mathrm{Rev}(r_k) = \{r_k\}$, the feasibility of Example 19.5 for $r_0, \ldots, r_{k-1}$ and $F_{r_k} = \{\texttt{force}\}$. The shortest path from r_0 to the non-admitting rule r_k has length k. A shortest path in a graph with $|\mathcal{R}|$ vertices has fewer than $|\mathcal{R}|$ edges. □

19.4 Capture of a quorum

Many amendment procedures require the consent of a number of independent parties, so the rule that governs at a given time depends on which parties have been captured. Take n parties and let the rules be r_S for $S \subseteq \{1, \ldots, n\}$, the set of captured parties, with r_S admitting the protected set C if and only if $|S| < q$ (the *quorum* q is the number of captured parties that is enough to rewrite the protection). The revision set is $\mathrm{Rev}(r_S) = \{r_S\} \cup \{r_{S \cup \{i\}} : i \notin S\}$, and capturing party i costs $c_i > 0$.

New result 19.7 (Depth of a quorum rule)**.** *For the quorum system above, $\mathrm{dep}_C(r_\varnothing) = q$ in steps, and the weighted depth is the sum of the q smallest capture costs. The protection of C holds against every amender with total capture cost below $c_{(1)} + \cdots + c_{(q)}$, where $c_{(1)} \leq c_{(2)} \leq \cdots$ are the costs in increasing order, and there are $\binom{n}{q}$ capture sets of size q, each of which reaches a nonadmitting rule when its parties are captured in any order.*

Assumptions: Capture costs additive and incurred once per party.

Proof. A path from $r_\varnothing$ to a nonadmitting rule ends at some r_S with $|S| \geq q$ and adds one party at each edge, so it has at least q edges, and q edges suffice. Its cost is $\sum_{i \in S} c_i$ at least the sum of the q smallest costs, and capturing exactly the q cheapest parties attains the minimum. The statement on protection is Result 19.3 with weights. The sets of size q number $\binom{n}{q}$. □

The numbers show what the depth measures. With $n = 5$ parties, $q = 3$ and costs $(1, 2, 4, 8, 16)$, a Dijkstra computation over the 32 nodes of the revision graph gives a weighted depth of 7 (the three cheapest parties), whereas the average cost of a party is 6.2 and the naive estimate $3 \times 6.2 = 18.6$ more than doubles the true figure. With equal costs of 1 the depth is 3 and with equal costs of 3 it is 9. The protection is as strong as the cheapest quorum, not the typical one: a procedure whose parties are expensive on average and cheap in a few cases is as weak as those few cases. Strengthening it means raising the cost of the cheapest q parties, for example by making them independent of one another, and not raising the quorum alone; a quorum of three among parties of cost $(1, 1, 1, 100, 100)$ has depth 3, the same as one with all costs 1.

Exercises

Exercise 19.8. Compute the sections Z_r of Result 19.2 for Example 19.5(a), and verify that Z_{r_1} is a proper subset of C or equal to it, as the case may be.

Exercise 19.9. In Example 19.5(a) give amendment costs $w(r_0, r_1) = 1$ and $w(r_1, r_2) = 3$ and compute the weighted depth. What budget lets an amender capture the constraint?

Exercise 19.10. Show that the amendment depth is not monotone along the revision graph, and give a system with $\mathrm{dep}_S(r_0) < \mathrm{dep}_S(r_1)$ for $r_1 \in \mathrm{Rev}(r_0)$.

Exercise 19.11. Allow the amender to act only when the state lies in a set $Q \subseteq X$ (a procedural condition on when amendments may be proposed), and modify Result 19.2. Show that the protection can improve even when the revision graph is unchanged.

Exercise 19.12. Connect Result 19.3 to the return cost of Chapter 8: show that if every amendment is irreversible, the maximum number of amendments any amender can make is at most the number of components of the condensation of the revision graph minus one.

Exercise 19.13. Compute the weighted depth for $n = 6$, $q = 4$ and costs $(1, 1, 2, 3, 5, 8)$, and list the capture sets that attain it.

Exercise 19.14. Suppose a single veto player, party 1, must consent to every amendment, so that nonadmitting rules are the r_S with $1 \in S$ and $|S| \geq q$. Find the weighted depth and show that it exceeds the quorum depth of Result 19.7 exactly when c_1 is not among the q smallest.

Exercise 19.15. Let capturing a party also reveals it, so that the second capture of a pair of related parties costs only $c_i/2$. Explain why the revision graph is no longer a shortest-path problem over subsets with additive weights, and what is needed to restore the algorithm.

Exercise 19.16. Compute the sections $Z_{r_0}, Z_{r_1}, Z_{r_2}$ of Result 19.2 for part (a) of Example 19.5 and show that all three are empty, although C is preserved against a single amendment. Then compute them for part (b) and show that $Z_{r_0} = Z_{r_1} = \{s_1, s_2\}$ and $Z_{r_2} = \varnothing$. Explain why the unbounded amender of the theorem is stronger than the bounded amender of the depth results.

Chapter 20

Proxies and the Deformation of Possibility

Question. When does optimizing a proxy transfer to the intended objective, and what happens when the transfer fails?

Established sources. The proxy-error bound for discounted MDPs (Puterman); taxonomies of Goodhart effects (Manheim and Garrabrant).

New results. The sup-norm bound is tight (Example 20.2); an on-policy refinement that charges the error on the occupancy of the two policies compared (Result 20.3); average error does not control transfer (Example 20.4); a constrained version in which both objective and constraint are proxied, with the loss measured against a tightened budget and bounded by concavity (Result 20.6); restriction to a viability domain separates safety from value (Result 20.8).

Proof obligations. None.

Depends on. Chapters 4 and 15.

An objective that cannot be measured directly is replaced by one that can. The replacement is a proxy, and the question is how much of what is gained under the proxy is gained under the original. The standard answer is a bound in terms of the largest discrepancy; its range of validity is narrow, and outside that range optimization changes which states are visited, which is a deformation of the set of futures that the system actually explores.

20.1 The uniform bound and its sharpness

Established result 20.1 (Proxy transfer under a uniform error bound)**.** *In a discounted MDP with discount $\gamma \in [0,1)$, let R and R' be reward functions with $\sup_{s,a} |R(s,a) - R'(s,a)| \le \varepsilon$. If π' is optimal for R' and π^* is optimal for R, then $V_R^{\pi'} \ge V_R^{\pi^*} - \frac{2\varepsilon}{1-\gamma}$ at every state.*

Source: A standard perturbation bound for discounted dynamic programming; see Puterman, *Markov Decision Processes*.

Proof. For every policy π and state s, $|V_R^\pi(s) - V_{R'}^\pi(s)| \le \sum_{t \ge 0} \gamma^t \varepsilon = \varepsilon/(1-\gamma)$. Therefore $V_R^{\pi'} \ge V_{R'}^{\pi'} - \frac{\varepsilon}{1-\gamma} \ge V_{R'}^{\pi^*} - \frac{\varepsilon}{1-\gamma} \ge V_R^{\pi^*} - \frac{2\varepsilon}{1-\gamma}$. □

Example 20.2 (The constant is attained)**.** One state, two actions a, b (each returning to the state), $R(a) = \varepsilon$, $R(b) = -\varepsilon$, $R'(a) = R'(b) = 0$. Then $\sup|R - R'| = \varepsilon$, every policy is optimal for R', and the policy that always plays b has $V_R = -\varepsilon/(1-\gamma)$ while $V_R^{\pi^*} = \varepsilon/(1-\gamma)$, a loss of exactly $2\varepsilon/(1-\gamma)$. With $R'(b) = \eta > 0$ for small η the proxy has a strict optimum b and the error is $\varepsilon + \eta$, so the loss $2\varepsilon/(1-\gamma)$ is approached by strict optima. The bound cannot be improved in general.

20.2 Transfer is governed by the error on the policies compared

The uniform bound charges the error at every state-action pair, including pairs that neither policy visits. A refinement charges only what the two policies do. For a policy π and an initial distribution μ write $\nu^\pi(s,a) = (1-\gamma)\sum_{t\geq 0}\gamma^t \Pr_\mu^\pi(s_t = s, a_t = a)$ for its normalized occupancy measure, so that $V_R^\pi(\mu) = \frac{1}{1-\gamma}\sum_{s,a}\nu^\pi(s,a)R(s,a)$, and set $e(\pi) = \sum_{s,a}\nu^\pi(s,a)\,|R(s,a) - R'(s,a)|$.

New result 20.3 (On-policy proxy bound)**.** *With π' optimal for R' and π^* optimal for R from the initial distribution μ,*

$$V_R^{\pi^*}(\mu) - V_R^{\pi'}(\mu) \;\leq\; \frac{e(\pi') + e(\pi^*)}{1-\gamma} \;\leq\; \frac{2\varepsilon}{1-\gamma}.$$

The first term charges the proxy for the error on the states the proxy-optimal policy visits, and the second for the error on the states the true optimum visits. The bound is attained when both terms are positive in the pattern of Example 20.2, and the loss is 0 *when $R' = R$ on the support of both occupancy measures, whatever R' does elsewhere.*

Assumptions: Finite state and action sets; π', π^* optimal from μ for the respective rewards.

Proof. By the occupancy formula, $|V_R^\pi(\mu) - V_{R'}^\pi(\mu)| \leq e(\pi)/(1-\gamma)$ for every π. Then

$$V_R^{\pi'} \geq V_{R'}^{\pi'} - \tfrac{e(\pi')}{1-\gamma} \geq V_{R'}^{\pi^*} - \tfrac{e(\pi')}{1-\gamma} \geq V_R^{\pi^*} - \tfrac{e(\pi^*)+e(\pi')}{1-\gamma}.$$

Since $e(\pi) \leq \varepsilon$ for every π, the second inequality of the statement follows. If $R = R'$ on the supports of $\nu^{\pi'}$ and ν^{π^*} then both e vanish. □

The deformation is visible in the first term. The policy π' is selected by the proxy, so it is drawn toward the pairs where R' exceeds R, and $e(\pi')$ is larger than the error that a fixed reference policy would suffer. The error is evaluated on a distribution that the optimization itself has moved.

Example 20.4 (Mean error does not control transfer)**.** One state and actions $a_0, a_1, \ldots, a_n$ with $R(a_0) = 1$, $R(a_k) = 0$ for $k \geq 1$, and $R'(a_0) = 0$, $R'(a_1) = 2$, $R'(a_k) = 0$ for $k \geq 2$. The average of $|R - R'|$ over the $n+1$ actions is $3/(n+1) \to 0$, but $\sup|R - R'| = 2$, and the proxy-optimal policy plays a_1 forever with $V_R = 0$ against $V_R^{\pi^*} = 1/(1-\gamma)$, the largest loss any policy pair can have in this system. Under the uniform distribution over actions, the proxy is nearly exact; under the optimizer it is wrong in the one place that matters.

20.3 Constrained problems with proxied constraints

Definition 20.5 (Constrained MDP and its proxy)**.** A *constrained MDP* has a reward R, a cost $C \geq 0$, a budget d, and the problem $\max_\pi V_R^\pi(\mu)$ subject to $V_C^\pi(\mu) \leq d$, with optimal value $f(d)$ (taken over randomized stationary policies, equivalently over occupancy measures; $-\infty$ if infeasible). A *proxy* is a pair R', C' with $\sup|R - R'| \leq \varepsilon_R$ and $\sup|C - C'| \leq \varepsilon_C$. Write $\eta = \varepsilon_C/(1-\gamma)$ and $\rho = \varepsilon_R/(1-\gamma)$.

The function f is concave and nondecreasing on its domain $[d_{\min}, \infty)$, where $d_{\min} = \min_\pi V_C^\pi(\mu)$: the feasible occupancy measures form a polytope that depends linearly on d, and the optimal value of a linear program is concave in its right-hand side.

New result 20.6 (Proxy transfer for constrained problems)**.** *Let π' solve the proxy problem with the tightened budget,*

$$\max_\pi V_{R'}^\pi(\mu) \quad \textit{subject to} \quad V_{C'}^\pi(\mu) \leq d - \eta.$$

Then:

(a) π' is feasible for the true problem: $V_C^{\pi'}(\mu) \leq d$.

(b) $V_R^{\pi'}(\mu) \geq f(d - 2\eta) - 2\rho$.

(c) If $d - 2\eta \geq a > d_{\min}$, then $f(d) - f(d-2\eta) \leq 2\eta \dfrac{f(a) - f(d_{\min})}{a - d_{\min}}$, hence $V_R^{\pi'}(\mu) \geq f(d) - 2\rho - 2\eta \dfrac{f(d-2\eta) - f(d_{\min})}{d - 2\eta - d_{\min}}$.

Without tightening, $V_{C'}^\pi \leq d$ does not imply $V_C^\pi \leq d$, and the true constraint can be violated by up to η.

Assumptions: Finite system; $d - 2\eta > d_{\min}$ for the statement of (b) and (c) to be non-vacuous.

Proof. (a) $V_C^{\pi'} \leq V_{C'}^{\pi'} + \eta \leq d$, as $|V_C^\pi - V_{C'}^\pi| \leq \eta$ for every π (the argument of Result 20.1).

(b) Let π° attain $f(d-2\eta)$, so $V_C^{\pi^\circ} \leq d - 2\eta$. Then $V_{C'}^{\pi^\circ} \leq d - 2\eta + \eta = d - \eta$, so π° is feasible for the tightened proxy problem and $V_{R'}^{\pi'} \geq V_{R'}^{\pi^\circ}$. Hence $V_R^{\pi'} \geq V_{R'}^{\pi'} - \rho \geq V_{R'}^{\pi^\circ} - \rho \geq V_R^{\pi^\circ} - 2\rho = f(d-2\eta) - 2\rho$.

(c) For a concave function the slope of the chord over $[d-2\eta, d]$ is at most the slope of the chord over $[d_{\min}, a]$ whenever $a \leq d - 2\eta$, because chord slopes of a concave function decrease as the interval moves right. The second inequality combines this with (b) and monotonicity of the difference quotient at $a = d - 2\eta$. □

The loss has two parts, the objective error 2ρ and the price $f(d) - f(d-2\eta)$ of the safety margin demanded by the constraint error. The second part is the cost of not knowing the constraint exactly, and it is bounded by the marginal value of budget at the tightened level, the quantity a Lagrange multiplier measures when one exists. If the constraint is binding with a large multiplier, a small error in its measurement is expensive; if the constraint is slack, the tightening is free.

Example 20.7 (A binding constraint with a steep marginal value)**.** One state, actions a (reward 1, cost 1) and b (reward 0, cost 0), $\gamma = \frac{1}{2}$, so $V_C^\pi = 2p$ and $V_R^\pi = 2p$ when a is played with probability p. With $d = 1$ one has $f(1) = 1$, $f(d) = d$ in general for $d \in [0, 2]$, so the marginal value is 1. With $\varepsilon_C = 0.1$, $\eta = 0.2$ and exact rewards, the tightened problem

has budget 0.8 and yields value $0.8 = f(d) - \eta$; (b) gives $f(d - 2\eta) = 0.6$ and the bound is conservative by 0.2, because the true loss is only η. Without tightening, a proxy that underestimates the cost by ε_C permits $p = 0.5 + 0.05 \cdot 2$, violating the true constraint by 0.2.

20.4 Safety need not depend on the proxy

The constraint in Result 20.6 is statistical, with a budget on an expected cost. A hard constraint, in the sense of a viability domain, behaves differently, and the difference is the reason to separate the two in the design of a system.

New result 20.8 (Hard safety is independent of the proxy)**.** *Let K be a viability domain of a finite MDP, in the sense that every $s \in K$ has an action whose successor distribution is supported in K, and let $\mathcal{M}_K$ be the MDP restricted to K and to such actions. For any reward R' let π' be optimal for R' in $\mathcal{M}_K$. Then (i) every trajectory of π' started in K stays in K almost surely, whatever R' is, and (ii) if $\sup|R - R'| \le \varepsilon$ then π' loses at most $2\varepsilon/(1-\gamma)$ against the best K-restricted policy for R.*

Assumptions: Finite state and action sets.

Proof. (i) is the definition of $\mathcal{M}_K$: every action available in it has successors in K. (ii) is Result 20.1 applied to $\mathcal{M}_K$. □

The proxy therefore affects the value obtained within K but cannot move the system outside it. What it can still do is choose among the options inside K in a way that shrinks later options, a loss of possibility that the bound of (ii) does not see when the true reward does not value the options; Chapter 15 bounds this loss for the preserved kernel and Chapter 16 for the regions of recoverable states.

20.5 Selection among many options: the winner's overstatement

The bounds of the previous sections concern the worst proxy of a given size. For a proxy that is noisy but unbiased in a precise sense the loss under selection can be computed exactly. Let there be n options with true values $U_1, \dots, U_n$ independent standard normal, and let the proxy score of option i be $P_i = U_i + \sigma Z_i$ with Z_i independent standard normal noise. The optimizer chooses $I = \arg\max_i P_i$. Write $m_n = \mathbb{E}\max_{i \le n} U_i$ for the expected maximum of n standard normal variables, so that an optimizer who sees U obtains m_n.

New result 20.9 (Selection on a noisy proxy)**.** *The true value obtained by the proxy-optimal choice is $\mathbb{E}\,U_I = m_n/\sqrt{1+\sigma^2}$, and the proxy value it reports is $\mathbb{E}\,P_I = \sqrt{1+\sigma^2}\,m_n$. Therefore the optimizer obtains the fraction $1/\sqrt{1+\sigma^2}$ of the attainable value for every n, and the proxy overstates the value it delivers by the factor exactly $1+\sigma^2$, independently of n. The absolute shortfall $m_n\big(1 - 1/\sqrt{1+\sigma^2}\big)$ and the absolute overstatement $m_n\sigma^2/\sqrt{1+\sigma^2}$ both grow without bound with n, at the rate $m_n \sim \sqrt{2\ln n}$.*

Assumptions: Independent Gaussian values and noise.

Proof. The pairs (U_i, P_i) are jointly Gaussian with $\operatorname{Var} P_i = 1 + \sigma^2$ and $\operatorname{Cov}(U_i, P_i) = 1$, so $\mathbb{E}[U_i \mid P_1, \dots, P_n] = P_i/(1+\sigma^2)$ by independence across options. The index I is a function of $P_1, \dots, P_n$, hence $\mathbb{E}\, U_I = \mathbb{E}\,\mathbb{E}[U_I \mid P] = \mathbb{E}\, P_I/(1+\sigma^2)$. The proxies P_i are independent $\mathcal{N}(0, 1+\sigma^2)$, so $\mathbb{E}\, P_I = \mathbb{E} \max_i P_i = \sqrt{1+\sigma^2}\, m_n$. Dividing gives $\mathbb{E}\, U_I = m_n/\sqrt{1+\sigma^2}$, and the ratio of the proxy value to the true value is $1+\sigma^2$. The asymptotics of m_n are those of the maximum of Gaussian variables. □

The identity was checked by simulation and numerical integration over four pool sizes ($n = 2, 10, 100, 1000$) and three noise levels ($\sigma = 0.5, 1, 2$); the simulated true values agreed with the formula to within 0.01 in every case (for example, at $\sigma = 1$ the formula gives 1.088, 1.773 and 2.292 for $n = 10, 100, 1000$, the simulations 1.089, 1.773 and 2.292). At $\sigma = 1$ the oracle values m_n are 1.539, 2.506 and 3.241 for the three sizes, so the proxy-optimal choice delivers about 71% of the oracle, while the proxy reports 2.176, 3.546 and 4.584, which are twice what is delivered. Two features need emphasis. The captured fraction does not improve as the search widens, because the extra options come with extra noise in the same proportion; and the overstatement grows with the width of the search, so that the larger the pool the more convincing the proxy figure is and the less it means. The mean error of the proxy is zero, and it is the selection that creates the loss: this is the sense in which Example 20.4 says that a small average error does not control transfer, here in a case where the average error vanishes exactly.

The relation to Result 20.3 is that the Gaussian proxy has unbounded uniform error, so the uniform bound is vacuous, while the on-policy signed error of the selected option is exactly the overstatement $\mathbb{E}(P_I - U_I) = m_n \sigma^2/\sqrt{1+\sigma^2}$: the error is evaluated on the distribution that the optimization has moved, and it is larger there than on a randomly chosen option, where it is zero in expectation.

Exercises

Exercise 20.10. Verify Example 20.2 with $\gamma = \frac{1}{2}$ and $\varepsilon = \frac{1}{4}$ by computing all three values explicitly.

Exercise 20.11. In Example 20.4 compute $e(\pi')$ and $e(\pi^*)$ and check Result 20.3.

Exercise 20.12. Construct a two-state example in which $\sup |R - R'|$ is large but $e(\pi') = e(\pi^*) = 0$, so that the proxy transfers exactly.

Exercise 20.13. Prove that f is concave by showing that a convex combination of feasible occupancy measures for budgets d_1, d_2 is feasible for the combined budget.

Exercise 20.14. Show that the tightening η in Result 20.6 can be replaced by $\varepsilon_C^{\mathrm{on}}(\pi)/(1-\gamma)$ evaluated on the occupancy of the policy being certified, and explain why this makes the tightened problem nonlinear.

Exercise 20.15. For $n = 2$ compute $m_2 = 1/\sqrt{\pi} \approx 0.5642$ and verify Result 20.9 at $\sigma = 1$ by computing $\mathbb{E}\, U_I$ directly from the joint density of (P_1, P_2).

Exercise 20.16. Replace the Gaussian values by values in $\{0, 1\}$ with $\Pr[U_i = 1] = p$ and a proxy that equals U_i with probability $1-q$ and is resampled independently otherwise. Compute the true value of the proxy-optimal choice among n options, with ties broken uniformly, and find the limit as $n \to \infty$.

Exercise 20.17. Show that if the optimizer screens the pool to its top k options by the proxy and then measures the true value of those k exactly at a cost c each, the expected true value and the expected cost can be computed from the order statistics of P, and say when screening is worthwhile compared with Result 20.9.

Exercise 20.18. Relate Result 20.9 to the Pandora problem of Chapter 10: show that inspecting an option corresponds to replacing a proxy by the true value at a cost, and compute the reservation value for an option whose proxy is $P_i = p$ and whose true value has conditional distribution $\mathcal{N}(p/(1+\sigma^2), \sigma^2/(1+\sigma^2))$.

Part VII

Learning and design

Chapter 21

Exploration and Unattempted Alternatives

Question. How is an infeasible alternative distinguished from an untested one?

Established sources. Bandits (Lai and Robbins; Auer, Cesa-Bianchi and Fischer); Bayesian experimental design; safe exploration.

New results. A three-way classification of alternatives into attempted, certified infeasible, and untested, with the untested set shrinking monotonically (Result 21.1); data on attempted alternatives carry no information about untested ones (Result 21.2); the exact threshold horizon above which attempting an untested alternative pays (Result 21.3); the knowledge-indexed viability kernel is monotone in knowledge, and safe attempts are those whose every possible outcome lies in it (Result 21.5).

Proof obligations. None.

Depends on. Chapters 5, 9, and 3.

A system that has never done something does not know whether it can. An institution that has never attempted a reform, an agent that has never tried an action, a laboratory that has never run an assay, each holds a set of alternatives about which the history is silent. The silence is ambiguous between two situations that call for opposite responses: the alternative is excluded by a constraint, so that attempting it is wasted or harmful, or it is available and merely unused, so that failing to attempt it forfeits whatever it would have yielded. This chapter shows what separates the two, what cannot be inferred from experience alone, and what exploration costs and returns.

21.1 Three epistemic statuses

Let A be a finite set of alternatives in a context. The world fixes a feasible subset $F \subseteq A$ that is unknown to the agent. An *attempt* of $a \in A$ costs $c_a \geq 0$ and reveals whether $a \in F$ (and, if so, its value). A *certificate of infeasibility* for a is a proof, from the known structure of the system, that $a \notin F$: for example a conservation law violated by the transition (Chapter 6), a kernel exclusion (Chapter 3), or an unreachable capability (Chapter 9).

New result 21.1 (Trichotomy)**.** *At each time the alternatives are partitioned into A_{ok} (attempted and found feasible), $A_{\times}$ (attempted and found infeasible, or certified infeasible), and*

$A_?$ (neither). Over time $A_?$ is non-increasing, $A_{\text{ok}} \cup A_\times$ is non-decreasing, and the three sets stay disjoint, provided that certificates are sound (a certified alternative is infeasible) and that feasibility does not change. An alternative lies in $A_?$ at time t if and only if no attempt of it has occurred and no available certificate decides it; consequently $A_? = \varnothing$ exactly when every alternative has been attempted or certified.

Assumptions: Feasibility is static; attempts are truthful; certificates are sound.

Proof. Attempting a places it in A_{ok} or $A_\times$ according to the outcome and never removes it from there, since feasibility is static. A sound certificate places a in $A_\times$, and no attempt can contradict it. Disjointness of A_{ok} and $A_\times$ follows from truthfulness and soundness. The remaining statements are the definition of $A_?$. □

If feasibility can change (Chapter 9) the status of an attempted alternative expires with the change, and the chapter's analysis applies per epoch. The practical lesson of the trichotomy is that the set $A_\times \cup A_?$ is what ordinary language calls "not done", and that the two parts of it are separated only by a certificate or an attempt.

New result 21.2 (Experience on attempted alternatives is silent on the rest)**.** *Suppose the feasibility indicators $(1\{a \in F\})_{a \in A}$ are independent under the prior. Then for every policy, the posterior probability that an unattempted a is feasible, given the history, equals its prior probability. In particular, no quantity of data on the attempted alternatives changes the prior of the untested one. If the indicators are dependent, the posterior of a changes with the data only through the dependence, and an untested alternative whose feasibility is independent of all the attempted ones remains at its prior.*

Assumptions: Observations of a policy depend on the world only through the attempted alternatives' outcomes.

Proof. The history is a function of the attempted alternatives' indicators (and independent randomization). By independence the indicator of a is independent of that history, so its conditional law is its prior. In the dependent case, Bayes' rule gives the posterior as the conditional law given the attempted indicators. □

This is the formal sense of the statement that absence of evidence is weak evidence of absence: the evidence is absent for a reason that is not informative, namely that nobody looked. Where the indicators are dependent, evidence about related alternatives is informative in the amount of the dependence, and this is the sole route by which a system infers the feasibility of what it has not tried.

21.2 When an attempt pays

New result 21.3 (Threshold horizon)**.** *An agent has a known best attempted option of per-round value μ^* and an untested alternative a that is feasible with probability $p \in (0, 1)$, in which case it yields $\mu^* + \Delta$ per round ($\Delta > 0$) from the round after it is attempted. Attempting a costs $c > 0$ once, and reveals feasibility exactly. Over a horizon of n rounds, with the attempt taking one round (in which the agent earns $\mu^* - c$ on the attempt round, and an infeasible*

attempt leaves the agent to continue with μ^*), the policy "attempt in the first round, then use the best option" has expected total value larger than "never attempt" if and only if

$$p\,\Delta\,(n-1) > c, \qquad \textit{that is,} \qquad n > n^* := 1 + \frac{c}{p\Delta}.$$

Among policies that attempt at most once, attempting at a fixed time is optimal at time 1 or never: delaying the attempt to round t changes the gain to $p\Delta(n-t) - c$, which is decreasing in t. Writing $G = ((n-1)\Delta - c)^+$, the regret of a policy in a world is its shortfall against the best policy for that world; the minimax regret over the two worlds (a feasible, a infeasible) is $cG/(c+G)$ when randomization is allowed, attained by attempting in round 1 with probability $q = G/(c+G)$, and it is $\min(c, G)$ over pure policies.

Assumptions: Feasibility revealed by one attempt; static; value additive across rounds.

Proof. Never attempting yields $n\mu^*$. Attempting in round 1 yields $\mu^* - c$ in that round, then $\mu^* + \Delta$ in each of the remaining $n-1$ rounds if a is feasible and μ^* if not, for an expected total $n\mu^* - c + p\Delta(n-1)$. The comparison is the stated inequality. Attempting in round t gains $p\Delta(n-t) - c$ in expectation, decreasing in t, so the best attempt time is 1 if this is positive and no attempt otherwise.

For regret, the best policy in the feasible world attempts at round 1 (value $n\mu^* - c + (n-1)\Delta$) and in the infeasible world never attempts (value $n\mu^*$). Never attempting therefore has regret G when a is feasible and 0 otherwise; attempting at round 1 has regret 0 when a is feasible and c otherwise. A policy attempting at round 1 with probability q has worst-case regret $\max(qc, (1-q)G)$, minimal where the two are equal, at $q = G/(c+G)$, with value $cG/(c+G)$; among pure policies the minimum of c and G is the smaller endpoint. A policy that attempts later has the same regret c in the infeasible world and a larger regret in the feasible one, so it does no better. □

The threshold $n^* = 1 + c/(p\Delta)$ expresses the exploration premium in the three quantities that matter: the cost of the attempt, the prior chance of feasibility, and the size of the gain. It is linear in the cost and inversely proportional to both probability and gain. Two corollaries are immediate. A system with a short remaining horizon rationally attempts nothing new, and a system with a long one attempts almost everything not certified infeasible. The second is the justification for the first volume's emphasis on separating the untested from the infeasible: certifying infeasibility removes alternatives from the set that a long-lived system should attempt.

Example 21.4 (Several alternatives)**.** With m independent untested alternatives with parameters (p_i, Δ_i, c_i) and the ability to adopt only the best feasible one, the problem is the Pandora's box problem of Weitzman, whose optimal rule is an index policy in reservation values (Chapter 10). The single-alternative threshold above is the case $m = 1$ in which the reservation value reduces to $\mu^* + p\Delta - c/(n-1)$. For $m \geq 2$ with additional structure (shared costs, correlated feasibility) the index property can fail, and exploration order becomes a combinatorial problem.

21.3 Safe exploration and the knowledge-indexed kernel

Exploration is not free of risk when attempts can move the state. Let a system have states X, actions U, and an unknown transition function; the agent knows a set $\Theta \subseteq X \times U$ of

state-action pairs whose outcome is known and deterministic: $\delta(x,u)$ for $(x,u) \in \Theta$. For $(x,u) \notin \Theta$ the agent knows only a set $Z(x,u) \subseteq X$ that contains the outcome. Let $C \subseteq X$ be the constraint set.

New result 21.5 (Knowledge-indexed kernel and safe attempts)**.** *Let $K(\Theta)$ be the largest $S \subseteq C$ such that every $x \in S$ has u with $(x,u) \in \Theta$ and $\delta(x,u) \in S$ (viability using known transitions only).*

(a) $K(\Theta)$ is monotone in Θ: if $\Theta \subseteq \Theta'$ (with δ agreeing) then $K(\Theta) \subseteq K(\Theta')$.

(b) An attempt of $(x,u) \notin \Theta$ at $x \in K(\Theta)$ is safe *if $Z(x,u) \subseteq K(\Theta)$: whatever the outcome, the agent can continue to satisfy the constraint with known transitions forever. Executing safe attempts, and adding each revealed pair to Θ, yields $\Theta_0 \subseteq \Theta_1 \subseteq \cdots$ and kernels $K(\Theta_0) \subseteq K(\Theta_1) \subseteq \cdots$, all constraint-satisfying along the way, and the process stabilizes after at most $|X||U|$ attempts.*

(c) The stable knowledge Θ_∞ is the set of pairs reachable by safe attempts. An unattempted pair (x,u) with $x \in K(\Theta_\infty)$ and $Z(x,u) \not\subseteq K(\Theta_\infty)$ is not safely testable from the current knowledge; it may be tested only if some other source of information shrinks $Z(x,u)$.

Assumptions: Finite system; the true outcome of an unknown pair lies in Z; revealed outcomes are deterministic.

Proof. (a) The predecessor operator $\Gamma_\Theta(S) = \{x \in C : \exists u,\ (x,u) \in \Theta,\ \delta(x,u) \in S\}$ is monotone in both S and Θ, so the greatest fixed point is monotone in Θ by the Knaster–Tarski characterization (Result 3.2).

(b) At $x \in K(\Theta_k)$ a safe attempt leads to a state in $K(\Theta_k) \subseteq C$, so the constraint holds after the attempt, and the agent then follows the known policy from the new state, which lies in the kernel. After the attempt, $\Theta_{k+1} = \Theta_k \cup \{(x,u)\}$ and (a) gives $K(\Theta_k) \subseteq K(\Theta_{k+1})$. Each attempt adds a new pair, so there are at most $|X||U|$ attempts.

(c) By construction, when the process stops, no remaining pair with $x \in K(\Theta_\infty)$ satisfies the safety condition. Pairs with $x \notin K(\Theta_\infty)$ cannot be attempted from a kernel state. The last statement is the definition. □

The theorem makes precise a distinction the previous section could not draw: an alternative may be untested because attempting it is unsafe given the present knowledge, which is different from being untested because nobody tried. The set of safely testable pairs is the exploration frontier, and exploration enlarges the kernel in the sense that the system learns its way to a larger safe region. Chapter 15 gives the price of restricting to the kernel; here the kernel is also the limit of what can be learned without risk.

Example 21.6 (A frontier that does not advance)**.** States $X = \{0,1,2,3\}$ on a line with $C = \{0,1,2\}$, known actions `stay` at every state, and the unknown action `right` with $Z(x, \texttt{right}) = \{x, x+1\}$. Then $K(\Theta_0) = C$, the pair $(0, \texttt{right})$ is safe to attempt ($Z = \{0,1\} \subseteq C$), as is $(1, \texttt{right})$ ($Z = \{1,2\}$), but $(2, \texttt{right})$ has $Z = \{2,3\} \not\subseteq C$ and is not safely testable. If moreover `right` at state 1 turns out to reach 2, nothing changes for $(2, \texttt{right})$: its uncertainty is not reduced by information on other pairs, unless the model relates them (for example by a translation-invariance assumption, which would make $(2, \texttt{right})$ known to be 3 and so certified unsafe). The exploration frontier is stopped by the edge of the constraint set.

21.4 Learning while exploring: a Bayesian two-armed instance

Result 21.3 treats feasibility as revealed by one attempt. When an alternative returns an informative but noisy signal, the agent may learn from repeated attempts, and the single-attempt threshold becomes a sufficient condition and not the exact one. Consider the standard instance: a known arm that pays λ per round, an untested arm that pays 1 with unknown probability θ and 0 otherwise, a uniform prior $\theta \sim \text{Beta}(1, 1)$, and a horizon of n rounds, with the posterior updated after every pull of the unknown arm.

New result 21.7 (Threshold for attempting once, and for exploring)**.** *For $\frac{1}{3} \le \lambda < \frac{2}{3}$, the policy that pulls the untested arm once and then keeps the better of the two arms by posterior mean beats never exploring if and only if $n > (3\lambda - 1)/(2 - 3\lambda)$. For $\lambda \ge \frac{2}{3}$ this policy never beats never exploring, since a single success moves the posterior mean to $\frac{2}{3} \le \lambda$.*

Assumptions: Uniform prior; arms as above.

Proof. After one pull, a success (probability $\frac{1}{2}$) gives the posterior mean $\frac{2}{3}$ and a failure gives $\frac{1}{3}$. For $\lambda \in [\frac{1}{3}, \frac{2}{3})$ the agent commits to the untested arm after a success and reverts after a failure, so the value is $\frac{1}{2}\big(1 + (n-1)\frac{2}{3}\big) + \frac{1}{2}(n-1)\lambda$, and subtracting the value $n\lambda$ of never exploring gives the gain $\frac{1}{2} + \frac{n-1}{3} - \frac{\lambda(n+1)}{2}$. This is positive if and only if $2n + 1 > 3\lambda(n + 1)$, that is $n(2 - 3\lambda) > 3\lambda - 1$. For $\lambda \ge \frac{2}{3}$ the commit rule never selects the untested arm, so the value is $\frac{1}{2} + \frac{1}{2} \cdot 0 + (n-1)\lambda < n\lambda$. □

Dynamic programming over the posterior (a, b) gives the exact optimal value with unrestricted adaptation. For $\lambda = \frac{1}{2}$ the values over horizons $n = 1, 2, 3, 5, 10, 20, 50$ are as follows (rounded to three places).

n	never explore	attempt once	optimal
1	0.500	0.500	0.500
2	1.000	1.083	1.083
3	1.500	1.667	1.667
5	2.500	2.833	2.850
10	5.000	5.750	5.824
20	10.000	11.583	11.912
50	25.000	29.083	30.350

The first horizon at which exploring has positive value is $n = 2$ for $\lambda = 0.5$, $n = 4$ for $\lambda = 0.6$, and $n = 11$ for $\lambda = 0.7$. For $\lambda = 0.6$ the attempt-once policy needs $n > 4$, that is $n = 5$, so the optimal policy explores one round of horizon earlier; for $\lambda = 0.7$ the attempt-once policy never pays while the optimal policy does, because it can pull again after a success to separate a good arm from a lucky one. The gap between the third and fourth columns widens with the horizon: at $n = 50$ the optimal policy earns 4.4% more than the policy that attempts once, and 21% more than never exploring.

The lesson for the trichotomy of this chapter is that the untested set should be explored not only when a single attempt pays but when the knowledge it begets pays over a sequence of decisions; the threshold of Result 21.3 understates the case for exploration whenever signals are noisy and the agent can respond to them.

Exercises

Exercise 21.8. Show that the threshold horizon of Result 21.3 is the exact break-even point by computing the expected value of both policies for $p = \frac{1}{2}$, $\Delta = 2$, $c = 3$, and $n \in \{3, 4, 5\}$.

Exercise 21.9. Extend Result 21.3 to the case in which an infeasible attempt costs an additional penalty c' (an attempt of the infeasible alternative damages the system). Find the new threshold.

Exercise 21.10. Prove that in Result 21.5 the order in which safe attempts are made does not affect Θ_∞.

Exercise 21.11. Give an example with dependent feasibility indicators in which data on attempted alternatives push the posterior of an untested alternative to 0, and an example where they push it to 1. Which of the two corresponds to a certificate of infeasibility?

Exercise 21.12. Combine Result 21.3 with a discount factor γ: find the threshold on (p, Δ, c, γ) for an infinite horizon, and show that the finite-horizon threshold n^* is recovered as $\gamma \to 1$ when n replaces $1/(1-\gamma)$.

Exercise 21.13. Verify the entries of the table for $n = 2$ and $n = 3$ by enumerating the two or four outcome sequences under the optimal policy.

Exercise 21.14. For $\lambda = 0.7$ and $n = 11$ describe the optimal policy that makes exploration worthwhile (which pull sequence it follows after a first success), and show that the gain over never exploring is $13/560 \approx 0.0232$.

Exercise 21.15. Extend Result 21.7 to the prior $\mathrm{Beta}(a_0, b_0)$ and find the threshold on n for the attempt-once policy in terms of λ, a_0 and b_0 when both post-pull means straddle λ.

Exercise 21.16. Explain why the optimal value in the last column of the table is bounded above by the value $\mathbb{E}\max(\theta, \lambda)\, n$ of an agent who knows θ in advance, compute that bound for $\lambda = \frac{1}{2}$, and report the fraction of the bound attained at $n = 50$.

Chapter 22

Maintaining and Acquiring Revision Sufficiency

Question. What does it cost to keep or to restore the distinctions that revision requires?

Established sources. Informational and physical reachability as developed in the first volume; Myhill–Nerode (Appendix A); online rent-or-buy problems (Karlin, Manasse, Rudolph and Sleator).

New results. The least maintainable representation sufficient for a family of anticipated properties is the intersection of their Nerode congruences, with between the largest and the product of their class counts (Result 22.1); the marginal cost of anticipating one more property (Result 22.2); a rent-or-buy theorem with competitive ratio two and its optimality among deterministic rules (Result 22.5); irreversible loss forces indefinite maintenance (Result 22.6).

Proof obligations. None. This chapter closes the cost-bound obligation for maintaining a representation sufficient for an anticipated policy family, under the cost model of Definition 22.4.

Depends on. Chapters 7, 12, 14, and Appendix A.

A system that may later be asked to revise a rule must retain the distinctions that the revised rule will depend on. Which rule will be adopted is not known in advance, so the question is how many distinctions to keep for a family of anticipated rules and what it costs to restore one that was discarded. The first volume proves that sufficiency for a property is exactly refinement of its continuation equivalence; this chapter adds costs to that statement.

22.1 Sizes of sufficient representations for families

Fix a history system $\mathcal{H}$ and properties $P_1, \dots, P_m : \mathcal{H} \to B_i$. Write $\approx_i$ for the continuation equivalence of P_i and $Q_i = \mathcal{H}/{\approx_i}$ for its set of classes, and let $\approx_F = \bigcap_i \approx_i$ for the family $F = \{P_1, \dots, P_m\}$.

New result 22.1 (Least maintainable sufficient representation for a family)**.** *A maintainable representation is sufficient for every P_i if and only if its kernel is contained in $\approx_F$. The relation $\approx_F$ is itself a right congruence, so the representation $h \mapsto ([h]_{\approx_1}, \dots, [h]_{\approx_m})$ is maintainable*

and sufficient, and its number of values $|Q_F|$ satisfies

$$\max_i |Q_i| \;\le\; |Q_F| \;\le\; \prod_i |Q_i|,$$

with both bounds attained: the lower bound when the $\approx_i$ coincide, the upper bound for the parities below. Any maintainable sufficient representation takes at least $|Q_F|$ values, hence at least $\lceil \log_2 |Q_F| \rceil$ bits.

Assumptions: Finite Q_i.

Proof. By Result A.5 a maintainable r is sufficient for P_i iff $\ker r \subseteq \approx_i$; this holds for all i iff $\ker r \subseteq \bigcap_i \approx_i$. An intersection of right congruences is a right congruence: if $h \approx_i h'$ for all i and $ha \in \mathcal{H}$ then $ha \approx_i h'a$ for every i. Hence the product representation, whose kernel is $\approx_F$, is maintainable. The map from Q_F to Q_i induced by $\approx_F \subseteq \approx_i$ is onto, which gives $|Q_F| \ge |Q_i|$; the map $Q_F \to \prod Q_i$ is injective, giving the upper bound. If all $\approx_i$ coincide, $|Q_F| = |Q_1|$. For the parities, let $\mathsf{E} = \{a, b\}$, $P_1(h)$ = parity of the number of a in h, $P_2(h)$ = parity of the number of b. Then $\approx_1$ has two classes (by parity of a's), $\approx_2$ two classes, and $\approx_F$ four, which is $2 \cdot 2$. A lower bound on values is the number of classes, and on bits its logarithm. □

The theorem identifies the memory needed to keep m distinctions open and shows that the cost is additive in bits only in the independent case. When anticipated properties are correlated through the dynamics, the family representation can be much smaller than the product, down to the single largest factor.

New result 22.2 (Marginal cost of anticipating one more property)**.** *Let $F' = F \cup \{P\}$. Then $|Q_{F'}|/|Q_F| \le |Q_P|$, so the additional memory is at most $\log_2 |Q_P|$ bits, and it is zero if and only if $\approx_F \subseteq \approx_P$, that is, if the representation already maintained for F is sufficient for P.*

Assumptions: As above.

Proof. $\approx_{F'} = \approx_F \cap \approx_P$ refines $\approx_F$, and each class of $\approx_F$ splits into at most $|Q_P|$ classes. The additional memory is zero exactly when no class splits, that is, $\approx_F \subseteq \approx_P$. □

Example 22.3 (An evidence archive)**.** An archive receives records, each with a source $s \in \{1, \ldots, k\}$ and a label. The rule currently in force requires, for admission of a claim, that at least one record from each of two sources has been received (2 bits suffice: two flags). A revised rule under consideration requires that the number of records from source 1 be at least 3 (a counter capped at 3: 4 values). The naive family representation is the pair (two flags, capped counter) with $4 \cdot 4 = 16$ values, but the flag for source 1 is the indicator that the counter is positive, so the representation actually needs the flag for source 2 and the capped counter, $2 \cdot 4 = 8$ values, 3 bits rather than 4. The saving is the dependence between the properties, and a design that stored each rule's state separately would pay for it.

22.2 The cost of a discarded distinction

A representation that has discarded a distinction cannot recover it by computation (Result A.2). Recovery requires new information from outside, and it has a cost.

Definition 22.4 (Cost model)**.** Maintaining a representation with b bits of distinctions costs $\rho = \varrho\, b$ per unit time, where ϱ is the rent per bit. If a distinction is not maintained and is later needed, restoring it costs a one-time amount $C \in [0, \infty]$, the *acquisition cost*; $C = \infty$ means the distinction cannot be restored. The need arises at an unknown time $T \in [0, \infty)$ (the *demand time*), after which the distinction is needed and maintained or acquired as the case may be. The acquisition cost C is the optimal value of the inspection problem of Chapter 10 when the distinction can be recovered by inspection, and is ∞ when the information has left the system (Chapter 8).

A strategy for a single distinction is a time $s \in [0, \infty]$ up to which it is maintained; if $T \le s$ the cost is ρT, and if $T > s$ the cost is $\rho s + C$. The offline optimum, knowing T, is $\min(\rho T, C)$.

New result 22.5 (Rent or buy)**.** *The strategy $s^* = C/\rho$ (maintain until the rent paid equals the acquisition cost) satisfies* $\mathrm{cost}(s^*, T) \le 2\min(\rho T, C)$ *for every demand time $T \ge 0$: it is* 2*-competitive. No deterministic strategy has a competitive ratio below* 2*. If the demand time has an exponential law with rate h, the expected-cost optimal strategy is $s = \infty$ (always maintain) if $\rho \le hC$ and $s = 0$ (never maintain) if $\rho \ge hC$.*

Assumptions: Known ρ and C; demand time unknown, adversarial in the competitive statement and exponential in the last.

Proof. For $T \le s^*$ the cost is $\rho T = \min(\rho T, C)$, since $\rho T \le C$. For $T > s^*$ it is $\rho s^* + C = 2C = 2\min(\rho T, C)$.

Let s be any deterministic strategy. If $s = 0$, take demand time $T \downarrow 0$: the cost is C and the optimum $\rho T \to 0$, so the ratio is unbounded. If $s = \infty$ take $T \to \infty$: the cost ρT against the optimum C, unbounded as $T \to \infty$ when $C < \infty$. Otherwise take $T = s + \epsilon$: the cost is $\rho s + C$ and the optimum is $\min(\rho s + \rho\epsilon, C)$, so the ratio tends to $(\rho s + C)/\min(\rho s, C) \ge 2$ as $\epsilon \downarrow 0$, since the numerator is the sum of C and ρs and the denominator is the smaller of the two.

For exponential demand with rate h: the expected cost of strategy s is $\int_0^s \rho t e^{-ht} h\, dt + e^{-hs}(\rho s + C)$. Differentiating in s gives $e^{-hs}(\rho - hC)$, which is nonpositive for all s iff $\rho \le hC$ (so $s = \infty$ is best) and nonnegative iff $\rho \ge hC$ (so $s = 0$ is best). □

The result shows that the choice between maintaining and acquiring is a rent-or-buy decision whose break-even is C/ρ, and that a rule which maintains a distinction exactly as long as the rent paid is less than the cost of re-acquisition loses at most a factor of two to a clairvoyant. When demand has a constant hazard h, no intermediate strategy is optimal: the distinction is either kept always or not at all, and the boundary is $\rho = hC$.

New result 22.6 (Irreversible loss forces indefinite maintenance)**.** *If $C = \infty$ and the demand time is finite with positive probability, every strategy with $s < \infty$ has infinite expected cost, so the only finite-cost strategy is to maintain indefinitely, at cost $\rho\,\mathbb{E}[T]$ (finite if $\mathbb{E}[T] < \infty$). If maintenance is itself impossible beyond a budget, $\rho s \le B$ for a finite B, the distinction is lost with probability at least* $\Pr(T > B/\rho)$.

Assumptions: As stated.

Proof. For $s < \infty$ the event $\{T > s\}$ has positive probability when T has unbounded support, or in general whenever it is positive, and on it the cost is infinite. For $s = \infty$ the cost is ρT

with expectation $\rho\mathbb{E}[T]$. With a budget, maintenance stops at $s = B/\rho$, and the distinction is lost on $\{T > s\}$. □

This is the cost-side statement of the irreversibility of Chapter 8: a representation that has coarsened beyond recovery imposes the maintenance obligation on every earlier time at which the distinction might later be needed. The design question is therefore which distinctions are cheap to restore and which are not, and rent-or-buy applies only to the former.

22.3 Many distinctions at once

For a family of m distinctions with rents ρ_i, acquisition costs C_i and independent demand times, Result 22.5 applies distinction by distinction when the costs are additive. They are not additive when the representation is shared, since by Result 22.2 the rent of a set of distinctions is $\varrho \log_2 |Q_S|$ for the maintained subset S and not the sum of the individual rents.

New result 22.7 (Shared rent reduces the break-even time)**.** *Let a set S of distinctions share a representation with rent $\rho_S = \varrho \log_2 |Q_S|$, and suppose demands arrive for each $i \in S$ at independent exponential rates h_i with acquisition costs C_i. The always-maintain strategy for S has lower expected cost than never maintaining if $\rho_S \le \sum_{i \in S} h_i C_i$. The sum of the individual break-even conditions $\rho_i \le h_i C_i$ implies this condition whenever $\rho_S \le \sum_i \rho_i$, which holds because $|Q_S| \le \prod |Q_i|$.*

Assumptions: Independent exponential demands; the cost of never maintaining is the sum of acquisition costs paid at the first demand for each distinction, with each distinction acquired at most once per demand; the comparison is made per unit time in the stationary regime in which each demand repeats at its rate.

Proof. In the stationary regime, never maintaining costs $\sum_i h_i C_i$ per unit time and maintaining S costs ρ_S. The first claim is the comparison. If $\rho_i \le h_i C_i$ for all i then $\sum_i h_i C_i \ge \sum_i \rho_i \ge \rho_S$, the last by the product bound of Result 22.1. □

22.4 Counters on a common event: the size of the family representation

The bounds of Result 22.1 are attained at both ends by simple families, and a third family fills in the interior exactly. Let the only event be a and let the property P_i be the count of a modulo m_i.

New result 22.8 (Counters on one event)**.** *For the properties $P_i(h) = \#_a(h) \bmod m_i$, $i = 1, \dots, r$, the continuation equivalence $\approx_i$ has m_i classes and $\approx_F$ has exactly $\operatorname{lcm}(m_1, \dots, m_r)$ classes. Hence $|Q_F| = \operatorname{lcm}(m_i)$ lies between $\max_i m_i$ (attained when each m_i divides the largest) and $\prod_i m_i$ (attained when the m_i are pairwise coprime), and the memory needed is $\log_2 \operatorname{lcm}(m_i)$ bits against $\sum_i \log_2 m_i$ bits for separate counters.*

Assumptions: One event; counts modulo m_i.

Proof. Two histories with counts n, n' are equivalent for P_i iff $n \equiv n' \pmod{m_i}$, since appending k further events shifts both counts by k and the residues differ for some k otherwise. They are equivalent for all i iff $n - n'$ is divisible by every m_i, that is, by the least common multiple. So $\approx_F$ identifies counts modulo $\operatorname{lcm}(m_i)$, giving exactly that many classes. The bounds follow from $\max m_i \le \operatorname{lcm} \le \prod m_i$, with equality on the right iff the moduli are pairwise coprime and on the left iff every modulus divides the largest. □

For the moduli $(2, 3, 4, 6)$ the family representation has lcm $= 12$ values, against a maximum of 6 and a product of 144; it needs 3.58 bits where separate counters would need 7.17. For the moduli $(2, 3, 5)$ the three counters are coprime and the saving vanishes: 30 values and 4.91 bits in both accounts. The arithmetic character of the dependence is the point: properties that are correlated through the dynamics share state, and the amount of sharing is a divisibility relation among the moduli, so a designer who chooses the moduli of related counters (retention periods, audit cycles, rotation schedules) with a common multiple in mind pays the memory of the largest cycle and not of the product.

22.5 Randomized maintenance

Result 22.5 shows that no deterministic strategy beats a competitive ratio of 2. If the strategy may be randomized, with the adversary choosing T without seeing the random draw, the ratio improves.

Established result 22.9 (Randomized rent-or-buy)**.** *In the discrete version with acquisition cost B times the unit rent, the best randomized strategy has competitive ratio $\big(1-(1-1/B)^B\big)^{-1}$, which tends to $e/(e-1) \approx 1.582$ as $B \to \infty$. The optimal distribution of the switching time k is proportional to $(1-1/B)^{B-k}$ for $k = 1, \dots, B$. The best deterministic strategy has ratio $2 - 1/B$.*

Source: Karlin, Manasse, McGeoch and Owicki (1990); Karlin, Manasse, Rudolph and Sleator (1988).

For $B = 10$ the randomized ratio is 1.5353 and the deterministic ratio is 1.9, as a linear program over the switching distribution confirms; the optimal distribution places probability 0.0595 on switching at the first step and increases geometrically to 0.1535 at the tenth. The randomized strategy matters here because the demand for a distinction is not announced: a system that maintains each of a large set of distinctions for a randomly drawn time, from a distribution of this shape, loses at most about 1.5 times the clairvoyant cost on each of them, against about 2 for the deterministic threshold, and the improvement does not depend on the demand times being drawn from any known law.

Exercises

Exercise 22.10. For the parities in Result 22.1 verify that $\approx_1 \cap \approx_2$ has four classes by exhibiting a representative for each and a separating continuation for each pair.

Exercise 22.11. Compute $|Q_F|$ for P_1 = "the number of a is even" and P_2 = "the number of a is divisible by 3". Compare with the product bound and explain the discrepancy.

Exercise 22.12. In Example 22.3 replace the revised rule by "at least 3 records from source 1 and at least one from source 2" and recompute $|Q_F|$.

Exercise 22.13. In the continuous version of rent-or-buy, with rent rate ρ, purchase price C and unknown demand length T, let the player buy at the random time S with density proportional to $e^{\rho s/C}$ on $[0, C/\rho]$. Show that the expected cost is at most $\frac{e}{e-1}\min(\rho T, C)$ for every fixed T. Explain why the guarantee requires the adversary to fix T without seeing S, and why an adversary who ends the demand just after the purchase forces ratio 2.

Exercise 22.14. Combine Result 22.5 with a finite horizon N for the demand: demands beyond N are of no concern. Show that the break-even changes from C/ρ to $\min(C/\rho, N)$ and that maintaining until N is then optimal whenever $\rho N \le C$.

Exercise 22.15. For the moduli $(4, 6, 10)$ compute lcm, the product, and the two bit counts, and say whether the representation can be built by combining counters modulo 4 and 6 into one counter modulo 12 plus the residue modulo 10.

Exercise 22.16. Show that in Result 22.8 adding a property P_{r+1} with modulus m_{r+1} costs $\log_2\big(\operatorname{lcm}(m_1,\dots,m_{r+1})/\operatorname{lcm}(m_1,\dots,m_r)\big)$ additional bits, and that this is zero iff m_{r+1} divides the previous lcm.

Exercise 22.17. For $B = 3$ solve the randomized rent-or-buy linear program by hand for the switching distribution on $k \in \{1, 2, 3\}$ and verify the ratio $(1 - (2/3)^3)^{-1} = 27/19$.

Exercise 22.18. Combine the shared rent of Result 22.7 with the randomized strategy: if m distinctions share a representation whose rent is the logarithm of the lcm of their moduli, describe a randomized schedule that drops them all at one random time, and compute its competitive ratio against the clairvoyant that pays only for the distinctions that are demanded.

Chapter 23

Architecture for Revisable Systems

Question. How are immediate efficiency and later intervention traded off?

Established sources. Modularity; reversible and deferred commitment; real options; vertex cover (Karp).

New results. The design problem as a constrained optimization over early commitments with flexibility requirements stated as admissible-intervention constraints (Definition 23.1); a commit-or-defer criterion $g > hR$ (Result 23.2); the constrained problem is polynomial for single-route requirements and NP-hard with alternative routes (Result 23.3); the efficiency price of flexibility is the total gain of the modules that interventions touch, and refining modules reduces it (Result 23.5).

Proof obligations. None.

Depends on. Chapters 15, 18, and 22.

Efficiency usually comes from commitment: a module that is fixed can be optimized, specialized, and fused with its neighbours. Intervention requires the opposite: a module that may later be replaced, reconfigured, or inspected must remain open. A design is therefore a choice of which modules to commit early, and the choice is constrained by the interventions that the system must still admit. This chapter states the choice as an optimization problem and solves it in the cases where it can be solved.

23.1 The design problem

Definition 23.1 (Design problem)**.** A *design problem* consists of a finite set M of modules, a gain $g_i \geq 0$ per unit time for committing module i (the efficiency obtained from fixing it), a reopening cost $R_i \in [0, \infty]$ (the cost of undoing the commitment when an intervention touches it), and a finite set $\mathcal{J}$ of required interventions. Each intervention $j \in \mathcal{J}$ has a set of *routes* $W_{j1}, \ldots, W_{jq_j} \subseteq M$; a route is a set of modules that must be uncommitted for the intervention to be carried out by that route. A *commitment set* $S \subseteq M$ is *flexible* if every required intervention has a route disjoint from S. The *flexibility-constrained design problem* is

$$\max_{S \subseteq M} \sum_{i \in S} g_i \quad \text{subject to} \quad \forall j \in \mathcal{J}\ \exists l :\ W_{jl} \cap S = \varnothing.$$

In the language of earlier chapters, the routes are the admissible ways of reaching an altered configuration, a committed module removes the actions that touch it, and a flexible

commitment set is one for which the set of admissible interventions retains every required one. The constraint is an admissibility constraint, and the objective is the ordinary one.

23.2 When to commit

If the interventions arrive at random, the constraint is replaced by a cost. A committed module i is reopened when an intervention touching it is demanded, at a time T_i that is unknown.

New result 23.2 (Commit or defer)**.** *Suppose the demand time T for an intervention touching module i is exponential with rate $h > 0$, a horizon $N \in (0, \infty]$ is fixed, the module yields gain g per unit time while committed and costs R to reopen, and after reopening it yields no further gain. The expected net value of committing at time 0 and not otherwise is*

$$V = \left(1 - e^{-hN}\right)\left(\frac{g}{h} - R\right) \qquad (\textit{with } 1 - e^{-hN} = 1 \textit{ if } N = \infty),$$

so committing is optimal if and only if $g > hR$, independently of the horizon. By memorylessness the same criterion applies at every later time, so the optimal policy is to commit at once or never. If $R = \infty$ and $h > 0$, the module should never be committed. If the demand time is deterministic, $T = t_0$, the module is committed if and only if $g \min(t_0, N) > R\,1\{t_0 \le N\}$.

Assumptions: Independent exponential demand; additive gain.

Proof. The net value is $g \min(T, N) - R\,1\{T \le N\}$. Its expectation is $g \int_0^N \Pr(T > t)\,dt - R \Pr(T \le N) = g\frac{1-e^{-hN}}{h} - R(1 - e^{-hN})$. This is positive if and only if $g/h > R$. For a later commitment time t the horizon becomes $N - t$ and, by memorylessness, the remaining demand time is again exponential with rate h, so the sign is unchanged. If $R = \infty$ the value is $-\infty$ for every $N > 0$ when $h > 0$. For deterministic T the value is $g \min(t_0, N)$ minus R if the demand falls inside the horizon. □

The criterion has the form of the rent-or-buy criterion of Chapter 22, with the hazard rate h the ratio between how often the module is needed and how much it earns. Two design levers appear. Raising the gain g (better specialization) favours commitment. Lowering the reopening cost R favours it too, and a good interface in the sense of Chapter 18 is one that lowers R by confining the effect of an intervention to few modules.

23.3 The constrained problem

New result 23.3 (Complexity of the flexibility-constrained design)**.** *(a) If every intervention has a single route W_j, the unique optimum is $S^* = M \setminus \bigcup_j W_j$ with value $\sum_{i \notin \bigcup W_j} g_i$, computed in linear time.*

(b) If interventions may have two routes of size one each, the decision problem "is there a flexible S with $\sum_{i \in S} g_i \ge G$?" is NP-complete, even when all $g_i = 1$.

(c) The value of the optimum is nonincreasing in the set of required interventions, and the flexible commitment sets form a down-closed family (subsets of flexible sets are flexible).

Assumptions: Finite sets; the gains are additive and nonnegative.

Proof. (a) With one route per intervention, S is flexible iff $S \cap W_j = \varnothing$ for all j, iff $S \subseteq M \setminus \bigcup_j W_j$. Since gains are nonnegative the maximum is attained at the largest such set.

(b) Membership in NP is clear: a flexible S is a certificate. For hardness reduce from VERTEX COVER: given a graph (V, E) and an integer k, let $M = V$, $g_i = 1$, and for each edge $\{u, v\} \in E$ add an intervention with the two routes $\{u\}$ and $\{v\}$. A set S is flexible iff for each edge at least one endpoint lies outside S, that is, iff $V \setminus S$ is a vertex cover. Thus $\sum_{i \in S} g_i = |V| - |V \setminus S| \geq |V| - k$ for a flexible S iff there is a vertex cover of size at most k.

(c) Adding an intervention adds a constraint, so the feasible set can only shrink. If S is flexible and $S' \subseteq S$ then $W_{jl} \cap S' \subseteq W_{jl} \cap S = \varnothing$ for the route that works for S. □

The dichotomy has an engineering reading. When each intervention has one known route, the problem is a trivial reservation of modules. When an intervention can be done in alternative ways, deciding which modules to keep open so that at least one way survives is as hard as covering. The practical approach is therefore either to make routes unique by design or to use an approximation, such as the factor-two approximation for vertex cover obtained from a maximal matching.

Example 23.4 (Three modules)**.** Modules $\{1, 2, 3\}$ with gains $(5, 3, 2)$. Interventions: j_1 with routes $\{1\}$ and $\{2\}$; j_2 with the single route $\{2, 3\}$. A set S is flexible iff $S \cap \{2, 3\} = \varnothing$ and ($1 \notin S$ or $2 \notin S$). The first condition forces $S \subseteq \{1\}$, which satisfies the second. The optimum is $S^* = \{1\}$ with value 5. If j_2 is withdrawn, the flexible sets are those omitting 1 or 2 (or both): the maximal ones are $\{1, 3\}$ (value 7) and $\{2, 3\}$ (value 5), so the optimum rises from 5 to 7. The requirement j_2 costs 2 units of gain.

23.4 The price of flexibility

New result 23.5 (Efficiency lost to flexibility)**.** *For single-route interventions, the efficiency lost relative to committing everything is* $\Lambda = \sum_{i \in \bigcup_j W_j} g_i$*. It is nondecreasing in the family of required interventions, and it is nonincreasing under* refinement*: if a module i is split into submodules i', i'' with $g_i = g_{i'} + g_{i''}$ and every route through i now contains only the submodules that the intervention actually touches, then the new loss is at most the old, with a strict decrease by $g_{i''}$ if no route touches i''.*

Assumptions: As stated.

Proof. The loss is the gain of the union of routes, which grows with the union. Under refinement the union of routes is replaced by a subset of the refined modules, whose total gain is at most that of the original union; if i'' is touched by no route it is no longer in the union and the loss decreases by $g_{i''}$. □

Example 23.6 (Refinement pays)**.** A controller module has gain 10 per unit time and an intervention that touches only its logging subroutine, which accounts for 2 of that gain. With the controller committed as a single module, flexibility costs 10. After refinement into a core (gain 8) and a logging part (gain 2), flexibility costs 2. The same intervention is carried out; the architecture changes only the grain at which commitment is applied.

23.5 A weighted instance: the price of flexibility as a vertex cover

When every intervention has two routes consisting of one module each, the interventions form the edges of a graph on the modules, and the design problem has a direct graph-theoretic form.

New result 23.7 (Flexibility price equals minimum weight vertex cover)**.** *Let the interventions be edges $\{u, v\}$ of a graph H on M, each with routes $\{u\}$ and $\{v\}$. A commitment set S is flexible if and only if S is an independent set of H, and the efficiency lost to flexibility, $\Lambda = \sum_{i \in M} g_i - \max_S \sum_{i \in S} g_i$, equals the minimum total gain of a vertex cover of H.*

Assumptions: Single-module routes; additive gains.

Proof. S is flexible iff each edge has an endpoint outside S, that is, iff no edge has both endpoints in S, so S is independent. The complement of an independent set is a vertex cover and conversely, so maximizing the gain of S is minimizing the gain of the complementary cover. □

Because exact minimum weight cover is NP-hard, the quality of approximations matters, and the unit-gain bound of the maximal matching does not carry over to unequal gains. The following instance makes the point. Eight modules have gains $(6, 5, 5, 4, 3, 3, 2, 1)$, and the required interventions are the nine pairs

$$\{1,2\}, \{2,3\}, \{3,4\}, \{4,5\}, \{5,6\}, \{6,7\}, \{7,8\}, \{2,7\}, \{1,5\}.$$

The total gain is 29. Exhaustive search over the 256 commitment sets gives the optimal flexible set $\{1, 3, 6, 8\}$ with value 15, so $\Lambda = 14$; the next best flexible sets have value 14. A greedy rule that adds modules in decreasing order of gain while preserving independence finds the optimum here. The matching heuristic that takes the first maximal matching in the listed order, $\{1,2\}, \{3,4\}, \{5,6\}, \{7,8\}$, is perfect, so it uncommits all eight modules and loses 29, which is 2.07 times the optimal loss: with unequal gains the factor two of the unit-gain argument fails.

Established result 23.8 (Local-ratio cover)**.** *For a weighted graph, process the edges in any order, and for each edge $\{u, v\}$ subtract $\min(w_u, w_v)$ from the residual weights of both endpoints. The vertices of zero residual weight form a vertex cover of total weight at most twice the minimum.*

Source: Bar-Yehuda and Even (1981).

On the instance above the procedure leaves the residual weights $(1, 0, 1, 0, 0, 0, 1, 0)$, so the cover is $\{2, 4, 5, 6, 8\}$ with weight $5 + 4 + 3 + 3 + 1 = 16$ and no member is redundant. The commitment set is its complement $\{1, 3, 7\}$ with value 13 against the optimum 15, and the loss is $16 \le 2 \cdot 14 = 28$ (in fact 1.14 times optimal). The design reading is that the approximation guarantee for unit gains does not transfer to graded gains without the local-ratio step, and that a design process that ranks modules by gain and commits greedily should be compared against the cover bound before its result is relied upon.

For the commit-or-defer rule of Result 23.2, a module with gain $g = 3$, hazard $h = \frac{1}{2}$, reopening cost $R = 5$ and horizon $N = 4$ has $g/h = 6 > 5 = R$, so committing is optimal, and the expected net value is $(1 - e^{-2})(6 - 5) = 0.865$. The same module with $R = 7$ has expected value -0.865 and should not be committed, however long the horizon.

Exercises

Exercise 23.9. For Result 23.2 compute V with $g = 3$, $h = 1/2$, $R = 5$, $N = 4$ and decide.

Exercise 23.10. Allow partial reopening: with probability θ an intervention needs only a repair of cost $R' < R$. Replace R by the expected cost and restate the criterion.

Exercise 23.11. Prove that in Result 23.3(b) the problem remains NP-hard when each module occurs in at most three routes, by reduction from vertex cover on graphs of maximum degree three.

Exercise 23.12. Give a greedy algorithm for the case in which each intervention has two routes of size one, based on a maximal matching in the graph whose edges are the interventions, and show that its value is within a factor 2 of the best flexible commitment in the sense of covering size. State what changes if the gains are unequal.

Exercise 23.13. In Example 23.4 compute the optimum if j_1 is replaced by an intervention with routes $\{1\}$, $\{2\}$, $\{3\}$, with and without j_2.

Exercise 23.14. Run the local-ratio procedure on the instance of Section 23.5 with the edges taken in reverse order, and compare the resulting cover weight with 14 and with 28.

Exercise 23.15. Show that for a path on n modules with unit gains the flexibility price is $\lfloor n/2 \rfloor$, and give the optimal commitment set. Compute the price for a cycle of length n.

Exercise 23.16. Show that when the intervention graph is bipartite the minimum weight vertex cover is computed in polynomial time by a minimum cut, and use it to solve the design problem for the bipartite graph with parts $\{1,2,3\}$, $\{4,5\}$, edges $\{1,4\},\{2,4\},\{3,5\},\{2,5\}$ and gains $(3,2,4,5,1)$.

Exercise 23.17. Modify Result 23.7 so that an intervention with routes $\{u\}$ and $\{v\}$ is required only with probability p_j, and each unmet required intervention costs a penalty P_j. State the new objective and say when it reduces to the cover problem.

Chapter 24

Integrated Case Studies and Research Directions

Question. What does a full analysis look like from specification through intervention and revision?

Established sources. Results of Parts I to VI.

New results. Four cases, each analysed from specification to a computed kernel or region: a resource-limited autonomous system (Result 24.1 with its robust form), a governed evidence archive (Results 24.2 and 24.3), an adaptive controller (Result 24.4), and an infrastructure network (Result 24.5).

Proof obligations. Each case ends with its own list of open obligations.

Depends on. Parts I to VI.

The preceding chapters treat one phenomenon at a time. A real system exhibits several together, and the value of the framework lies in how its pieces combine: the specification fixes the state and the constraint, the kernel says what is safe, the cost model says what safety costs, the revision structure says what can change. Each case below is small enough to compute exactly. The cases are not meant to be realistic; they show the order of steps and which result supplies which line of the analysis.

24.1 A resource-limited autonomous system

Specification. A vehicle moves on positions $\{0, 1, \dots, L\}$ with a charger at 0. State (x, b) with battery $b \in \{0, \dots, B\}$. Moving one step or waiting away from the charger costs one unit of battery; waiting at 0 restores $b = B$. An action is available only if the battery suffices. A state without an available action is a failure. The constraint is to never reach a failure.

New result 24.1 (Kernel of the vehicle)**.** *The viability kernel is* $\mathcal{K} = \{(x, b) : b \geq x\}$*. From* $(0, B)$ *the positions reachable inside the kernel are* $0, 1, \dots, \min(L, \lfloor B/2 \rfloor)$*, whereas without the kernel constraint positions up to* $\min(L, B)$ *are reachable. If each step costs an amount in* $[1, 1 + \delta]$ *chosen adversarially, the robust kernel is* $\{(x, b) : b \geq (1 + \delta)x\}$*.*

Assumptions: Integer batteries in the first statement; real batteries and costs in $[1, 1 + \delta]$ in the second.

Proof. The set $\{b \geq x\}$ is a viability domain: at (x, b) with $x \geq 1$ moving left gives $(x-1, b-1)$ with $b - 1 \geq x - 1$, and at $x = 0$ waiting gives $(0, B)$. Conversely if $b < x$, each step changes the position by at most one and costs at least one unit, so after b steps the battery is zero at a position $x' \geq x - b \geq 1$, where no action is available. Hence such states are not viable. From $(0, B)$ after m rightward moves the state is $(m, B - m)$, which lies in the kernel iff $m \leq B/2$, and the positions reachable ignoring the kernel are those with $m \leq B$. For the robust statement: the set $\{b \geq (1 + \delta)x\}$ is robustly invariant, since moving left with cost $c \leq 1 + \delta$ gives $b - c \geq (1 + \delta)x - (1 + \delta) = (1 + \delta)(x - 1)$, and at 0 waiting restores B; if $b < (1+\delta)x$ the adversary charges $1+\delta$ at every step, and after $b/(1+\delta) < x$ steps the vehicle is stranded. □

The reachable set shrinks by a factor of two under the safety requirement, and by a further factor $1/(1 + \delta)$ under cost uncertainty. Exploration of the corridor is then governed by Result 21.5: the unknown cell beyond position $\lfloor B/2 \rfloor$ is not safely testable from the charger with a single charge.

Obligations of this case. (i) The chance-constrained version with independent per-step costs is treated for a two-valued cost in Section 24.6; the general case with continuous costs, and the Boole allocation across many excursions, are not treated beyond the remark there. (ii) The multi-charger version, in which the kernel is the set of states within battery distance of some charger.

24.2 A governed evidence archive

Specification. An archive receives records from two sources and admits claims under a rule. The constitutional constraint is C: no claim is admitted without sourced evidence. The rule r_0 admits a claim when at least one record from each source has been received; r_1 admits it when at least three records from source 1 and one record from source 2 have been received; r_2 admits any claim. The revision graph is $r_0 \to r_1 \to r_2$; r_0, r_1 admit C and r_2 does not.

New result 24.2 (Protection and entrenchment)**.** *The amendment depth of C at r_0 is* 2*. If a rule r_1' equal to r_1 in its admission test is introduced with* $\mathrm{Rev}(r_1) = \{r_1, r_1'\}$ *and* $\mathrm{Rev}(r_1') = \{r_1', r_2\}$*, the depth is* 3*. The representation that supports both r_0 and r_1 has* 8 *values, namely the pair (indicator that source* 2 *has reported, count of source* 1 *capped at* 3*), against* 4 *values for r_0 alone.*

Assumptions: Finite revision graph.

Proof. The shortest path from r_0 to r_2, the only rule not admitting C, has length 2, and with the extra layer length 3 (Result 19.3). The Nerode classes of r_0's admission test are the pairs of indicators that sources 1 and 2 have reported (4 classes); those of r_1's are the capped count and the source-2 indicator (8 classes), and the indicator of source 1 is a function of the count. By Result 22.1 the family representation has the intersection of the kernels and therefore 8 values, which is the larger of the two and not their product 32. □

Supporting both rules costs one additional bit over r_0 alone (Result 22.2).

New result 24.3 (Keeping the extra bit)**.** *Let the extra bit cost rent $\rho = 1$ per month and let re-acquiring it after an amendment to r_1 cost $C = 12$ (re-requesting records). If amendments to r_1 arrive at rate h per month, the extra bit should be kept always if $h > 1/12$ and never otherwise; at $h = 1/24$ it is not kept, at $h = 1/6$ it is. A deterministic keep-then-drop rule that keeps it for 12 months is 2-competitive.*

Assumptions: Exponential demand.

Proof. Result 22.5 with $\rho = 1$, $C = 12$: the boundary is $\rho = hC$, that is $h = 1/12$; and $s^* = C/\rho = 12$. □

Obligations of this case. (i) A monitor for the bounded-response obligation "every submitted claim receives a decision within k steps" composed with the rules, whose kernel is given by Result 14.2; the interaction between amendment and open obligations is not analysed. (ii) Estimation of the amendment hazard h from the archive's own history.

24.3 An adaptive controller

Specification. A scalar state $x_{t+1} = x_t + 1 + b\,u_t$ on integers with $b \in \{-1, +1\}$ an unknown constant gain, inputs $u \in \{-1, 0, 1\}$, constraint $K = \{-3, \dots, 3\}$.

New result 24.4 (Three kernels)**.** *If b is known, the viability kernel is all of K. If b is unknown but constant, the controller observes the state and learns b from any step with $u \neq 0$; the set of initial states from which the constraint can be maintained is $\{-3, \dots, 1\}$. If b may change at every step, the robust kernel is empty.*

Assumptions: Observation of the state at each step; the controller knows the form of the dynamics.

Proof. Known b: the choice $u = -b$ gives $x_{t+1} = x_t + 1 - b^2 = x_t$, so every state in K is held. Unknown constant b, before learning: the action $u = 0$ gives $x + 1$ without information, and $u = \pm 1$ gives $x + 1 \pm b \in \{x, x + 2\}$, the two values of b giving different outcomes, so b is known afterwards and the state is held at its new value. Learning in one step succeeds without leaving K iff the worst outcome $x + 2$ lies in K, that is $x \leq 1$. For $x \in \{2, 3\}$ the action $u = \pm 1$ has a possible outcome $x + 2 \geq 4$, and the action $u = 0$ leads to $x + 1$ with b still unknown, from which the same dilemma recurs and the state leaves K within two steps. Hence the safe set is $\{-3, \dots, 1\}$ (states below -3 are outside K). Time-varying b: no learning is possible. Let S be a current candidate set with maximum m. From m, the action $u = 0$ leads to $m + 1 \notin S$, and each action $u = \pm 1$ has, for the adversarial sign of b, the outcome $m + 2 \notin S$. So m is removed, and the iteration of the predecessor operator removes the maximum at each stage until the set is empty. □

The price of learning is the loss of $\{2, 3\}$ from the kernel, and the price of an adversary able to change b at will is everything. The step with $u \neq 0$ is the exciting action, and its worst-case cost $x \mapsto x + 2$ is the explicit price of identification. When a handover to the automatic controller $u = -b$ follows identification, the arrangement is an instance of relegation as in Chapter 17 with handover time $\tau = 1$ and the common core K on which the automatic controller is invariant.

Obligations of this case. (i) Noisy identification: with observation noise the learning step may fail, and the kernel must be replaced by the probabilistic version of Chapter 13. (ii) Continuous parameter sets, where identification takes infinitely many steps and the guarantee requires a rate.

24.4 An infrastructure network

Specification. A service r requires n regions $e_1, \dots, e_n$; region e_i is supplied by any one of m_i lines $p_{i,1}, \dots, p_{i,m_i}$, each carrying load 1 when used; the capacity is $\Gamma = n$. This is the dependency system of Chapter 16, with the lines as primitives.

New result 24.5 (Redundancy is not capacity)**.** *The fallback region is* $\mathrm{FB}(n) = \{F : \forall i,\ F \not\supseteq \{p_{i,1}, \dots, p_{i,m_i}\}$ *and* $F \cap \{r, e_1, \dots, e_n\} = \varnothing\}$ *and* $\mathrm{FB}(n-1) = \varnothing$*. The system tolerates every set of k simultaneous line faults if and only if $m_i \geq k+1$ for all i, so tolerance of k faults requires $n(k+1)$ lines in total, while the capacity $\Gamma = n$ is unchanged. The number of maximal elements of* $\mathrm{FB}(n)$ *is* $\prod_i m_i$*.*

Assumptions: Faults hit primitives only.

Proof. A support must contain r, all e_i, and one line for each i, so its least load is n; hence $\mathrm{FB}(n-1) = \varnothing$ and, by Result 16.5(a), $F \in \mathrm{FB}(n)$ iff some support of load n is disjoint from F, i.e. iff each region retains a line and $r, e_i \notin F$. A set F of k line faults destroys region i iff it contains all m_i lines, which is impossible when $m_i \geq k+1$; if some $m_i \leq k$ then $F = \{p_{i,\cdot}\}$ is a k-fault pattern outside $\mathrm{FB}(n)$. The minimal supports choose one line per region, $\prod m_i$ of them, with pairwise incomparable complements. □

The redundancy needed is paid in components and not in capacity, and the number of maximal fault patterns that the system survives, $\prod m_i$, grows multiplicatively in the redundancy. This is the sense in which the fallback region carries more information than the single capacity number.

Obligations of this case. (i) Cascading failure, in which the load of a failed line moves to its neighbours and raises their hazard; the fallback region must then be defined along a trajectory rather than for a static fault set. (ii) Repair sequencing with the recoverable region of Result 16.1.

24.5 Research directions

The cases show where the present results run out. Four directions recur. First, quantitative robustness: the kernels above are worst-case, and the stochastic variants (chance constraints along trajectories) are available only for linear Gaussian systems, with the general case open. Second, learning of the model while preserving the kernel, of which the adaptive controller is the smallest example, requires identification rates in the continuous case. Third, institutional dynamics: the amendment hazard in the archive is exogenous, whereas in real institutions it depends on the state of the system, and the augmented system is then a game rather than a control problem. Fourth, computation: the NP-hardness results of Chapters 16 and 23 suggest studying structured subclasses (tree-like dependency, bounded interface width) where exact algorithms are polynomial.

24.6 The vehicle under random costs

The robust kernel of Result 24.1 charges every step at the worst cost. If the cost of each step is random, independent of the action, equal to 1 with probability $1-p$ and to 2 with probability p, the worst-case threshold $b \geq 2x$ is wasteful, and a chance constraint per excursion gives a smaller one. Fix a risk level ε and call (x, b) *ε-safe* if the vehicle, following the best policy, returns to the charger before the battery is exhausted with probability at least $1-\varepsilon$.

New result 24.6 (Chance kernel of the vehicle)**.** *The state (x, b) is ε-safe if and only if $b \geq x + q_{1-\varepsilon}(x)$, where $q_{1-\varepsilon}(x)$ is the least integer q with $\Pr[\mathrm{Bin}(x,p) \leq q] \geq 1-\varepsilon$. Returning immediately is optimal. The threshold satisfies $x \leq x + q_{1-\varepsilon}(x) \leq 2x$ and, as $x \to \infty$, $x + q_{1-\varepsilon}(x) = (1+p)x + z_{1-\varepsilon}\sqrt{p(1-p)x} + o(\sqrt{x})$ with $z_{1-\varepsilon} = \Phi^{-1}(1-\varepsilon)$.*

Assumptions: Costs independent of the action and of one another; the battery resets at the charger.

Proof. Reaching the charger from x takes at least x steps, and the cost of the first $k \geq x$ steps is at least the cost S_x of the first x steps, because costs are positive and do not depend on the action. The vehicle survives only if its battery covers all steps taken, so the survival probability of any policy is at most $\Pr[S_x \leq b]$, and moving left at every step attains it. Since $S_x = x + \mathrm{Bin}(x,p)$, survival holds with probability at least $1-\varepsilon$ iff $b - x \geq q_{1-\varepsilon}(x)$. The bounds follow from $0 \leq \mathrm{Bin}(x,p) \leq x$, and the expansion is the central limit theorem for the binomial distribution, whose quantile is $px + z\sqrt{p(1-p)x} + o(\sqrt{x})$. □

The expected form that Result 24.1 gave was a threshold of order $x + \kappa\sqrt{x}$; the result shows that the drift of the mean cost enters first, as $(1+p)x$, and the fluctuation term $\kappa\sqrt{x}$ is a correction on top of it. For $p = 0.2$ and $\varepsilon = 0.05$ the exact thresholds, computed from the binomial distribution, together with the Gaussian approximation $1.2x + 1.645\sqrt{0.16x}$ and the worst-case threshold $2x$ are as follows.

x	exact $x+q$	Gaussian	worst case $2x$
1	2	1.86	2
3	5	4.74	6
5	8	7.47	10
10	14	14.08	20
20	27	26.94	40
50	65	64.65	100
100	127	126.58	200

For small x the Gaussian approximation can fall below the exact threshold (at $x = 3$ it gives 4.74 where 5 is required), which is the reason the exact binomial quantile is stated. For $x = 100$ the chance-constrained vehicle ventures out to a battery requirement of 127, a saving of 36% over the worst-case 200, at the cost of a risk of 5% per excursion. Across N excursions the Boole allocation of Chapter 13 requires the per-excursion level ε/N, which raises z only logarithmically in N.

Exercises

Exercise 24.7. For the vehicle with $B = 7$ and $L = 10$, list the reachable positions inside the kernel and the positions reachable without it.

Exercise 24.8. Add a second charger at position L to the vehicle and compute the kernel. Where is the farthest position reachable from $(0, B)$?

Exercise 24.9. In the archive, find the representation size for the family $\{r_0, r_1\}$ if r_1 is replaced by "at least three records in total, one of them from source 2".

Exercise 24.10. For the adaptive controller extend the input set to $\{-2, \dots, 2\}$ and compute the three kernels.

Exercise 24.11. For the network with $n = 3$ and $m = (2, 2, 3)$, count the maximal elements of the fallback region and the number of line-fault patterns of size two that lie outside it.

Exercise 24.12. Verify the first three exact thresholds of Section 24.6 by computing $\Pr[\mathrm{Bin}(x, 0.2) \le q]$ for $x = 1, 2, 3$.

Exercise 24.13. Compute the exact threshold at $x = 10$ for $\varepsilon = 0.01$ and for $\varepsilon = 0.001$, and compare with $2x$.

Exercise 24.14. Allow the vehicle to detect the realized cost and to turn back at any step. Show that returning immediately remains optimal, and explain why the answer would change if costs depended on the direction of travel.

Exercise 24.15. Suppose the vehicle must make $N = 10$ excursions of length $x = 10$ with total failure probability at most 0.05. Compute the per-excursion level and the required battery by the Boole allocation, and by the exact independent allocation $1 - 0.95^{1/10}$.

Appendix A

Sufficiency Recap from the First Volume

This appendix collects, with proofs, the results of the first volume that this volume uses. It is self-contained, so the second volume can be read first. Statuses follow the first volume: results marked established are standard, and results marked new are elementary consequences proved here in full. Throughout, X is a nonempty set of cases, a *representation* is a function $r : X \to R$, and the *kernel* of a function f on X is the equivalence relation $\ker f = \{(x, x') : f(x) = f(x')\}$.

A.1 Kernels and factorization

Established result A.1 (Factorization)**.** *For functions $r : X \to R$ and $f : X \to B$ there is a function $g : r(X) \to B$ with $f = g \circ r$ if and only if $\ker r \subseteq \ker f$. The function g is then unique.*

Source: Standard quotient factorization.

Proof. If $f = g \circ r$ and $r(x) = r(x')$ then $f(x) = g(r(x)) = f(x')$. Conversely define $g(r(x)) = f(x)$; this is well defined because $r(x) = r(x')$ forces $f(x) = f(x')$, and it is unique because r maps onto $r(X)$. □

Established result A.2 (Joint kernel and coarsening)**.** *For functions $f_1, \dots, f_m$ on X the representation $(f_1, \dots, f_m)$ has kernel $\bigcap_i \ker f_i$. If $r' = g \circ r$ then $\ker r \subseteq \ker r'$: a coarsening can answer no question that r cannot, and discarded distinctions are not recovered by any later computation.*

Proof. $(f_i)(x) = (f_i)(x')$ if and only if every f_i agrees. If $r(x) = r(x')$ then $g(r(x)) = g(r(x'))$. □

A.2 Sufficiency

A *policy* is a function $\pi : X \to A$. A representation r is *sufficient* for π if $\pi = d \circ r$ for some decision rule d, equivalently if $\ker r \subseteq \ker \pi$.

New result A.3 (Minimal sufficient representation)**.** *The representation r is sufficient for π and takes the fewest possible values among sufficient representations if and only if* $\ker r = \ker \pi$*, and then it takes $|\pi(X)|$ values. For a family Φ of policies, r is sufficient for every member if and only if* $\ker r \subseteq \kappa(\Phi) := \bigcap_{\pi\in\Phi} \ker \pi$*, and the least number of values is $|X/\kappa(\Phi)|$.*

Proof. Sufficiency is Result A.1. A representation with $\ker r \subseteq \ker \pi$ has at least as many classes as $\ker \pi$, with equality exactly when the kernels are equal. The statement for a family follows by applying the same argument to the joint kernel. □

New result A.4 (Compression is free only for injective representations)**.** *A representation of X is sufficient for every policy $X \to \{0, 1\}$ if and only if it is injective.*

Proof. An injective r satisfies $\pi = (\pi \circ r^{-1}) \circ r$. If $r(x) = r(x')$ with $x \neq x'$ then the indicator of $\{x\}$ separates them, so $\ker r \not\subseteq \ker \pi$. □

A.3 Histories, continuation equivalence, and maintainable representations

Let E be an alphabet and $\mathcal{H} \subseteq \mathsf{E}^*$ a prefix-closed set of histories. A *property* is a function P on $\mathcal{H}$. The *continuation equivalence* is $h \approx_P h'$ if and only if $\{w : hw \in \mathcal{H}\} = \{w : h'w \in \mathcal{H}\}$ and $P(hw) = P(h'w)$ for all such w. A representation r of $\mathcal{H}$ is *maintainable* if $\ker r$ is a right congruence: $r(h) = r(h')$ implies $r(ha) = r(h'a)$ for every event a with $ha, h'a \in \mathcal{H}$.

Established result A.5 (Myhill–Nerode)**.** *The relation $\approx_P$ is the coarsest right congruence contained in* $\ker P$*. A maintainable representation r is sufficient for P if and only if* $\ker r \subseteq \approx_P$*, and the minimal number of values of a maintainable sufficient representation is the number of classes of $\approx_P$.*

Source: Myhill (1957); Nerode (1958).

Proof. Taking $w = \varepsilon$ shows $\approx_P \subseteq \ker P$. If $h \approx_P h'$ and $ha \in \mathcal{H}$ then $\{w : haw \in \mathcal{H}\} = \{w : h'aw \in \mathcal{H}\}$ and $P(haw) = P(h'aw)$, so $ha \approx_P h'a$. If $\sim$ is a right congruence inside $\ker P$ and $h \sim h'$ then $hw \sim h'w$ for every w, so $P(hw) = P(h'w)$, and the extensions agree because $\sim$ is a right congruence on a prefix-closed set; hence $\sim \subseteq \approx_P$. A maintainable representation sufficient for P has kernel a right congruence inside $\ker P$, hence inside $\approx_P$, which gives the counting statement. □

A.4 Safety and viable histories

Assume in this section that E is finite. A *safety condition* is given by a prefix-closed set $\mathcal{S} \subseteq \mathcal{H}$ of acceptable histories. A history $h \in \mathcal{S}$ is *viable* if for every n there is w with $|w| = n$ and $hw \in \mathcal{S}$.

Established result A.6 (Viable histories)**.** *The set* $\mathrm{Viable}(\mathcal{S})$ *of viable histories is prefix closed, contained in $\mathcal{S}$, and is the largest subset $\mathcal{V} \subseteq \mathcal{S}$ such that every $h \in \mathcal{V}$ has an extension $ha \in \mathcal{V}$ with $a \in \mathsf{E}$.*

Source: Alpern and Schneider (1985); Kechris on pruned trees.

Proof. Prefix closure: if hu is viable, extensions of hu of every length give extensions of h of every length, and $h \in \mathcal{S}$ by prefix closure of $\mathcal{S}$. Successor property: if h is viable then for each n there is an acceptable extension of length $n+1$, which begins with some event; since E is finite, one event a occurs as the first event for infinitely many n, and prefix closure of $\mathcal{S}$ then gives ha acceptable extensions of every length, so ha is viable. Maximality: if $\mathcal{V} \subseteq \mathcal{S}$ has the successor property then each $h \in \mathcal{V}$ has extensions in $\mathcal{V} \subseteq \mathcal{S}$ of every length, so h is viable. □

The set Viable($\mathcal{S}$) is the pruned tree of $\mathcal{S}$ in the language of the first volume; Chapter 3 identifies it with the set of histories whose reached state lies in the viability kernel.

Appendix B

Linear Algebra and Convexity

This appendix collects the facts from linear algebra and convex geometry used in Chapters 11 and 12. Statuses follow the main text.

B.1 Spectral radius and growth

For a square real matrix A the *spectral radius* is $\rho(A) = \max\{|\lambda| : \lambda \text{ an eigenvalue of } A\}$.

Established result B.1 (Gelfand's formula)**.** *For any submultiplicative matrix norm,* $\rho(A) = \lim_{n\to\infty} \|A^n\|^{1/n}$.

Source: Gelfand (1941); see Horn and Johnson, *Matrix Analysis*, Corollary 5.6.14.

New result B.2 (Counting walks grows at the spectral radius)**.** *Let* $A \in \mathbb{R}_{\geq 0}^{N\times N}$ *be the adjacency matrix of a directed multigraph and* $\mathbf{1}$ *the all-ones vector. Then* $\mathbf{1}^\top A^n \mathbf{1}$ *is the number of walks of length* n*, and*

$$\lim_{n\to\infty} \left(\mathbf{1}^\top A^n \mathbf{1}\right)^{1/n} = \rho(A),$$

with the convention that the limit is 0 *when* A *is nilpotent (no walks of some length). Consequently the number of walks grows exponentially with rate* $\log \rho(A)$ *if* $\rho(A) > 1$*, polynomially if* $\rho(A) = 1$*, and is eventually zero if* $\rho(A) = 0$*.*

Assumptions: Nonnegative integer entries.

Proof. Take the norm $\|B\|_\infty = \max_i \sum_j |B_{ij}|$, which is submultiplicative. For nonnegative B, $\|B\|_\infty = \max_i (B\mathbf{1})_i$, so $\|A^n\|_\infty \leq \mathbf{1}^\top A^n \mathbf{1} \leq N\|A^n\|_\infty$. Taking nth roots and using $N^{1/n} \to 1$ together with Result B.1 gives the limit. The classification follows: if $\rho > 1$ the count is at least $(\rho - \epsilon)^n$ eventually, and so on; if $\rho(A) = 0$ then A is nilpotent and $A^n = 0$ for $n \geq N$. $\square$

Established result B.3 (Perron–Frobenius)**.** *A nonnegative irreducible matrix* A *has a simple eigenvalue equal to* $\rho(A) > 0$ *with a strictly positive eigenvector, and no nonnegative eigenvector belongs to any other eigenvalue. For a general nonnegative* A*,* $\rho(A)$ *is an eigenvalue with a nonnegative eigenvector, and* $\rho(A)$ *equals the maximum of* ρ *over the irreducible diagonal blocks of its Frobenius normal form.*

Source: Perron (1907); Frobenius (1912); see Horn and Johnson, Chapter 8.

The last clause is the form used in the growth theorem of Chapter 11: the growth rate of the count of walks is the maximum, over strongly connected components reachable from the initial state, of the component's spectral radius.

B.2 Convex sets, support functions, and scalarization

Established result B.4 (Separating hyperplane)**.** *If $C \subseteq \mathbb{R}^n$ is closed and convex and $p \notin C$, there exist $w \in \mathbb{R}^n$ and β with $w \cdot p < \beta \le w \cdot c$ for all $c \in C$.*

Source: Rockafellar, *Convex Analysis*, Theorem 11.3.

The *support function* of a bounded set A is $h_A(w) = \sup_{a \in A} w \cdot a$.

New result B.5 (Support functions add under Minkowski sums and ignore convexification)**.** *For bounded $A, B \subseteq \mathbb{R}^n$: $h_{A+B} = h_A + h_B$ with $A + B = \{a + b\}$, and $h_{\mathrm{conv}A} = h_A$. A linear objective over a finite set attains its optimum at an extreme point of the convex hull.*

Assumptions: Boundedness.

Proof. $\sup_{a,b} w \cdot (a + b) = \sup_a w \cdot a + \sup_b w \cdot b$. For the convex hull, a convex combination has $w \cdot \sum \lambda_i a_i = \sum \lambda_i w \cdot a_i \le \max_i w \cdot a_i$. A finite set's hull is a polytope, a linear function on which is maximized at a vertex, and every vertex is a point of the set. □

New result B.6 (Scalarization misses interior points)**.** *Let $P \subset \mathbb{R}^n$ be finite and $p \in P$ a convex combination of other points of P with positive weights ($p \in \mathrm{relint}\,\mathrm{conv}(P \setminus \{p\})$). Then there is no $w \in \mathbb{R}^n$ such that p is the unique maximizer of $w \cdot x$ over P.*

Assumptions: Finite set.

Proof. If $p = \sum_i \lambda_i q_i$ with $\lambda_i > 0$, $\sum \lambda_i = 1$, $q_i \in P \setminus \{p\}$, then $w \cdot p = \sum \lambda_i w \cdot q_i \le \max_i w \cdot q_i$, so p is not the unique maximizer. □

B.3 Linear inequalities

Established result B.7 (Farkas' lemma)**.** *For a matrix A and a vector b, exactly one of the following holds: there exists $x \ge 0$ with $Ax = b$, or there exists y with $A^\top y \ge 0$ and $b^\top y < 0$.*

Source: Farkas (1902); Schrijver, *Theory of Linear and Integer Programming*, Section 7.3.

The lemma is the basis of the marking-equation certificates of Chapter 6: a vector y with $y^\top C \ge 0$, $y^\top (m - m_0) < 0$ is a linear invariant excluding the marking m.

B.4 Worked examples

Example B.8 (Growth of a two-component walk count)**.** Let $A = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$, the adjacency matrix of a graph with a loop of multiplicity two at vertex 1, an edge $1 \to 2$, a loop at 2, an edge $2 \to 3$ and a loop at 3. The strongly connected components are $\{1\}, \{2\}, \{3\}$ with

spectral radii $2, 1, 1$, so $\rho(A) = 2$ and the number of walks of length n from vertex 1 grows like 2^n (Result B.2). From vertex 2 the reachable components have radius 1 and the number of walks grows polynomially: the walks stay at 2 for i steps and then at 3 for the remaining $n - i$ steps, so there are $n + 1$ of them, linear in n.

Example B.9 (Scalarization on a three-point front)**.** The points $(1, 4), (2, 2), (4, 1)$ in the plane are all Pareto-minimal for the componentwise order. For weights $w = (w_1, w_2) \geq 0$ the minimizers of $w \cdot x$ are $(1, 4)$ for $w_1/w_2 < 1/2$, $(4, 1)$ for $w_1/w_2 > 2$, and $(2, 2)$ for $\frac{1}{2} < w_1/w_2 < 2$; the point $(2, 2)$ is a weighted minimizer because it lies below the segment from $(1, 4)$ to $(4, 1)$, whose midpoint $(2.5, 2.5)$ it dominates. Replace it by $(3, 3)$: now $(3, 3)$ lies above the segment, is Pareto-minimal, and is the minimizer of no weight vector, as Result B.6 predicts.

Exercises

Exercise B.10. Compute the spectral radius and the exact number of walks of length n from vertex 1 for the matrix of Example B.8, and verify that the ratio of consecutive counts tends to 2.

Exercise B.11. Prove the last clause of Result B.3 for a block upper-triangular matrix: the spectrum of the matrix is the union of the spectra of the diagonal blocks.

Exercise B.12. Show that $h_A(w) = \max_{a \in A} w \cdot a$ for a polytope A and compute the support function of the square $[0, 1]^2$ in the direction $(1, -1)$.

Exercise B.13. Using Farkas' lemma, show that the marking $(0, 2)$ is unreachable in the net with one transition consuming one token from place 1 and producing two tokens in place 2, from the initial marking $(1, 0)$ and with a linear invariant.

Appendix C

Graph Algorithms

The algorithms used throughout the volume, with the running times quoted in the text. Graphs are finite and directed, $G = (V, E)$ with $n = |V|$, $m = |E|$.

C.1 Components and shortest paths

Established result C.1 (Strongly connected components)**.** *The strongly connected components of G, and the acyclic condensation graph on them, are computed in time $O(n + m)$.*

Source: Tarjan (1972).

Established result C.2 (Shortest paths)**.** *Single-source shortest paths: breadth-first search in $O(n + m)$ for unit weights; Dijkstra's algorithm in $O(m + n \log n)$ for nonnegative weights; Bellman–Ford in $O(nm)$ for arbitrary weights without negative cycles; all pairs by Floyd–Warshall in $O(n^3)$.*

Source: Dijkstra (1959); Bellman (1958); Ford (1956); Floyd (1962); Warshall (1962); see Cormen, Leiserson, Rivest and Stein.

The weighted costs of Chapters 8 and 19 use the second statement; the quasi-metric property of the return cost is the triangle inequality of shortest-path distances.

C.2 Computing the viability kernel

Let a finite system be given by states X and a relation $E \subseteq X \times U \times X$ (a nondeterministic transition), and a constraint set $C \subseteq X$. For the deterministic case $|E| = |X||U|$.

New result C.3 (Linear-time kernel)**.** *The viability kernel* $\mathrm{Viab}(C)$ *is computed in time $O(|X| + |E|)$ by the following counter algorithm. Maintain for each state–action pair (x, u) a flag* alive*, initially true if all successors of (x, u) lie in C and false otherwise; maintain for each state $x \in C$ the number $c(x)$ of alive actions at x. Place in a queue every state of C with $c(x) = 0$. While the queue is nonempty, remove x, mark x* dead*, and for each pair (y, u) with x a successor of (y, u) and (y, u) alive, set (y, u) not alive and decrement $c(y)$, queuing y if $c(y)$ reaches 0 and y is not yet queued. The states of C never marked dead form the kernel.*

Assumptions: Finite system; the controller chooses the action and nature the successor (the nondeterministic case is the robust kernel).

Proof. Soundness. Let S be the set of surviving states. Every $x \in S$ has an alive action: $c(x) > 0$ at termination. An alive action has all its successors in C and in S (a successor marked dead would have killed the pair), so S is a viability domain contained in C, hence $S \subseteq \mathrm{Viab}(C)$ by the maximality in Result 3.2.

Completeness. Let x be marked dead; we show $x \notin \mathrm{Viab}(C)$ by induction on the order of marking. When x is queued, every action at x either has a successor outside C or a successor that was marked dead earlier, hence outside $\mathrm{Viab}(C)$ by induction. So no action at x keeps all successors in $\mathrm{Viab}(C)$, and $x \notin \mathrm{Viab}(C)$ (a state of the kernel needs such an action).

Time. Each pair is examined once when it is set not alive, and each state is dequeued once; the predecessor lists are built in $O(|X| + |E|)$. □

The algorithm also gives the kernel's *rank*: recording the dequeue time of each dead state gives the least number of steps within which an adversary can force a violation, the quantity used for the lookahead of Chapter 15.

C.3 Closure and hardness

Established result C.4 (Horn closure)**.** *The closure of a set of facts under a set of Horn rules (c, P) (derive c when all of P are derived) is computed in time linear in the total size of the rules.*

Source: Dowling and Gallier (1984).

This is the algorithm behind the capability closure of Chapter 10; it is the same as Result C.3 with counters of unsatisfied premises.

Established result C.5 (Classical hardness)**.** *SET COVER and VERTEX COVER are NP-complete, and VERTEX COVER has a polynomial-time 2-approximation (take both ends of every edge of a maximal matching).*

Source: Karp (1972); Garey and Johnson, *Computers and Intractability.*

The reductions of Chapters 16 and 23 start from these problems.

C.4 Worked example of the kernel algorithm

Example C.6 (Tracing the counter algorithm)**.** States $\{1, 2, 3, 4\}$, constraint $C = \{1, 2, 3\}$, state $4 \notin C$. Actions: at 1, $a \to 2$; at 2, $a \to 3$ and $b \to 4$; at 3, $a \to 4$. The flags: $(1, a)$ true, $(2, a)$ true, $(2, b)$ false, $(3, a)$ false. The counts are $c(1) = 1$, $c(2) = 1$, $c(3) = 0$. State 3 is queued. Removing 3 clears $(2, a)$, so $c(2) = 0$ and 2 is queued; removing 2 clears $(1, a)$, so $c(1) = 0$ and 1 is queued. All of $1, 2, 3$ are removed and the kernel is empty. The removal order $3, 2, 1$ is the order of exit steps: from state 3 the adversary wins in 1 step, from 2 in 2, from 1 in 3.

New result C.7 (Shortest exit and the rank function)**.** *For the robust kernel with disturbance, let* $\mathrm{rk}(x)$ *be the step of the iteration of Result C.3 at which x is removed (∞ for kernel states). Then* $\mathrm{rk}(x) \le |C|$ *and, for* $\mathrm{rk}(x) < \infty$*, the adversary can force a violation within* $\mathrm{rk}(x)$ *steps against any strategy, and not within* $\mathrm{rk}(x) - 1$ *steps against the best strategy.*

Assumptions: Finite system.

Proof. The first and second claims are the induction in the proof of Result C.3. For the third: if x survived $k-1$ iterations, where $k = \mathrm{rk}(x)$, then there is an action all of whose successors survived $k-2$ iterations, hence by induction the controller can avoid violation for $k-1$ steps. $\square$

Exercises

Exercise C.8. Run Tarjan's algorithm by hand on the graph with edges $1 \to 2, 2 \to 3, 3 \to 1, 3 \to 4, 4 \to 5, 5 \to 4$ and list the components in the order in which they are completed.

Exercise C.9. Give a graph on which Dijkstra's algorithm with a negative edge returns a wrong distance.

Exercise C.10. Show that the counter algorithm of Result C.3 specializes, for deterministic systems with one action per state, to the removal of all states from which C^c is reachable.

Exercise C.11. Write the Horn rules for the capability closure with three capabilities c_1, c_2, c_3 where c_3 needs both others and c_2 needs c_1, and compute the closure of $\{c_1\}$.

Appendix D

Dynamical Systems

Discrete-time systems $x_{t+1} = f(x_t, u_t)$ on a metric space, with invariance, Lyapunov functions, and the contraction principle.

D.1 Invariance and viability

A set S is *invariant* for $x_{t+1} = f(x_t)$ if $f(S) \subseteq S$. For a controlled system, S is a *viability domain* if every $x \in S$ has u with $f(x, u) \in S$ (Chapter 3).

New result D.1 (Invariant intervals of a scalar affine system)**.** *For $x_{t+1} = ax_t + u_t$ with $|u_t| \le 1$, the interval $[-c, c]$ is a viability domain iff $|a|c \le c + 1$. Hence it is one for every $c > 0$ if $|a| \le 1$, and for $c \le 1/(|a| - 1)$ if $|a| > 1$. The viability kernel of $K = [-k, k]$ is $[-c^*, c^*]$ with $c^* = k$ if $|a| \le 1$ and $c^* = \min(k, 1/(|a| - 1))$ otherwise.*

Assumptions: Real scalar state.

Proof. From $x \in [-c, c]$ the reachable next states form $[ax - 1, ax + 1]$, which meets $[-c, c]$ iff $|ax| \le c + 1$. The worst case is $|x| = c$, giving the condition. For $|a| > 1$ the condition reads $c(|a| - 1) \le 1$. The kernel of K is the largest viability domain inside it, which is $[-\min(k, c_{\max}),\ \min(k, c_{\max})]$ as every viability domain of the form $[-c, c]$ satisfies the condition, and any viability domain is contained in one of this form. □

The last claim is Result 13.3 in the deterministic case, and gives the example $|u| \le 1$, $K = [-3, 3]$ of Chapter 3.

D.2 Lyapunov functions

New result D.2 (Lyapunov functions give invariant sets)**.** *Let $V : X \to \mathbb{R}$ and suppose that for every x with $V(x) \le \beta$ there is a control with $V(f(x, u)) \le V(x)$. Then the sublevel set $\{V \le \beta\}$ is a viability domain. If in addition $V(f(x, u)) \le V(x) - \epsilon$ whenever $V(x) > \beta_0$, with $\epsilon > 0$, and $V \ge 0$, then from any x_0 the sublevel set $\{V \le \beta_0\}$ is reached within $\lceil V(x_0)/\epsilon \rceil$ steps, and a trajectory started in $\{V \le \beta\}$ with $\beta \ge \beta_0$ stays there.*

Assumptions: Controls chosen as stated.

Proof. The first statement is the definition of viability for the set. For the second, while the trajectory is above β_0 the function decreases by at least ϵ per step and is nonnegative, so after at most $V(x_0)/\epsilon$ steps it is at most β_0; below β_0 the nonincrease condition keeps it below β. □

This is the form of progress function used in Chapter 16: the bound on recovery time is $\lceil V(x_0)/\epsilon \rceil$.

Established result D.3 (Stability of linear systems)**.** *The system $x_{t+1} = Ax_t$ has $x_t \to 0$ for all x_0 if and only if $\rho(A) < 1$; if $\rho(A) > 1$ there are initial states with $\|x_t\| \to \infty$. If $\rho(A) < 1$ there is a norm $\|\cdot\|_*$ with $\|A\|_* < 1$, whose unit ball is invariant.*

Source: Standard; see Horn and Johnson, Lemma 5.6.10.

D.3 Contraction

Established result D.4 (Banach fixed-point theorem)**.** *Let (Y, d) be a complete metric space and $T : Y \to Y$ with $d(Ty, Ty') \le \gamma d(y, y')$ for some $\gamma < 1$. Then T has a unique fixed point y^*, and $d(T^k y_0, y^*) \le \gamma^k d(y_0, y^*)$.*

Source: Banach (1922).

The Bellman operator of a discounted MDP is a γ-contraction in the sup-norm (Appendix E), which gives existence and uniqueness of the value function.

D.4 Worked examples

Example D.5 (A Lyapunov recovery bound)**.** For $x^+ = \frac{1}{2}x + u$ with $|u| \le \frac{1}{4}$, take $V(x) = |x|$. If $|x| > \frac{1}{2}$, choose $u = -\frac{1}{4}\operatorname{sign}(x)$: then $|x^+| = \frac{1}{2}|x| - \frac{1}{4}$ and $|x| - |x^+| = \frac{1}{2}|x| + \frac{1}{4} \ge \frac{1}{2}$ for $|x| \ge \frac{1}{2}$. So V decreases by at least $\epsilon = \frac{1}{2}$ per step above $\beta_0 = \frac{1}{2}$, and from $x_0 = 3$ the set $\{|x| \le \frac{1}{2}\}$ is reached within $\lceil 3/(1/2) \rceil = 6$ steps; in fact two steps suffice ($3 \to 1.25 \to 0.375$), so the bound is conservative.

Example D.6 (Contraction and the Bellman operator)**.** For a single state with two actions of reward 1 and 0 and $\gamma = \frac{1}{2}$, $TV = \max(1 + \frac{1}{2}V, \frac{1}{2}V) = 1 + \frac{1}{2}V$. From $V_0 = 0$ the iterates are $0, 1, 1.5, 1.75, \cdots \to 2 = V^*$, and the error halves each step, as the contraction rate $\gamma = \frac{1}{2}$ predicts.

Exercises

Exercise D.7. Compute the viability kernel of $K = [-5, 5]$ for $x^+ = \frac{3}{2}x + u$, $|u| \le 1$, by Result D.1 and verify by iterating the predecessor operator twice.

Exercise D.8. Show that for $A = \begin{pmatrix} 0.9 & 1 \\ 0 & 0.9 \end{pmatrix}$ the Euclidean norm of $A^n x_0$ is not monotone in n although $\rho(A) < 1$, and find a norm with $\|A\|_* < 1$.

Exercise D.9. Prove the uniqueness part of Result D.4.

Exercise D.10. Give a system and a function V satisfying the nonincrease condition of Result D.2 for which the sublevel set is invariant but recovery from outside never happens (no strict decrease).

Appendix E

Optimization

E.1 Linear programming

Established result E.1 (Strong duality)**.** *If the primal* $\max\{c^\top x : Ax \le b,\ x \ge 0\}$ *is feasible and bounded, then so is the dual* $\min\{b^\top y : A^\top y \ge c,\ y \ge 0\}$*, and the optimal values are equal.*

Source: von Neumann; Gale, Kuhn and Tucker (1951); Schrijver, Section 7.4.

New result E.2 (Concavity of the optimal value in the right-hand side)**.** *Let* $f(b) = \max\{c^\top x : Ax \le b,\ x \ge 0\}$ *on the set* $\mathcal{B}$ *of* b *for which the problem is feasible and bounded. Then* $\mathcal{B}$ *is convex and* f *is concave on it. If* f *is nondecreasing in a scalar component* $d = b_k$ *and* $d_{\min}$ *is the least feasible* d *(others fixed), then for* $d_{\min} < a \le d_1 < d_2$*,*

$$\frac{f(d_2) - f(d_1)}{d_2 - d_1} \ \le\ \frac{f(a) - f(d_{\min})}{a - d_{\min}}.$$

Assumptions: Feasible and bounded for the b considered.

Proof. If x_1, x_2 are optimal for b_1, b_2 then $\lambda x_1 + (1-\lambda)x_2$ is feasible for $\lambda b_1 + (1-\lambda)b_2$ with value $\lambda f(b_1) + (1-\lambda) f(b_2)$, so f of the combination is at least that, which is concavity and shows feasibility is preserved. For a concave function of one variable the chord slopes are nonincreasing as the interval moves right: for $u < v < w < z$ the slope on $[u, v]$ is at least that on $[w, z]$. Apply this with $[d_{\min}, a]$ and $[d_1, d_2]$ (using $a \le d_1$). □

This is the inequality used in Result 20.6.

E.2 Dynamic programming

For a finite MDP with discount $\gamma \in [0, 1)$, rewards bounded by $\bar{r}$, the Bellman operator is $(TV)(s) = \max_a \big[r(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s')\big]$.

New result E.3 (Contraction and error bound)**.** T *is a* γ*-contraction in the sup-norm. Hence there is a unique fixed point* V^* *and, for the iterates* $V_k = T^k V_0$*,*

$$\|V_k - V^*\|_\infty \le \frac{\gamma}{1-\gamma}\|V_k - V_{k-1}\|_\infty, \qquad \|V_k - V^*\|_\infty \le \gamma^k \|V_0 - V^*\|_\infty.$$

Assumptions: Finite state and action sets.

Proof. For V, W and each s, $|TV(s) - TW(s)| \le \max_a \gamma \sum_{s'} P(s'|s,a)|V(s') - W(s')| \le \gamma \|V - W\|_\infty$, since $|\max_a \alpha_a - \max_a \beta_a| \le \max_a |\alpha_a - \beta_a|$. Existence, uniqueness and the second bound are Result D.4. For the first, $\|V_k - V^*\| \le \|V_k - V_{k+1}\| + \|V_{k+1} - V^*\| \le \|V_k - V_{k+1}\| + \gamma\|V_k - V^*\|$, whence $\|V_k - V^*\| \le \|V_{k+1} - V_k\|/(1-\gamma) \le \gamma\|V_k - V_{k-1}\|/(1-\gamma)$ using the contraction again. □

E.3 Convex optimization

Established result E.4 (KKT conditions under Slater's condition)**.** *For a convex problem* $\min f_0(x)$ *subject to convex* $f_i(x) \le 0$ *and affine equalities, if some point is strictly feasible then* x^* *is optimal if and only if there exist multipliers* $\lambda_i \ge 0$ *with* $\nabla f_0(x^*) + \sum \lambda_i \nabla f_i(x^*) = 0$ *and* $\lambda_i f_i(x^*) = 0$.

Source: Boyd and Vandenberghe, *Convex Optimization*, Section 5.5.

E.4 Submodular maximization

A set function F is *submodular* if $F(A \cup \{e\}) - F(A) \ge F(B \cup \{e\}) - F(B)$ for $A \subseteq B$, $e \notin B$.

Established result E.5 (Greedy approximation)**.** *For a monotone nonnegative submodular* F *with* $F(\varnothing) = 0$*, the greedy algorithm that adds the element of largest marginal gain until* k *elements are chosen achieves at least* $(1 - 1/e)$ *of the maximum of* F *over sets of size at most* k.

Source: Nemhauser, Wolsey and Fisher (1978).

This is the guarantee behind the acquisition results of Chapter 10.

E.5 Worked examples

Example E.6 (Concave value of a budget)**.** $\max x_1 + 2x_2$ subject to $x_1 + x_2 \le d$, $x_1 \le 1$, $x \ge 0$. For $d \le 1$ the optimum is $x_2 = d$ with value $2d$; for $1 \le d$ the optimum puts $x_2 = d$ still (since x_2 is worth more), so $f(d) = 2d$ for all $d \ge 0$. Add the constraint $x_2 \le 1$: then $f(d) = 2d$ for $d \le 1$, $f(d) = 2 + (d-1) = d + 1$ for $1 \le d \le 2$, and $f(d) = 3$ for $d \ge 2$. The slopes $2, 1, 0$ decrease, as concavity requires, and the chord inequality of Result E.2 holds, for instance $\frac{f(2) - f(1.5)}{0.5} = 1 \le \frac{f(1) - f(0)}{1} = 2$.

Example E.7 (Greedy on a coverage function)**.** Universe $\{1, \dots, 6\}$ and sets $S_1 = \{1, 2, 3, 4\}$, $S_2 = \{1, 2, 5\}$, $S_3 = \{3, 4, 6\}$. The coverage function is monotone submodular. For $k = 2$ greedy picks S_1 (4 elements) then S_2 or S_3 (adding 1 each), covering 5, while $S_2 \cup S_3$ covers 6. The ratio is $5/6 > 1 - 1/e$, consistent with Result E.5, and shows that greedy is not exact.

Exercises

Exercise E.8. Write the dual of the linear program in Example E.6 (first version) and verify strong duality at $d = 2$.

Exercise E.9. Run value iteration by hand for the two-state MDP with $\gamma = 0.9$ in which state 1 has actions (stay, reward 1) and (move to 2, reward 0), and state 2 has the single action (stay, reward 2). Report V_1, V_2, V_3 from $V_0 = 0$ and the stopping iteration for $\epsilon = 0.1$.

Exercise E.10. Prove that the coverage function is submodular.

Exercise E.11. Verify the KKT conditions for $\min x^2$ subject to $x \geq 1$.

Appendix F

Stochastic Processes

F.1 Finite Markov chains

New result F.1 (Absorption and hitting probabilities)**.** *Let a finite Markov chain with transition matrix P have a set B of target states. The vector $h(s) = \Pr_s(\text{the chain ever enters } B)$ is the minimal nonnegative solution of $h = 1$ on B, $h(s) = \sum_{s'} P(s, s')h(s')$ off B. If the states off B from which B is unreachable are removed, the solution is unique, and $h = (I - Q)^{-1}R\mathbf{1}$ where Q is P restricted to non-target states and R the one-step transitions into B.*

Assumptions: Finite chain.

Proof. Let $h_n(s)$ be the probability of entering B within n steps; $h_0 = 1_B$ and $h_{n+1} = 1_B + 1_{B^c}Ph_n$ and $h_n \uparrow h$. Any nonnegative solution g of the system satisfies $g \geq h_0$ and, inductively, $g \geq h_n$, so $g \geq h$; and h solves the system by monotone convergence, hence it is the minimal solution. If B is reachable from every non-target state, then $Q^n \to 0$ (the probability of remaining outside B for N steps is at most $1 - \epsilon$ for some N and $\epsilon > 0$), so $I - Q$ is invertible and the solution unique. □

F.2 Elementary inequalities and laws

Established result F.2 (Boole's inequality)**.** *For events $E_1, \ldots, E_T$: $\Pr(\bigcup E_t) \leq \sum_t \Pr(E_t)$.*

Source: Boole (1854).

Established result F.3 (Hoeffding's inequality)**.** *For independent $Y_1, \ldots, Y_N \in [0, 1]$ with mean μ, $\Pr\big(|\frac{1}{N}\sum Y_i - \mu| \geq \epsilon\big) \leq 2e^{-2N\epsilon^2}$.*

Source: Hoeffding (1963).

New result F.4 (Memoryless laws)**.** *A positive random variable T with $\Pr(T > s + t \mid T > s) = \Pr(T > t)$ for all $s, t \geq 0$ and $\Pr(T > t)$ right-continuous is exponential. A discrete positive-integer variable with the same property is geometric.*

Assumptions: As stated.

Proof. Let $G(t) = \Pr(T > t)$. Then $G(s+t) = G(s)G(t)$. By right-continuity and monotonicity, the only nonincreasing right-continuous solutions of this Cauchy equation with $G(0) = 1$ are $G(t) = e^{-ht}$ for some $h \in [0, \infty]$ (if $G(t_0) = 0$ for some $t_0 > 0$ then $G(t_0) = G(t_0/2)^2$ forces $G(t_0/2) = 0$ and by iteration $G(t) = 0$ for all $t > 0$, the degenerate $h = \infty$). In the discrete case $G(n) = G(1)^n$. □

Established result F.5 (Second Borel–Cantelli lemma)**.** *If $E_1, E_2, \ldots$ are independent events with $\sum_t \Pr(E_t) = \infty$ then almost surely infinitely many of them occur.*

Source: Borel (1909); Cantelli (1917).

This is the mechanism for the absence of forever-safety under unbounded independent noise in Chapter 13: if each step has a probability bounded below of leaving the constraint set, the leaving event recurs.

F.3 Gaussian tails

Established result F.6 (Gaussian quantile)**.** *If $X \sim \mathcal{N}(\mu, \sigma^2)$ then $\Pr(X \le \mu + z\sigma) = \Phi(z)$; in particular $\Phi^{-1}(0.95) \approx 1.645$ and $\Phi^{-1}(0.99) \approx 2.326$. Moreover, $1 - \Phi(z) \le \frac{1}{2}e^{-z^2/2}$ for $z \ge 0$.*

Source: Standard.

F.4 Worked examples

Example F.7 (Hitting probability in a gambler's chain)**.** States $0, \ldots, 4$ with 0 and 4 absorbing, and from $s \in \{1, 2, 3\}$ a move up or down with probability $\frac{1}{2}$. The target is $B = \{4\}$. The system $h(0) = 0$, $h(4) = 1$, $h(s) = \frac{1}{2}h(s-1) + \frac{1}{2}h(s+1)$ has the unique solution $h(s) = s/4$ by Result F.1: the harmonic equation on $1, 2, 3$ with the boundary values $h(0) = 0$, $h(4) = 1$ is linear in s. For a bias $p > \frac{1}{2}$ upward the solution is $h(s) = \frac{1-r^s}{1-r^4}$ with $r = (1-p)/p$.

Example F.8 (Boole allocation over a horizon)**.** To keep the total failure probability over $T = 20$ steps below 0.05 it suffices, by Result F.2, that each step fails with probability at most 0.0025; for a Gaussian margin this is $z = \Phi^{-1}(0.9975) \approx 2.81$, against 1.645 for a single step at level 0.05. The price of the horizon is the increase of the margin from 1.645 to 2.81 standard deviations.

Exercises

Exercise F.9. Compute the hitting probability of the target $\{4\}$ from state 2 in the biased gambler's chain with $p = 0.6$.

Exercise F.10. Show that the sum of two independent exponential random variables with the same rate is not memoryless.

Exercise F.11. Using Hoeffding's inequality, find how many samples ensure that the empirical frequency is within 0.02 of the true probability with confidence 0.99.

Exercise F.12. Prove the first Borel–Cantelli lemma and explain why independence is not needed there.

Appendix G

Numerical Methods

The algorithms of the volume are exact in exact arithmetic. This appendix states what is needed to run them in floating point and what guarantees survive.

G.1 Fixed-point iteration for kernels and values

New result G.1 (Finite termination of kernel iteration)**.** *For a finite system with $|X|$ states the iteration $S_0 = C$, $S_{k+1} = \Gamma(S_k) \cap S_k$ (with Γ the one-step predecessor operator) stabilizes after at most $|X| - |C^c|$ steps, and the stable set is the kernel. A state's removal time is the least number of steps within which an adversary can force it out.*

Assumptions: Finite state set.

Proof. The sets decrease, and the iteration stops as soon as two consecutive sets agree, since then $\Gamma(S_k) \supseteq S_k$ and S_k is a viability domain; each non-final step removes at least one state, so there are at most $|C|$ steps. The removal-time statement is the induction of Result C.3. □

New result G.2 (Stopping rule for value iteration)**.** *Value iteration for a discounted MDP stopped at the first k with $\|V_k - V_{k-1}\|_\infty \le \frac{1-\gamma}{\gamma}\epsilon$ returns V_k with $\|V_k - V^*\|_\infty \le \epsilon$. The number of iterations needed from $V_0 = 0$ is at most $\lceil \log(\bar{r}/((1-\gamma)\epsilon))/\log(1/\gamma) \rceil$, where $\bar{r}$ bounds $|r|$. The policy greedy with respect to V_k has value within $2\gamma\epsilon/(1-\gamma)$ of optimal at every state.*

Assumptions: Exact arithmetic.

Proof. The first bound is Result E.3. For the count, $\|V_0 - V^*\| \le \bar{r}/(1-\gamma)$ and $\gamma^k \bar{r}/(1-\gamma) \le \epsilon$ once $k \ge \log(\bar{r}/((1-\gamma)\epsilon))/\log(1/\gamma)$. For the policy let π be greedy for V_k, so $T_\pi V_k = TV_k$. Then $\|V^\pi - V^*\| \le \|V^\pi - T_\pi V_k\| + \|TV_k - TV^*\| \le \gamma\|V^\pi - V_k\| + \gamma\|V_k - V^*\|$, and $\|V^\pi - V_k\| \le \|V^\pi - V^*\| + \|V^* - V_k\|$, so $(1-\gamma)\|V^\pi - V^*\| \le 2\gamma\|V_k - V^*\| \le 2\gamma\epsilon$. □

G.2 Spectral radius

Established result G.3 (Power iteration)**.** *If A has a dominant eigenvalue λ_1 with $|\lambda_1| > |\lambda_2| \ge \cdots$ and the starting vector has a nonzero component along the dominant eigenvector, then $\|A^{k+1}v\|/\|A^k v\| \to |\lambda_1|$ at rate $O(|\lambda_2/\lambda_1|^k)$.*

Source: Golub and Van Loan, *Matrix Computations*, Section 7.3.

For a nonnegative reducible matrix with several components of equal spectral radius, the iteration should be run per strongly connected component (Result C.1) and the maximum taken, as in the growth theorem of Chapter 11. For a periodic irreducible matrix the ratio oscillates and a geometric mean over a period is used.

G.3 Monte Carlo certification of chance constraints

New result G.4 (Sample size for a chance constraint)**.** *To decide, from N independent simulated trajectories, whether the probability of constraint violation q is at most q_0, estimate $\hat{q}$ as the violation frequency. Accepting when $\hat{q} \le q_0 - \epsilon$ certifies $q \le q_0$ with confidence at least $1 - \delta$ provided $N \ge \ln(2/\delta)/(2\epsilon^2)$, in the sense that the probability of accepting when $q > q_0$ is at most $\delta/2$. The sample size does not depend on the dimension of the state or the length of the horizon.*

Assumptions: Independent trajectories from the true model.

Proof. If $q > q_0$ and $\hat{q} \le q_0 - \epsilon$, then $\hat{q} \le q - \epsilon$, which by Result F.3 has probability at most $e^{-2N\epsilon^2} \le \delta/2$ for the stated N (one-sided form of the inequality). □

The guarantee is about the model that is simulated. It transfers to the real system only through the model error bounds of Chapter 20.

G.4 Floating-point and thresholds

Decisions that compare a computed quantity to a threshold (the kernel membership $b \ge (1+\delta)x$, the break-even $g > hR$) are discontinuous in the inputs. Under floating-point error of size η, the decision is correct whenever the margin exceeds η times the condition number of the computation. The reliable remedy is interval arithmetic: compute an interval containing the exact value and decline to decide when the interval contains the threshold. All kernel computations in the examples of this volume are over integers or rationals and therefore exact.

G.5 Worked examples

Example G.5 (Iterations to ϵ)**.** With $\gamma = 0.9$, rewards in $[0, 1]$ and $\epsilon = 0.01$, Result G.2 requires at most $\lceil \log(1/(0.1 \cdot 0.01)) / \log(1/0.9) \rceil = \lceil 6.91/0.1054 \rceil = 66$ iterations from $V_0 = 0$. A tolerance of 10^{-6} needs about 131, so the cost is logarithmic in the accuracy and grows like $1/(1-\gamma)$ in the discount.

Example G.6 (Samples for a chance constraint)**.** To certify a violation probability below $q_0 = 0.05$ with margin $\epsilon = 0.02$ and confidence $\delta = 0.01$, Result G.4 requires $N \ge \ln(200)/(2 \cdot 0.0004) = 5.30/0.0008 \approx 6623$ trajectories. Halving the margin to 0.01 quadruples this to about 26 500.

Exercises

Exercise G.7. Compute the number of iterations of Result G.2 for $\gamma = 0.99$, $\bar{r} = 1$ and $\epsilon = 10^{-3}$.

Exercise G.8. Run the power iteration by hand for $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ from $v = (1, 0)^{\top}$ for three steps and compare the ratios with $\rho(A) = 3$.

Exercise G.9. Explain why the power iteration fails for a permutation matrix of a cycle, and how a geometric mean over a period repairs the growth estimate.

Exercise G.10. Design an interval-arithmetic test for the kernel condition $b \geq (1 + \delta)x$ when δ is known only to within $\pm 10^{-3}$.